PUC Minas

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Sumário

- Hospedagem de aplicações Web
  - Plataformas em nuvem para hospedagem
  - Preparando a aplicação
  - Publicando a aplicação em uma solução PaaS (Platform as a Service)

Sumário

- Hospedagem de aplicações Web
  - **Plataformas em nuvem para hospedagem**
  - Preparando a aplicação
  - Publicando a aplicação em uma solução PaaS (Platform as a Service)

Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**
  - Plataformas em nuvem para hospedagem representam uma revolução no modo como empresas e indivíduos gerenciam e implantam suas aplicações e dados.

  - Essa tecnologia inovadora proporciona uma abordagem flexível, escalável e eficiente para a gestão de recursos computacionais, permitindo que organizações alcancem seus objetivos de forma mais ágil e econômica.

Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**
  - Plataformas em nuvem referem-se a ambientes virtuais oferecidos por provedores de serviços, nos quais usuários podem armazenar, processar e acessar dados e aplicativos pela internet.

  - Essas plataformas eliminam a necessidade de infraestrutura física, como servidores locais, proporcionando uma abordagem mais dinâmica e baseada em recursos virtuais.

Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**

Plataformas em nuvem

- **Principais características**
  - **Escalabilidade**: As plataformas em nuvem oferecem a capacidade de dimensionar recursos de maneira dinâmica conforme a demanda. Isso significa que os usuários podem facilmente aumentar ou diminuir a capacidade de armazenamento, processamento e rede, garantindo eficiência operacional.

  - **Acessibilidade**: Os dados e aplicativos hospedados na nuvem podem ser acessados de qualquer lugar do mundo, desde que haja uma conexão com a internet. Isso facilita a colaboração remota e o acesso aos recursos da empresa em qualquer dispositivo.

Plataformas em nuvem

- **Principais características**
  - **Gerenciamento simplificado**: Os provedores de nuvem geralmente oferecem interfaces de gerenciamento intuitivas que simplificam tarefas como provisionamento, monitoramento e manutenção, permitindo que mesmo usuários com menos conhecimento técnico possam gerenciar recursos de maneira eficaz.

  - **Pagamento por uso**: Um dos benefícios mais atrativos das plataformas em nuvem é o modelo de pagamento por uso. Isso significa que os usuários pagam apenas pelos recursos que realmente utilizam, tornando a infraestrutura mais econômica e flexível.

Plataformas em nuvem

- **Aplicações práticas**
  - **Hospedagem de websites**: As plataformas em nuvem oferecem uma solução eficaz para hospedar websites, garantindo alta disponibilidade e desempenho consistente.
  - **Desenvolvimento de aplicações**: Desenvolvedores podem usar ambientes em nuvem para criar, testar e implantar aplicações de forma rápida e eficiente, aproveitando os recursos sob demanda.

Plataformas em nuvem

- **Aplicações práticas**
  - **Armazenamento de dados**: Empresas podem utilizar a nuvem para armazenar grandes volumes de dados de maneira segura, acessível e escalável.
  - **Análise de dados**: Plataformas em nuvem fornecem poder computacional para análise de dados complexa, permitindo insights valiosos para tomada de decisões.

Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**
  - As plataformas em nuvem são ferramentas essenciais na era digital, oferecendo flexibilidade, eficiência e economia. Ao compreender esses conceitos, as organizações podem transformar suas operações e otimizar recursos, garantindo uma posição competitiva no cenário tecnológico atual.

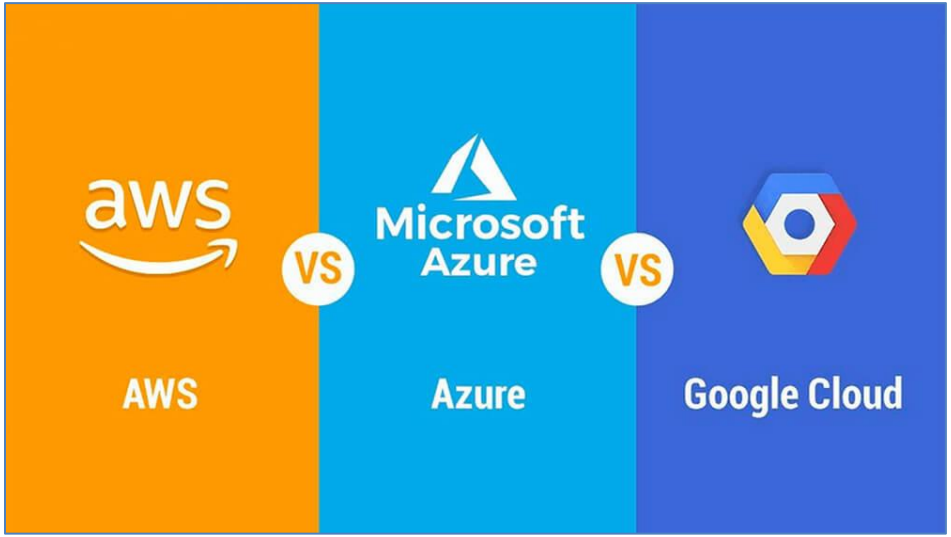
Hospedagem de aplicações Web

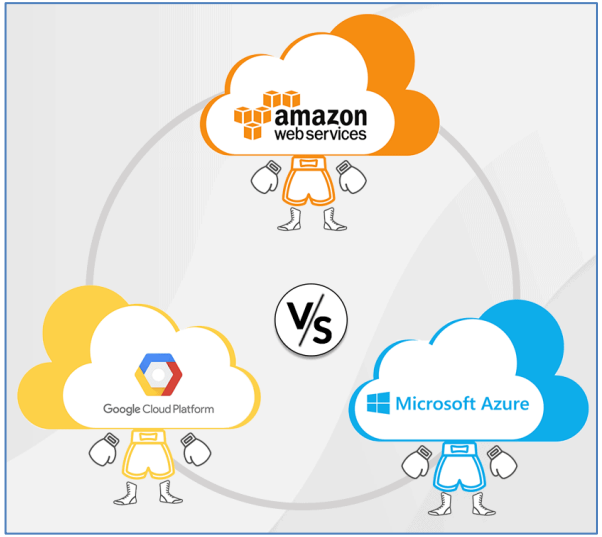
- **Plataformas em nuvem para hospedagem**
  - Existem várias plataformas de nuvem populares no mercado, cada uma oferecendo uma variedade de serviços e recursos.

  - Vamos começar pelo “The big 3”.

Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**
  - “The big 3”:





Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**

	amazon web services	Microsoft Azure	
Compute Services	Elastic Compute Cloud (EC2)	Virtual Machines	Compute Engine
Object Storage	Amazon S3	Azure Blob Storage	Cloud Storage
Networking	Amazon VPC	Azure Virtual Network	Cloud Virtual Network

Hospedagem de aplicações Web

- **Plataformas em nuvem para hospedagem**

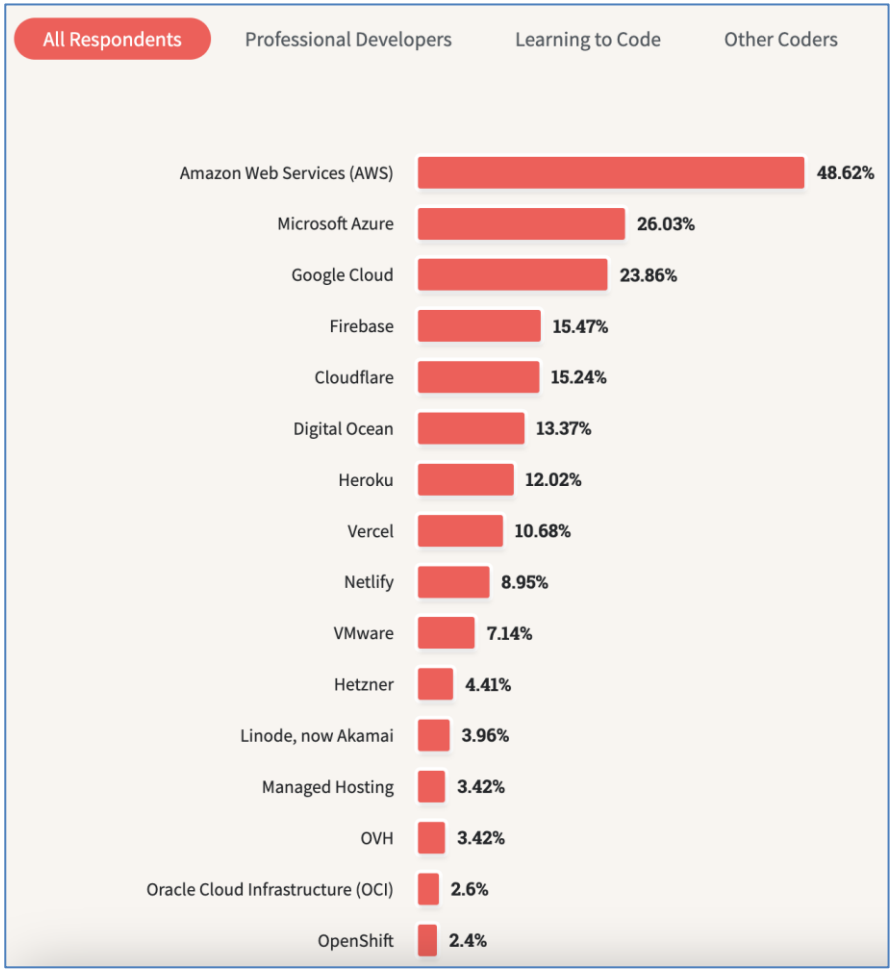
Produto	AWS	Azure	GCP
Machine learning	Amazon SageMaker	Azure Machine Learning	Google Cloud AI Platform
Image recognition	Amazon Rekognition	Azure Cognitive Services	Google Cloud Vision
Speach	Amazon Polly, Amazon Transcribe	Azure Cognitive Services	Google Cloud Speech-to-Text and Text-to-Speech
Natural language processing	Amazon Comprehend	Azure Cognitive Services	Google Cloud Natural Language API
Big Data Analytics	Databricks	Databricks	Databricks
Chat Bot	AWS Chatbot	Azure Bot Service	Dialogflow
Pricing	Per hour	Per minute	Per minute

Plataformas em nuvem

- Fonte: **Stack Overflow Survey 2023**

  https://survey.stackoverflow.co/2023/#cloud-platforms

- Tipo de pesquisa:
  - Todos os respondentes
  - Respondentes: **69.549**

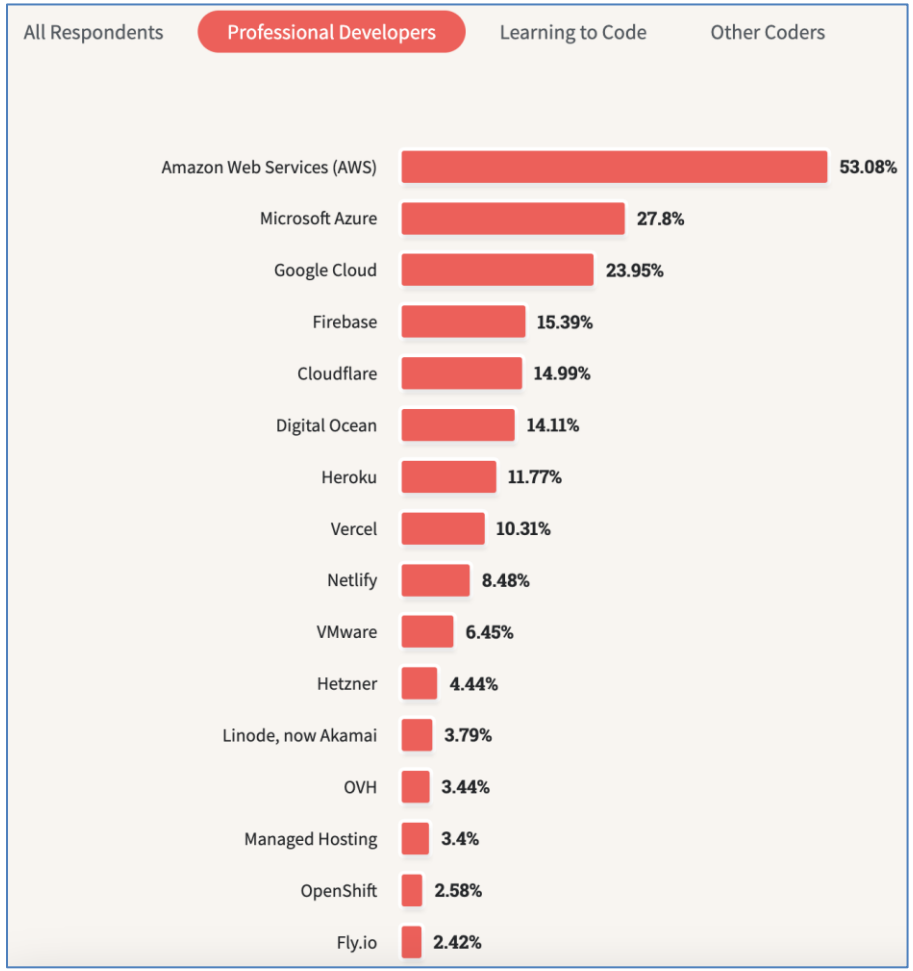


Plataformas em nuvem

- Fonte: **Stack Overflow Survey 2023**

  https://survey.stackoverflow.co/2023/#cloud-platforms

- Tipo de pesquisa:
  - Devs profissionais
  - Respondentes: **55.540**

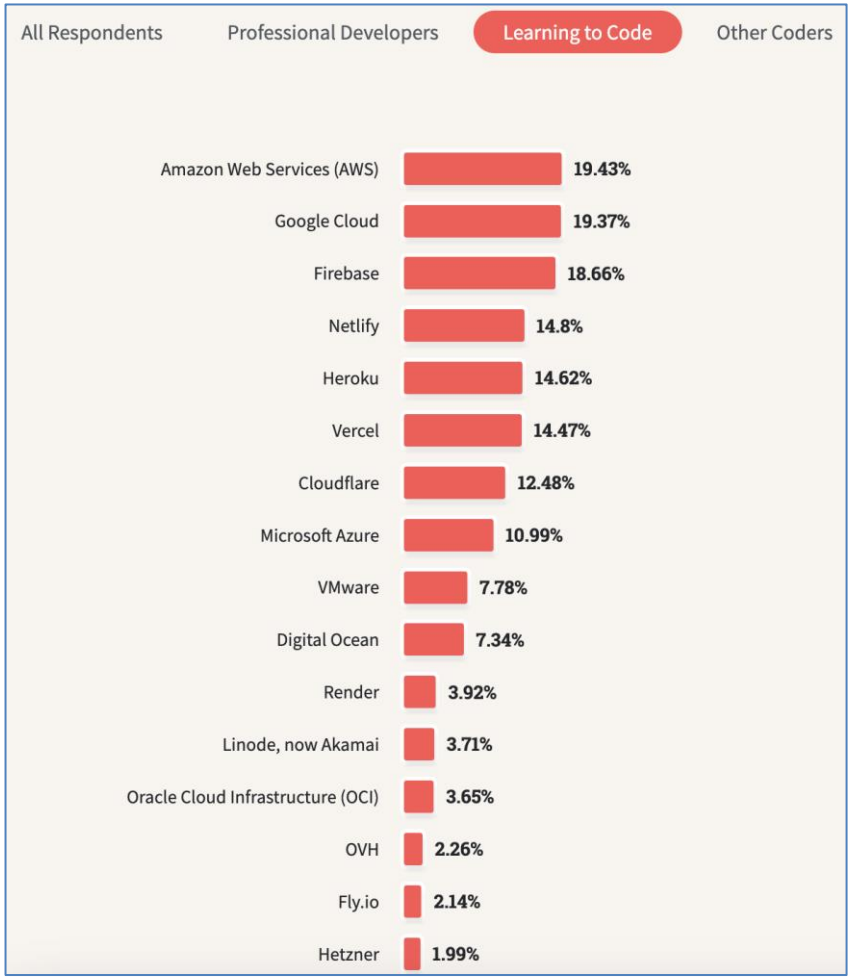


Plataformas em nuvem

- Fonte: **Stack Overflow Survey 2023**

  https://survey.stackoverflow.co/2023/#cloud-platforms

- Tipo de pesquisa:
  - Aprendendo a programar
  - Respondentes: **3.366**



Plataformas em nuvem

- **Stack Overflow Survey 2023**
  - A AWS permanece como a plataforma de nuvem mais utilizada por todos os entrevistados. Ela conquista facilmente o primeiro lugar, quase dobrando a porcentagem da segunda plataforma de nuvem mais utilizada por todos os entrevistados, o Azure.

Plataformas em nuvem

- **Stack Overflow Survey 2023**
  - As pessoas que estão aprendendo a programar ainda estão usando a AWS (26%) em maior número, mas há uma maior equidade entre as três plataformas de nuvem (26% Google Cloud e 25% Firebase).

Plataformas em nuvem

- **Stack Overflow Survey 2023**
  - Curiosamente, o Heroku foi a plataforma de nuvem mais utilizada no ano passado (2022) por aqueles que estão aprendendo a programar, mas caiu para a quinta posição este ano (2023).

Plataformas em nuvem

- **Stack Overflow Survey 2023**
  - É possível observar a crescente influência do Azure nas organizações — o dobro de Desenvolvedores Profissionais estão utilizando o Azure em comparação com pessoas que estão aprendendo a programar (30% vs. 15%).

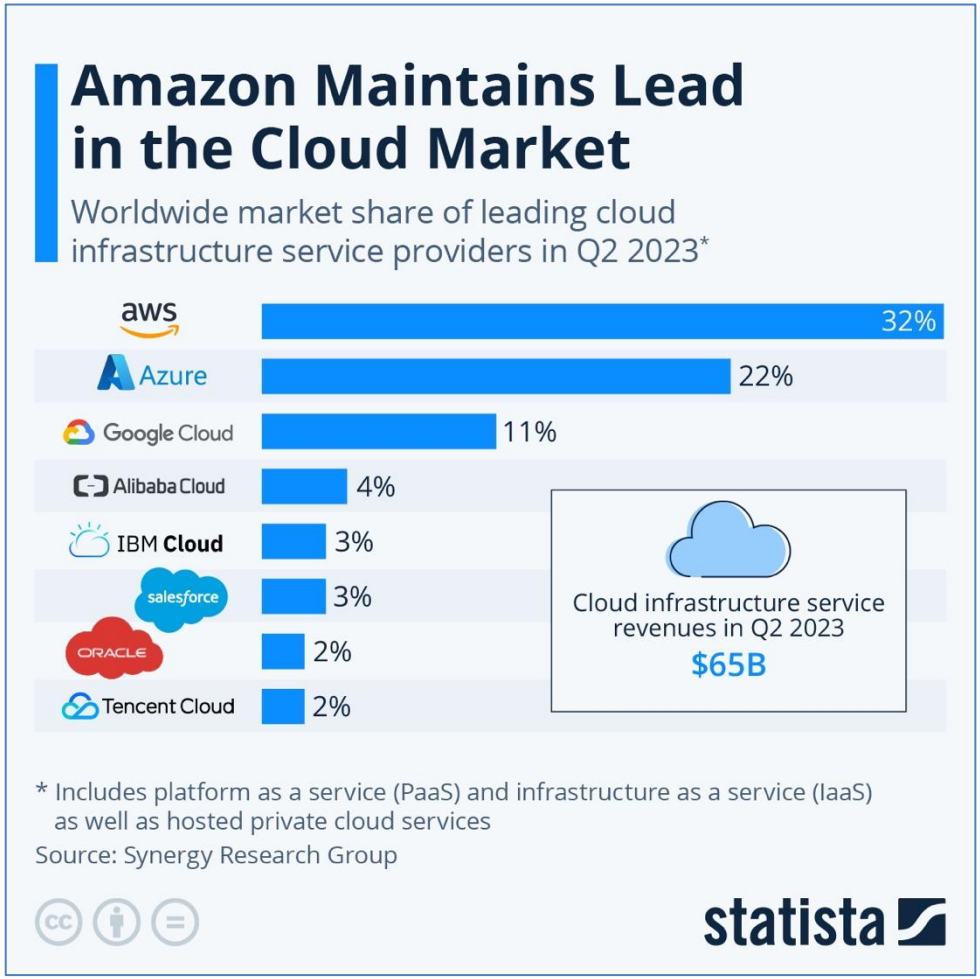
Plataformas em nuvem

- Fonte: **Statista**

  https://www.statista.com/chart/18819/worldwide-market-share-of-leading-cloud-infrastructure-service-providers/

- Leitura sugerida:

  https://k21academy.com/amazon-web-services/aws-solutions-architect/aws-vs-azure-vs-gcp/



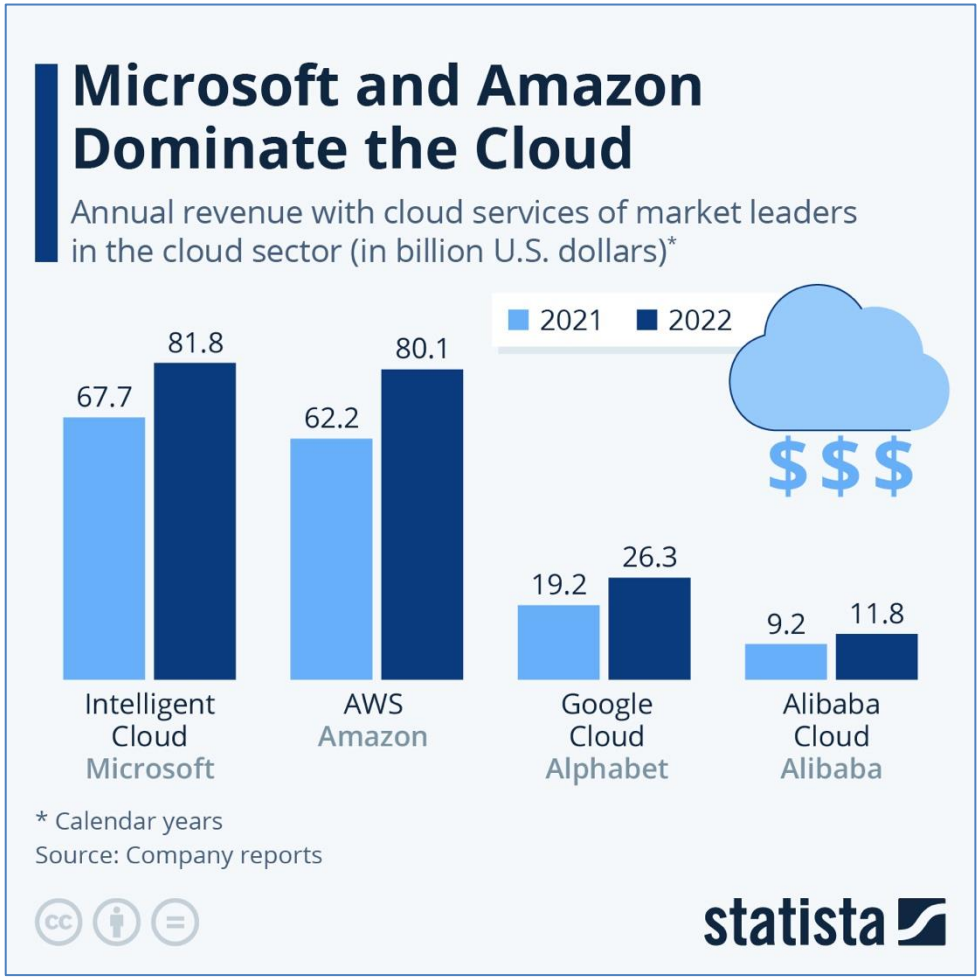
Plataformas em nuvem

- Fonte: **Statista**

  https://www.statista.com/chart/30489/revenue-from-cloud-services-by-cloud-sector-market-leaders/

- Leitura sugerida:

  https://cloud.google.com/docs/get-started/aws-azure-gcp-service-comparison?hl=pt-br



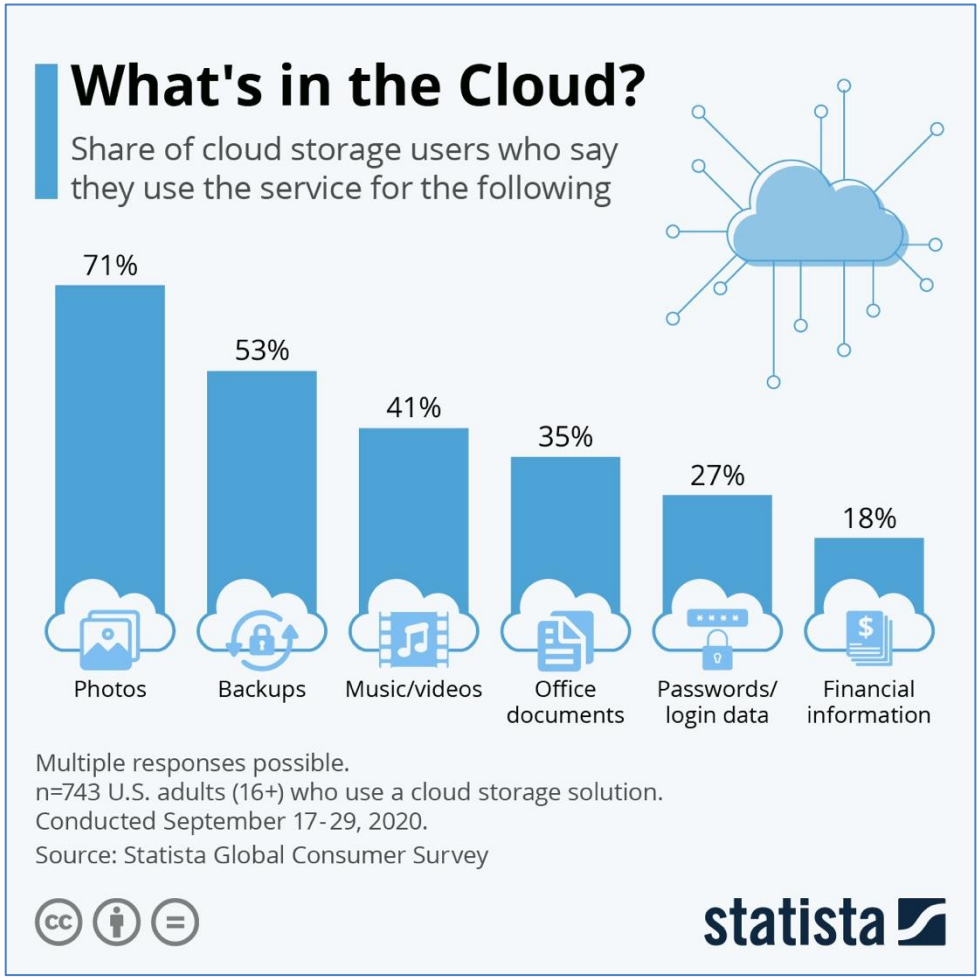
Plataformas em nuvem

- Fonte: **Statista**

  https://www.statista.com/chart/25896/gcs-cloud-storage-services-usage/

- Leitura sugerida:

  https://www.digitalocean.com/resources/article/comparing-aws-azure-gcp



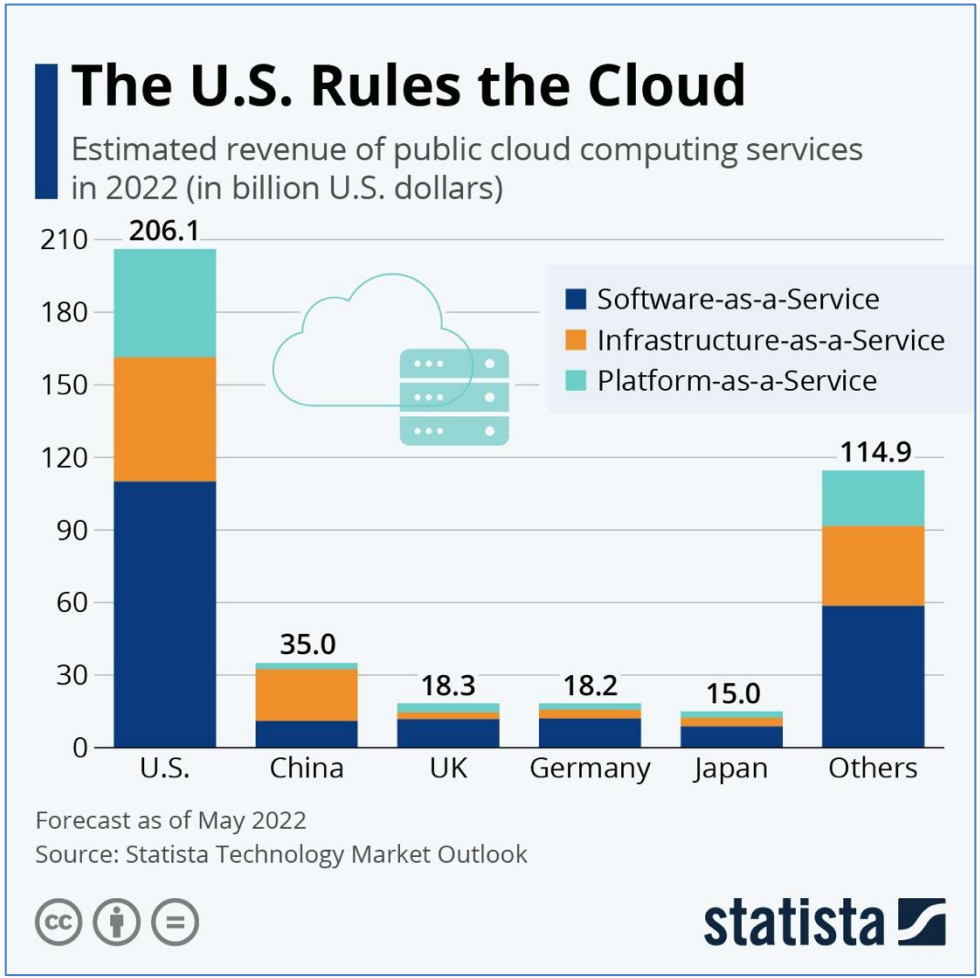
Plataformas em nuvem

- Fonte: **Statista**

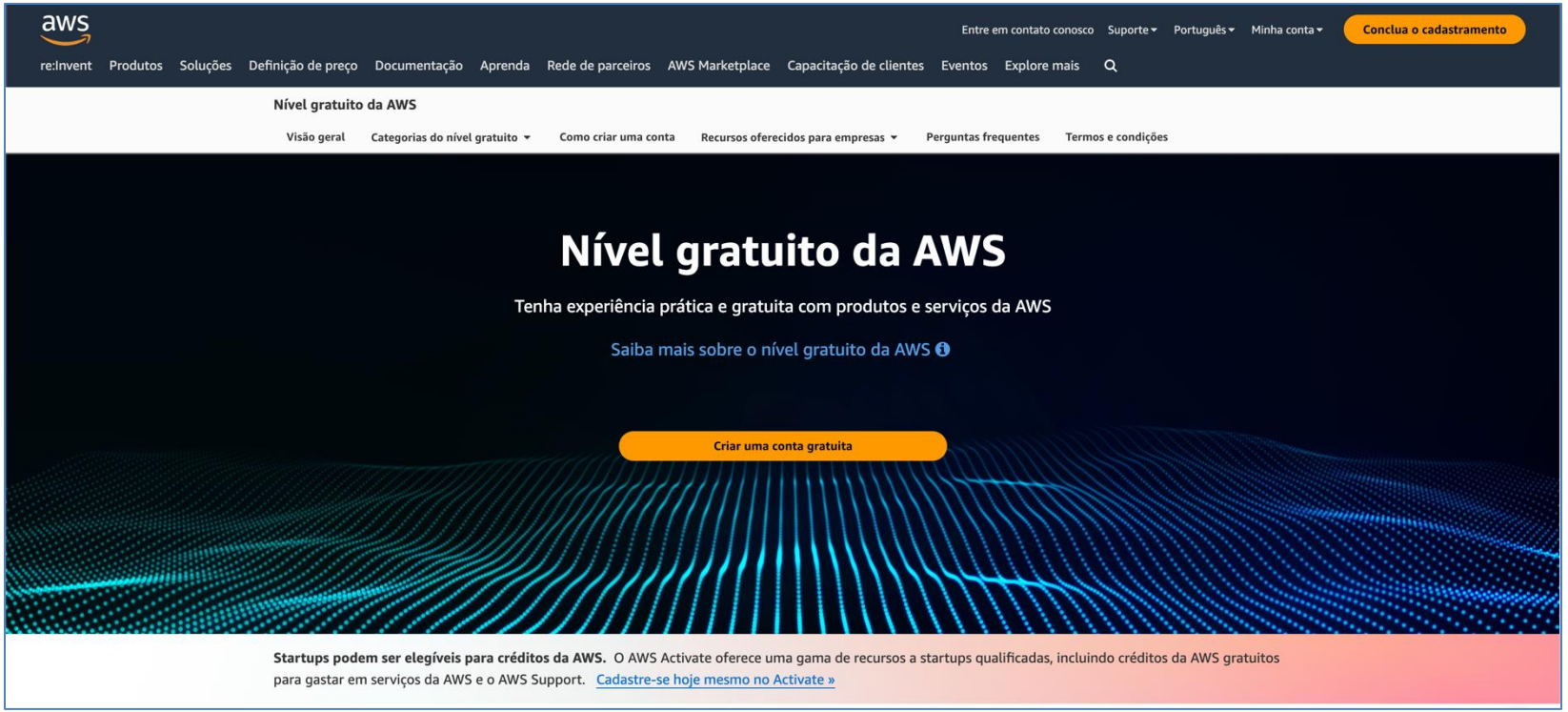
  https://www.statista.com/chart/22253/estimated-revenue-public-cloud-computing-top-markets/

- Leitura sugerida:

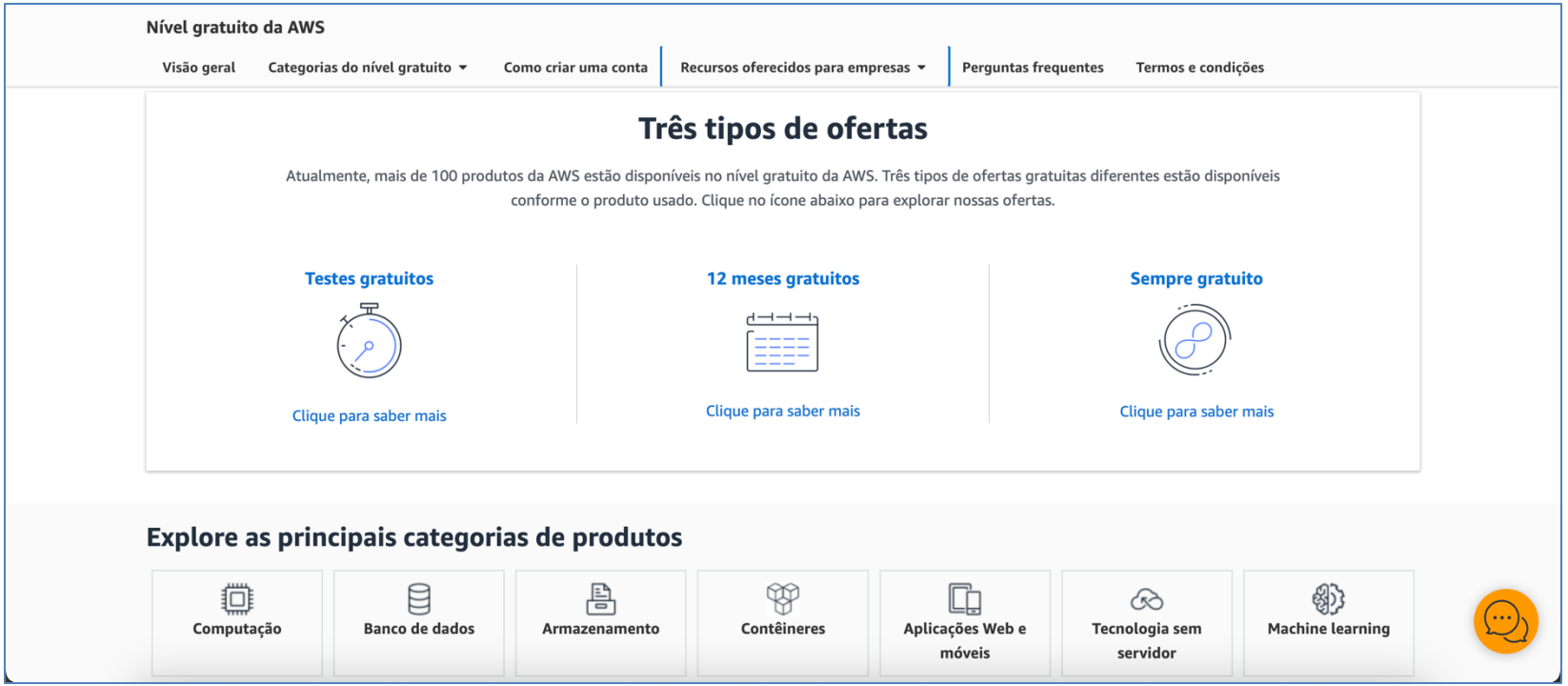
  https://www.cloudzero.com/blog/aws-vs-azure-vs-google-cloud/



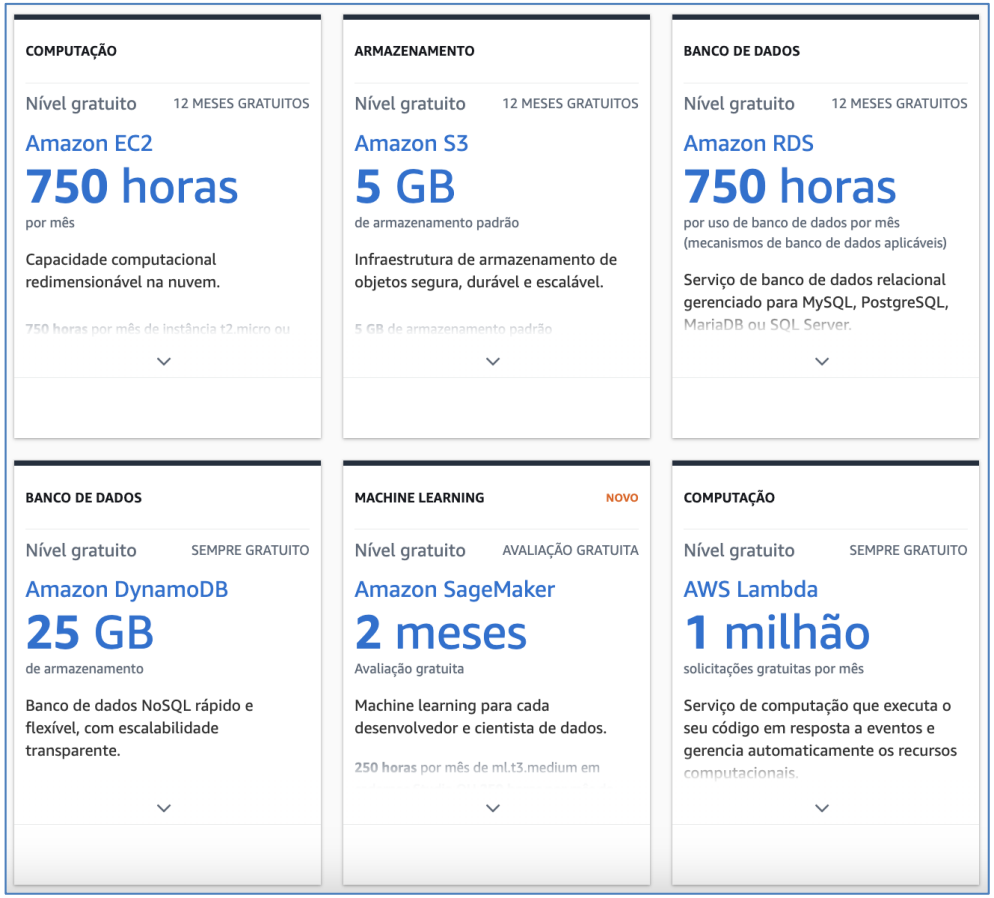
- **Ferramentas**: Plataformas de nuvem
  - **AWS**: https://aws.amazon.com/pt/free/



- **Ferramentas**: Plataformas de nuvem
  - **AWS**: https://aws.amazon.com/pt/free/

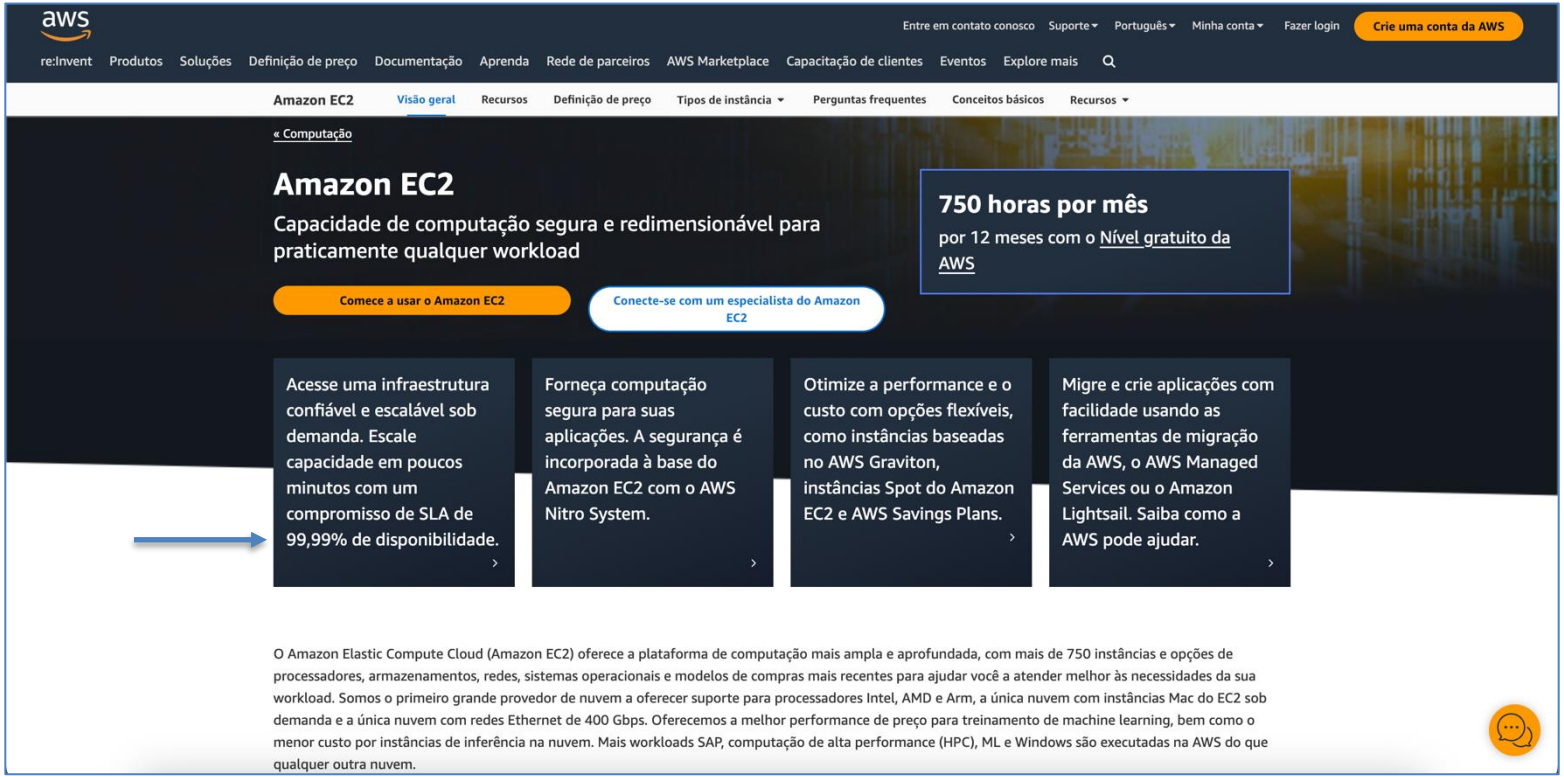


- **Ferramentas**: Plataformas de nuvem
  - **AWS**: https://aws.amazon.com/pt/free/



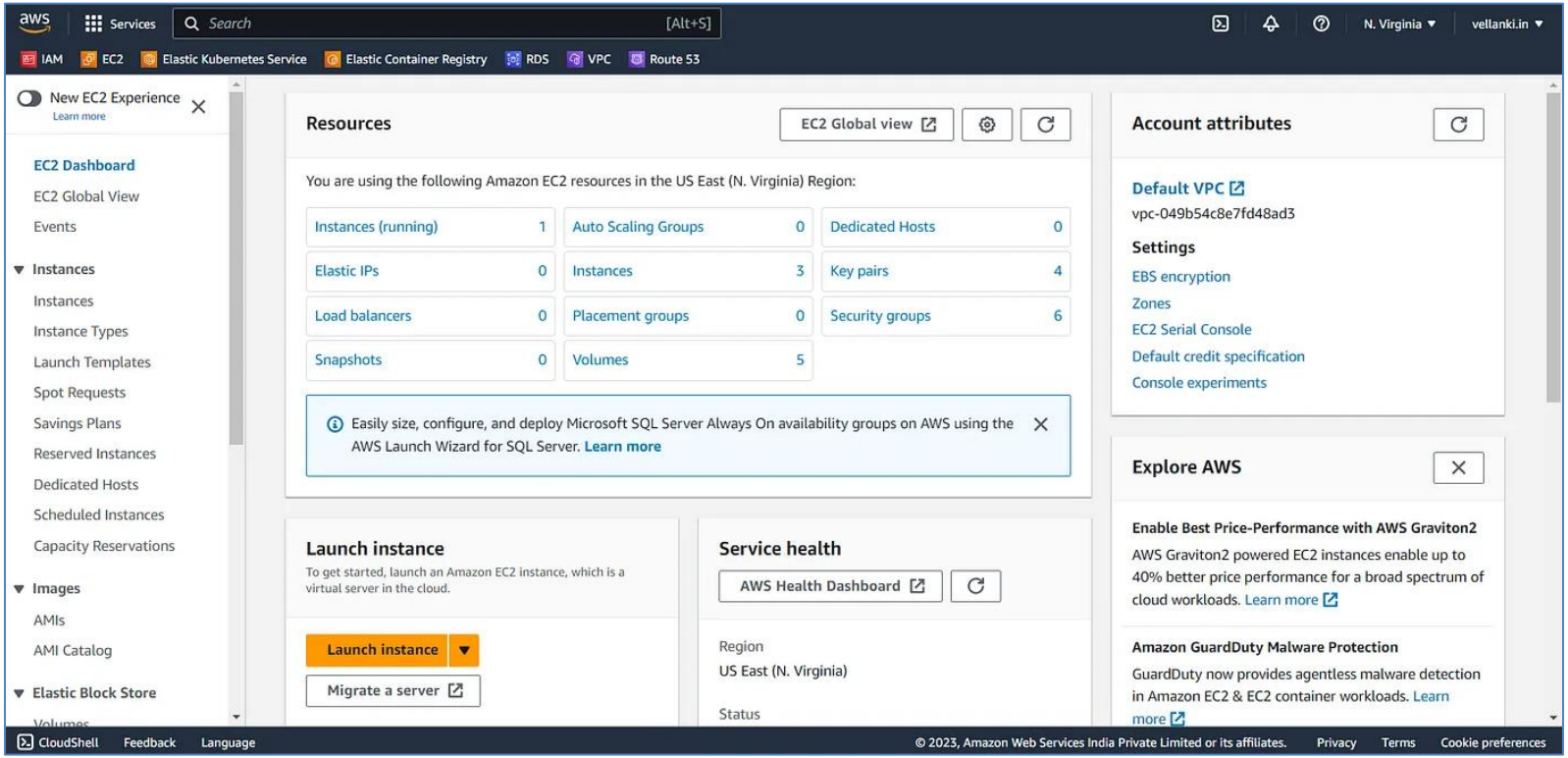
- https://aws.amazon.com/pt/free/free-tier-faqs/

- **Ferramentas**: Plataformas de nuvem
  - **Amazon EC2**: https://aws.amazon.com/pt/ec2/

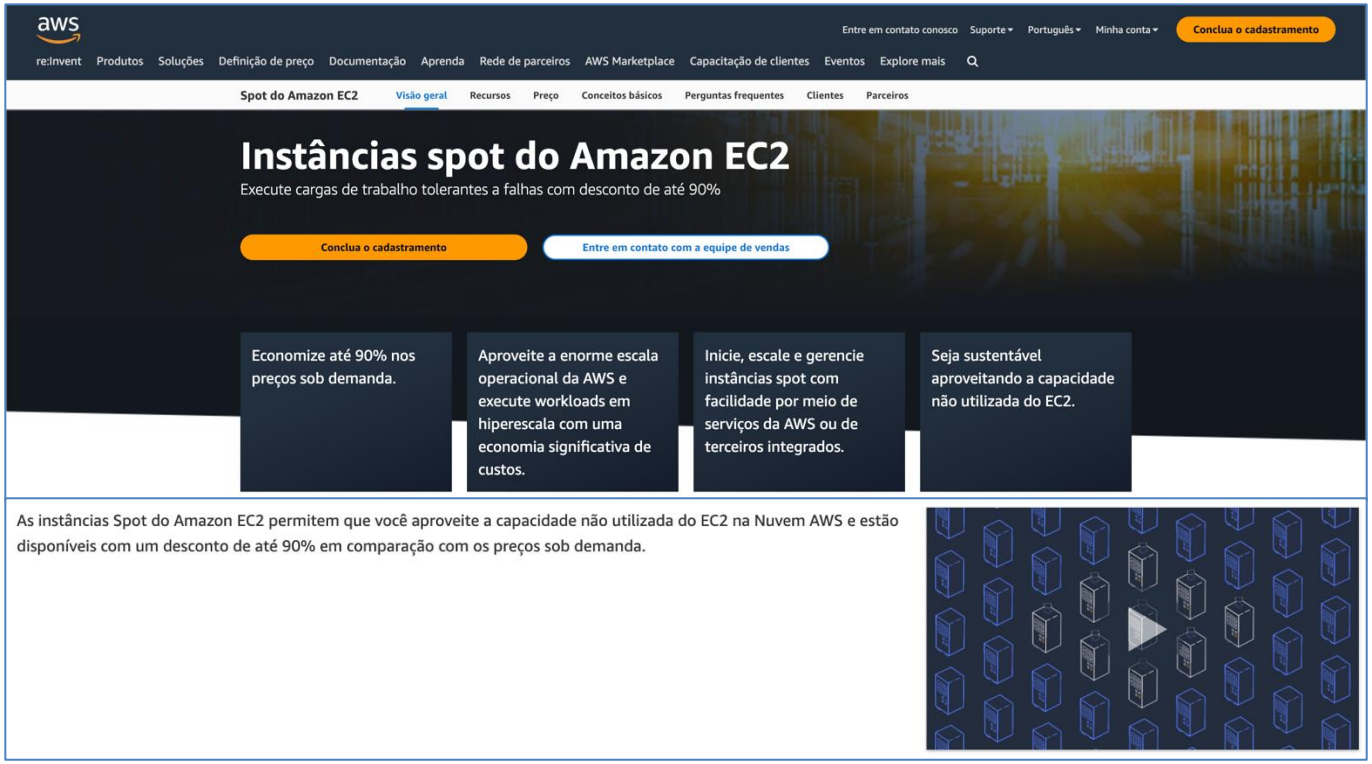


- **Ferramentas**: Plataformas de nuvem
  - **Amazon EC2**: https://aws.amazon.com/pt/ec2/

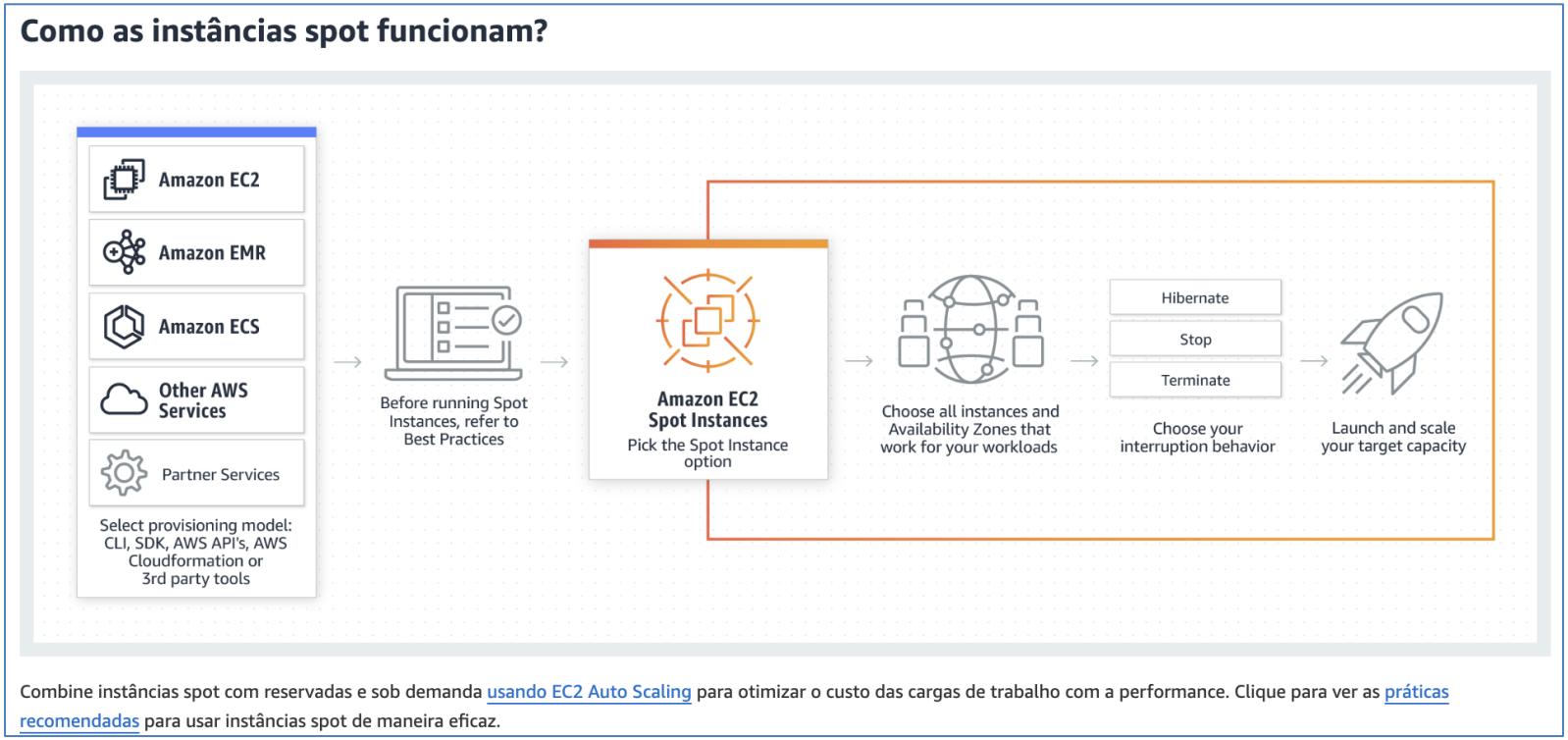
- Leitura sugerida:
  - https://medium.com/@kotivellanki



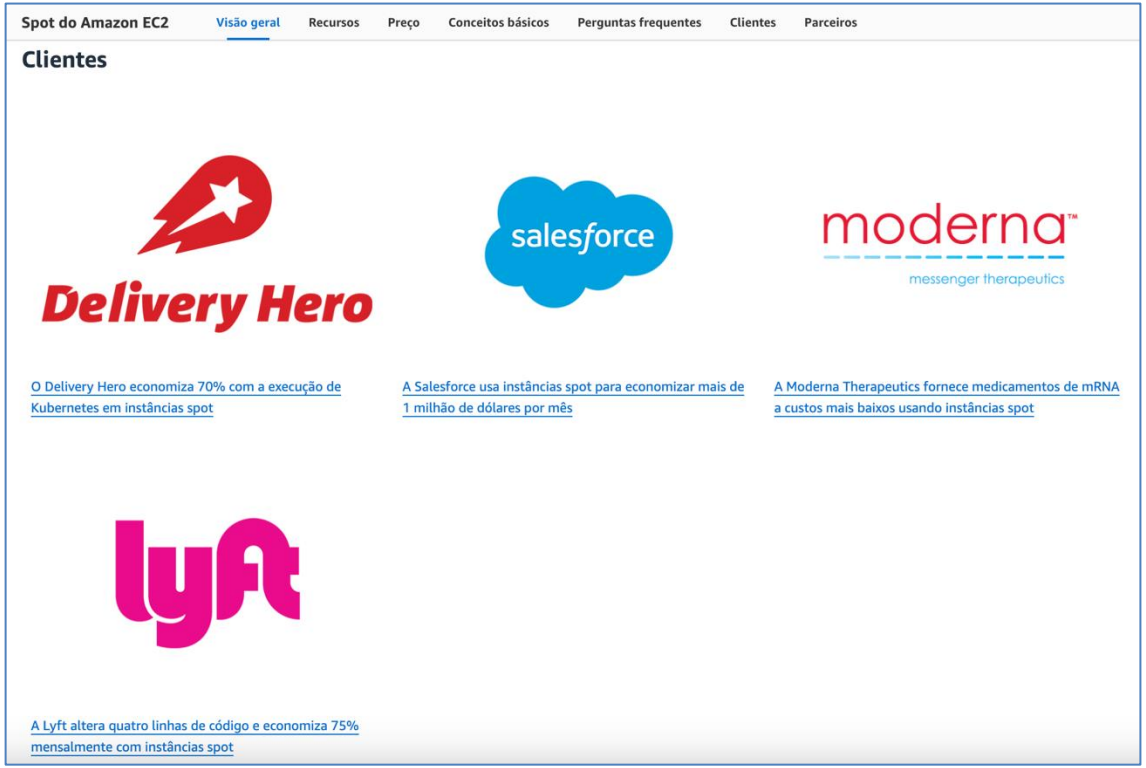
- **Ferramentas**: Plataformas de nuvem
  - **Amazon EC2 Spot**: https://aws.amazon.com/pt/ec2/spot/



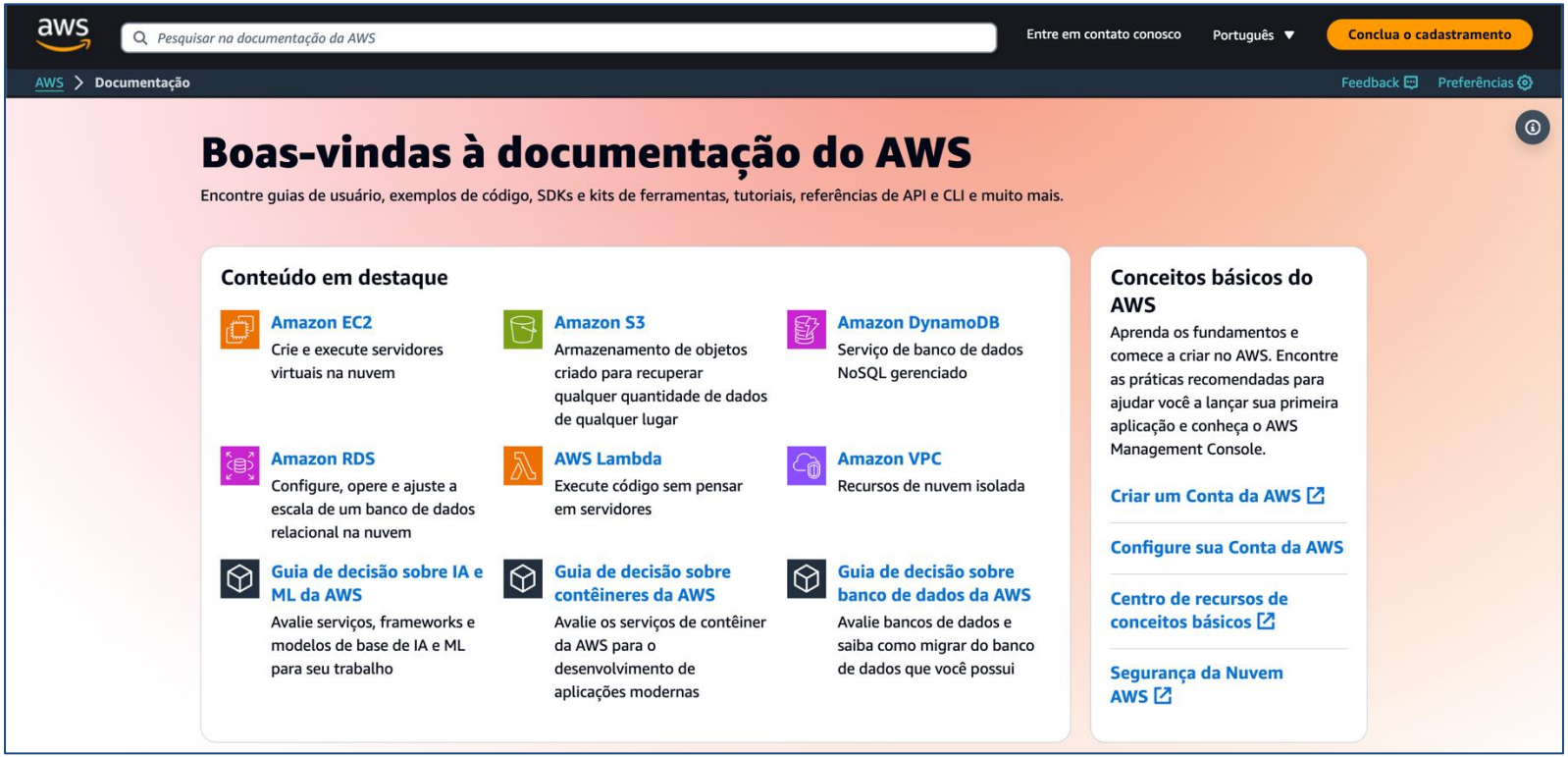
- **Ferramentas**: Plataformas de nuvem
  - **Amazon EC2 Spot**: https://aws.amazon.com/pt/ec2/spot/



- **Ferramentas**: Plataformas de nuvem
  - **Amazon EC2 Spot**: https://aws.amazon.com/pt/ec2/spot/



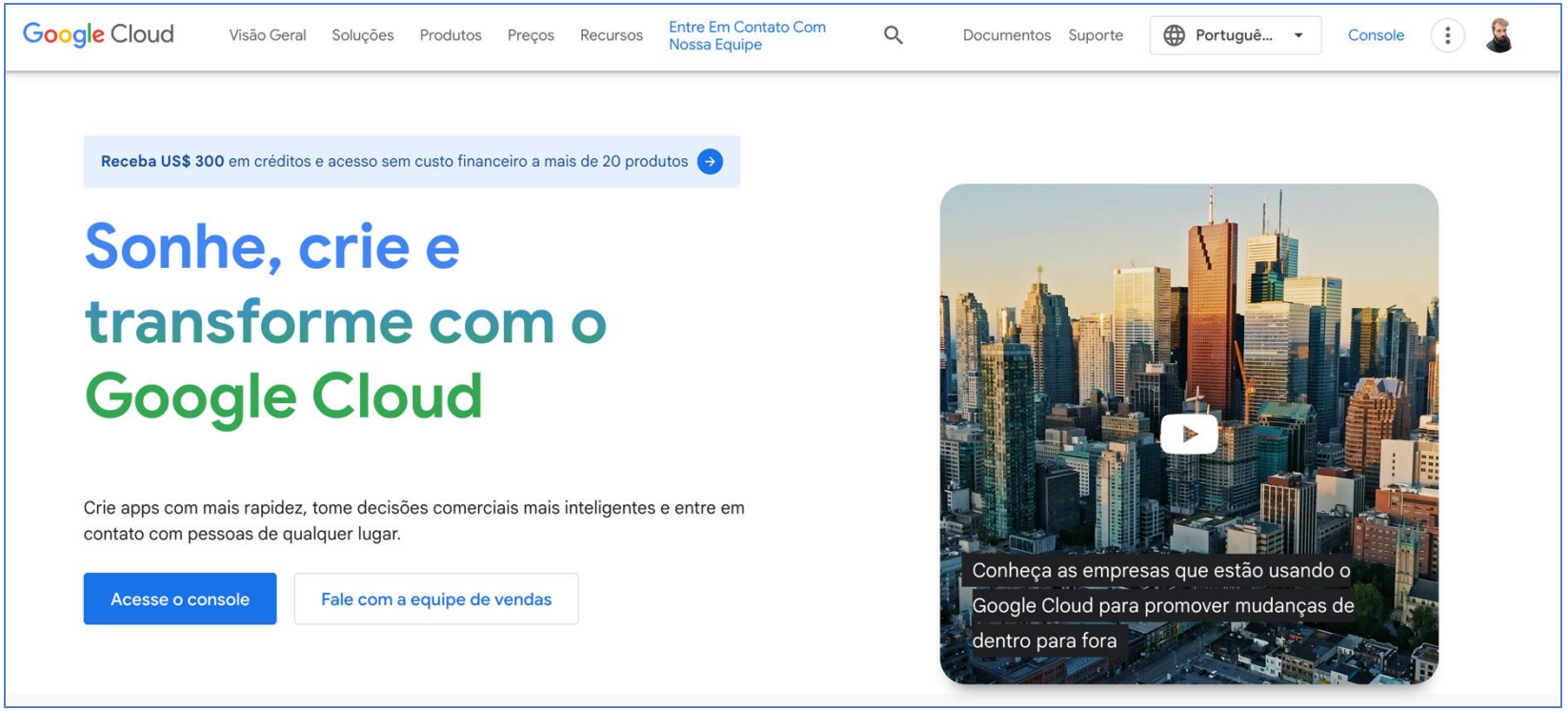
- Documentação da AWS:
  - https://docs.aws.amazon.com/pt_br/



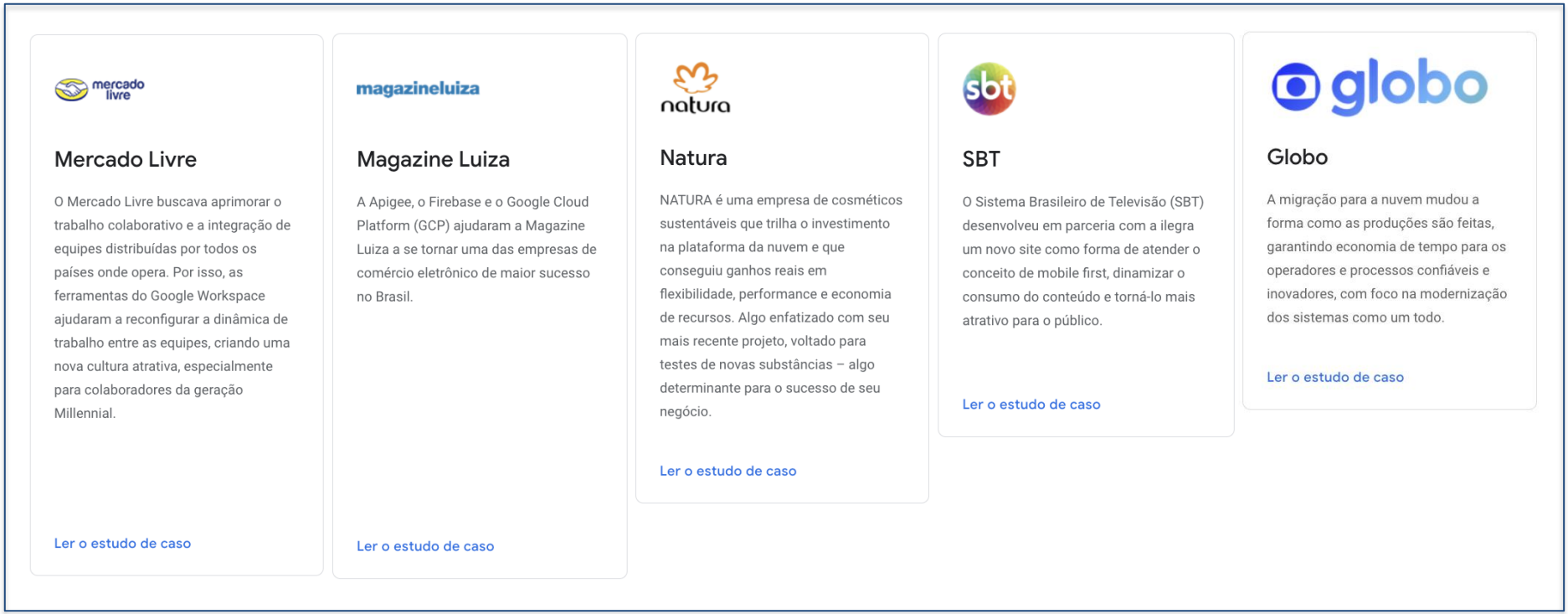
- Documentação da AWS:
  - https://docs.aws.amazon.com/pt_br/

- Evento Cloud Experience da AWS:
  - https://aws.amazon.com/pt/events/cloudexperience/

- **Ferramentas**: Plataformas de nuvem
  - **Google Cloud Platform (GCP)**: https://cloud.google.com/?hl=pt_br



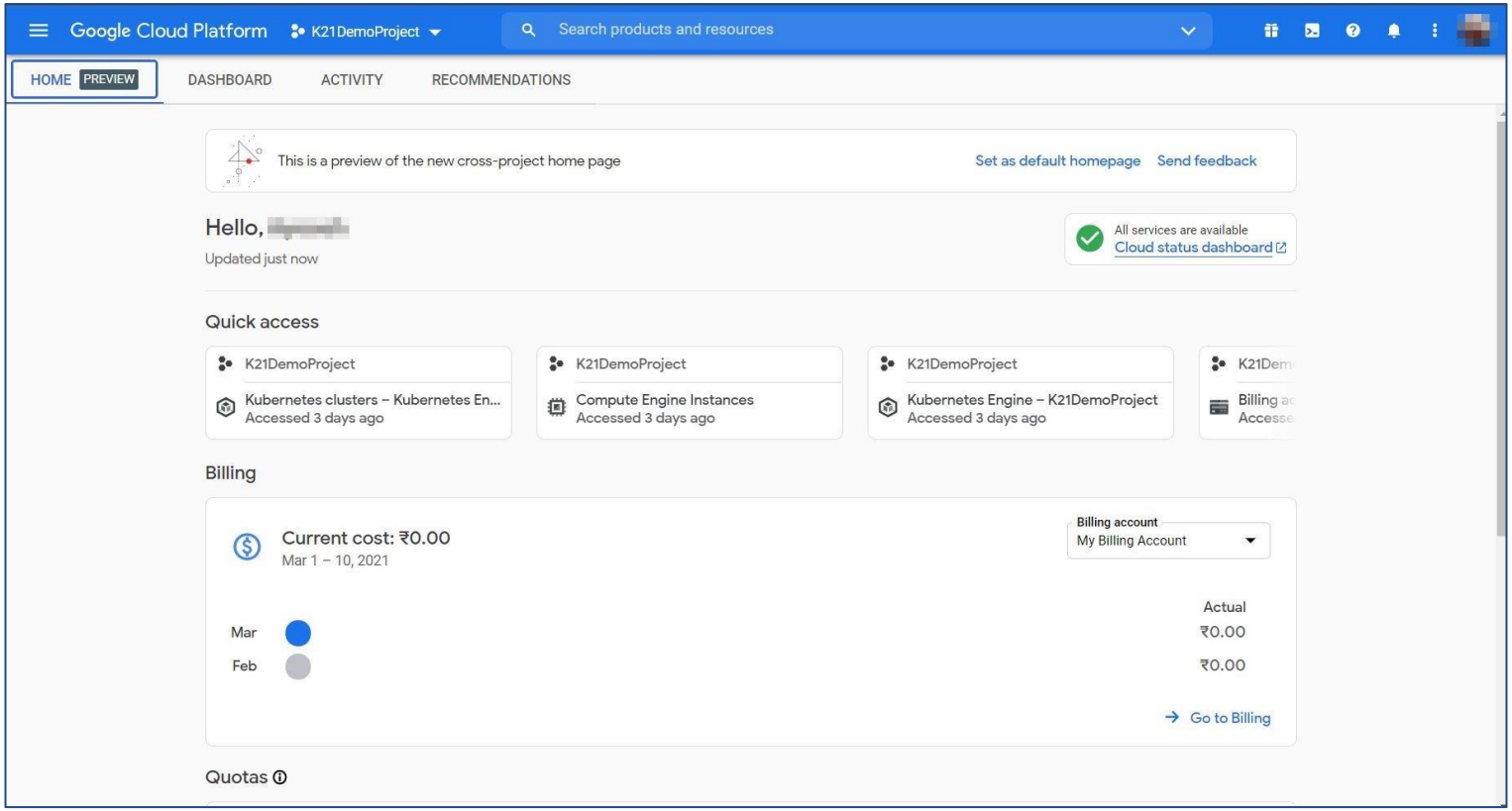
- **Ferramentas**: Plataformas de nuvem
  - **Google Cloud Platform (GCP)**: https://cloud.google.com/?hl=pt_br



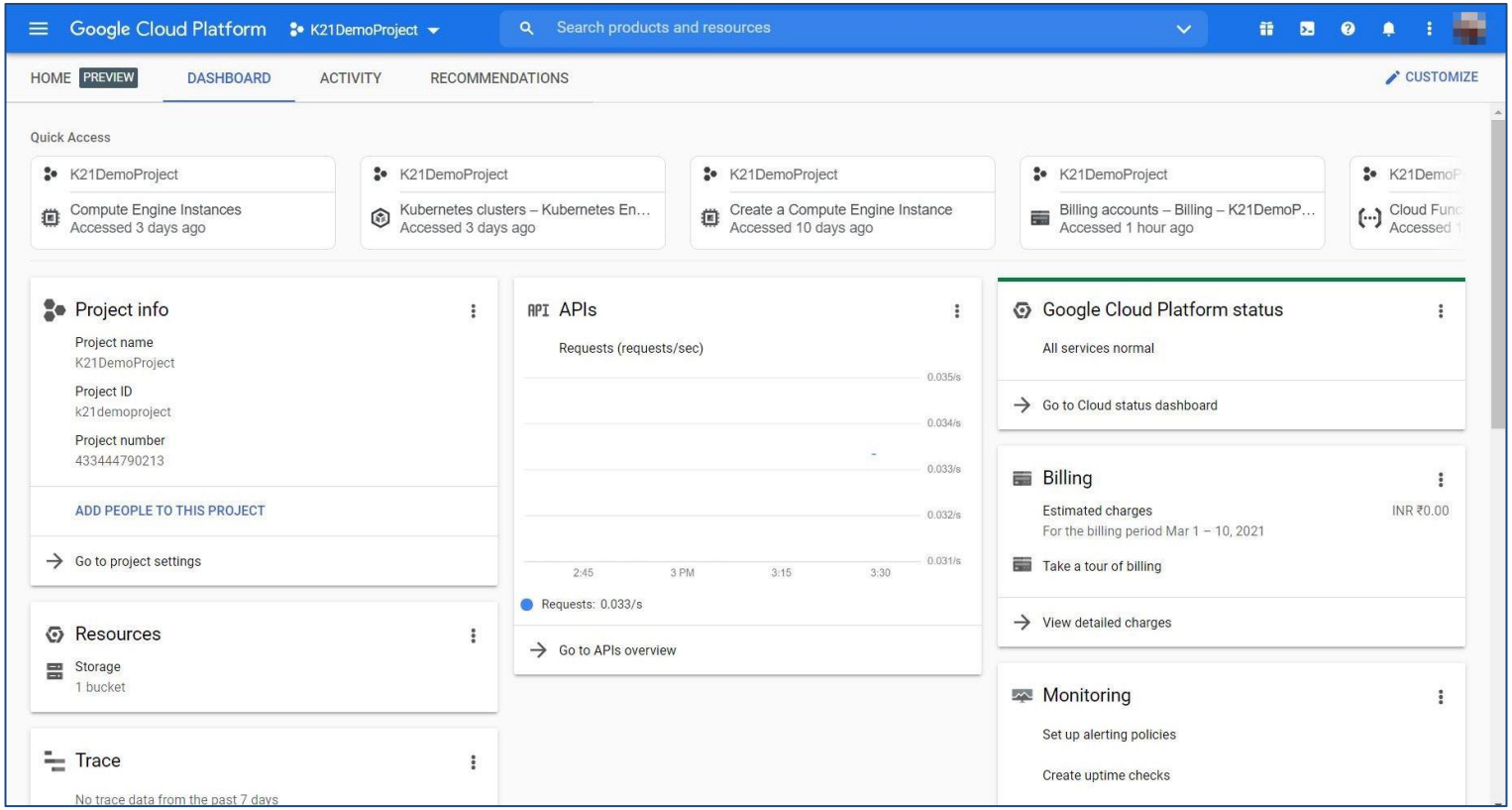
- **Ferramentas**: Plataformas de nuvem
  - **Google Cloud Platform (GCP)**: https://cloud.google.com/?hl=pt_br

- **Ferramentas**: Plataformas de nuvem
  - **Google Cloud Platform (GCP)**: https://cloud.google.com/free?hl=pt_br

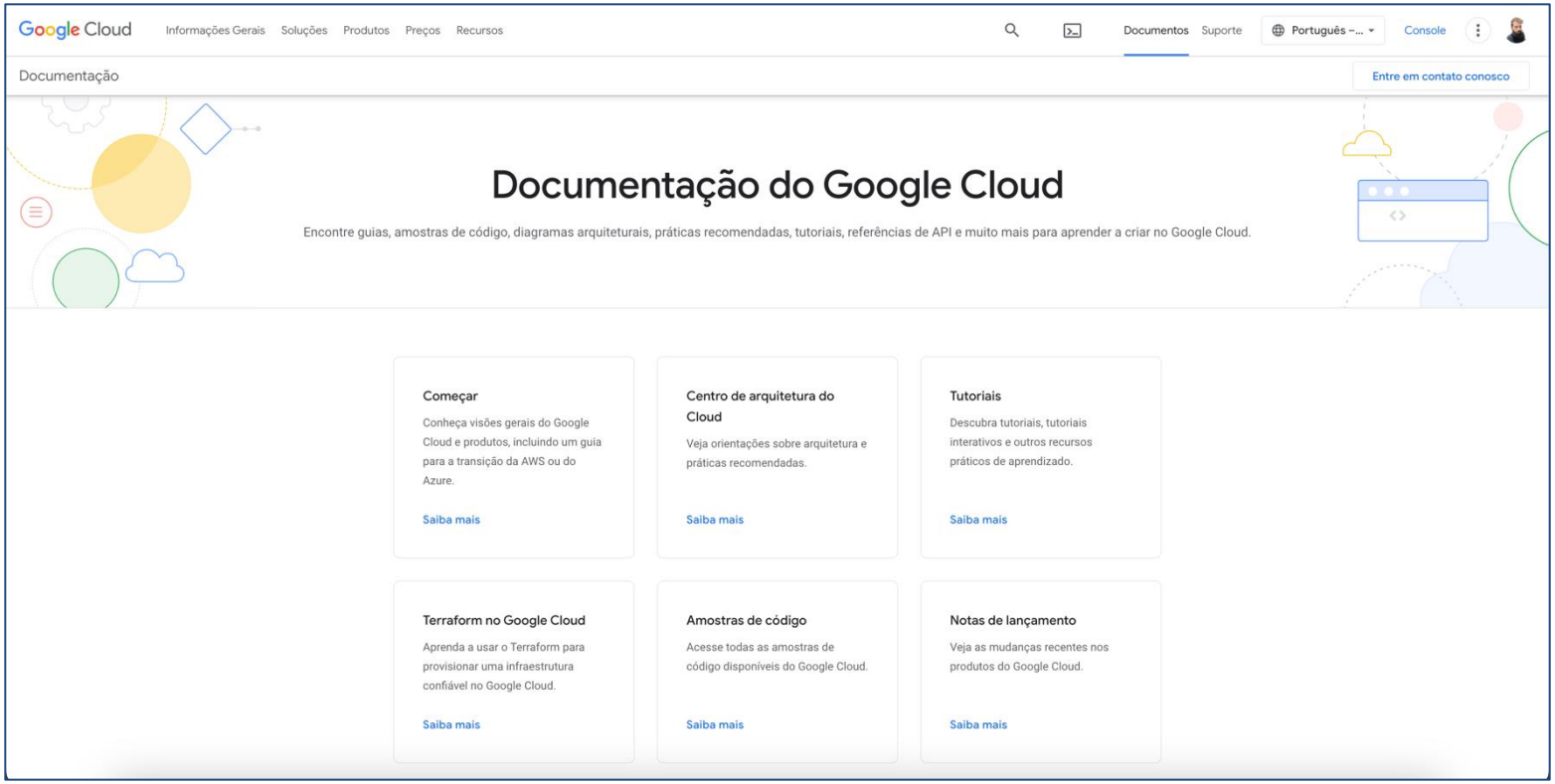
- Leitura sugerida:
  - https://k21academy.com/google-cloud/google-cloud-platform-console-walkthrough/



- Leitura sugerida:
  - https://k21academy.com/google-cloud/google-app-engine/

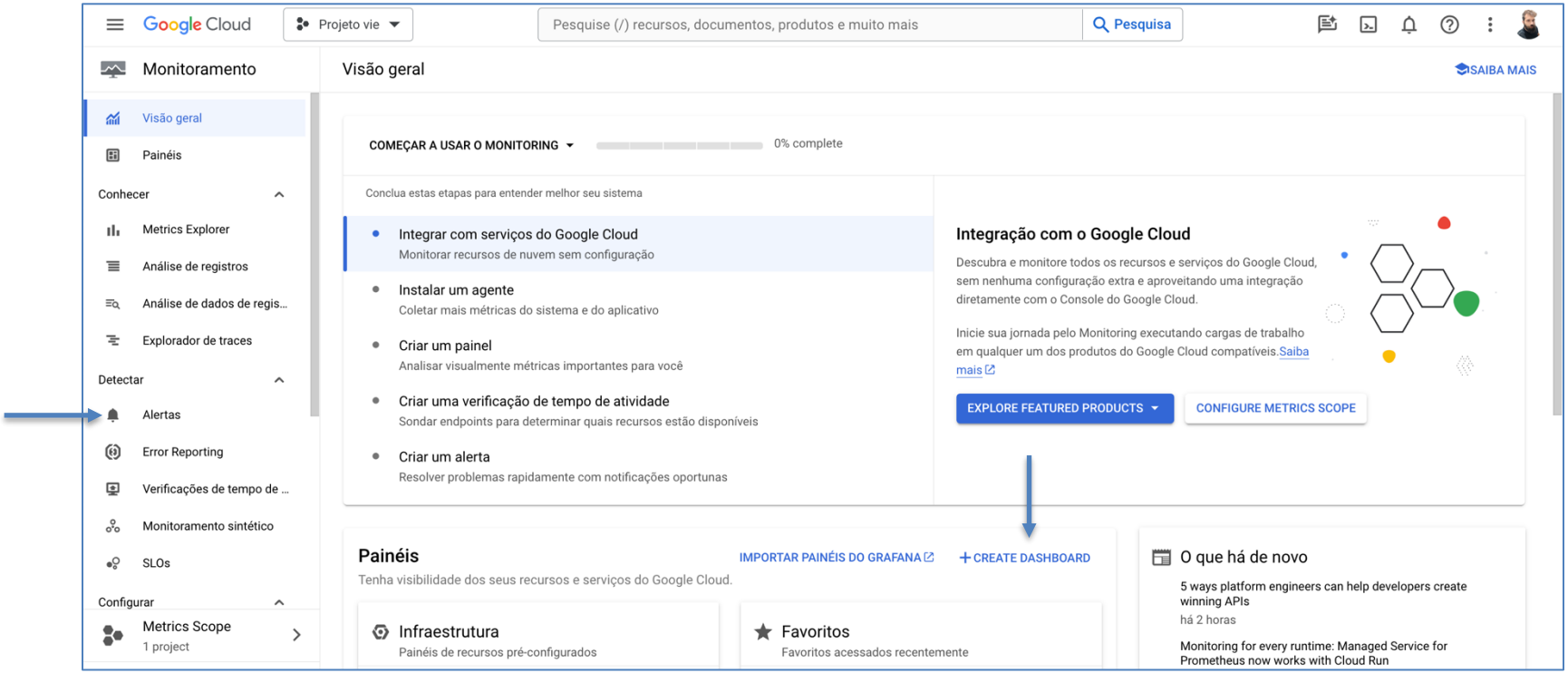


- Documentação do GCP:
  - https://cloud.google.com/docs?hl=pt-br

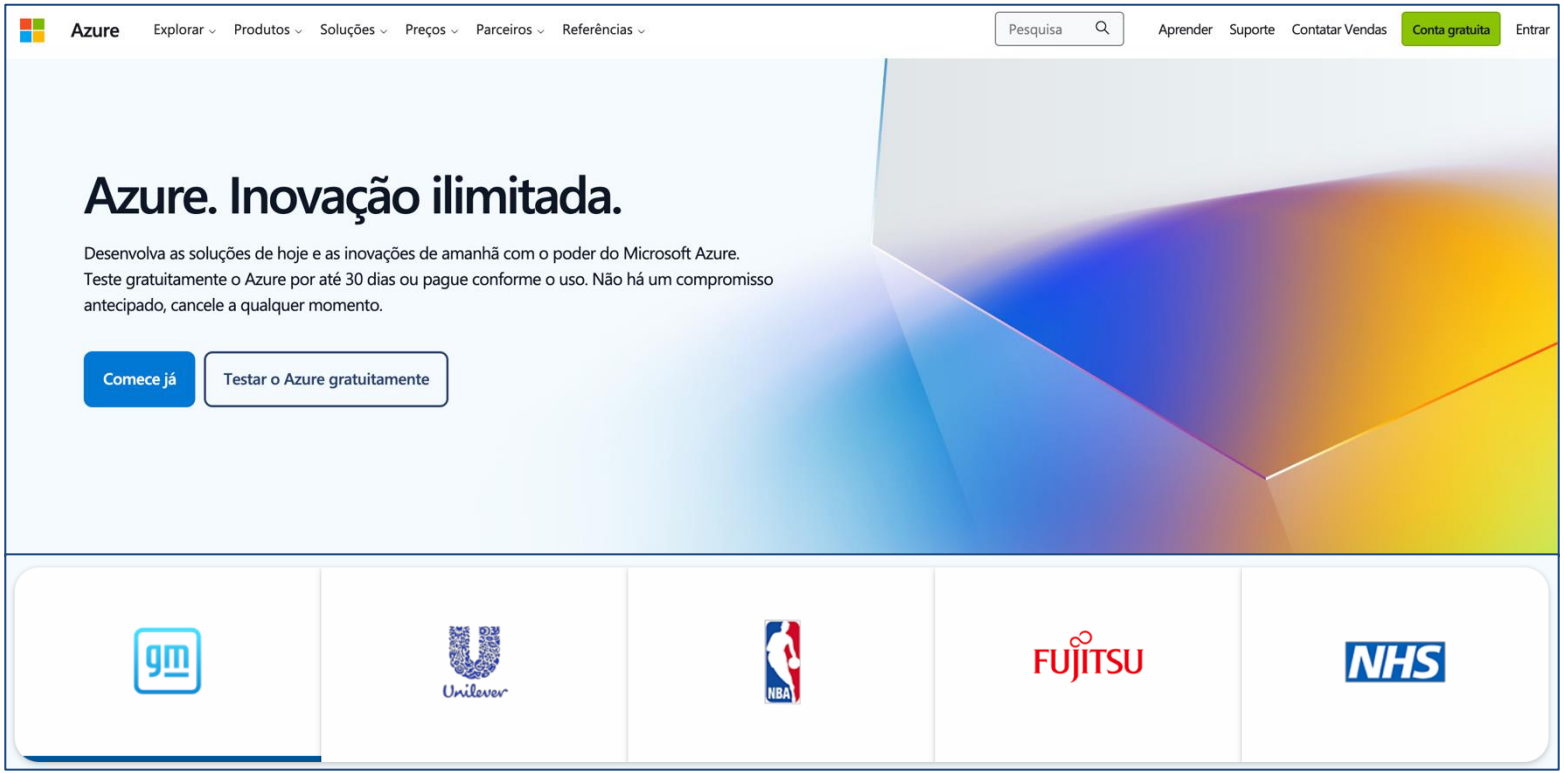


- Documentação do GCP:
  - https://cloud.google.com/monitoring?hl=pt_br
  - https://cloud.google.com/monitoring/docs?hl=pt-br

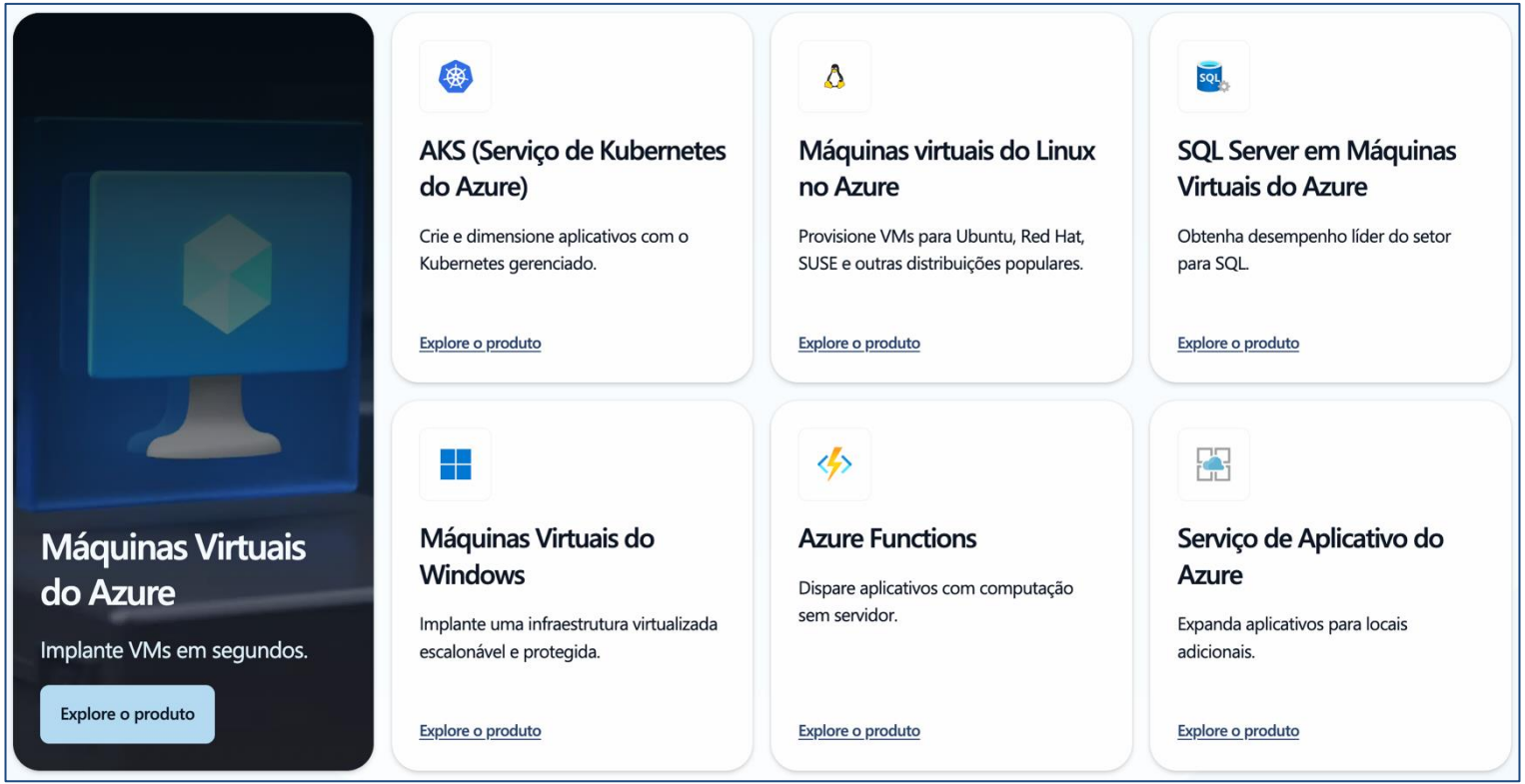
- Documentação do GCP:
  - https://cloud.google.com/monitoring?hl=pt_br
  - https://cloud.google.com/monitoring/docs?hl=pt-br



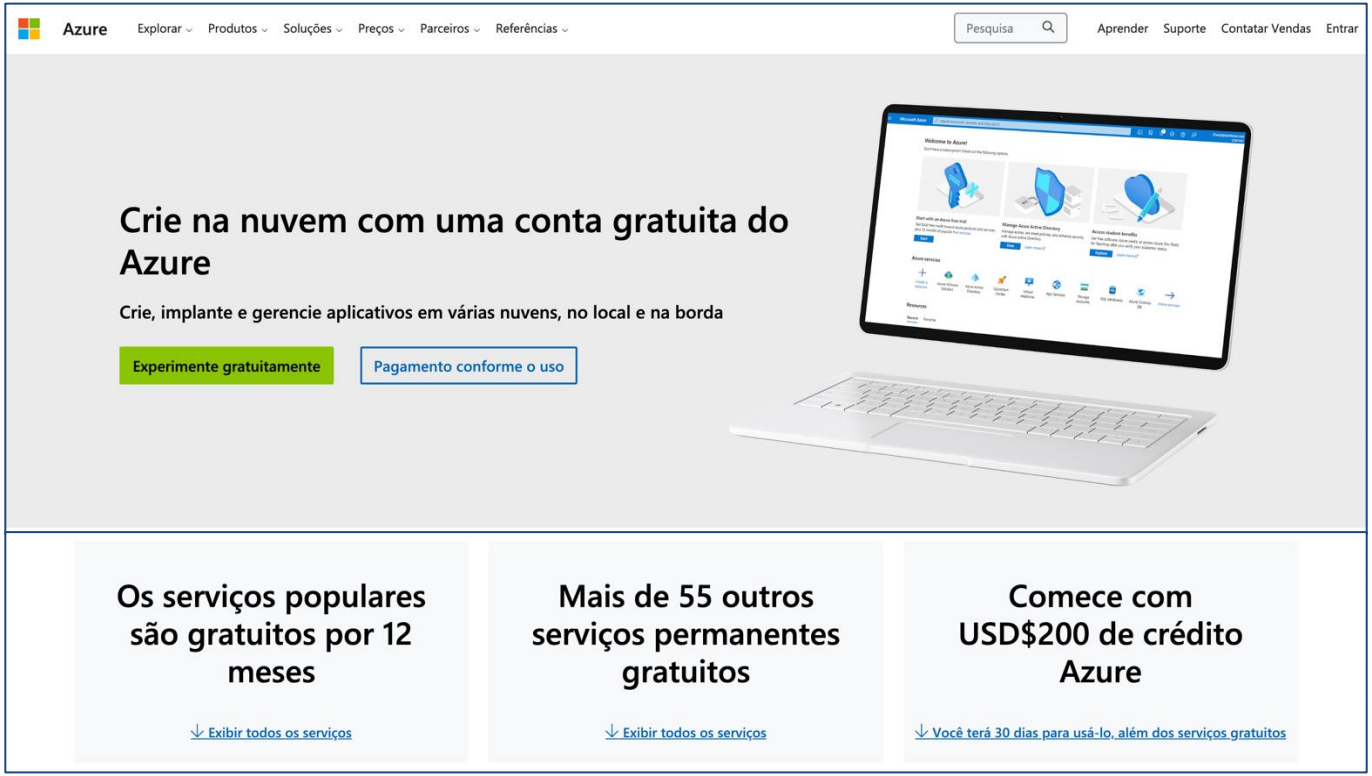
- **Ferramentas**: Plataformas de nuvem
  - **Microsoft Azure**: https://azure.microsoft.com/pt-br/



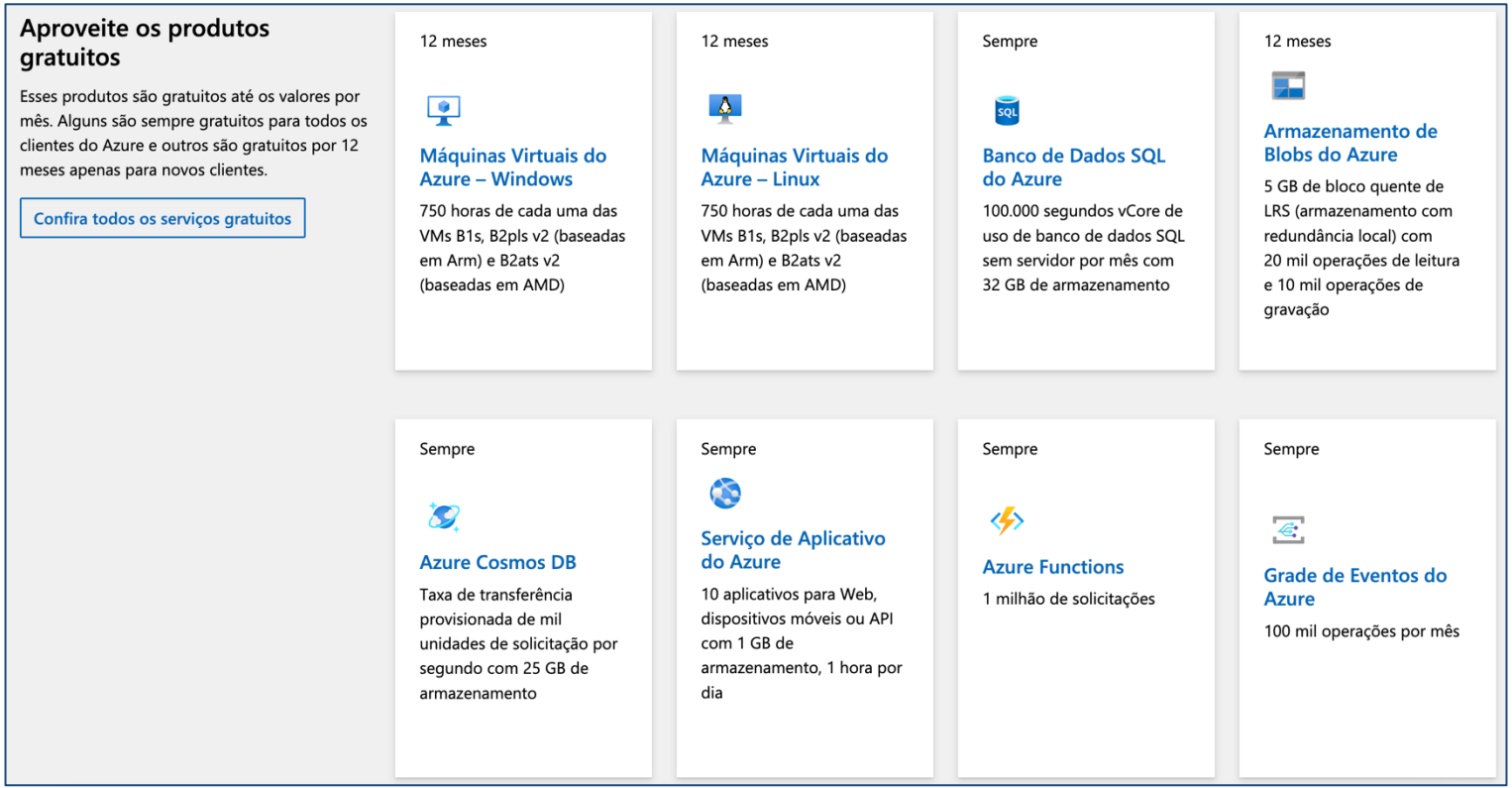
- **Ferramentas**: Plataformas de nuvem
  - **Microsoft Azure**: https://azure.microsoft.com/pt-br/



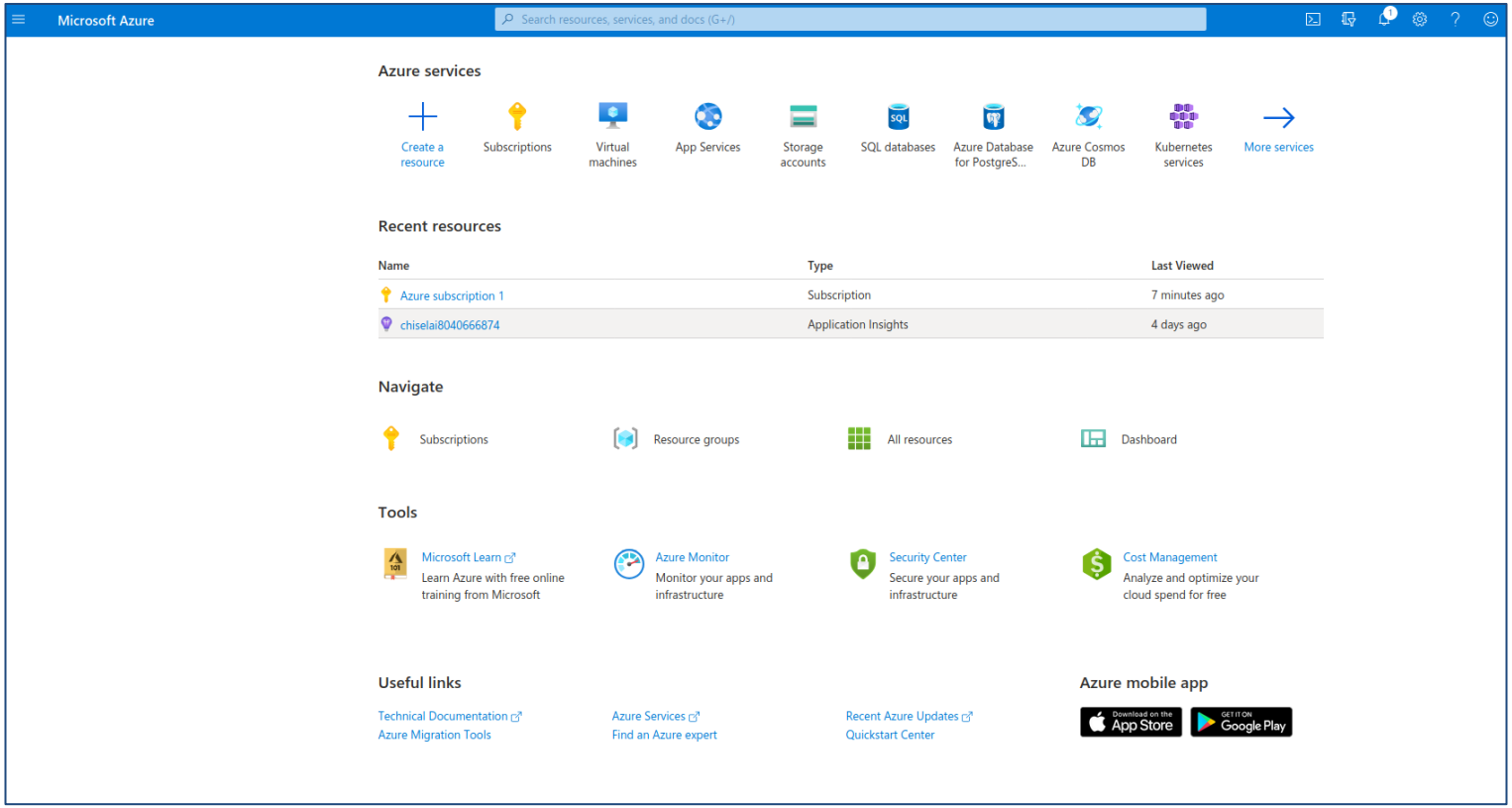
- **Ferramentas**: Plataformas de nuvem
  - **Microsoft Azure**: https://azure.microsoft.com/pt-br/free/



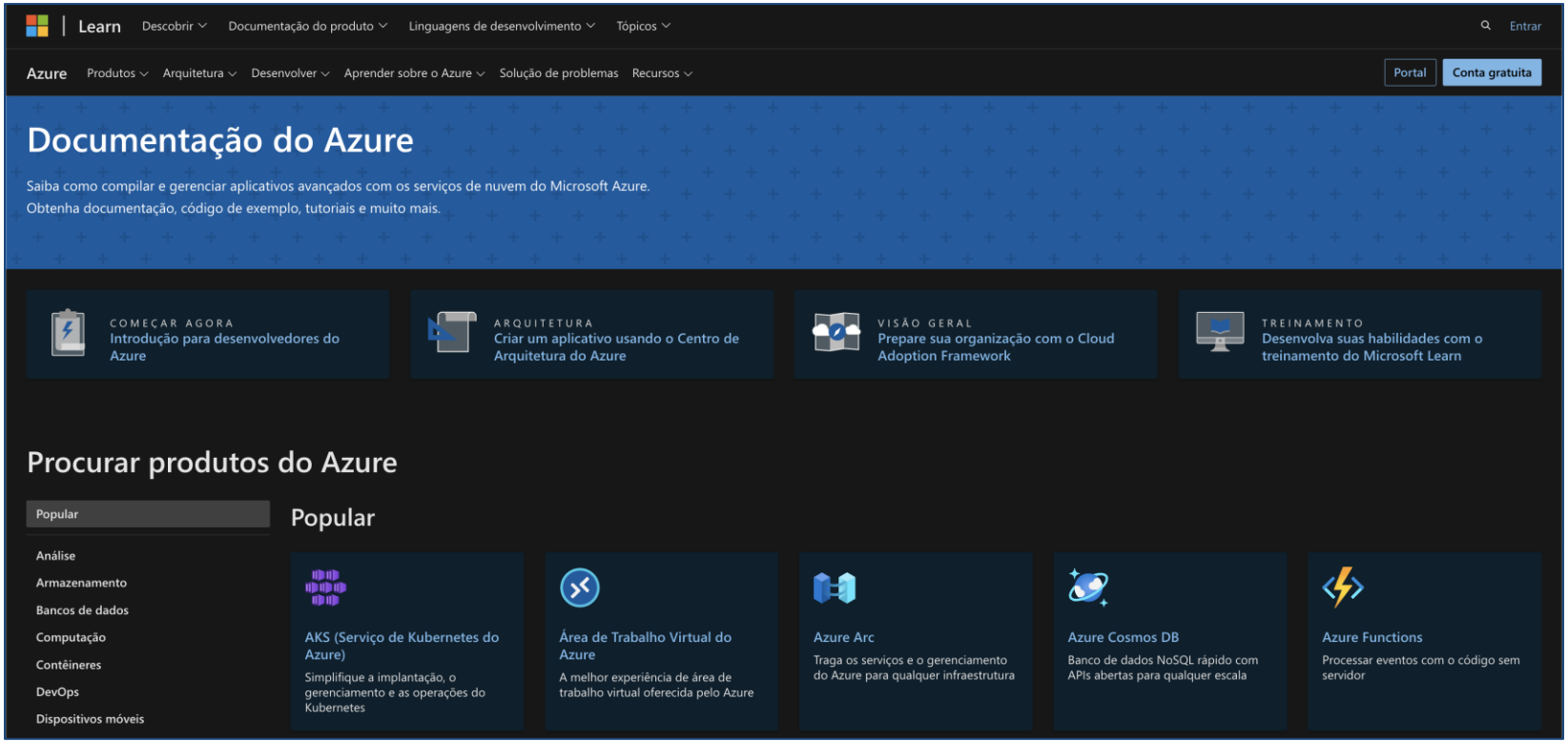
- **Ferramentas**: Plataformas de nuvem
  - **Microsoft Azure**: https://azure.microsoft.com/pt-br/free/



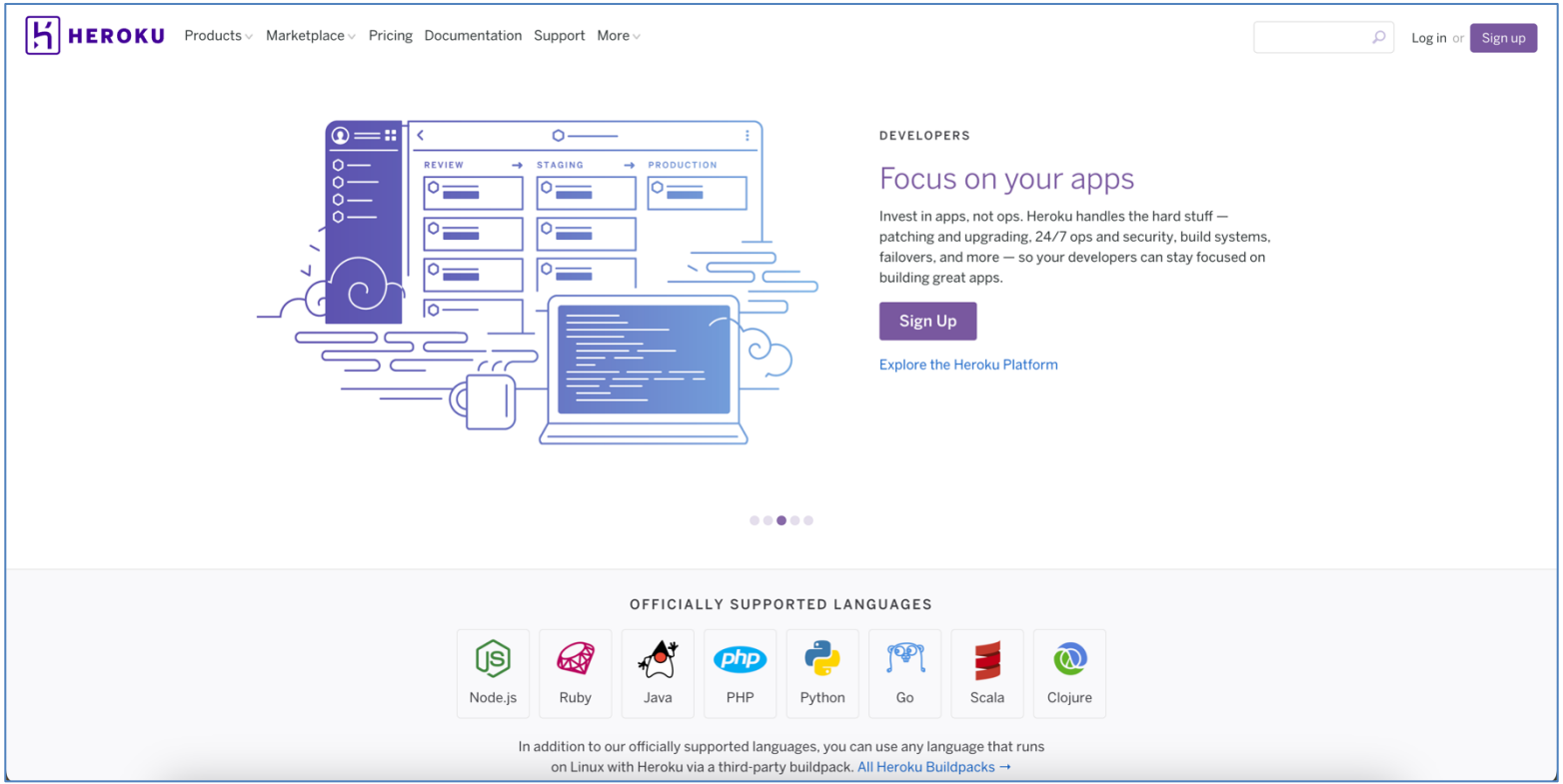
- Leitura sugerida:
  - https://azure.microsoft.com/pt-br/get-started



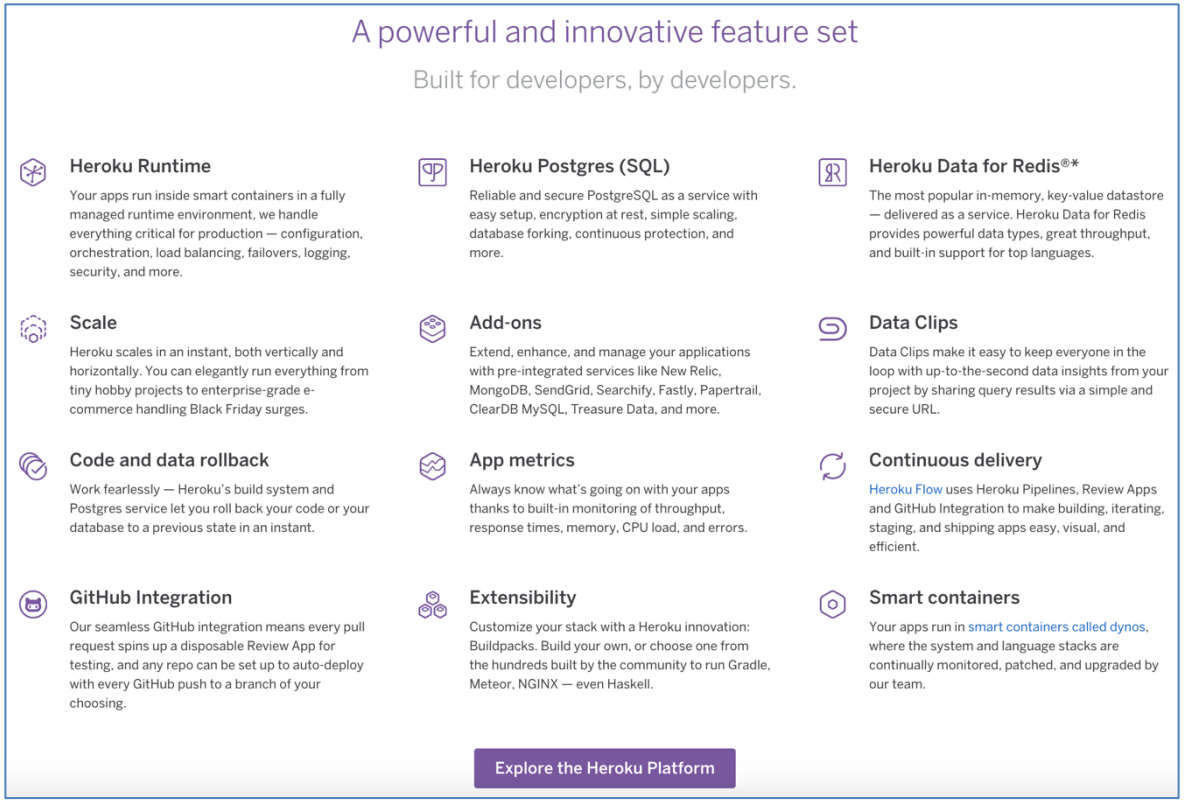
- Documentação do Azure:
  - https://learn.microsoft.com/pt-br/azure/?product=popular



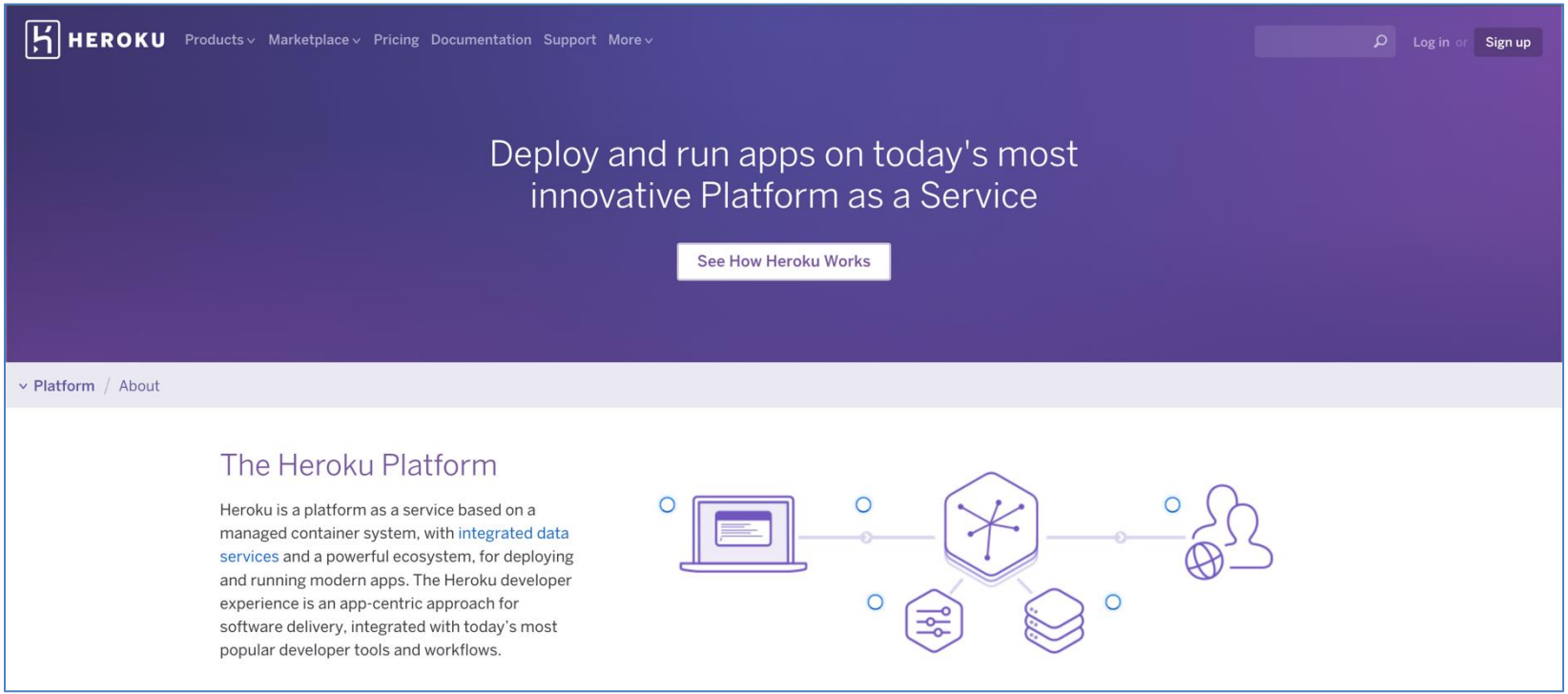
- **Ferramentas**: Plataformas de nuvem
  - **Heroku**: https://www.heroku.com/



- **Ferramentas**: Plataformas de nuvem
  - **Heroku**: https://www.heroku.com/

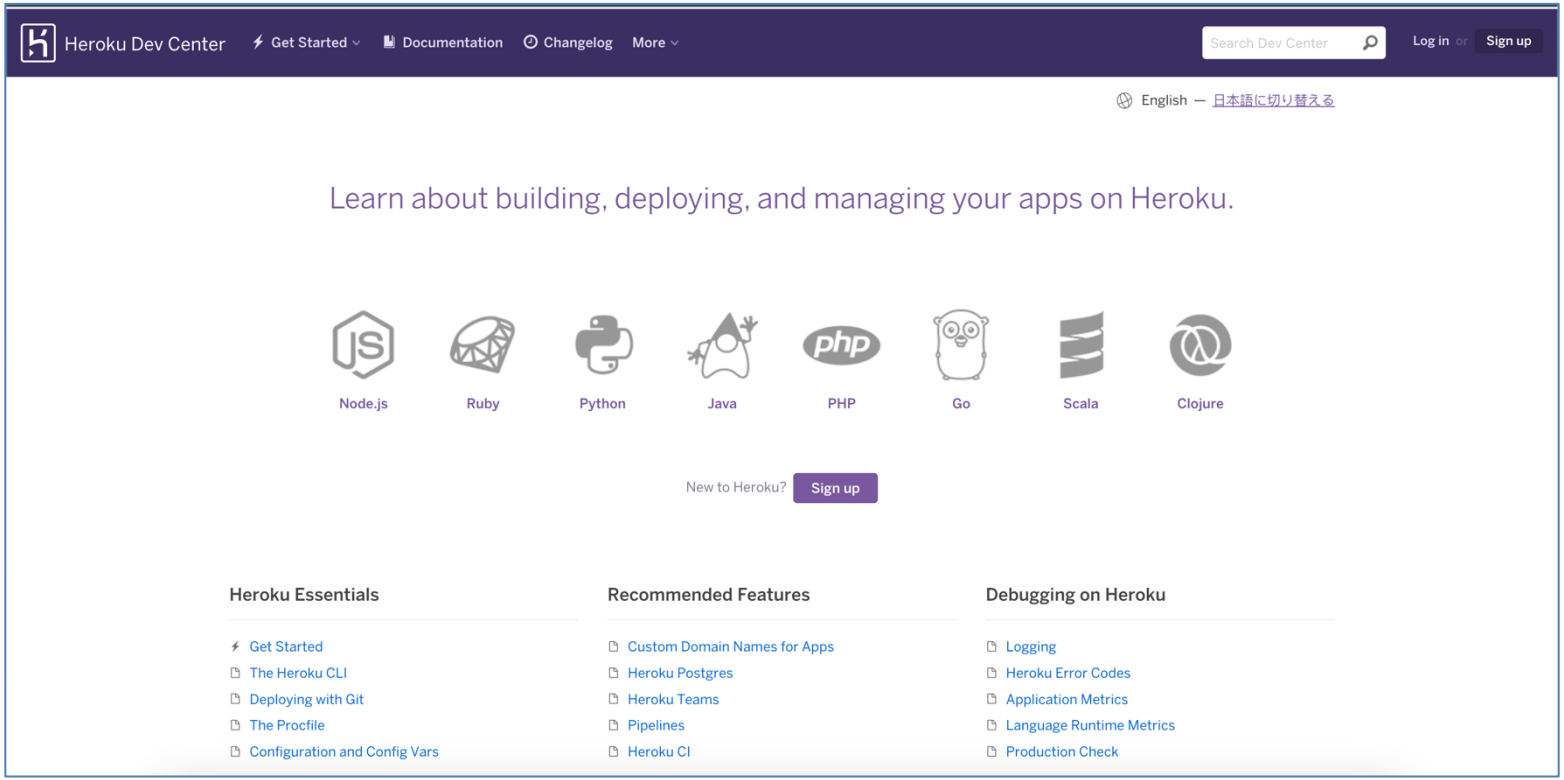


- **Ferramentas**: Plataformas de nuvem
  - **Heroku**: https://www.heroku.com/platform

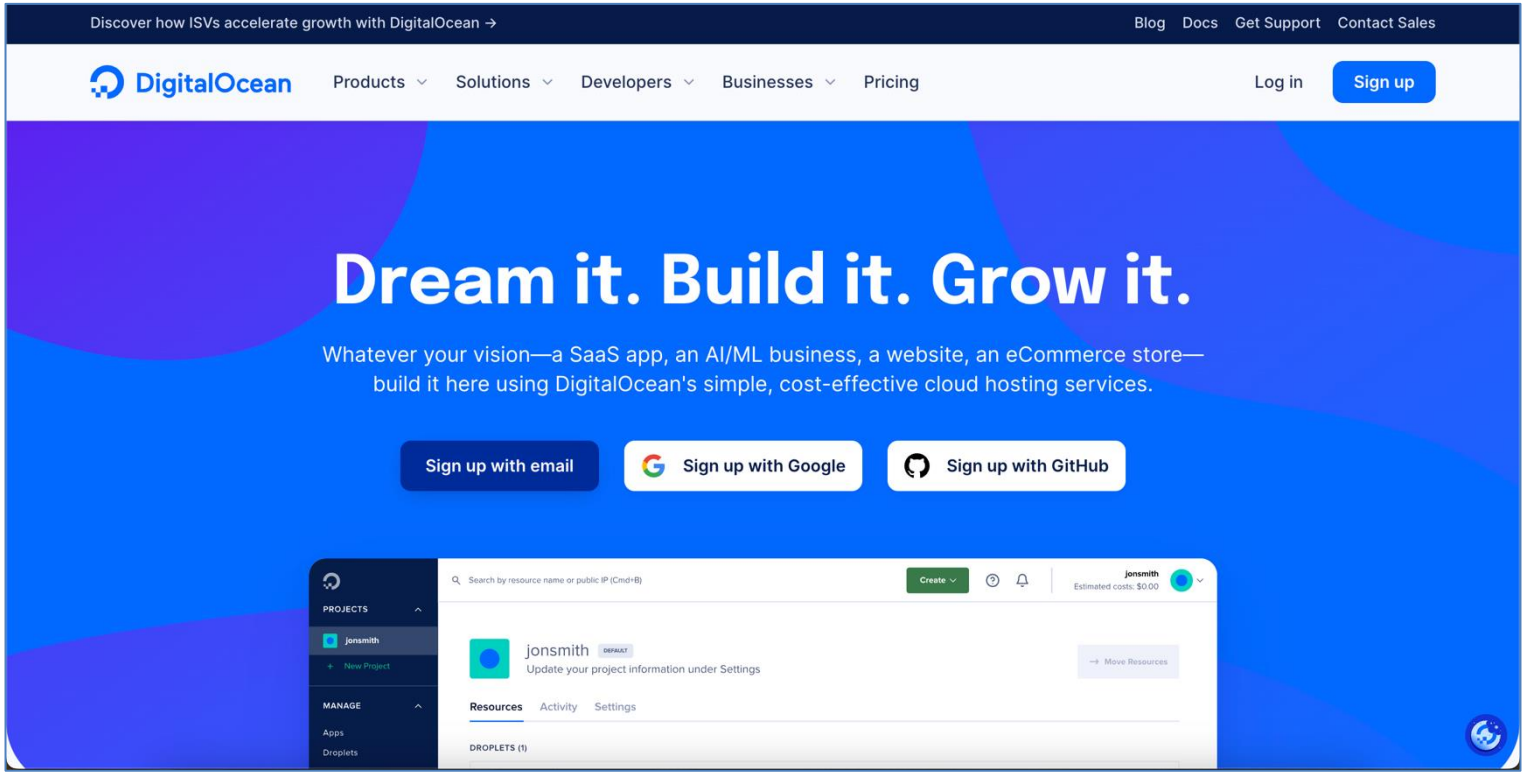


- **Ferramentas**: Plataformas de nuvem
  - **Heroku**: https://www.heroku.com/platform

- **Ferramentas**: Plataformas de nuvem
  - **Heroku**: https://devcenter.heroku.com/

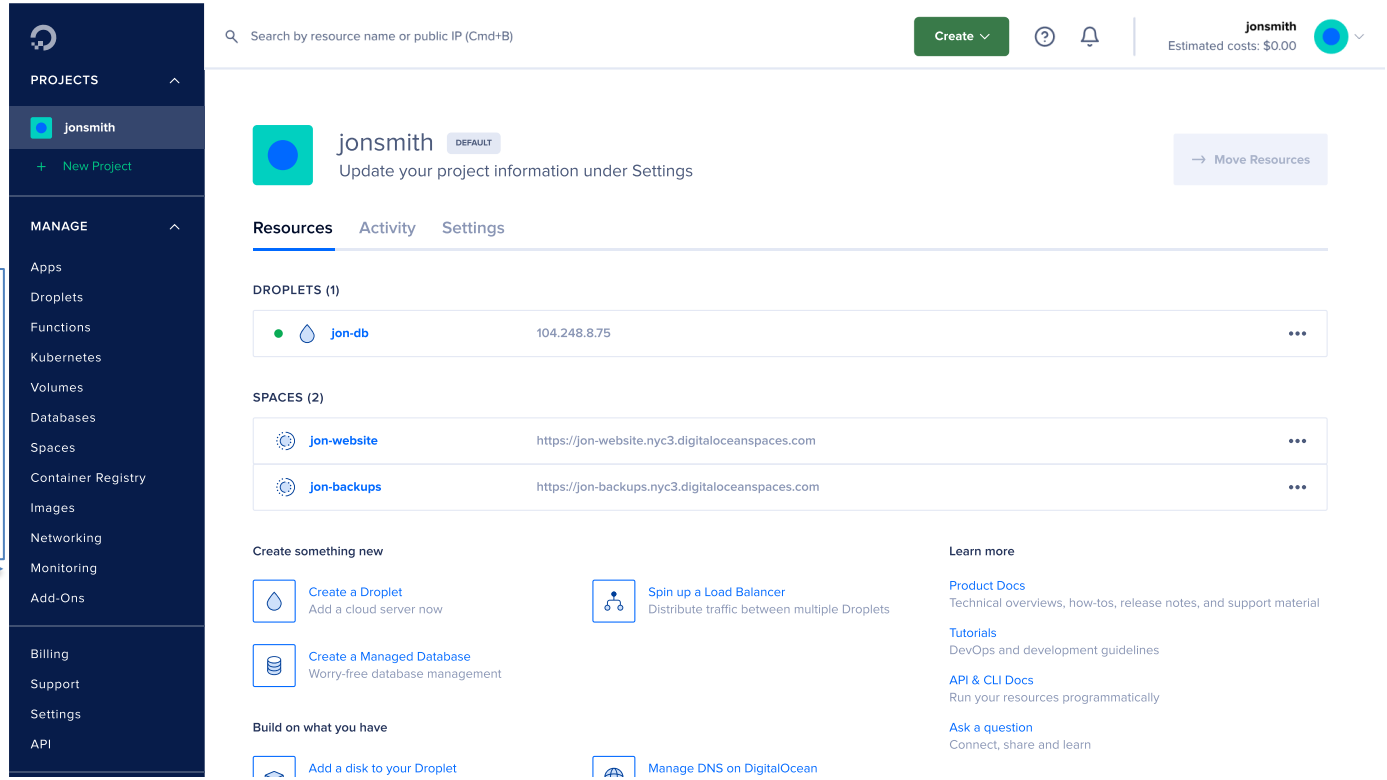


- **Ferramentas**: Plataformas de nuvem
  - **DigitalOcean**: https://www.digitalocean.com/

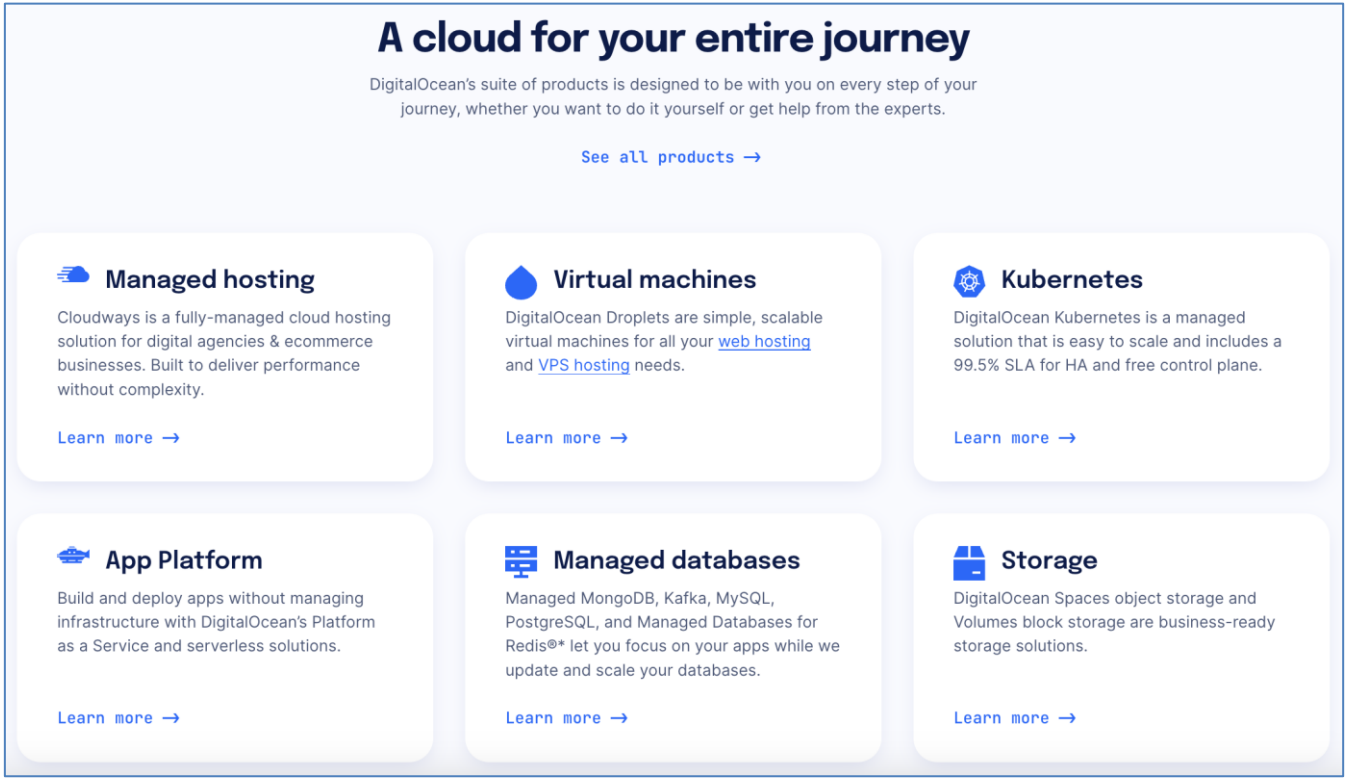


- **Ferramentas**: Plataformas de nuvem
  - **DigitalOcean**: https://www.digitalocean.com/

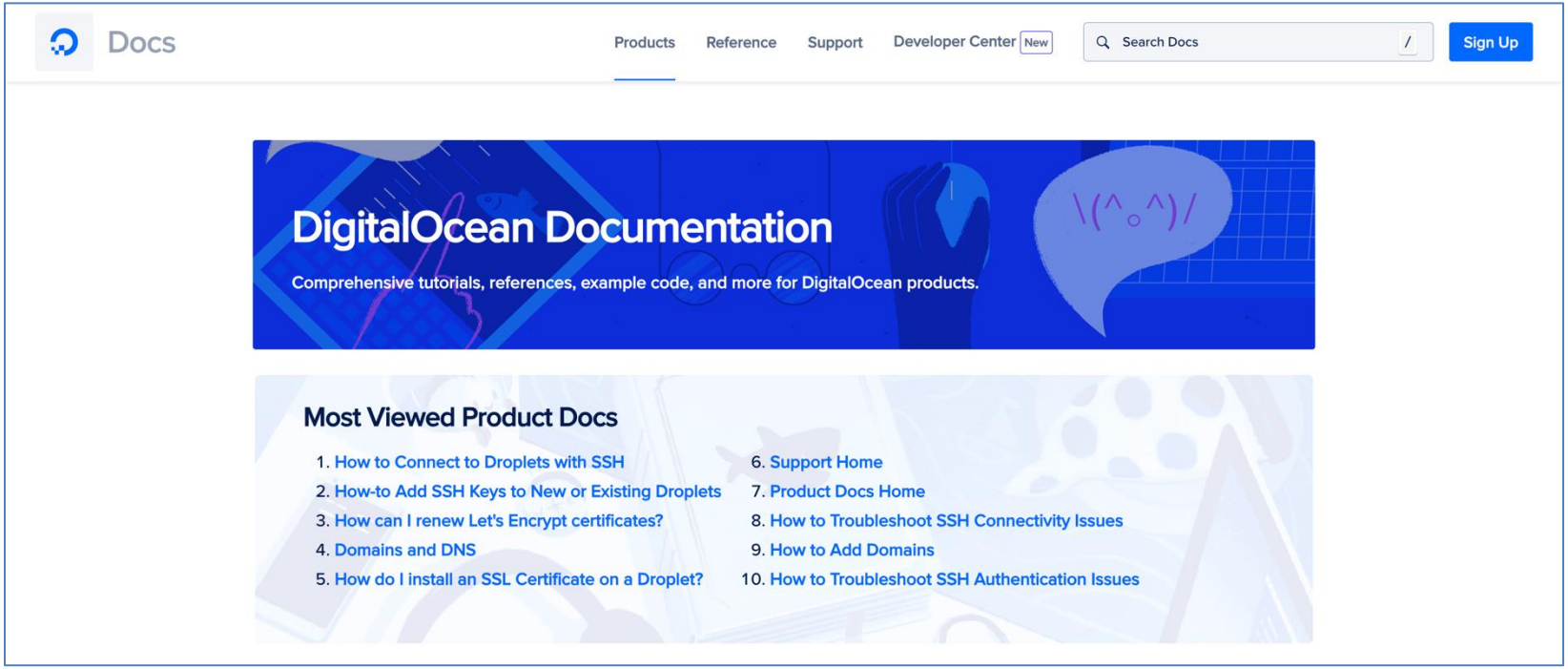
Assim como a AWS possui o **Amazon CloudWatch** e o **Azure Monitor**, as outras plataformas de nuvem também costumam ter ferramentas de monitoramento similares.



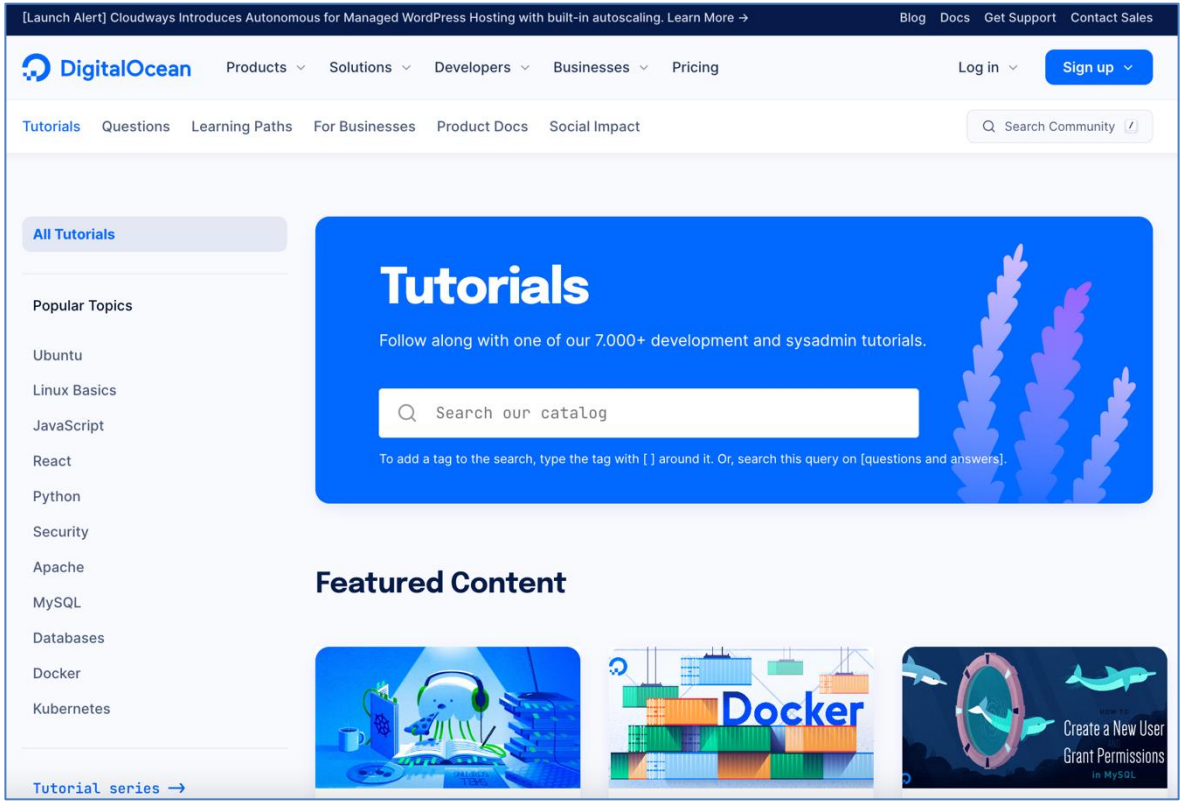
- **Ferramentas**: Plataformas de nuvem
  - **DigitalOcean**: https://www.digitalocean.com/



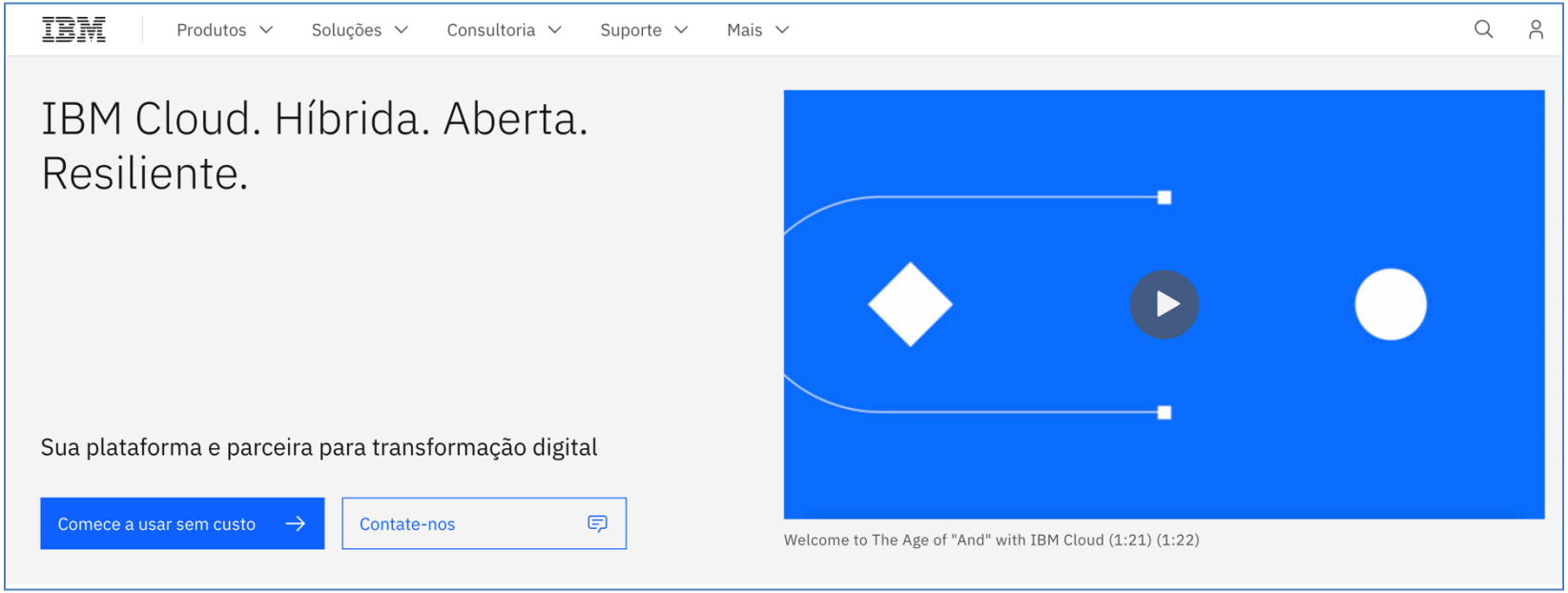
- Documentação do DigitalOcean:
  - https://docs.digitalocean.com/
  - https://docs.digitalocean.com/products/



- Tutoriais do DigitalOcean:
  - https://www.digitalocean.com/community/tutorials

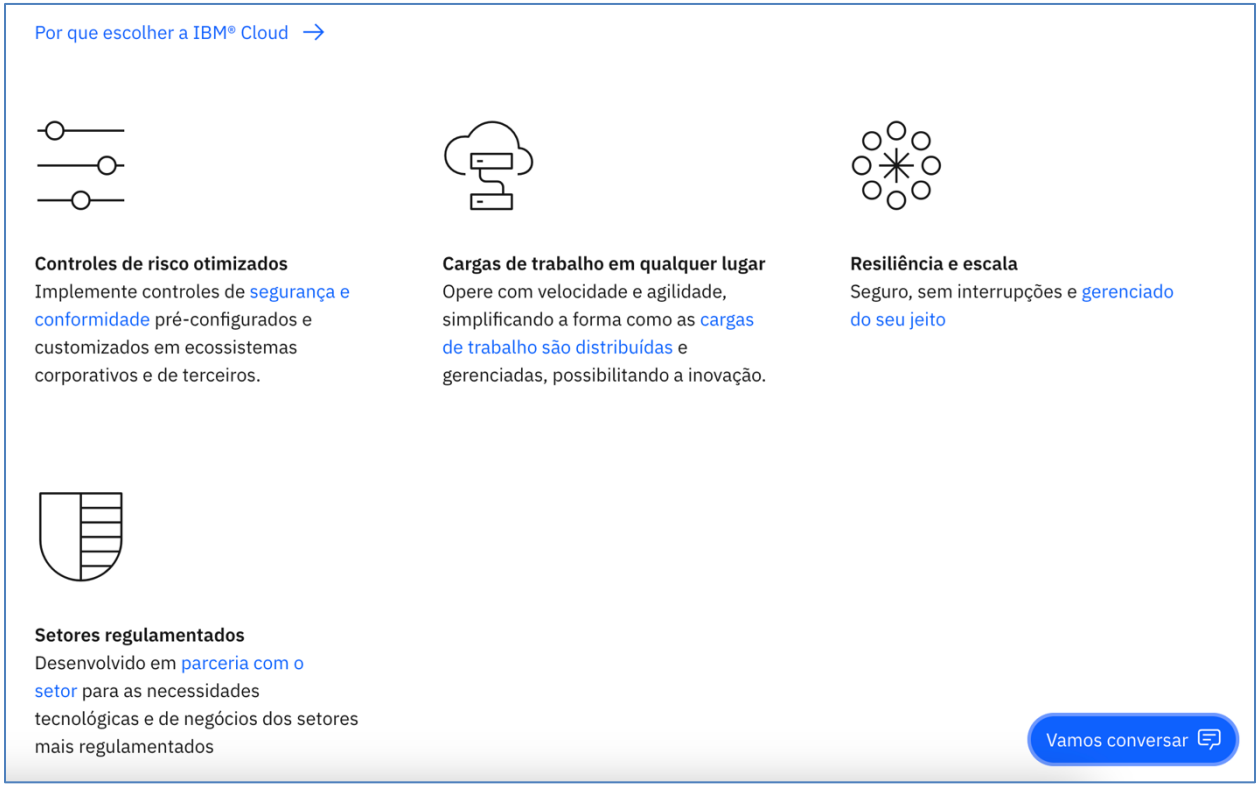


- **Ferramentas**: Plataformas de nuvem
  - **IBM Cloud**: https://www.ibm.com/br-pt/cloud

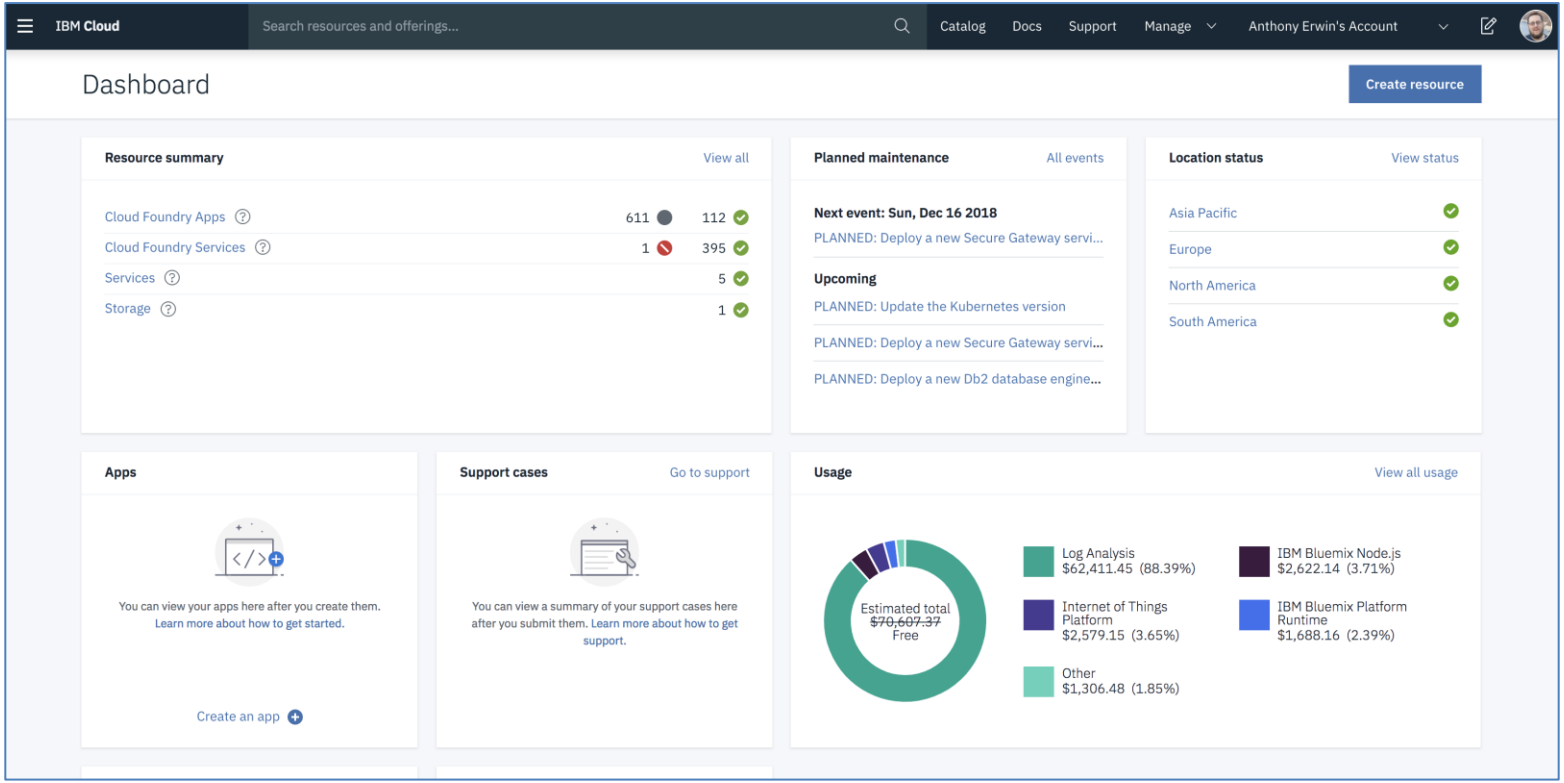


- **Ferramentas**: Plataformas de nuvem
  - **IBM Cloud**: https://www.ibm.com/br-pt/cloud

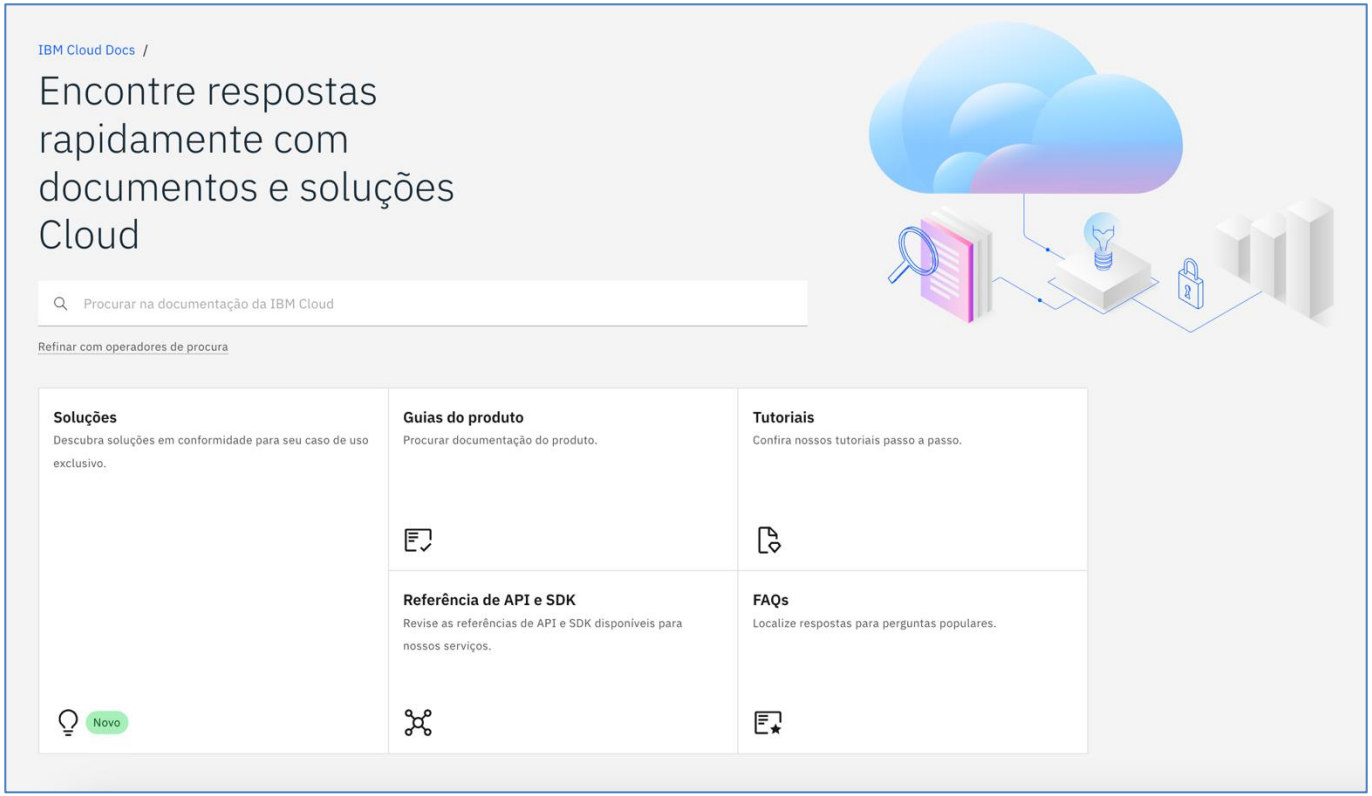
- **Ferramentas**: Plataformas de nuvem
  - **IBM Cloud**: https://www.ibm.com/br-pt/cloud



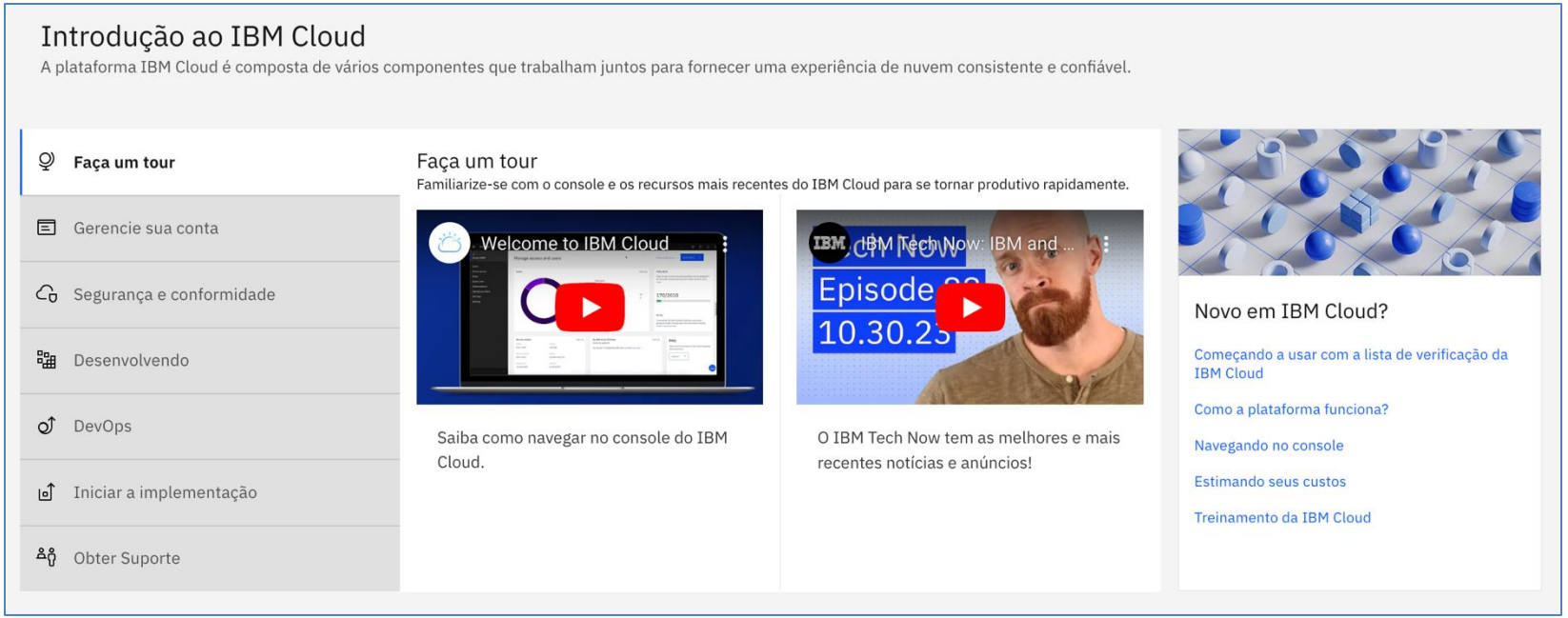
- Leitura sugerida:
  - https://www.ibm.com/br-pt/cloud/get-started



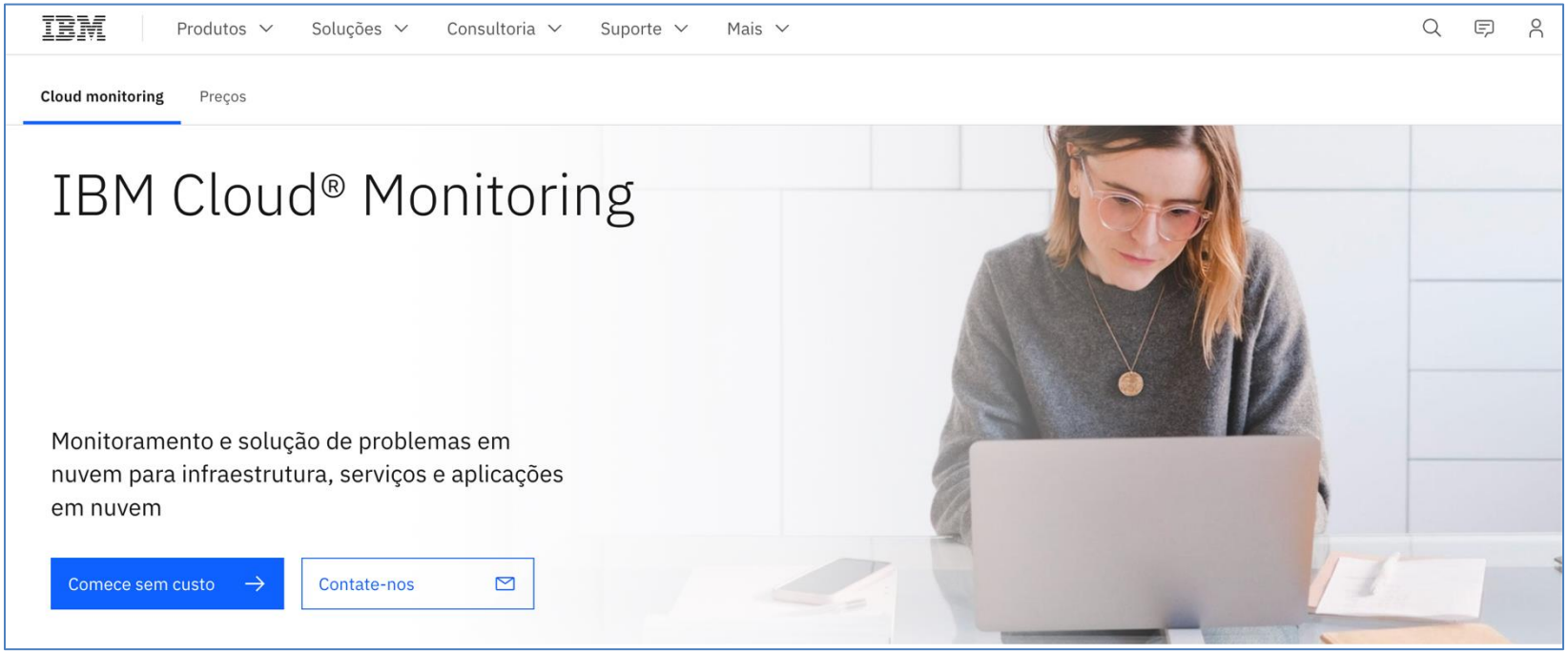
- Documentação do IBM Cloud:
  - https://cloud.ibm.com/docs



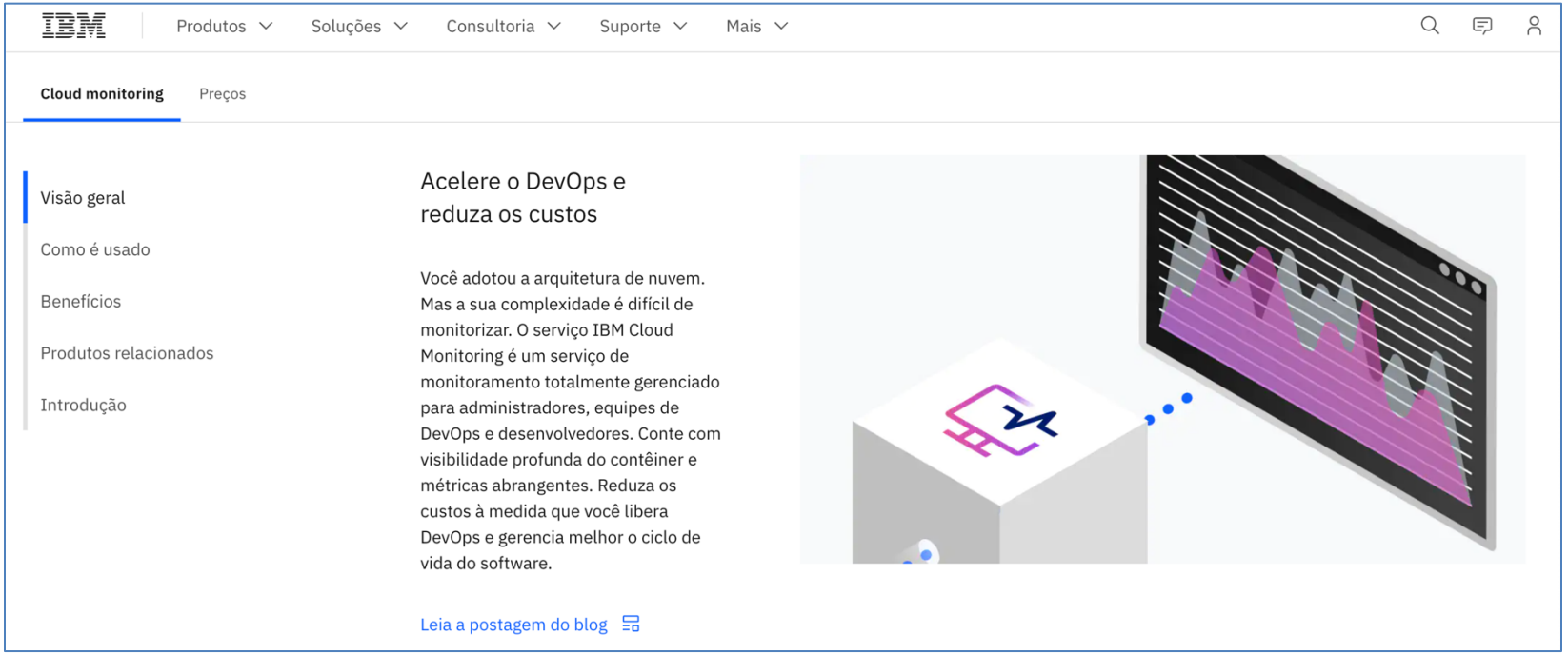
- Documentação do IBM Cloud:
  - https://cloud.ibm.com/docs



- **Ferramentas**: Plataformas de nuvem
  - **IBM Cloud Monitoring**:
    - https://www.ibm.com/br-pt/products/cloud-monitoring



- **Ferramentas**: Plataformas de nuvem
  - **IBM Cloud Monitoring**:
    - https://www.ibm.com/br-pt/products/cloud-monitoring

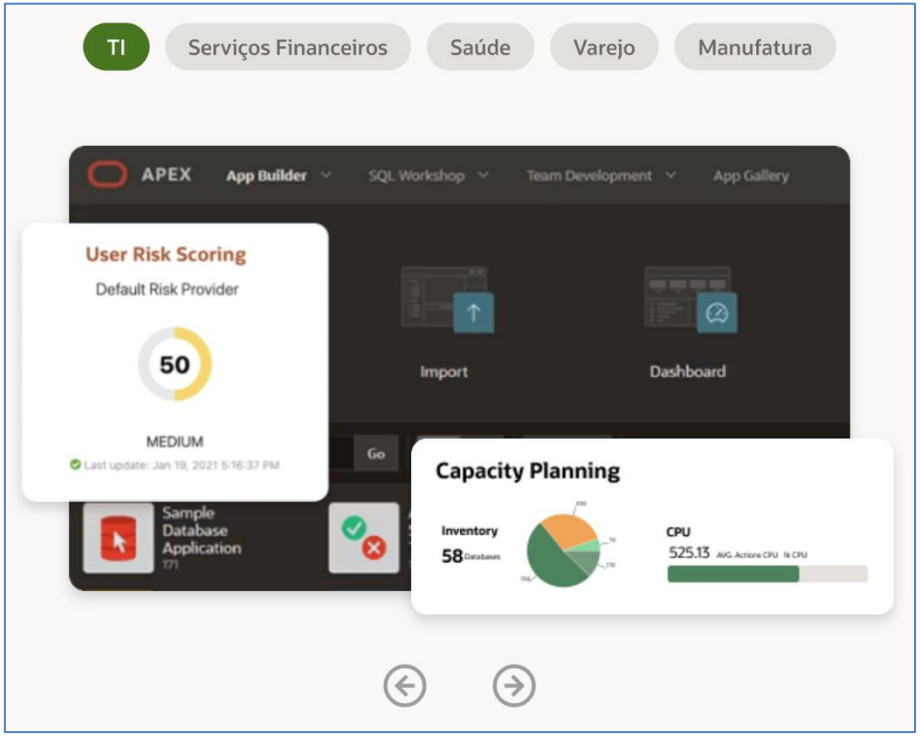


- **Ferramentas**: Plataformas de nuvem
  - **Oracle Cloud**: https://go.oracle.com/LP=138330

OCI

Uma plataforma completa de infraestrutura de nuvem para cada carga de trabalho

A Oracle Cloud Infrastructure (OCI) é a nuvem de última geração projetada para executar qualquer aplicação de forma mais rápida e segura, com custo menor.



- **Ferramentas**: Plataformas de nuvem
  - **Oracle Cloud**: https://go.oracle.com/LP=138330

Oracle Cloud Infrastructure para a empresa moderna

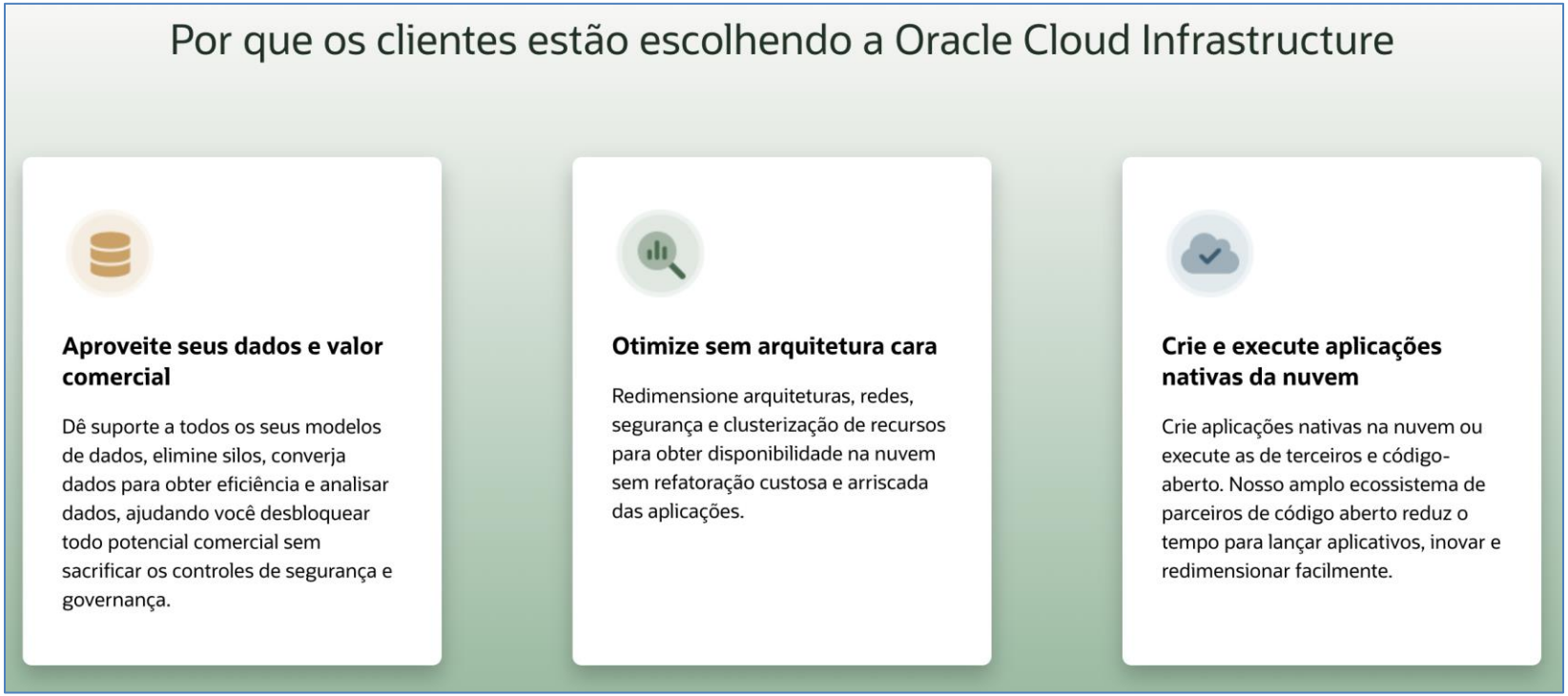
Saiba como a Oracle Cloud Infrastructure oferece custo-benefício incomparável. Junte-se a marcas líderes como 7-Eleven, Mazda e Cisco com a OCI. Inscreva-se agora para acessar o PDF!

Faça o download do guia de negócios

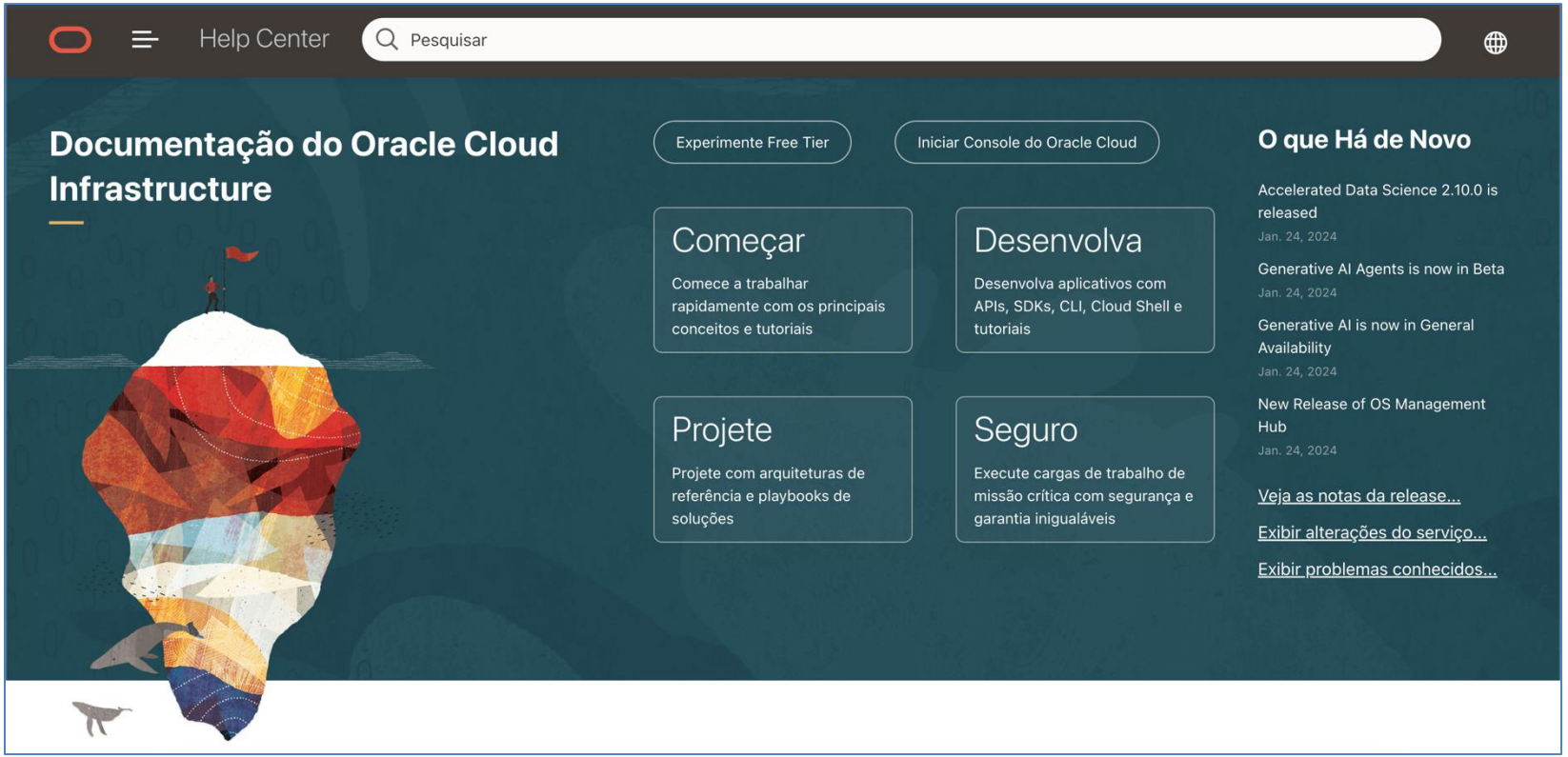
Líderes do setor que confiam na Oracle

vodafone ZOOM TOYOTA mazda FedEx

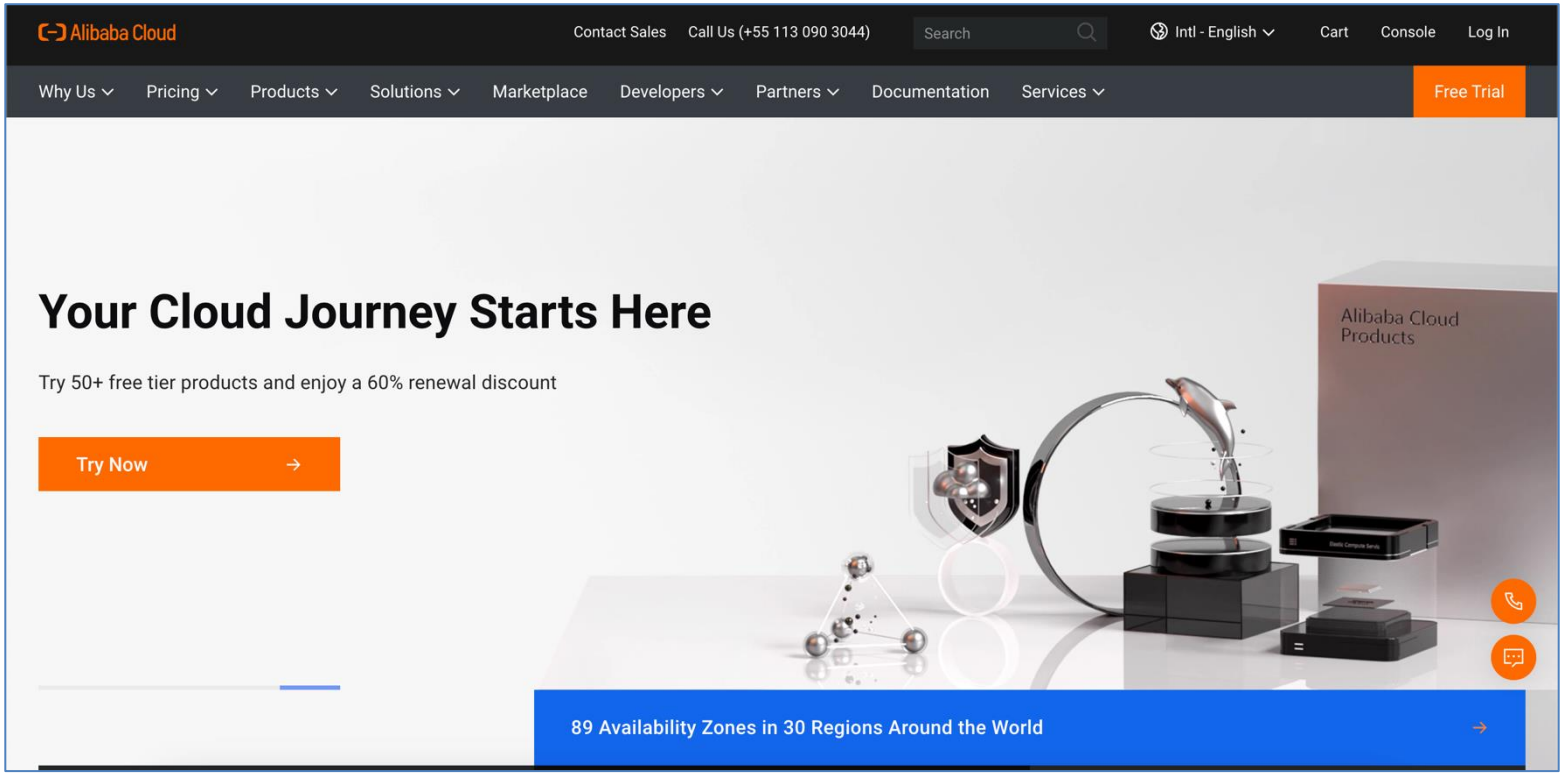
- **Ferramentas**: Plataformas de nuvem
  - **Oracle Cloud**: https://go.oracle.com/LP=138330



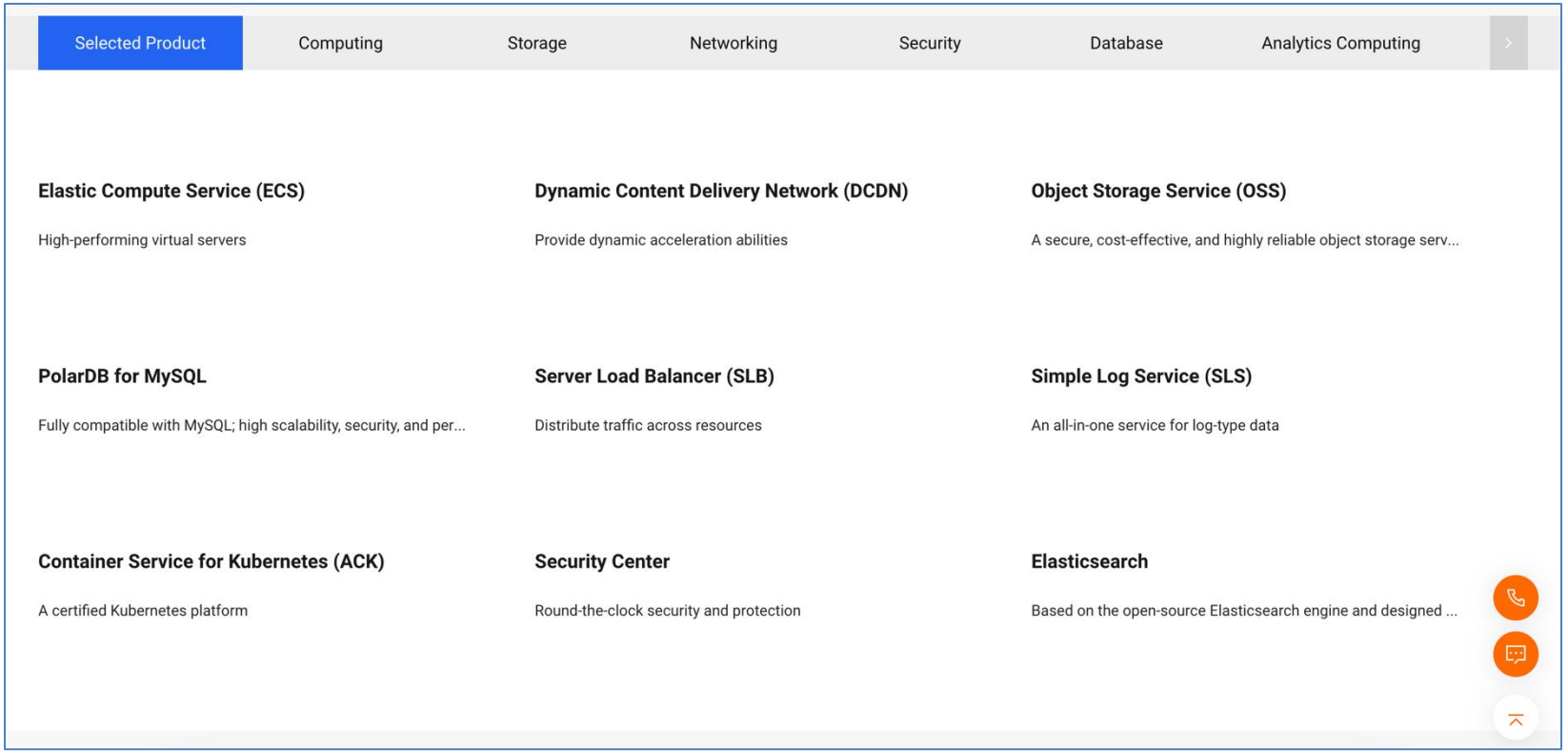
- Documentação do Oracle Cloud:
  - https://docs.oracle.com/pt-br/iaas/Content/home.htm



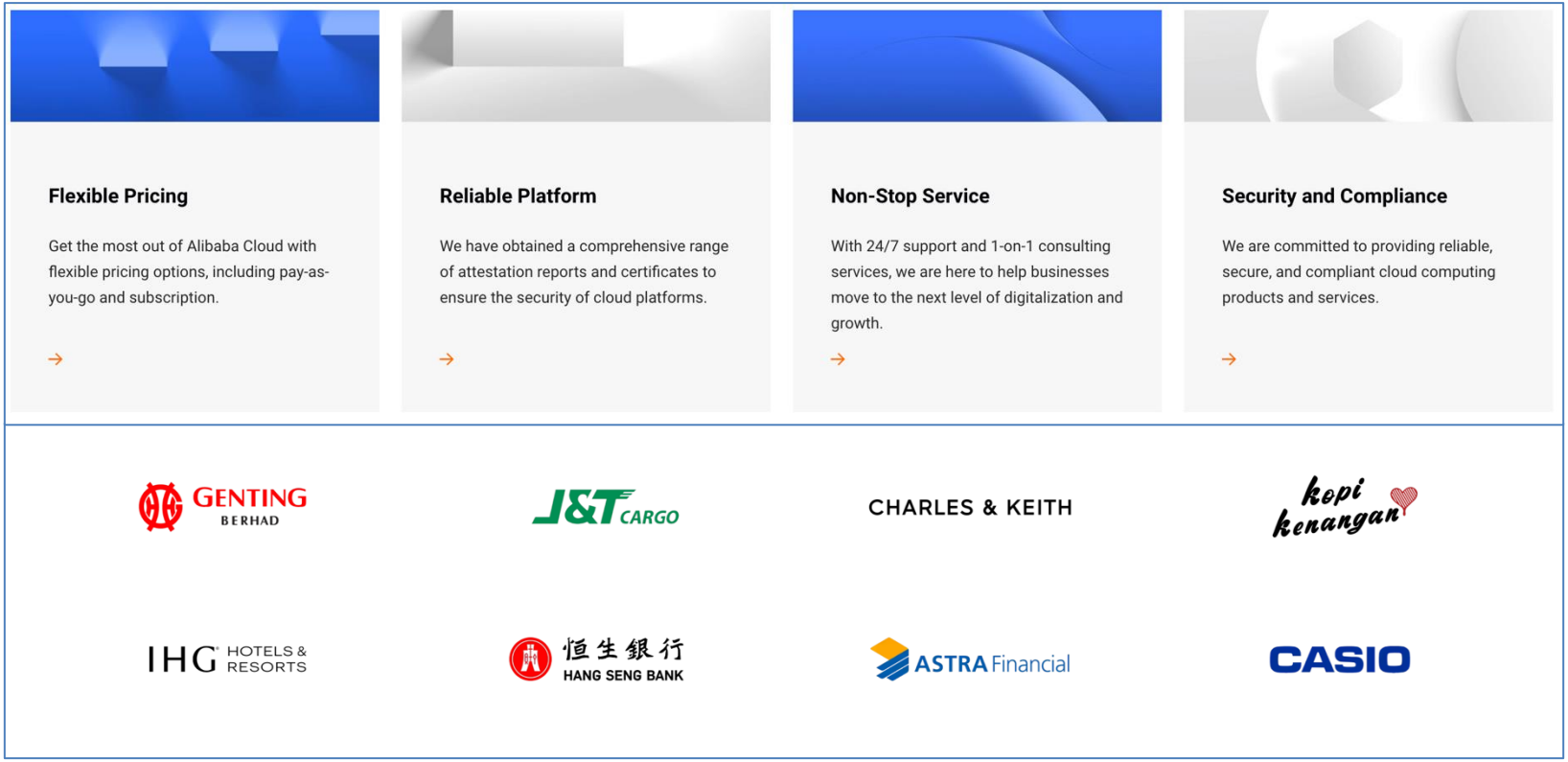
- **Ferramentas**: Plataformas de nuvem
  - **Alibaba Cloud**: https://www.alibabacloud.com/



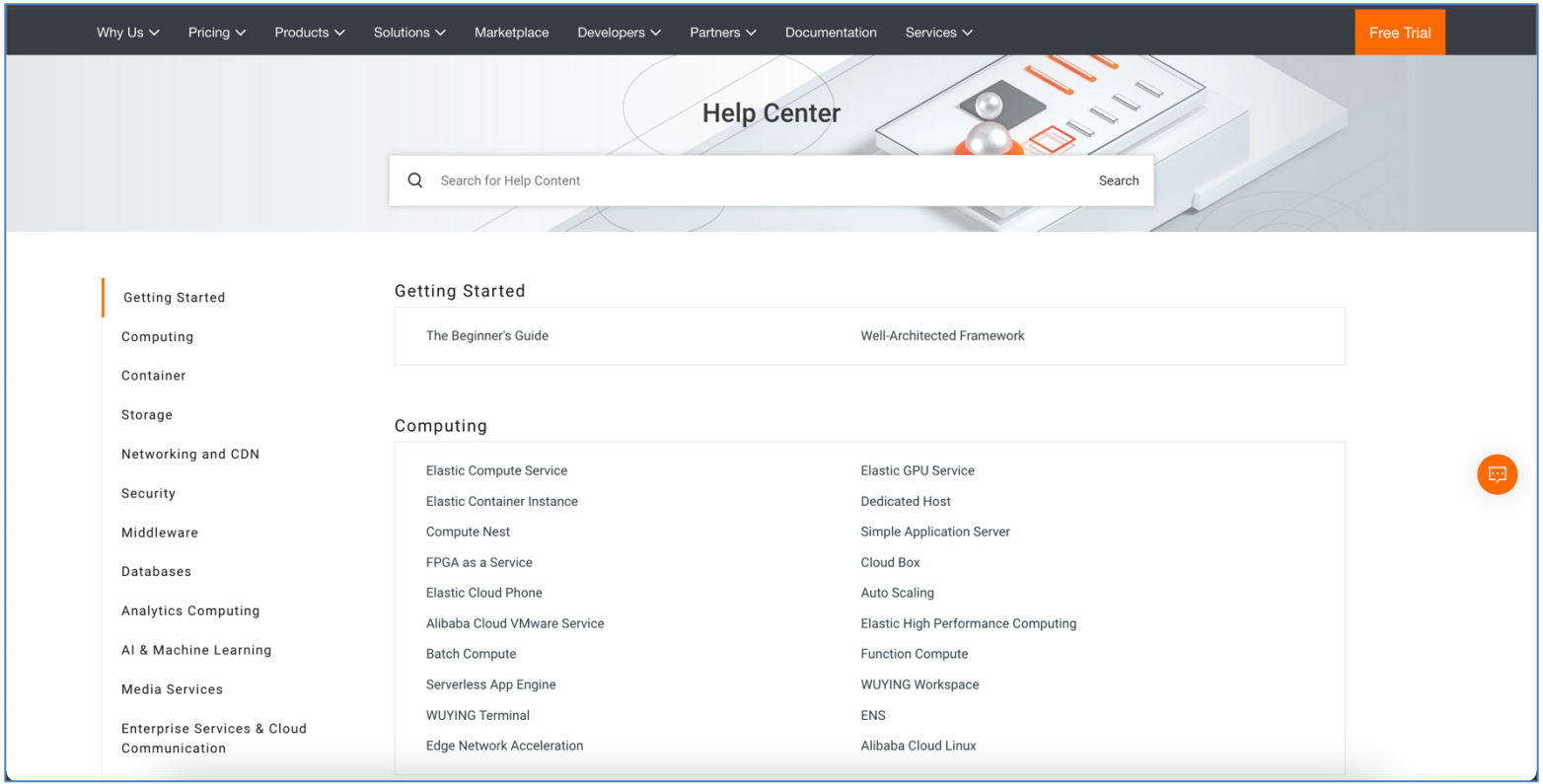
- **Ferramentas**: Plataformas de nuvem
  - **Alibaba Cloud**: https://www.alibabacloud.com/



- **Ferramentas**: Plataformas de nuvem
  - **Alibaba Cloud**: https://www.alibabacloud.com/

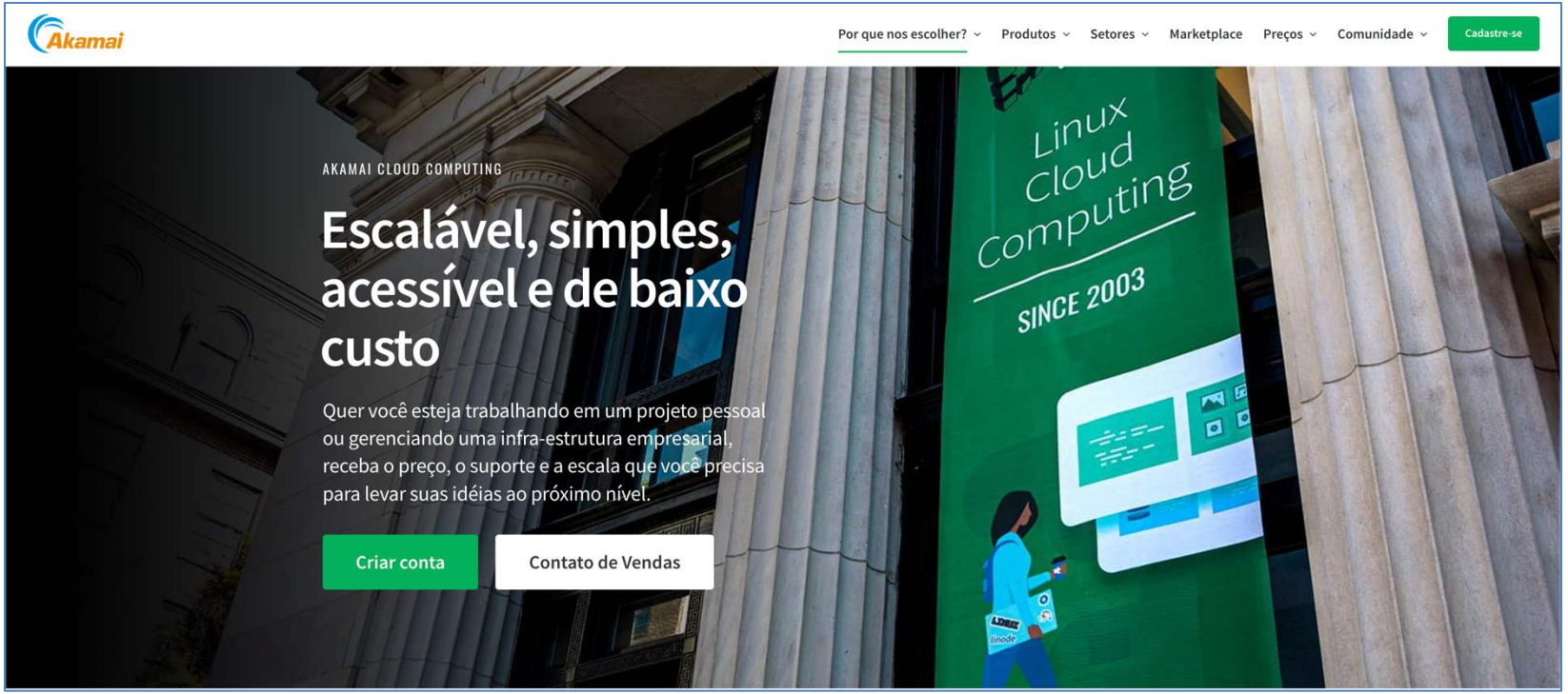


- Documentação do Alibaba Cloud:
  - https://www.alibabacloud.com/help/en



- **Ferramentas**: Plataformas de nuvem
  - **Linode/Akamai Cloud**: https://www.linode.com/pt-br/

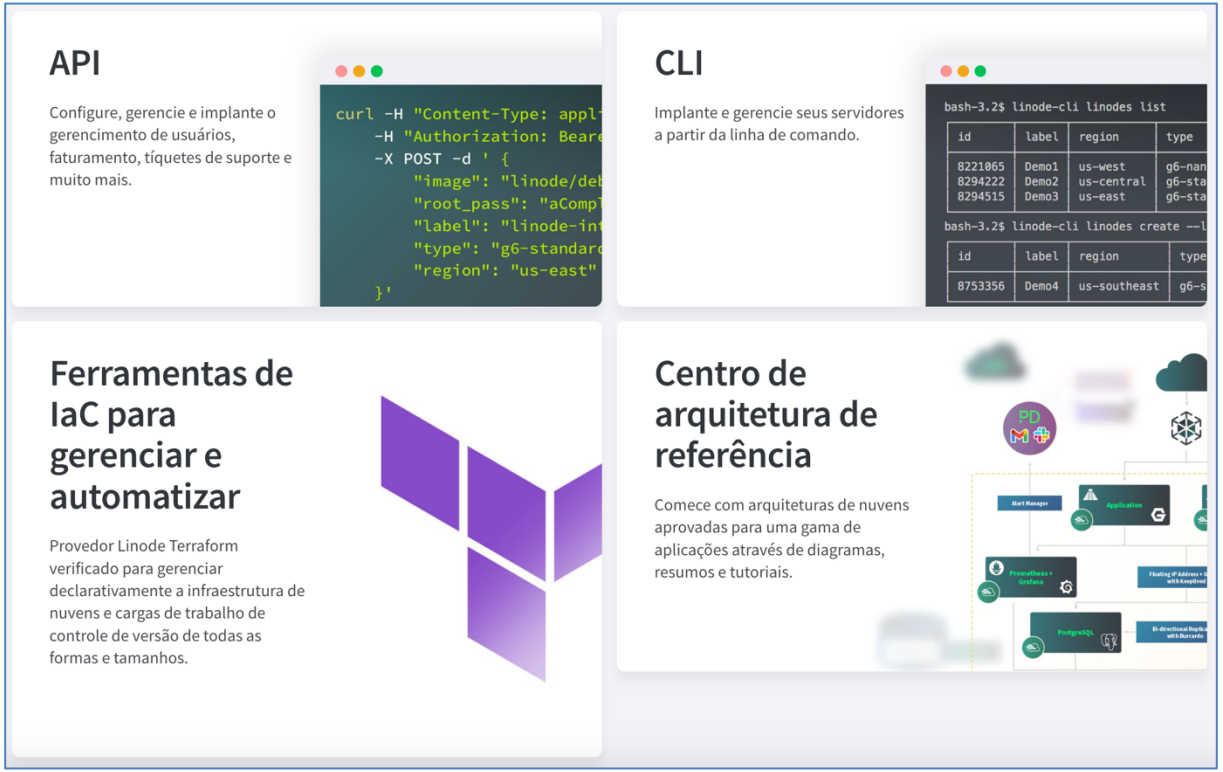
- **Ferramentas**: Plataformas de nuvem
  - **Linode/Akamai Cloud**: https://www.linode.com/pt-br/choosing-linode/



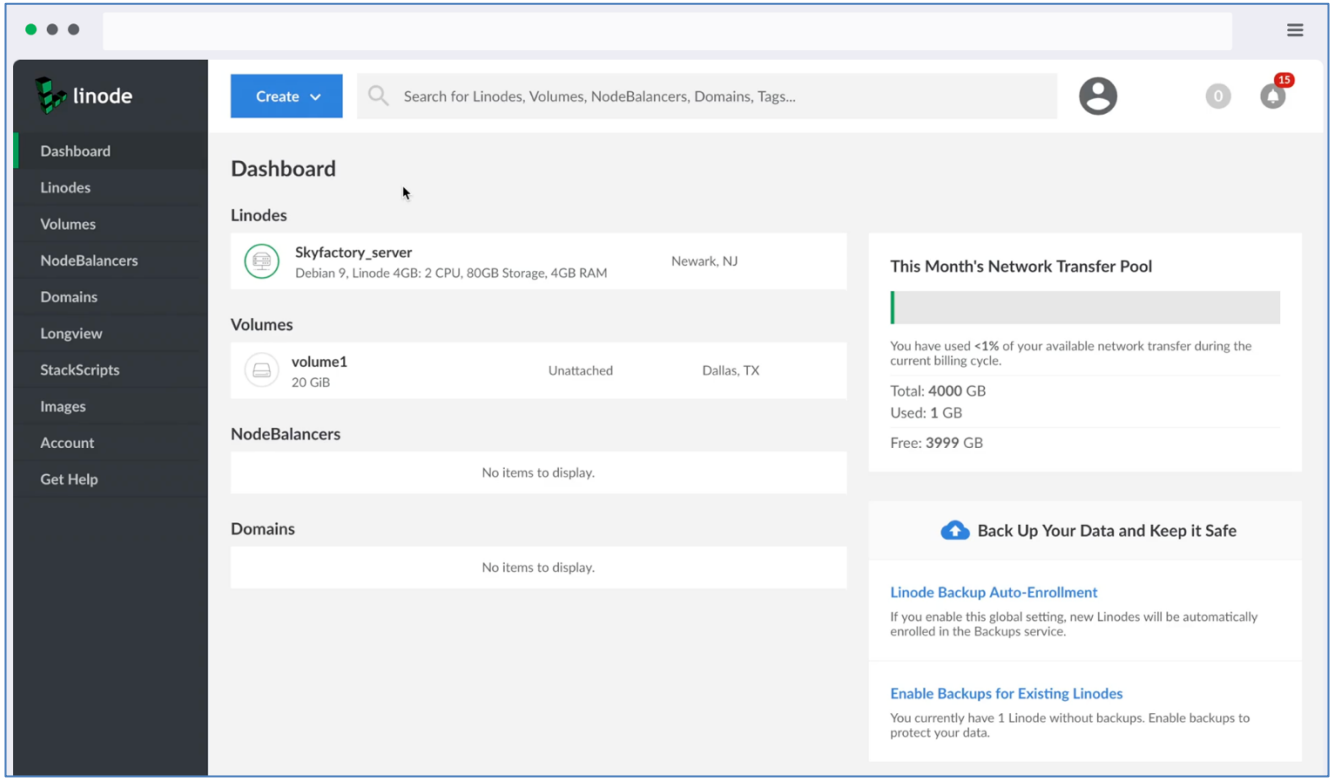
- **Ferramentas**: Plataformas de nuvem
  - **Linode/Akamai Cloud**: https://www.linode.com/pt-br/

- **Ferramentas**: Plataformas de nuvem
  - **Linode/Akamai Cloud**: https://www.linode.com/pt-br/

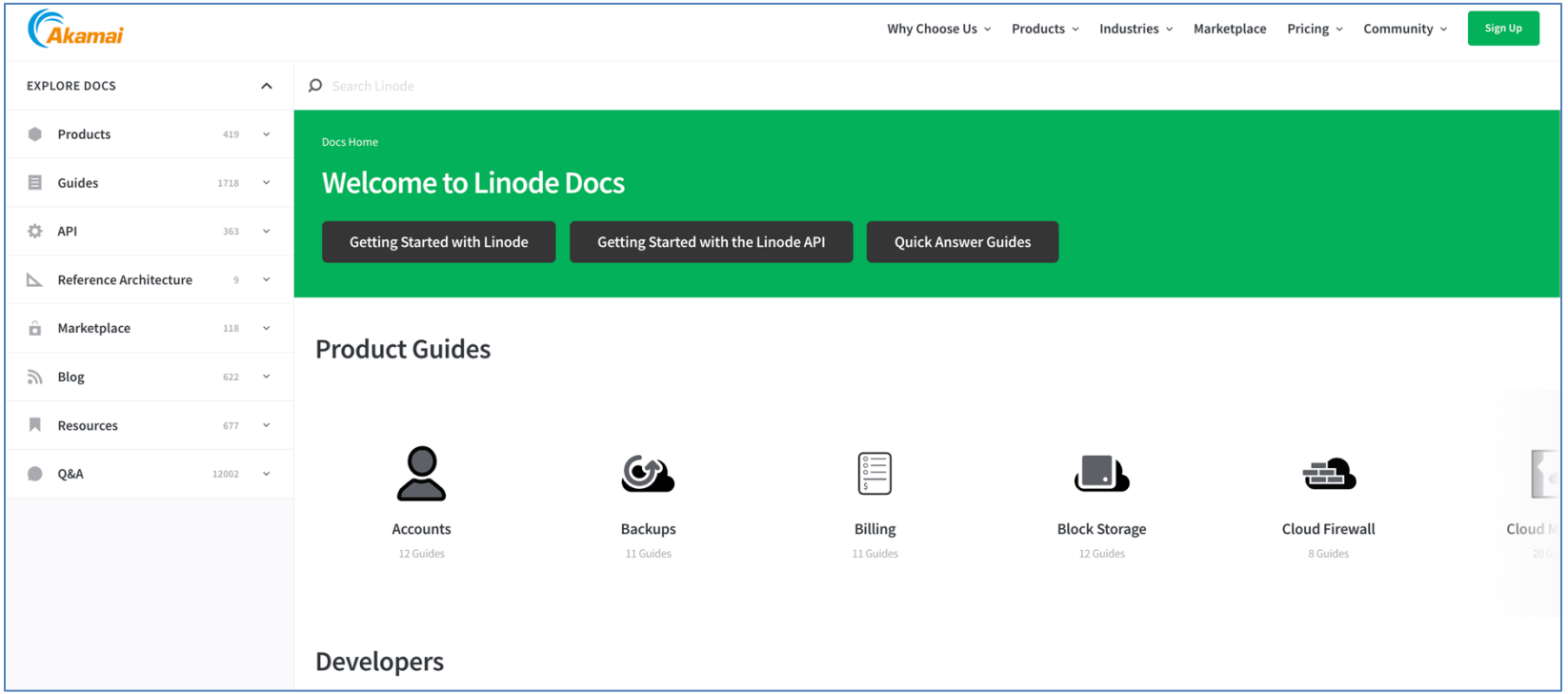
- **Ferramentas**: Plataformas de nuvem
  - **Linode/Akamai Cloud**: https://www.linode.com/pt-br/



- **Ferramentas**: Plataformas de nuvem
  - **Linode/Akamai Cloud**: https://www.linode.com/pt-br/

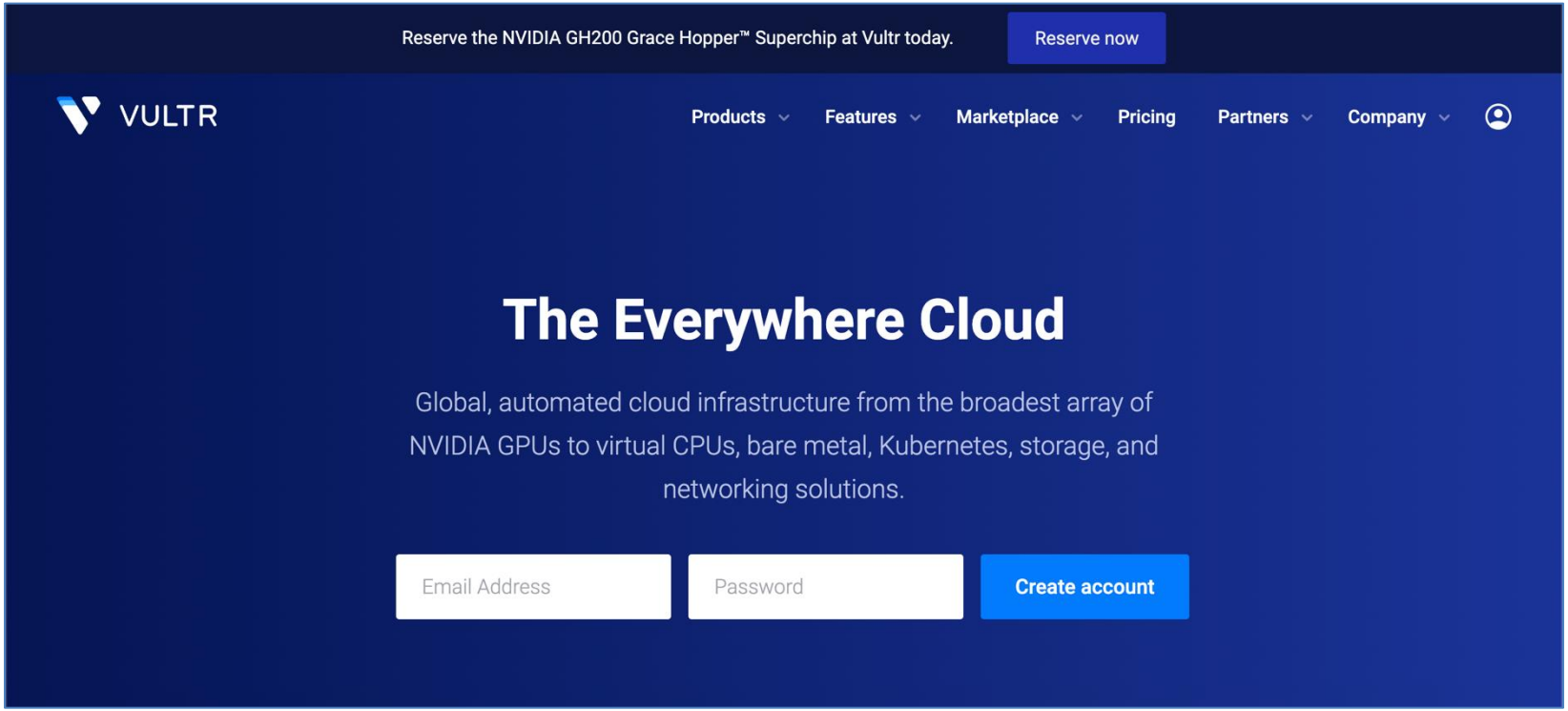


- Documentação do Linode/Akamai Cloud:
  - https://www.linode.com/docs/

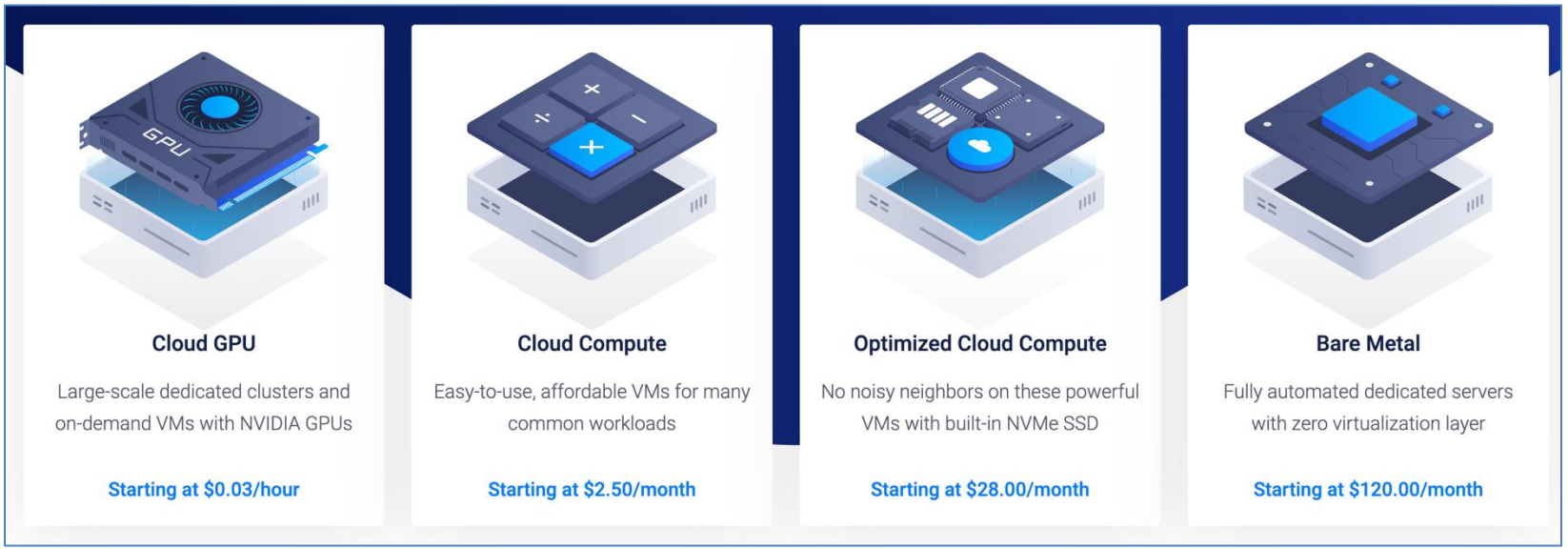


- Comunidade do Linode/Akamai:
  - https://www.linode.com/pt-br/cloud-computing-community/

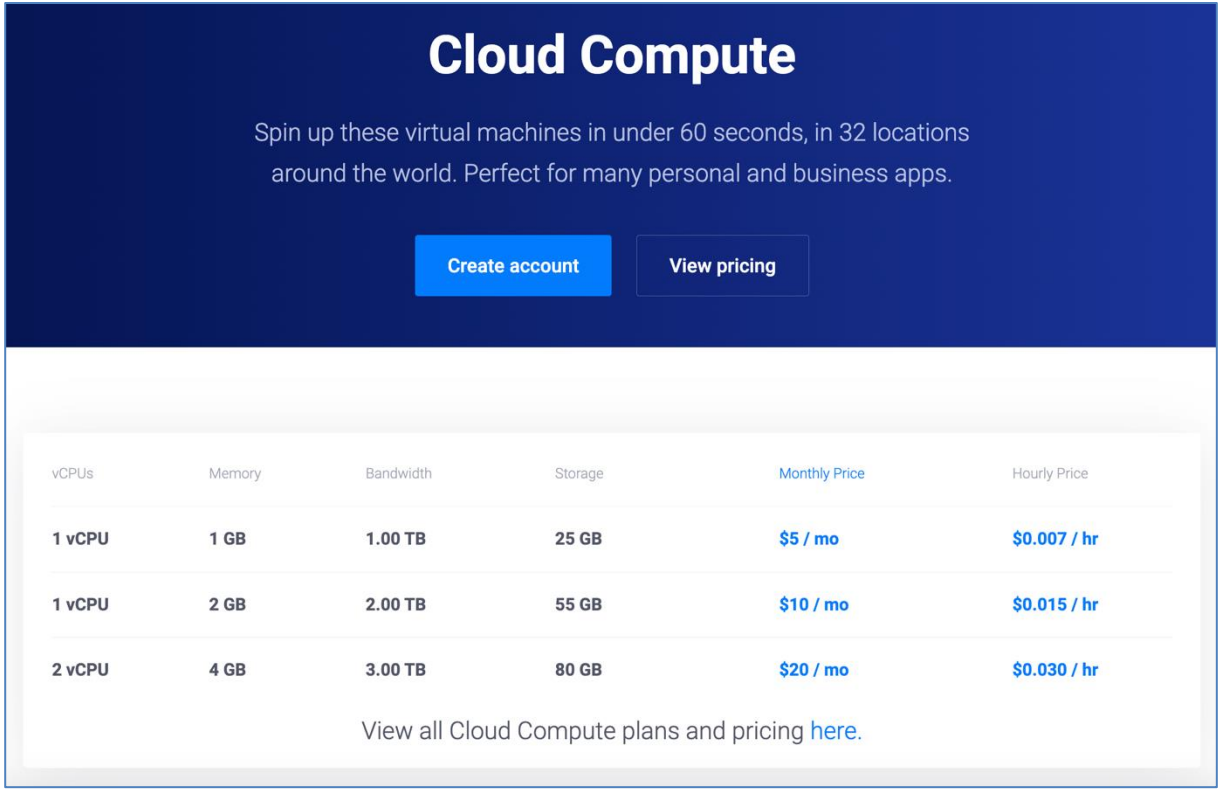
- **Ferramentas**: Plataformas de nuvem
  - **Vultr Cloud**: https://www.vultr.com/



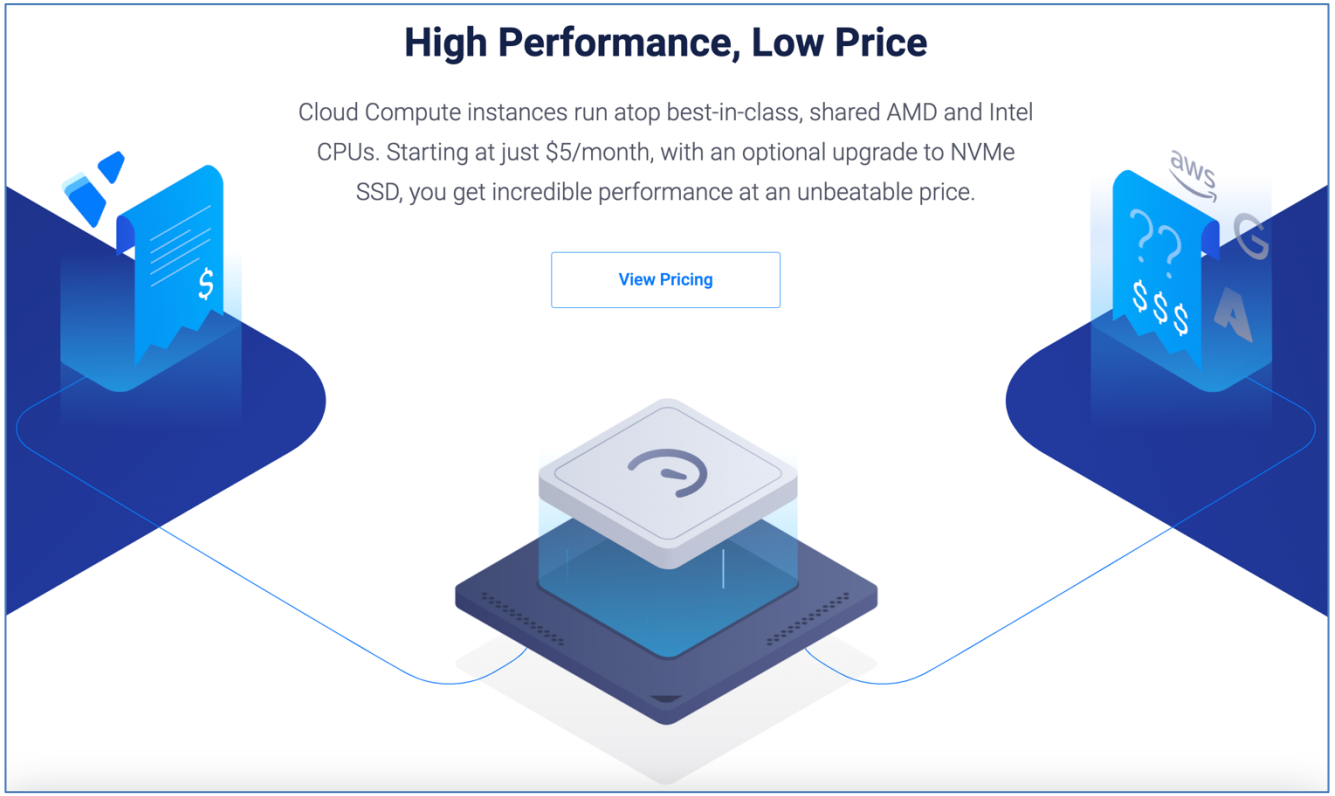
- **Ferramentas**: Plataformas de nuvem
  - **Vultr Cloud**: https://www.vultr.com/



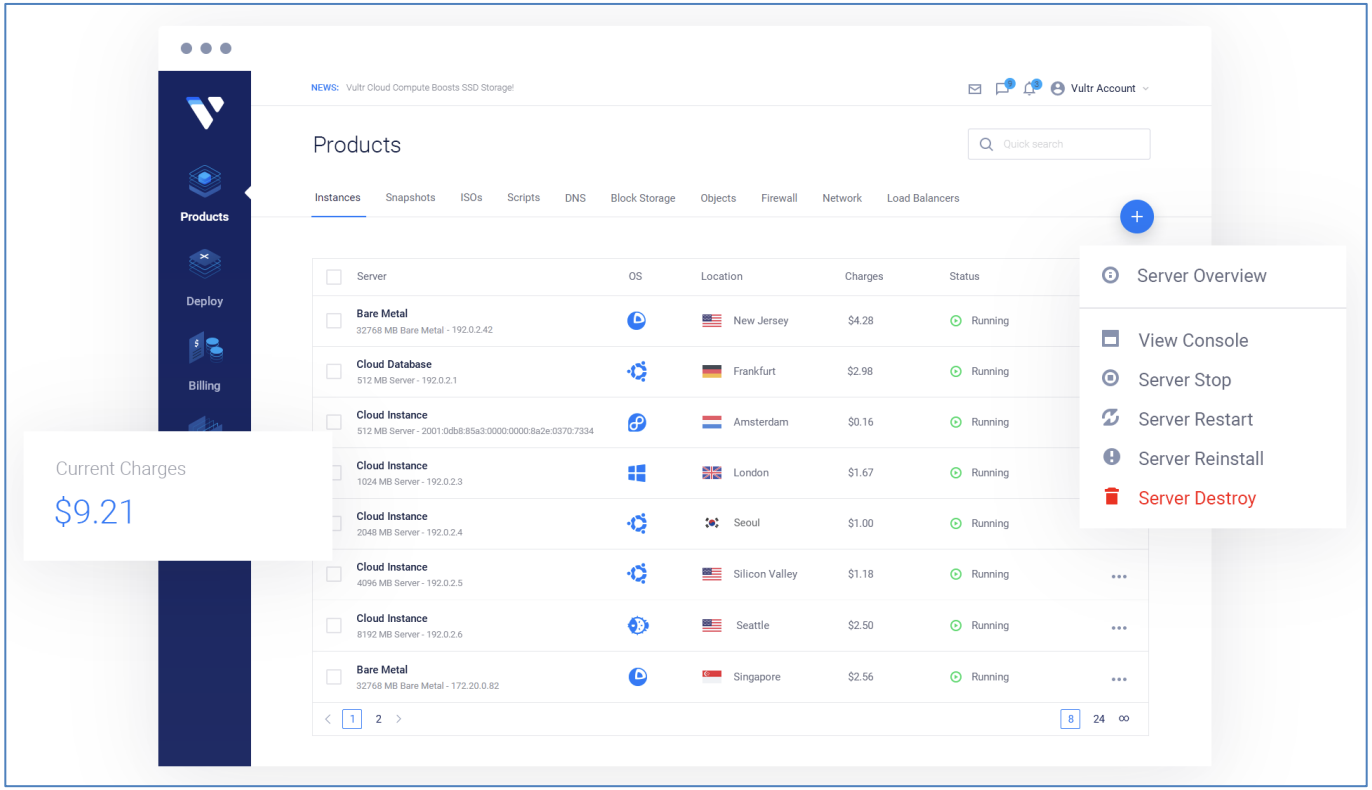
- **Ferramentas**: Plataformas de nuvem
  - **Vultr Cloud**: https://www.vultr.com/



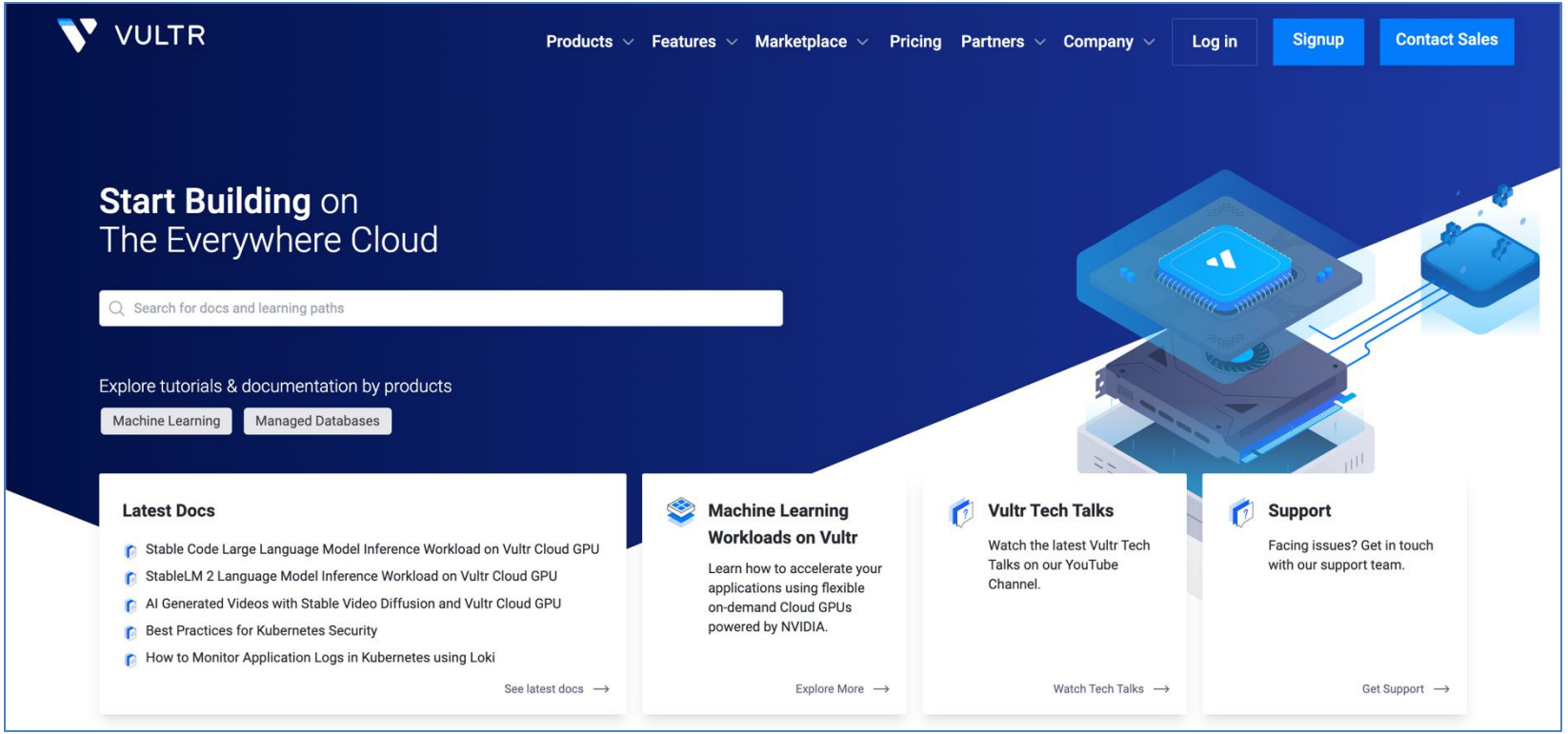
- **Ferramentas**: Plataformas de nuvem
  - **Vultr Cloud**: https://www.vultr.com/



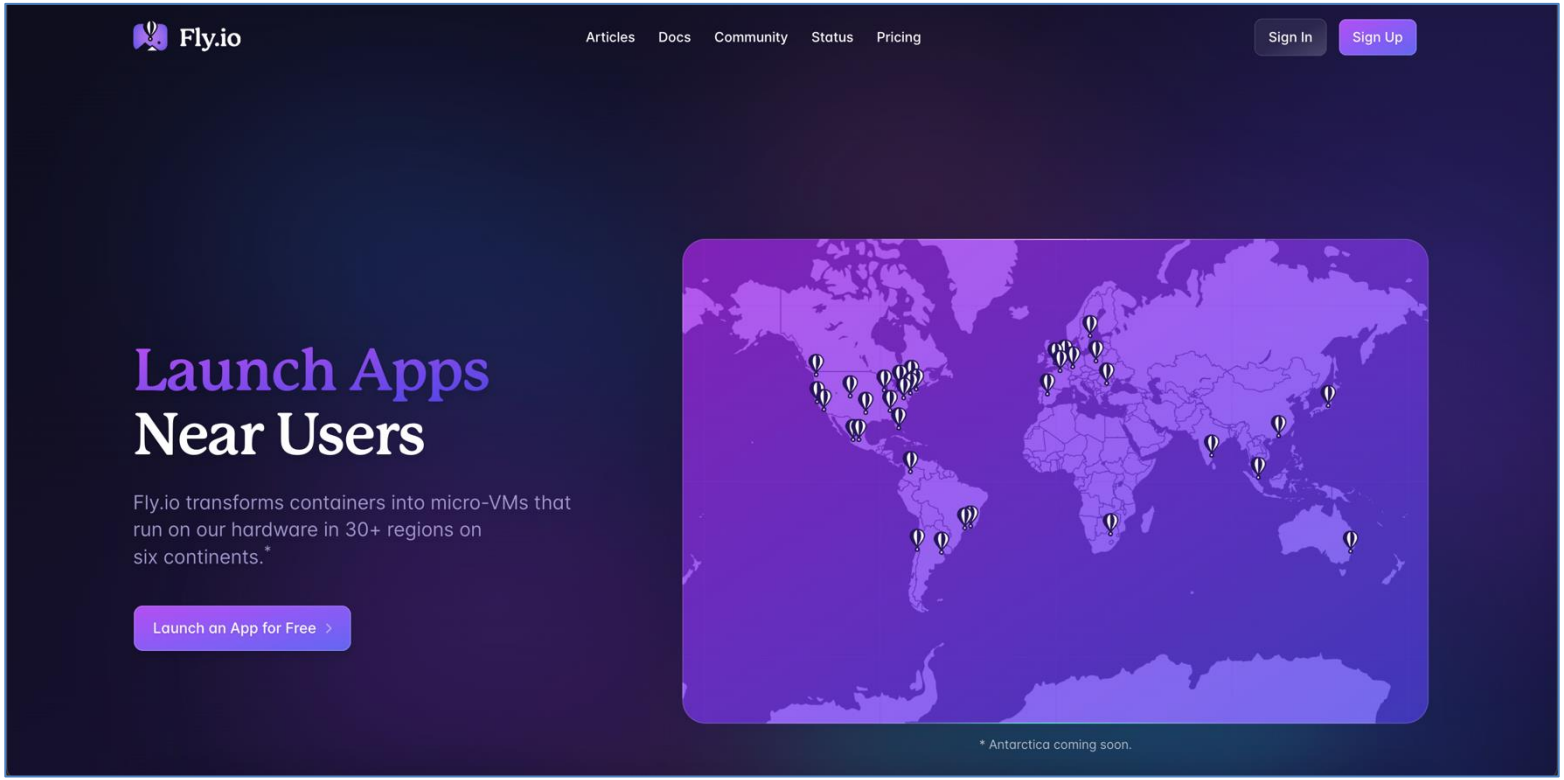
- **Ferramentas**: Plataformas de nuvem
  - **Vultr Cloud**: https://www.vultr.com/



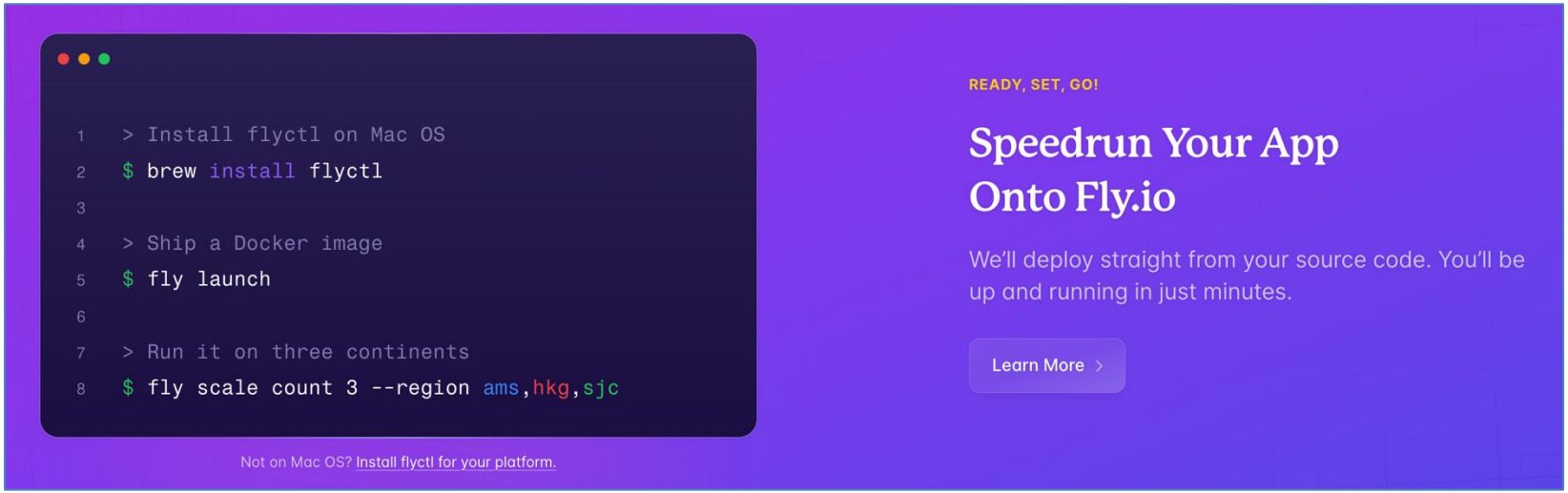
- Documentação do Vultr:
  - https://docs.vultr.com/



- **Ferramentas**: Plataformas de nuvem
  - **Fly.io**: https://fly.io/



- **Ferramentas**: Plataformas de nuvem
  - **Fly.io**: https://fly.io/

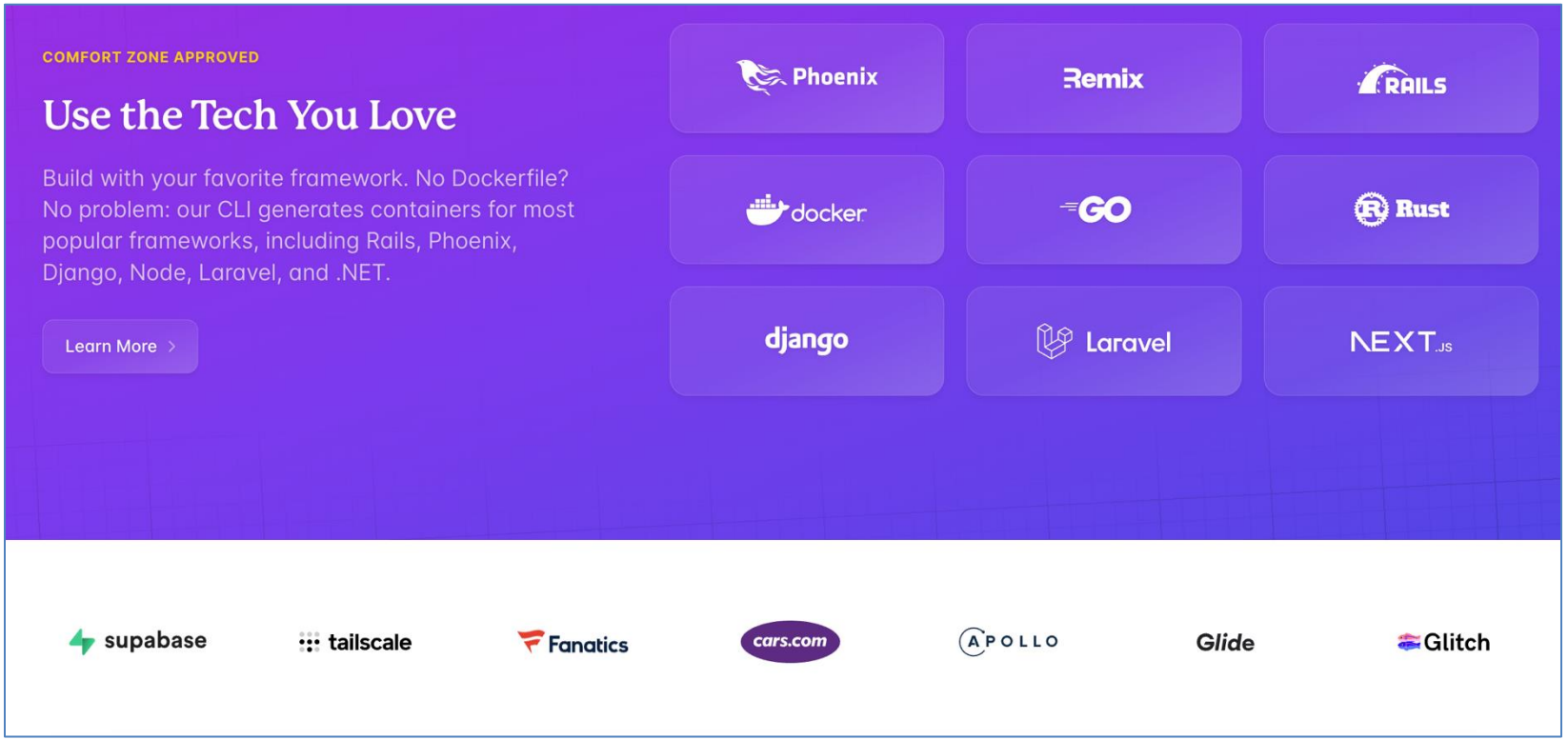


  - Leitura sugerida:

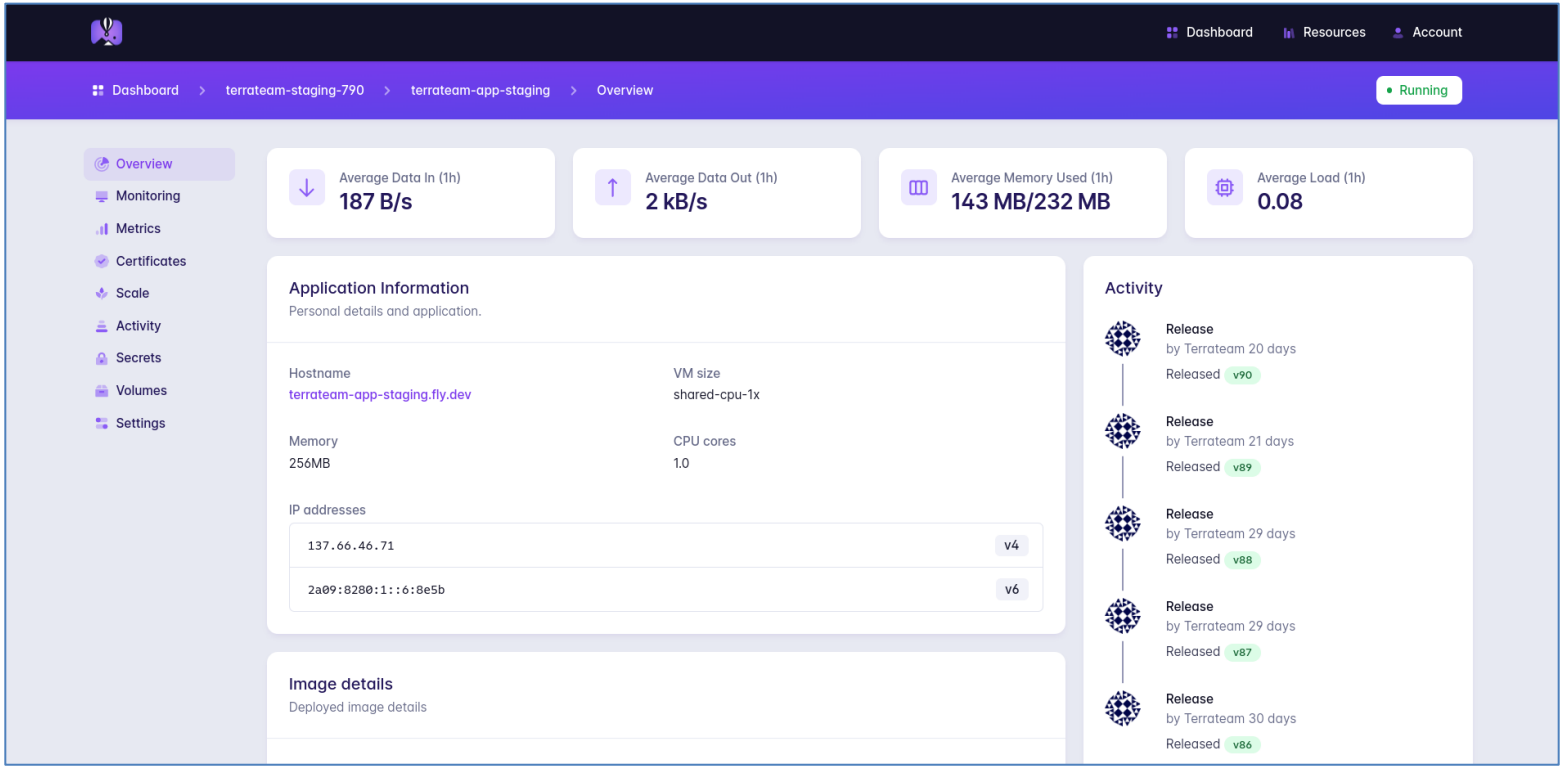
    https://terrateam.io/blog/flying-away-from-aws

    https://guilhermeabreucc.medium.com/heroku-deixou-de-ser-gratuito-onde-vou-hospedar-meu-container-net-46cc996c96b7

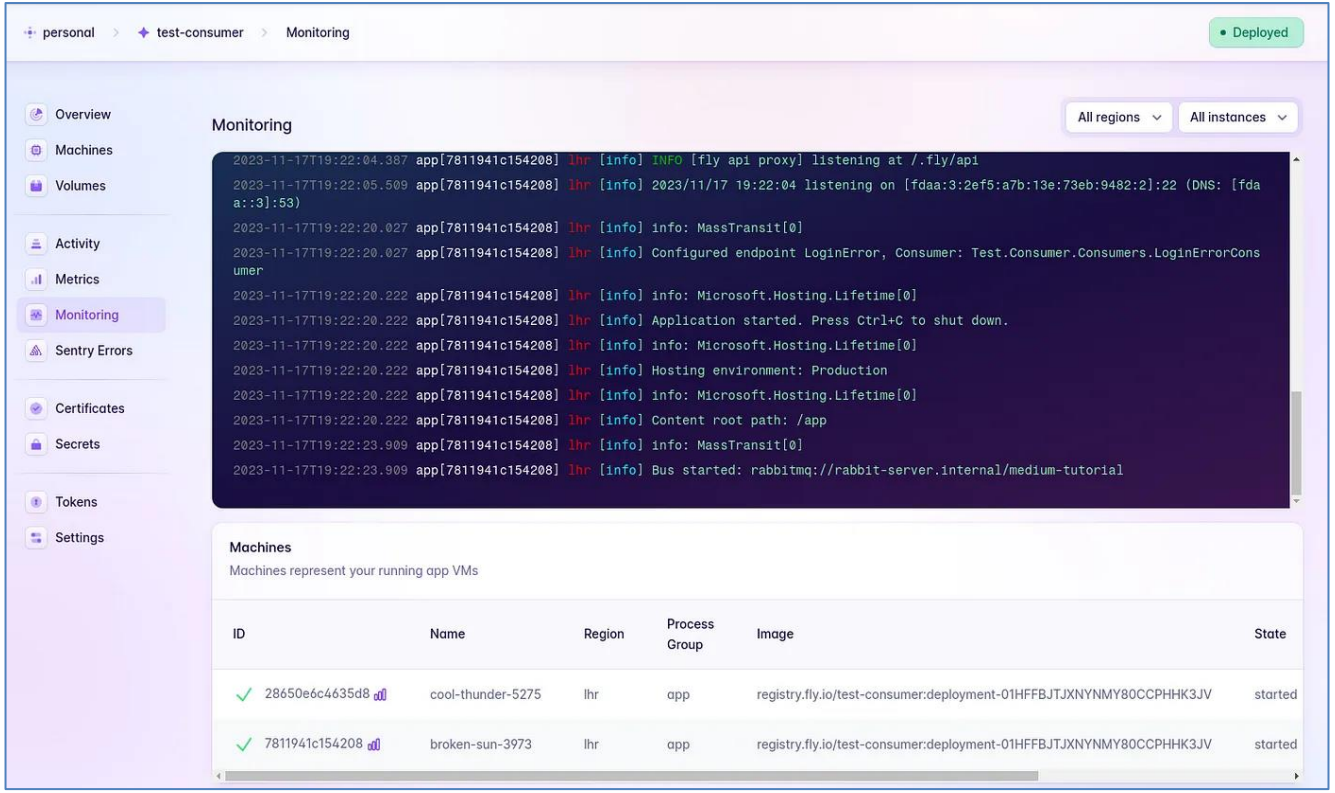
- **Ferramentas**: Plataformas de nuvem
  - **Fly.io**: https://fly.io/



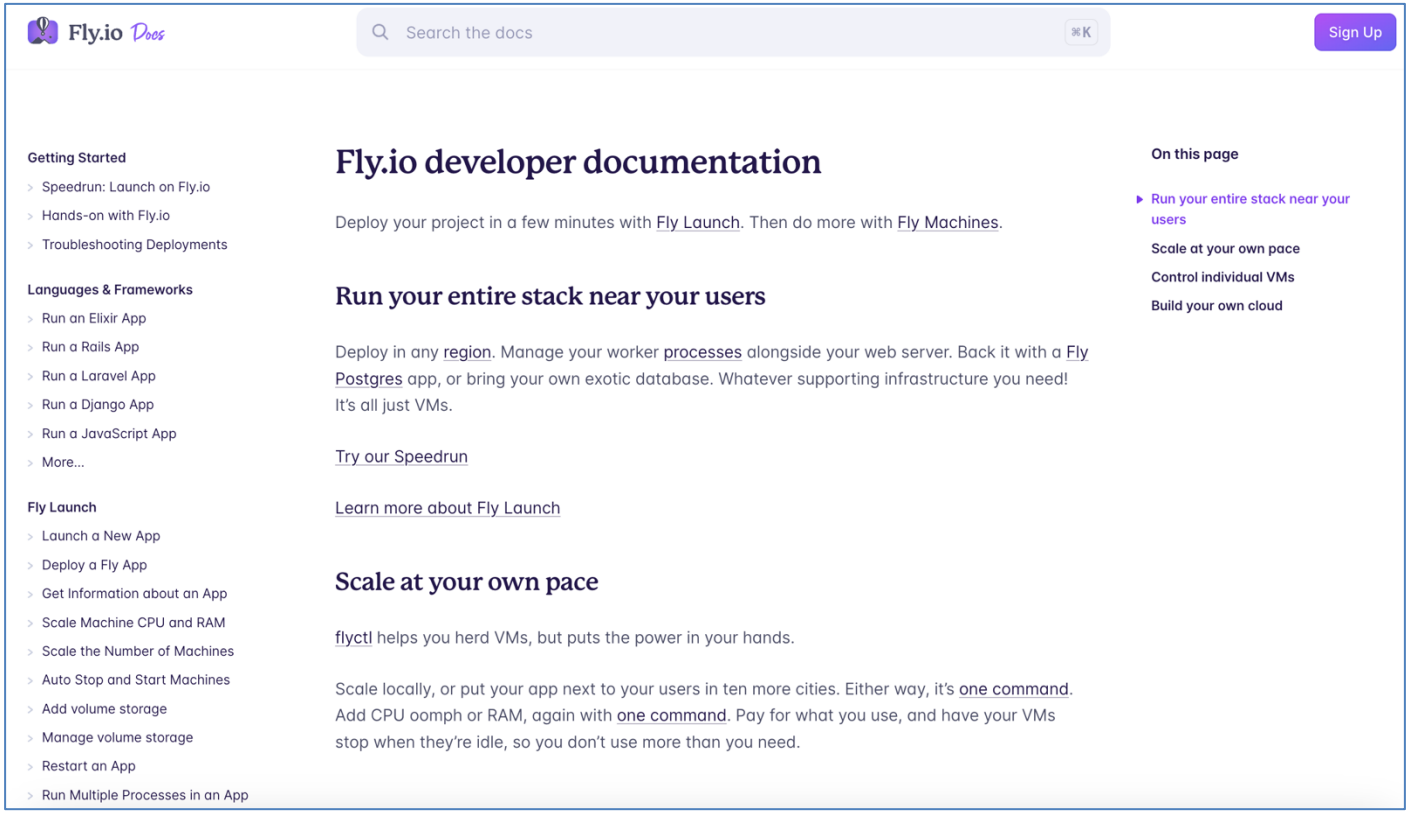
- **Ferramentas**: Plataformas de nuvem
  - **Fly.io**: https://fly.io/



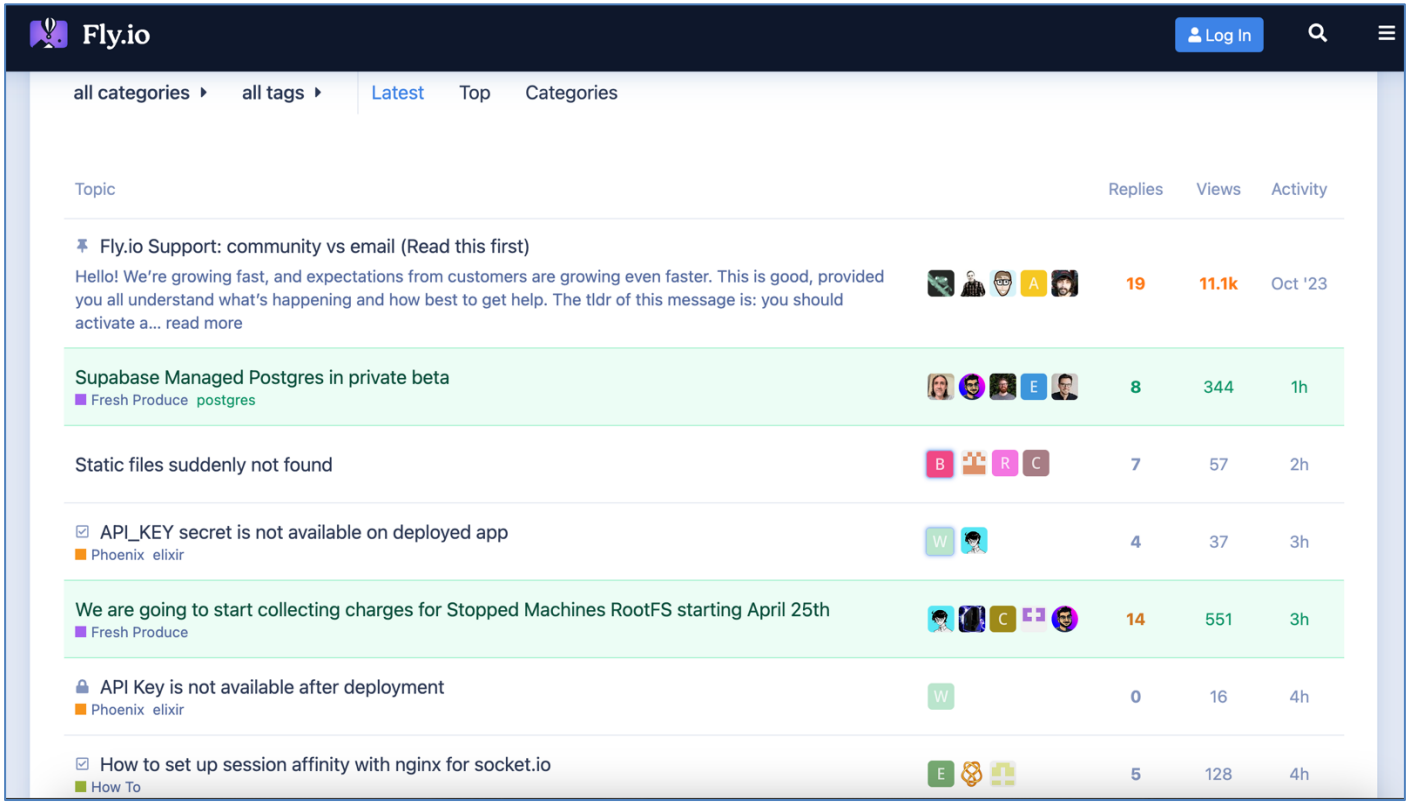
- **Ferramentas**: Plataformas de nuvem
  - **Fly.io - Metrics**: https://fly.io/docs/reference/metrics/



- Documentação do Fly.io:
  - https://fly.io/docs/



- Comunidade do Fly.io:
  - https://community.fly.io/



Sumário

- Hospedagem de aplicações Web
  - Plataformas em nuvem para hospedagem
  - **Preparando a aplicação**
  - Publicando a aplicação em uma solução PaaS (Platform as a Service)

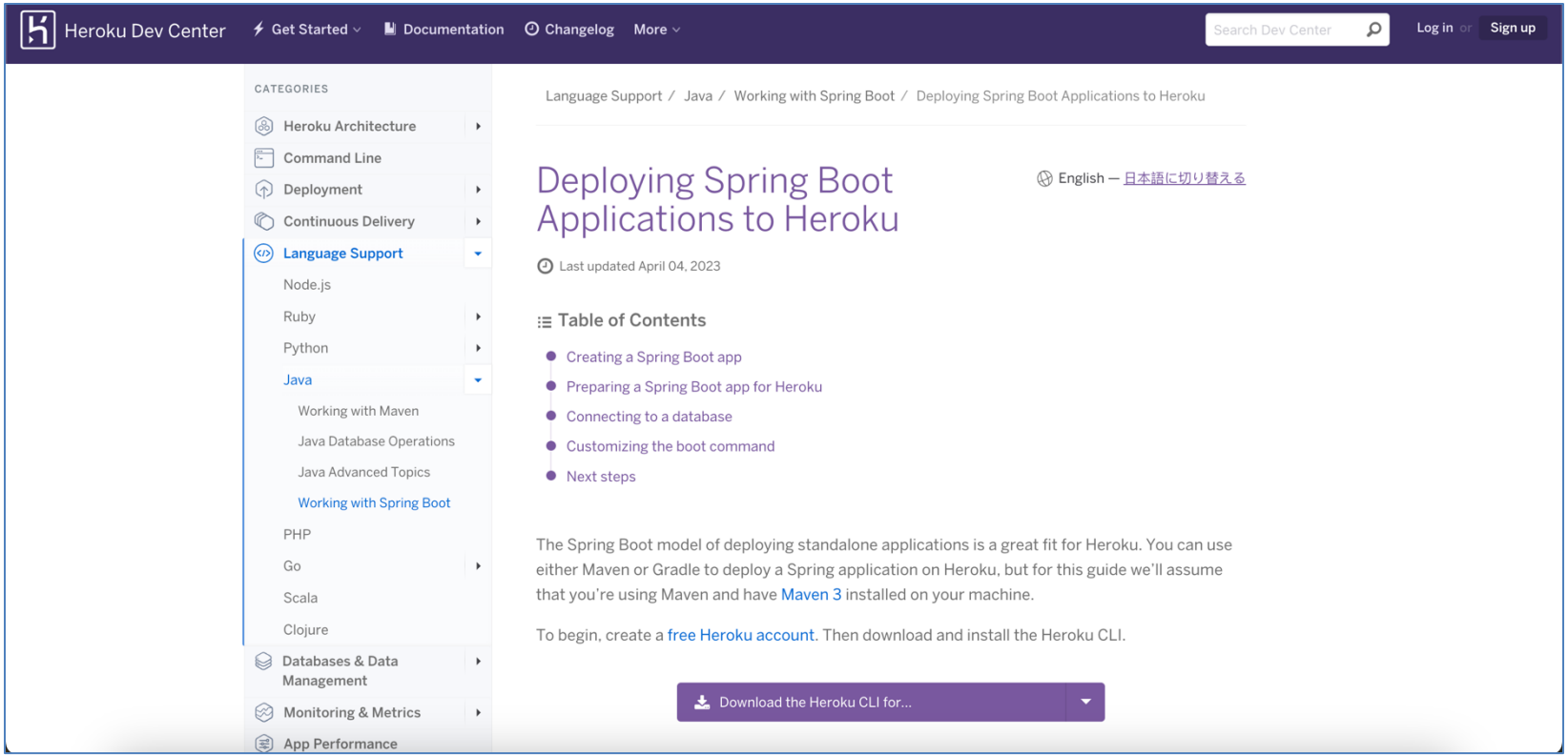
Hospedagem de aplicações Web

Fly.io

- **Preparando a aplicação**
  - No caso desta disciplina, usaremos o **Fly.io** para hospedar nossas aplicações Spring Boot.

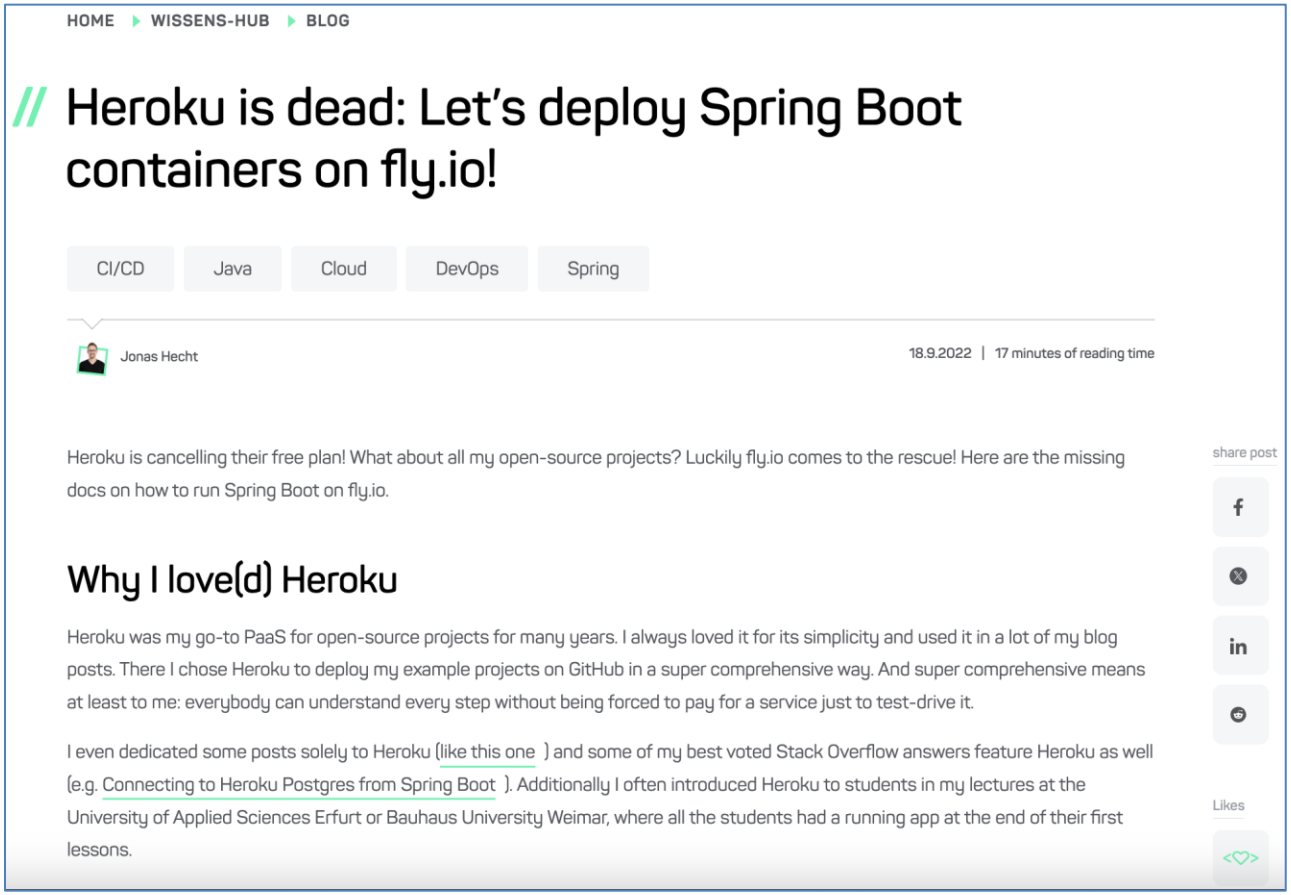
  - O Heroku vinha sendo uma escolha popular para cursos de desenvolvimento web, porém em agosto de 2022 foi anunciado que o serviço **gratuito** de hospedagem de aplicações web seria removido do Heroku.
    - Isto deixou a comunidade dev bem triste.

- Leitura sugerida:
  - https://devcenter.heroku.com/articles/deploying-spring-boot-apps-to-heroku



Leitura sugerida:

https://www.codecentric.de/wissens-hub/blog/spring-boot-flyio



HOME ▸ WISSENS-HUB ▸ BLOG

// Heroku is dead: Let's deploy Spring Boot containers on fly.io!

CI/CD | Java | Cloud | DevOps | Spring

Jonas Hecht

18.9.2022 | 17 minutes of reading time

Heroku is cancelling their free plan! What about all my open-source projects? Luckily fly.io comes to the rescue! Here are the missing docs on how to run Spring Boot on fly.io.

Why I love(d) Heroku

Heroku was my go-to PaaS for open-source projects for many years. I always loved it for its simplicity and used it in a lot of my blog posts. There I chose Heroku to deploy my example projects on GitHub in a super comprehensive way. And super comprehensive means at least to me: everybody can understand every step without being forced to pay for a service just to test-drive it.

I even dedicated some posts solely to Heroku (like this one) and some of my best voted Stack Overflow answers feature Heroku as well (e.g. Connecting to Heroku Postgres from Spring Boot). Additionally I often introduced Heroku to students in my lectures at the University of Applied Sciences Erfurt or Bauhaus University Weimar, where all the students had a running app at the end of their first lessons.

share post

Likes

Hospedagem de aplicações Web

- **Preparando a aplicação**
  - Antes de mais nada, vamos entender o que é o processo de **deployment** ou, simplesmente, **deploy** da aplicação.

  - O deploy, ou deployment, refere-se ao processo de colocar uma aplicação ou uma versão específica dela em um ambiente de execução, tornando-a acessível e funcional para os usuários finais.

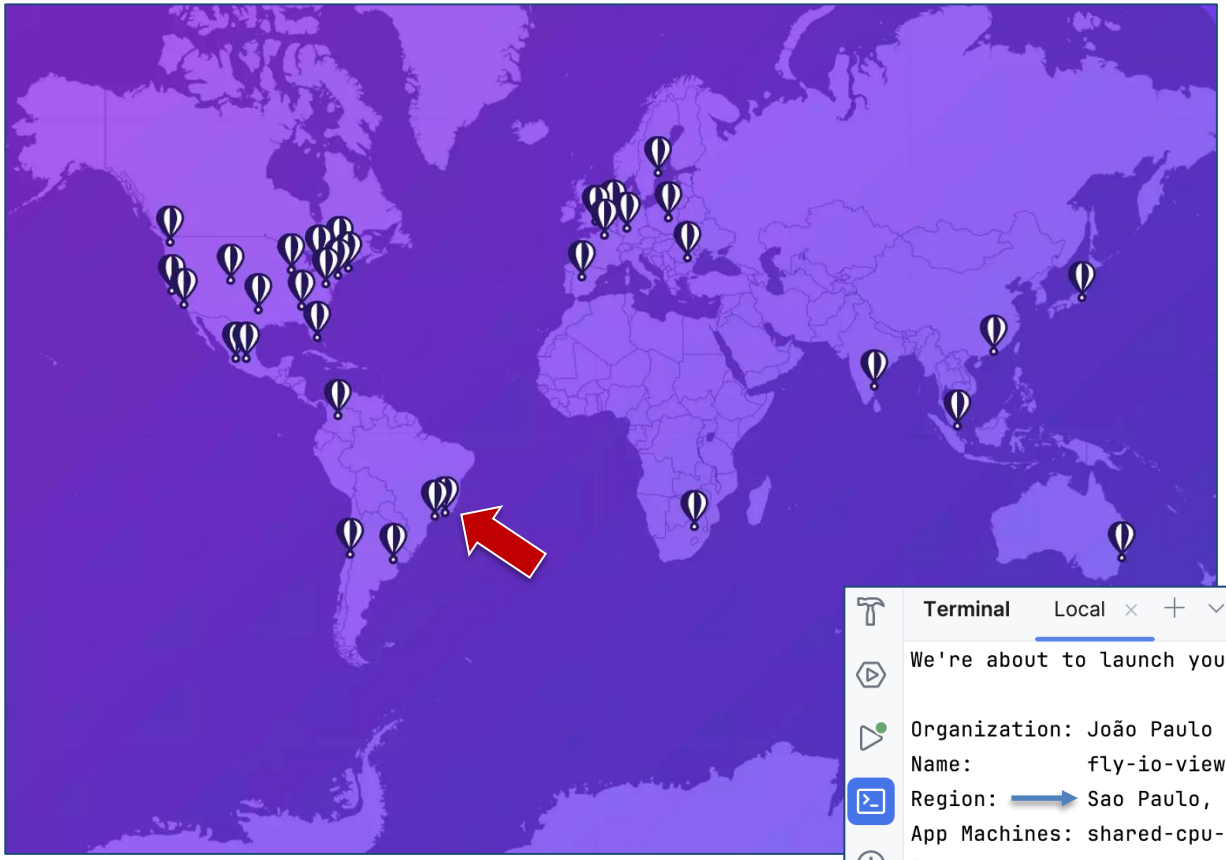
Hospedagem de aplicações Web

- **Preparando a aplicação**
  - Esse processo envolve a transferência do código-fonte da aplicação para um ambiente de produção, onde será compilado, empacotado e configurado para execução.

  - Ao realizar o deploy, as alterações ou atualizações no código-fonte da aplicação podem ser implementadas, garantindo que os usuários tenham acesso às últimas funcionalidades ou correções de bugs. O deployment é uma etapa essencial no ciclo de vida do desenvolvimento de software.

- Aplicação rodando localmente na máquina:
  - http://localhost:8080/

- Aplicação rodando na nuvem em uma PaaS (Platform as a Service) - **Fly.io**:
  - https://fly-io-view-restapi.fly.dev/



Launch Apps Near Users

Fly.io transforms containers into micro-VMs that run on our hardware in 30+ regions on six continents.*

Launch an App for Free ›

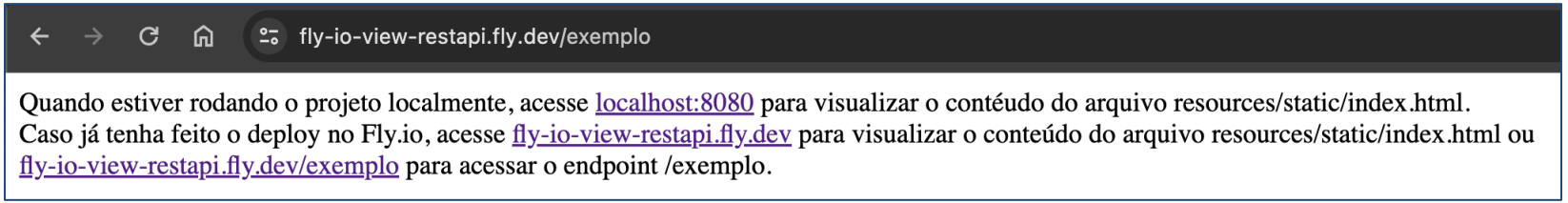
Terminal Local

```
We're about to launch your app on Fly.io. Here's what you're getting:

Organization: João Paulo Aramuni    (fly launch defaults to the personal org)
Name:         fly-io-view-restapi    (derived from your directory name)
Region:       Sao Paulo, Brazil      (this is the fastest region for you)
App Machines: shared-cpu-1x, 1GB RAM (most apps need about 1GB of RAM)
Postgres:     <none>                 (not requested)
Redis:        <none>                 (not requested)
```

- Acessando o endpoint **/exemplo** localmente na máquina:
  - http://localhost:8080/exemplo

- Acessando o endpoint **/exemplo** na nuvem em uma PaaS (Platform as a Service) - **Fly.io**:
  - https://fly-io-view-restapi.fly.dev/exemplo

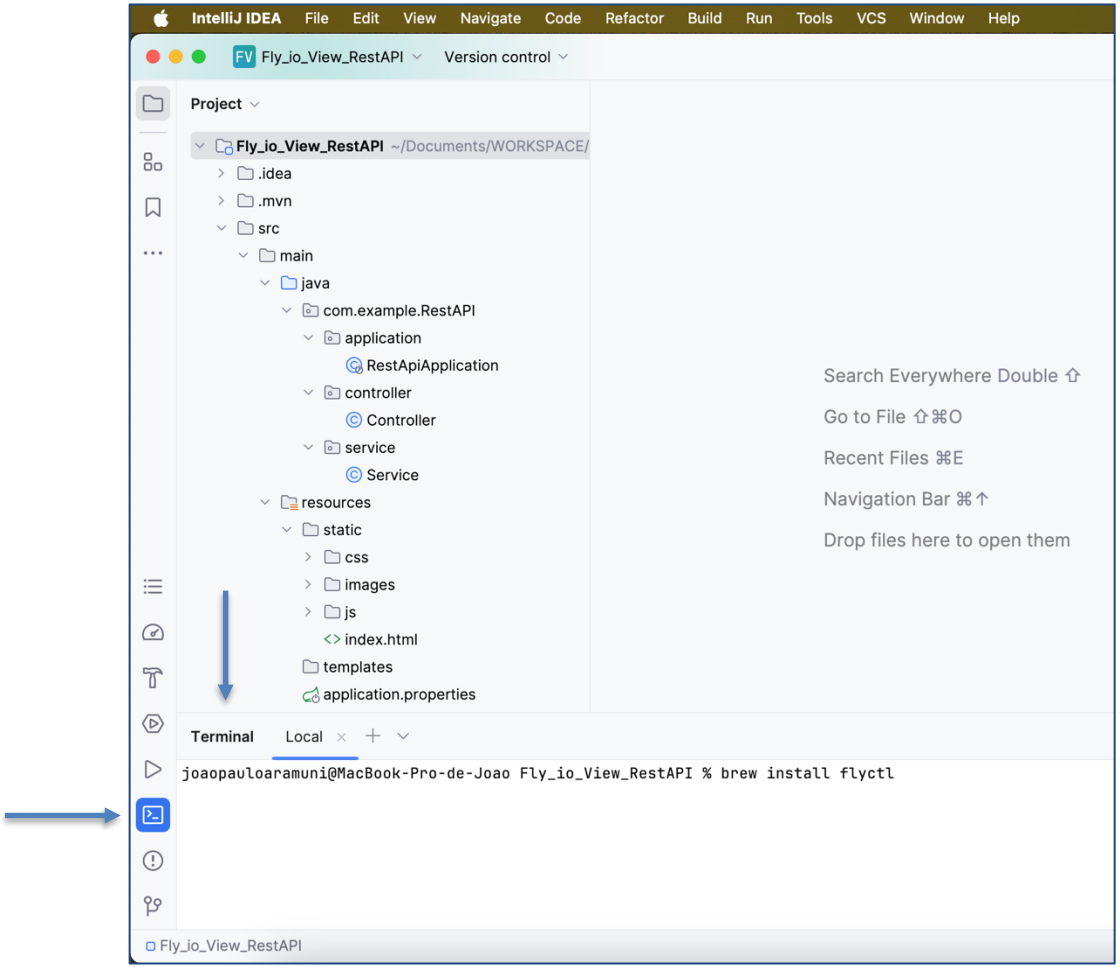


Hospedagem de aplicações Web

- **Preparando a aplicação**
  - Instalando o **Fly.io**:
    - https://fly.io/launchpad
    - https://fly.io/docs/flyctl/
    - https://fly.io/docs/hands-on/install-flyctl/

    - Abra o terminal e execute:
      - No macOS: brew install flyctl ou curl -L https://fly.io/install.sh | sh
      - No Linux: curl -L https://fly.io/install.sh | sh
      - No Windows: pwsh -Command "iwr https://fly.io/install.ps1 -useb | iex"

- **Preparando a aplicação**
  - Instalando o **Fly.io**:
    - Você pode utilizar o próprio terminal do IntelliJ IDEA para isso:
    - Ou um terminal de sua preferência:
      - https://iterm2.com/



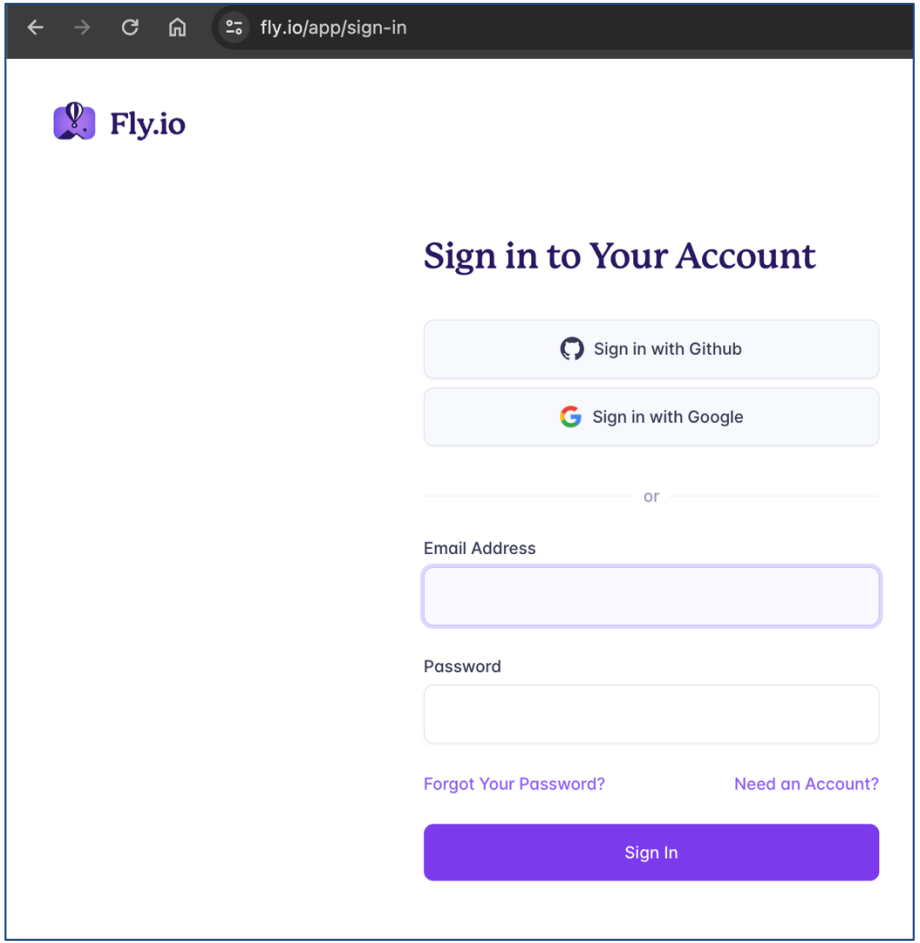
- **Preparando a aplicação**
  - Instalando o **Fly.io**:
    - Em seguida, faça o cadastro e/ou login no Fly.io:
      - https://fly.io/app/sign-up
      - https://fly.io/docs/hands-on/sign-up/
      - https://fly.io/docs/hands-on/sign-in/

Sign In

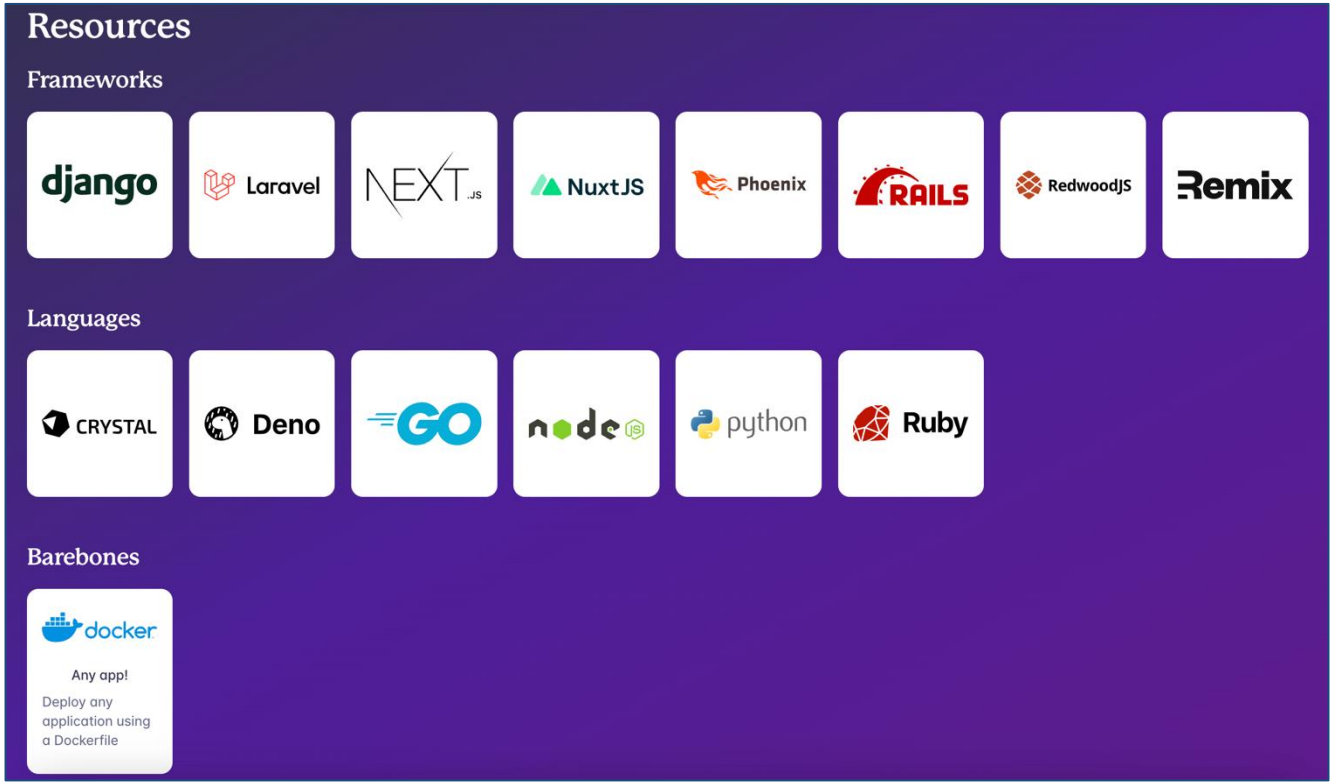
If you already have a Fly.io account, all you need to do is sign in with flyctl. To sign in, run:

```
$ fly auth login
```

When your browser opens to the Fly.io sign-in screen, enter your user name and password to sign in. If you signed up with GitHub, then click `Sign in with GitHub` to sign in.

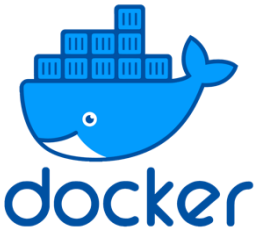


- Infelizmente, o Fly.io não possui suporte nativo para o Java.
  - Por conta disso, teremos que “**dockerizar**” a nossa aplicação.



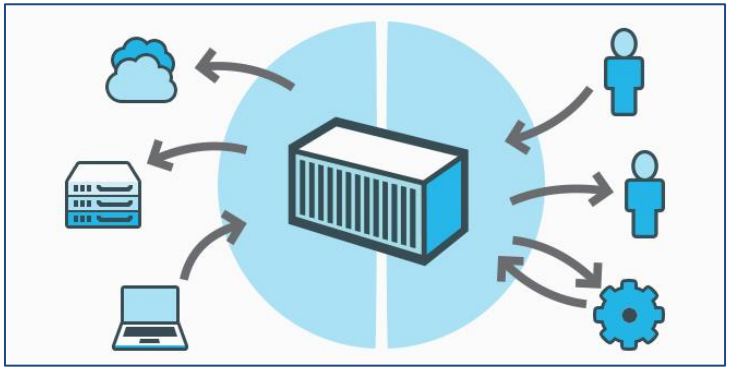
Hospedagem de aplicações Web

- **Preparando a aplicação**
  - **Docker**:
    - O Docker é uma plataforma de software que permite a criação, implantação e execução de aplicações em contêineres.
    - Contêineres são unidades leves e portáteis que empacotam tudo o que é necessário para executar um software, incluindo o código, as bibliotecas e as dependências.



Hospedagem de aplicações Web

- **Preparando a aplicação**
  - **Docker**:
    - Os contêineres são isolados uns dos outros e do ambiente do sistema operacional host, o que torna o Docker uma solução eficiente para desenvolvimento, teste e implantação de aplicações.



Hospedagem de aplicações Web

- **Preparando a aplicação**
  - **Docker**:
    - Uma imagem Docker é um artefato empacotado que contém um sistema de arquivos e parâmetros necessários para a execução de uma aplicação em um contêiner Docker.

    - Em outras palavras, uma imagem Docker é uma unidade leve e independente que encapsula todos os componentes essenciais para que um software seja executado, incluindo o sistema operacional, bibliotecas, código-fonte, dependências e configurações.

Hospedagem de aplicações Web

- **Preparando a aplicação**
  - **Docker**:
    - De forma resumida, uma imagem Docker é usada como um modelo para criar contêineres, e cada contêiner é uma instância em execução de uma única imagem.

    - Isso fornece isolamento e consistência, permitindo que você desenvolva, distribua e execute aplicações de maneira confiável em diferentes ambientes.

- **Preparando a aplicação**
  - **Docker**:
    - Algumas características importantes do Docker incluem:
      - **Portabilidade**: Os contêineres são executados em qualquer lugar onde o Docker esteja instalado, independentemente do ambiente do sistema operacional ou da infraestrutura subjacente. Isso facilita a movimentação de aplicativos entre diferentes ambientes, desde o desenvolvimento até a produção.

      - **Orquestração**: O Docker pode ser integrado a ferramentas de orquestração, como Kubernetes, para facilitar a implantação e o gerenciamento de aplicativos em escala.

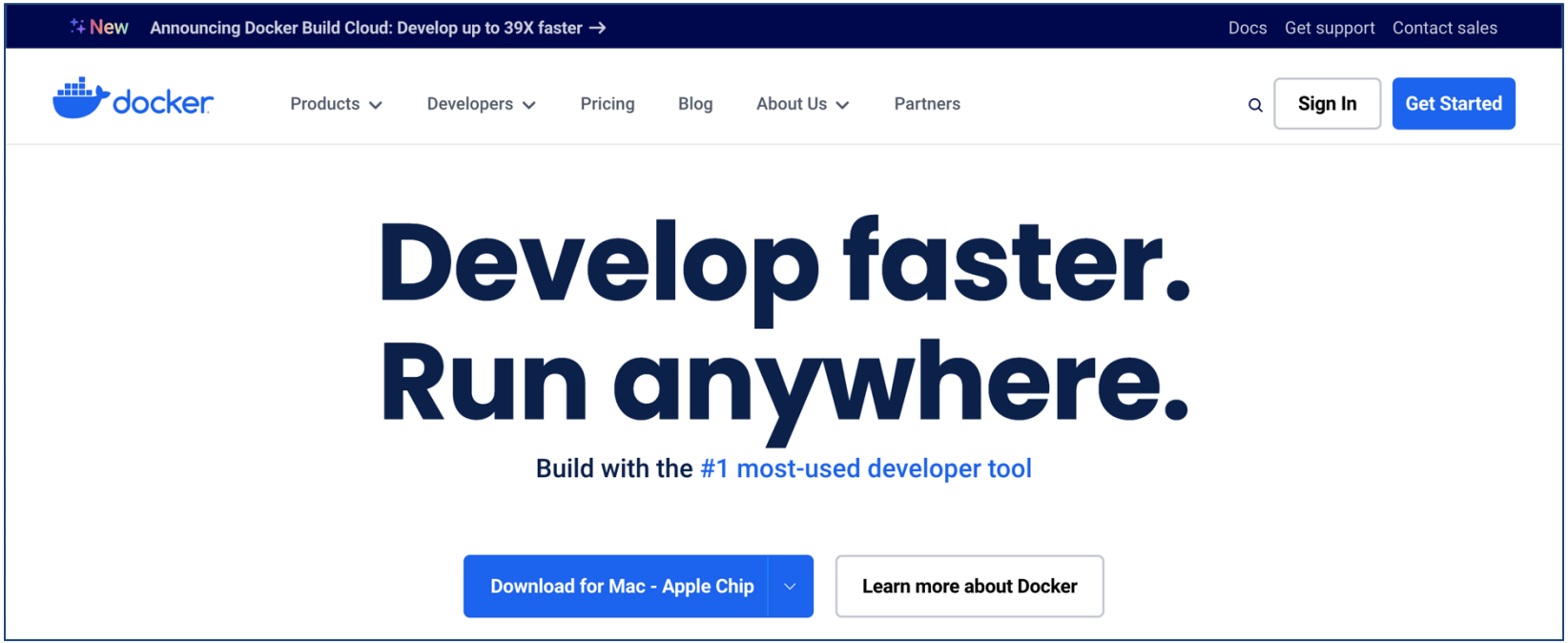
- **Preparando a aplicação**
  - **Docker**:
    - Algumas características importantes do Docker incluem:
      - **Isolamento**: Os contêineres fornecem isolamento de recursos, garantindo que um aplicativo em um contêiner não interfira nos outros ou no sistema operacional host. Isso é alcançado usando recursos como namespaces e cgroups do kernel do Linux.

      - **Eficiência**: Os contêineres compartilham o mesmo kernel do sistema operacional host, o que torna seu desempenho mais eficiente do que a virtualização tradicional.

- **Preparando a aplicação**
  - **Docker**:
    - Algumas características importantes do Docker incluem:
      - **Rápido provisionamento**: Os contêineres podem ser iniciados e parados rapidamente, facilitando o escalonamento dinâmico de aplicativos conforme necessário.

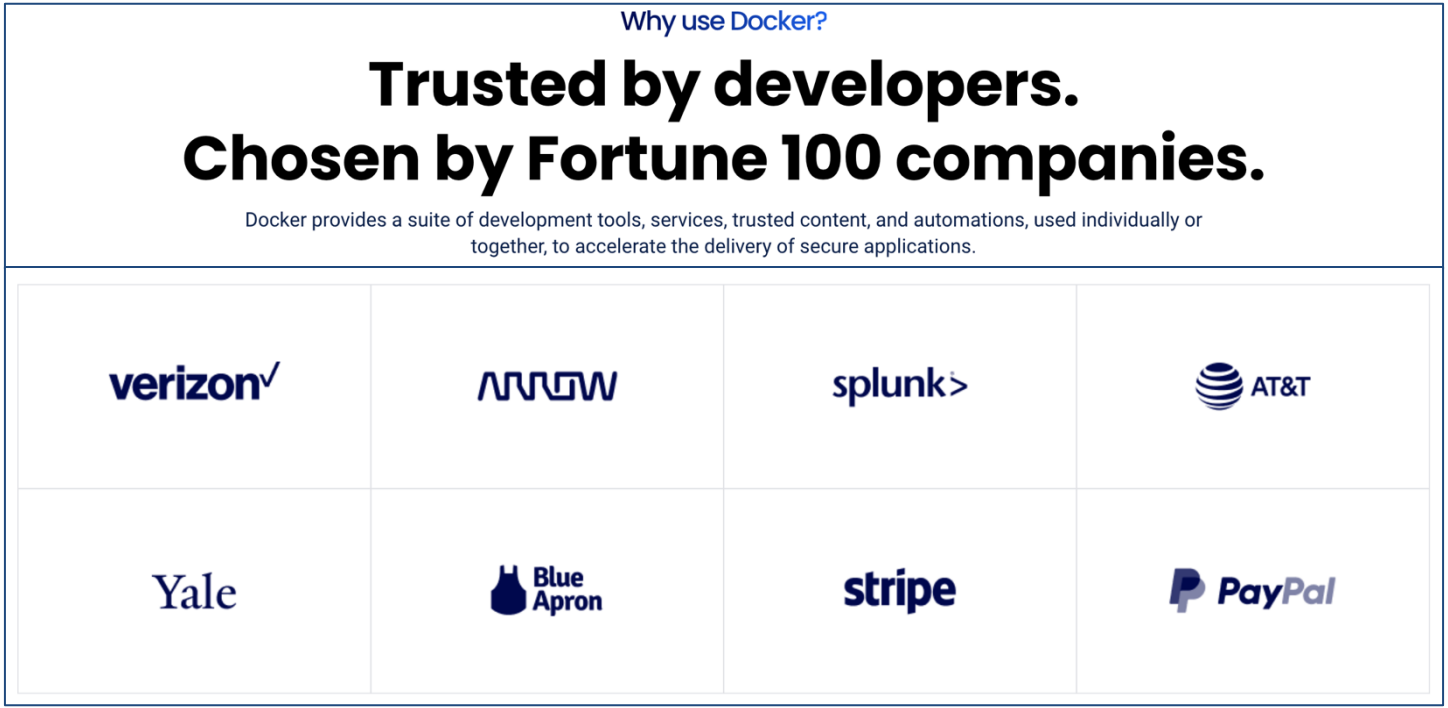
      - **Gestão de imagens**: O Docker usa um sistema de imagens para empacotar e distribuir aplicativos com todas as suas dependências. Essas imagens podem ser versionadas e compartilhadas por meio do Docker Hub ou repositórios privados.

- **Preparando a aplicação**
  - **Docker**:
    - https://www.docker.com/



- **Preparando a aplicação**
  - **Docker**:
    - https://www.docker.com/

20M+ monthly developers | **7M+** applications | **20B+** monthly image pulls

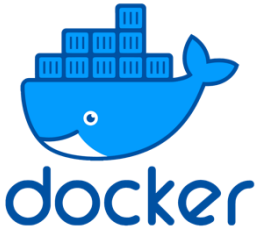


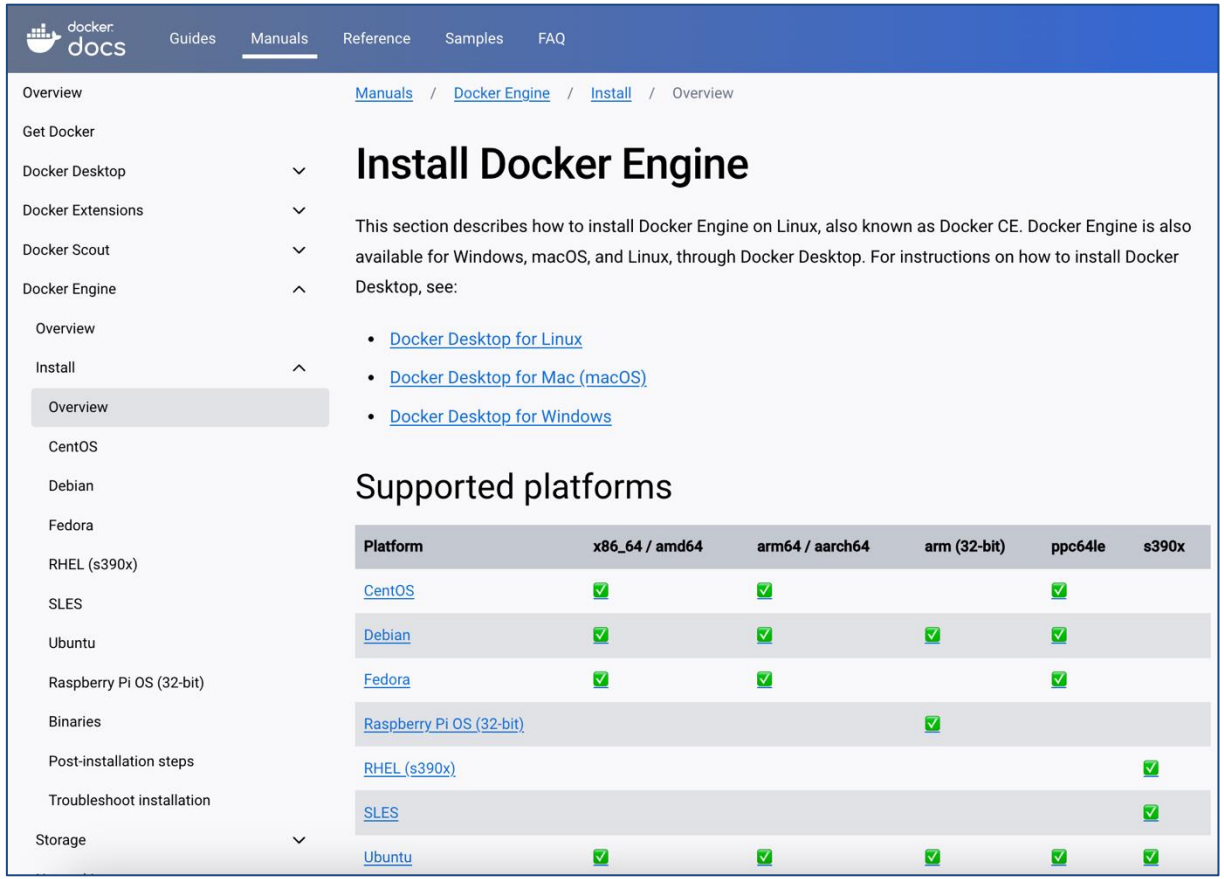
- **Stack Overflow Survey 2023**:
  - Em 2023, o Docker foi a ferramenta (na pesquisa *other tools*) mais utilizada entre todos os entrevistados (53%), subindo do segundo lugar do ano de 2022.
  - A pesquisa obteve 80.249 respondentes na categoria *All Respondents*.
  - https://survey.stackoverflow.co/2023/#other-tools

Instalando o **Docker**:

- https://docs.docker.com/engine/install/

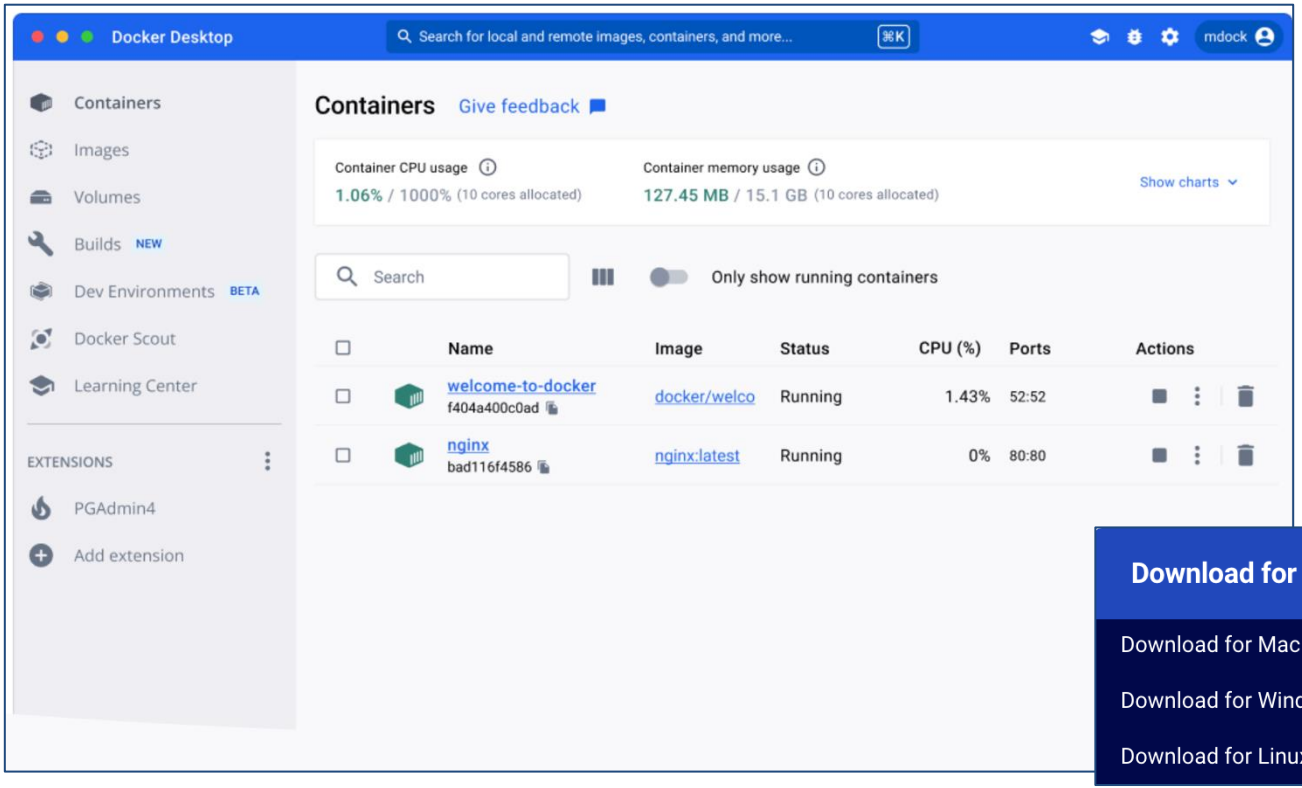
- Instale o Docker no terminal para poder usar a sua CLI (Interface de linha de comando):
- https://docs.docker.com/engine/reference/commandline/cli/
- https://docs.docker.com/engine/reference/commandline/docker/

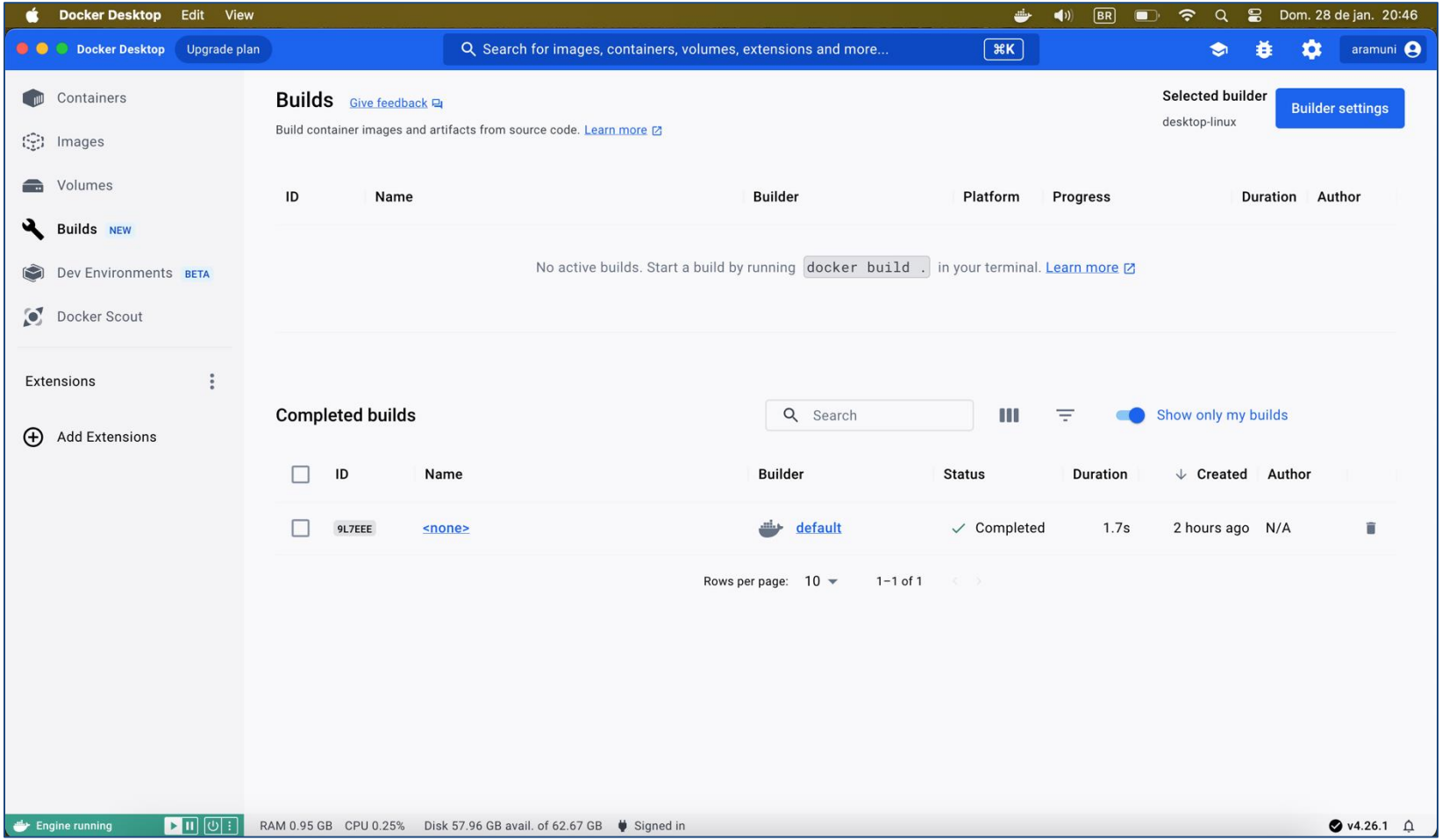




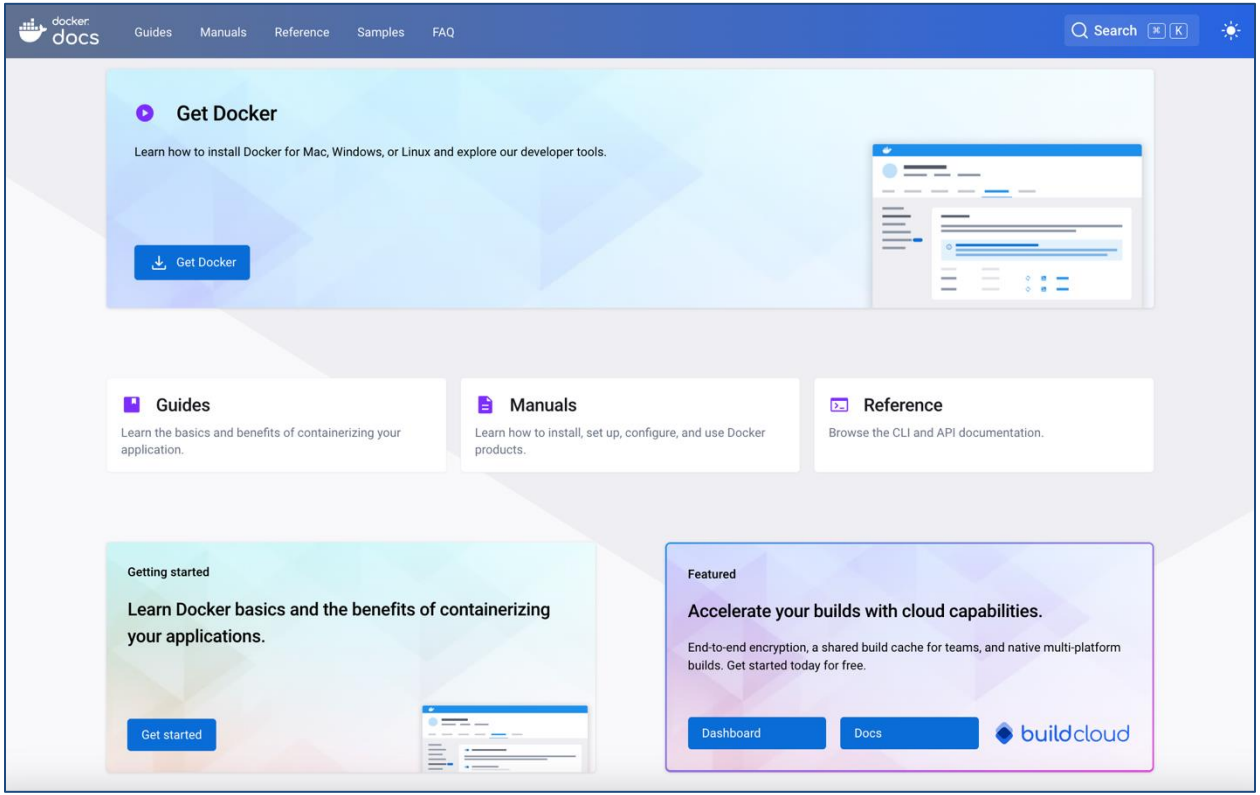
Instalando o **Docker**:

- Instale também a versão do Docker para **Desktop**: https://www.docker.com/



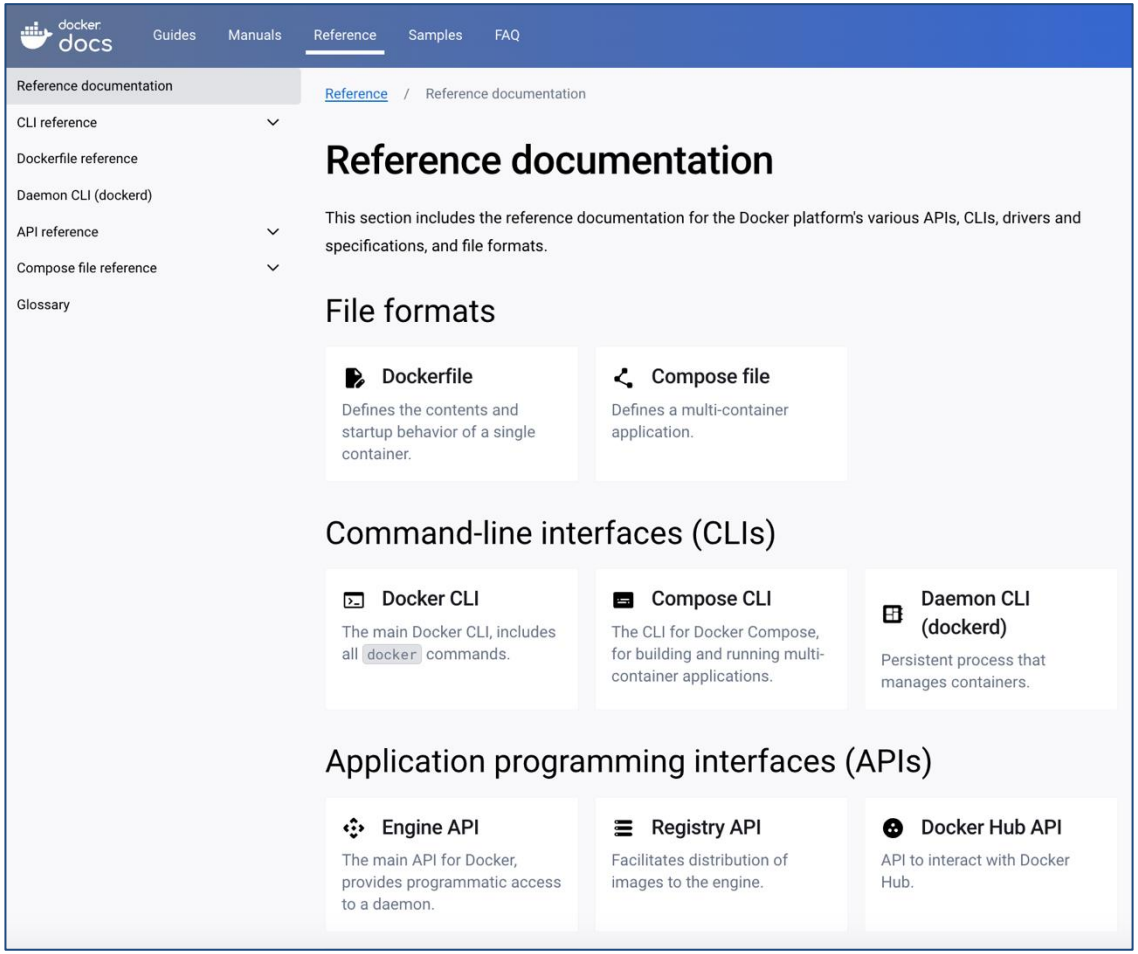

Docker Desktop Edit View
Dom. 28 de jan. 20:46
Docker Desktop
Upgrade plan
Search for images, containers, volumes, extensions and more...
⌘K
aramuni
Containers
Images
Volumes
Builds NEW
Dev Environments BETA
Docker Scout
Extensions
Add Extensions
Builds
Give feedback
Build container images and artifacts from source code. Learn more
Selected builder
desktop-linux
Builder settings
ID
Name
Builder
Platform
Progress
Duration
Author
No active builds. Start a build by running docker build . in your terminal. Learn more
Completed builds
Search
Show only my builds
ID
Name
Builder
Status
Duration
Created
Author
9L7EEE
<none>
default
Completed
1.7s
2 hours ago
N/A
Rows per page: 10
1–1 of 1
Engine running
RAM 0.95 GB CPU 0.25% Disk 57.96 GB avail. of 62.67 GB
Signed in
v4.26.1

- Documentação do Docker:
  - https://docs.docker.com/



Documentação do Docker:

- https://docs.docker.com/
- https://docs.docker.com/reference/
- https://docs.docker.com/engine/reference/builder/
- https://docs.docker.com/build/



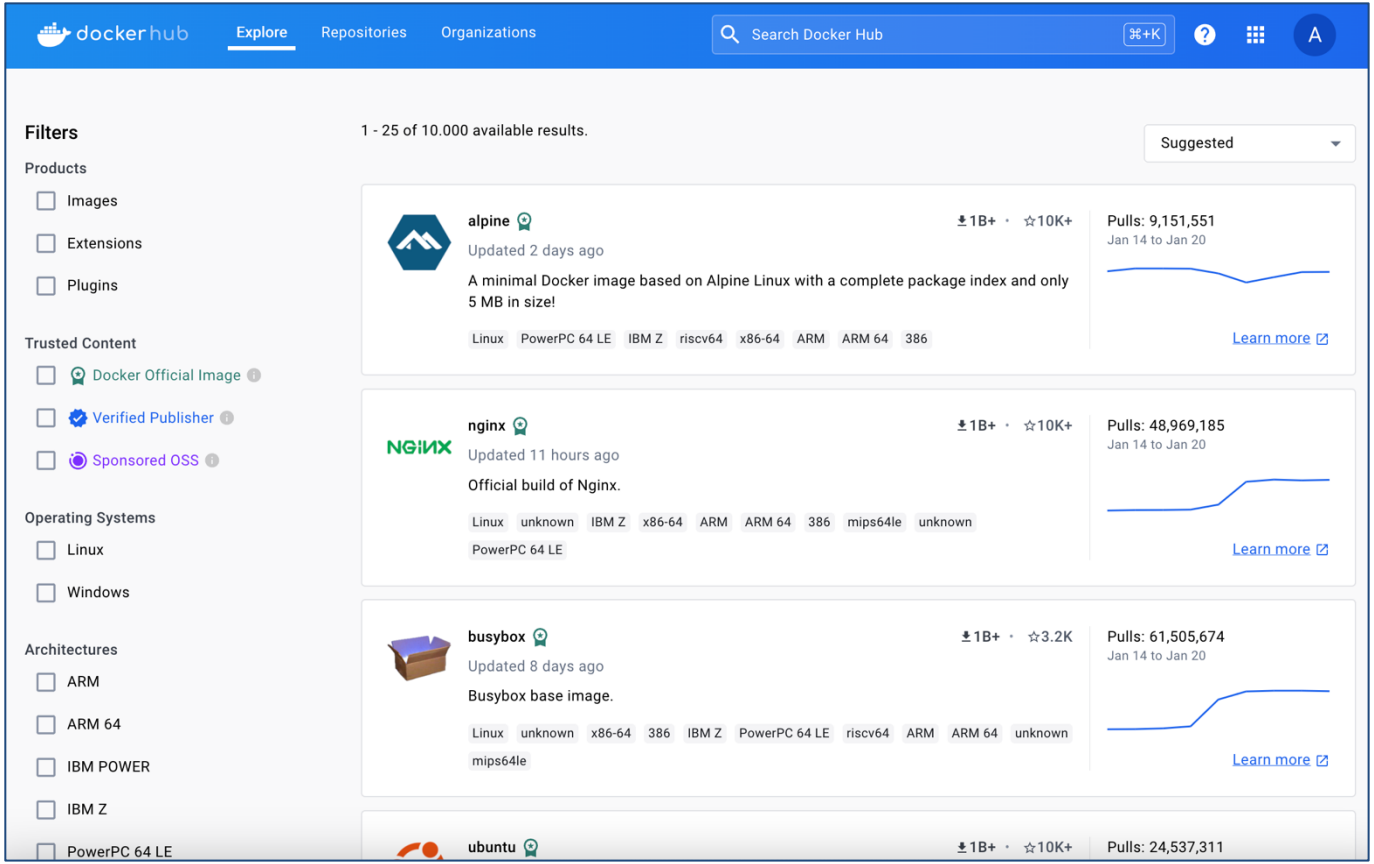
Hospedagem de aplicações Web

- **Docker Hub**:
  - Acesse também Docker Hub: https://hub.docker.com/
    - O **Docker Hub** é um serviço de registro de contêineres que permite aos desenvolvedores compartilhar, armazenar e distribuir imagens de contêineres Docker.
    - Ele é um componente fundamental no ecossistema Docker, uma plataforma de virtualização de contêineres.

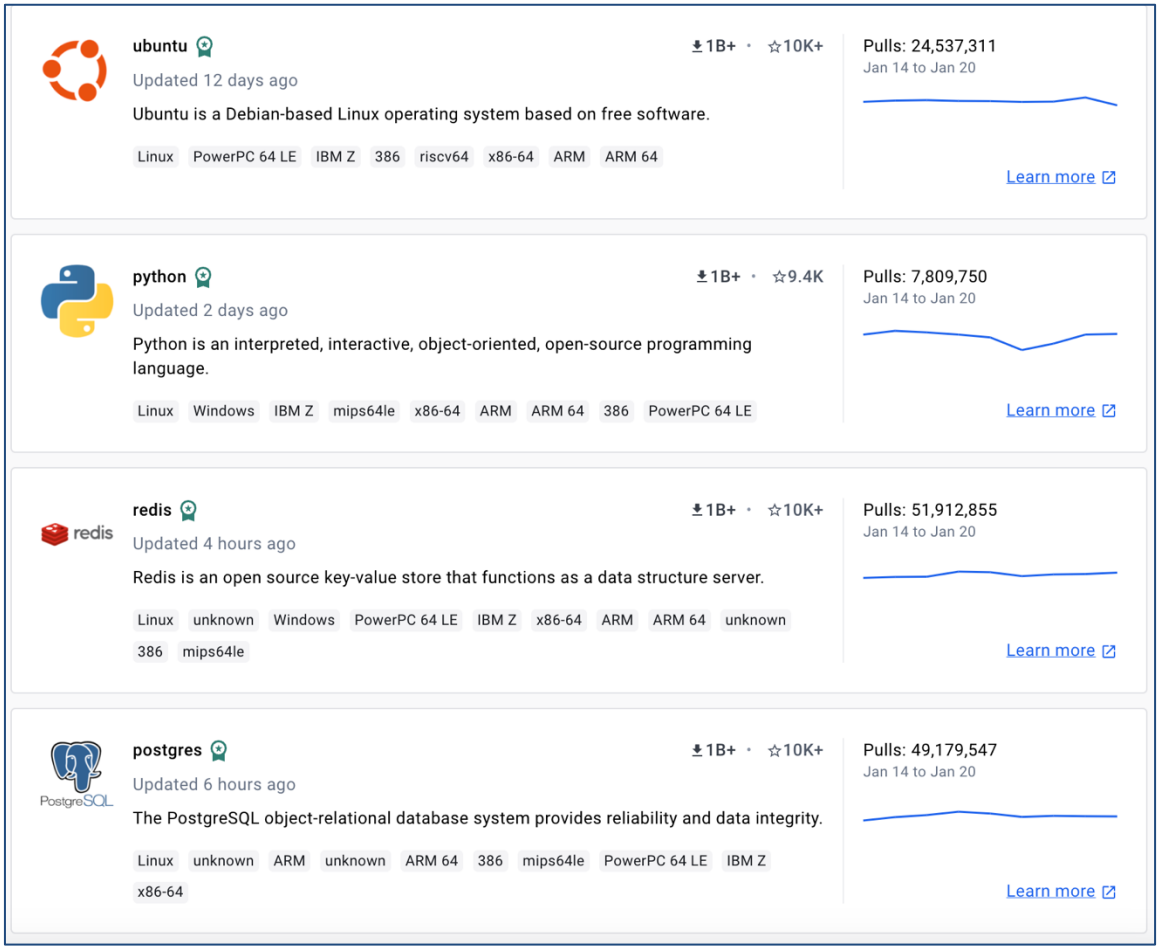
Hospedagem de aplicações Web

- **Docker Hub**:
  - Acesse também Docker Hub: https://hub.docker.com/
    - O **Docker Hub** funciona como um repositório centralizado para imagens de contêineres Docker.
    - Desenvolvedores podem fazer o upload de suas imagens para o Docker Hub e compartilhá-las com a comunidade ou com membros específicos de uma equipe.

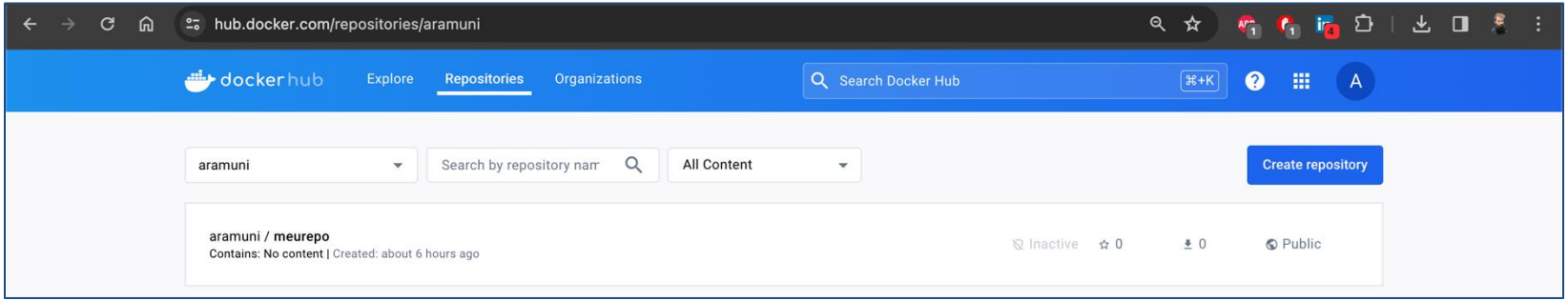
- **Docker Hub:**


docker hub
Explore
Repositories
Organizations
Search Docker Hub
⌘+K
A
Filters
Products
Images
Extensions
Plugins
Trusted Content
Docker Official Image
Verified Publisher
Sponsored OSS
Operating Systems
Linux
Windows
Architectures
ARM
ARM 64
IBM POWER
IBM Z
PowerPC 64 LE
1 - 25 of 10.000 available results.
Suggested
alpine
1B+ · ☆10K+
Updated 2 days ago
A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!
Linux PowerPC 64 LE IBM Z riscv64 x86-64 ARM ARM 64 386
Pulls: 9,151,551
Jan 14 to Jan 20
Learn more
nginx
1B+ · ☆10K+
Updated 11 hours ago
Official build of Nginx.
Linux unknown IBM Z x86-64 ARM ARM 64 386 mips64le unknown PowerPC 64 LE
Pulls: 48,969,185
Jan 14 to Jan 20
Learn more
busybox
1B+ · ☆3.2K
Updated 8 days ago
Busybox base image.
Linux unknown x86-64 386 IBM Z PowerPC 64 LE riscv64 ARM ARM 64 unknown mips64le
Pulls: 61,505,674
Jan 14 to Jan 20
Learn more
ubuntu
1B+ · ☆10K+
Pulls: 24,537,311

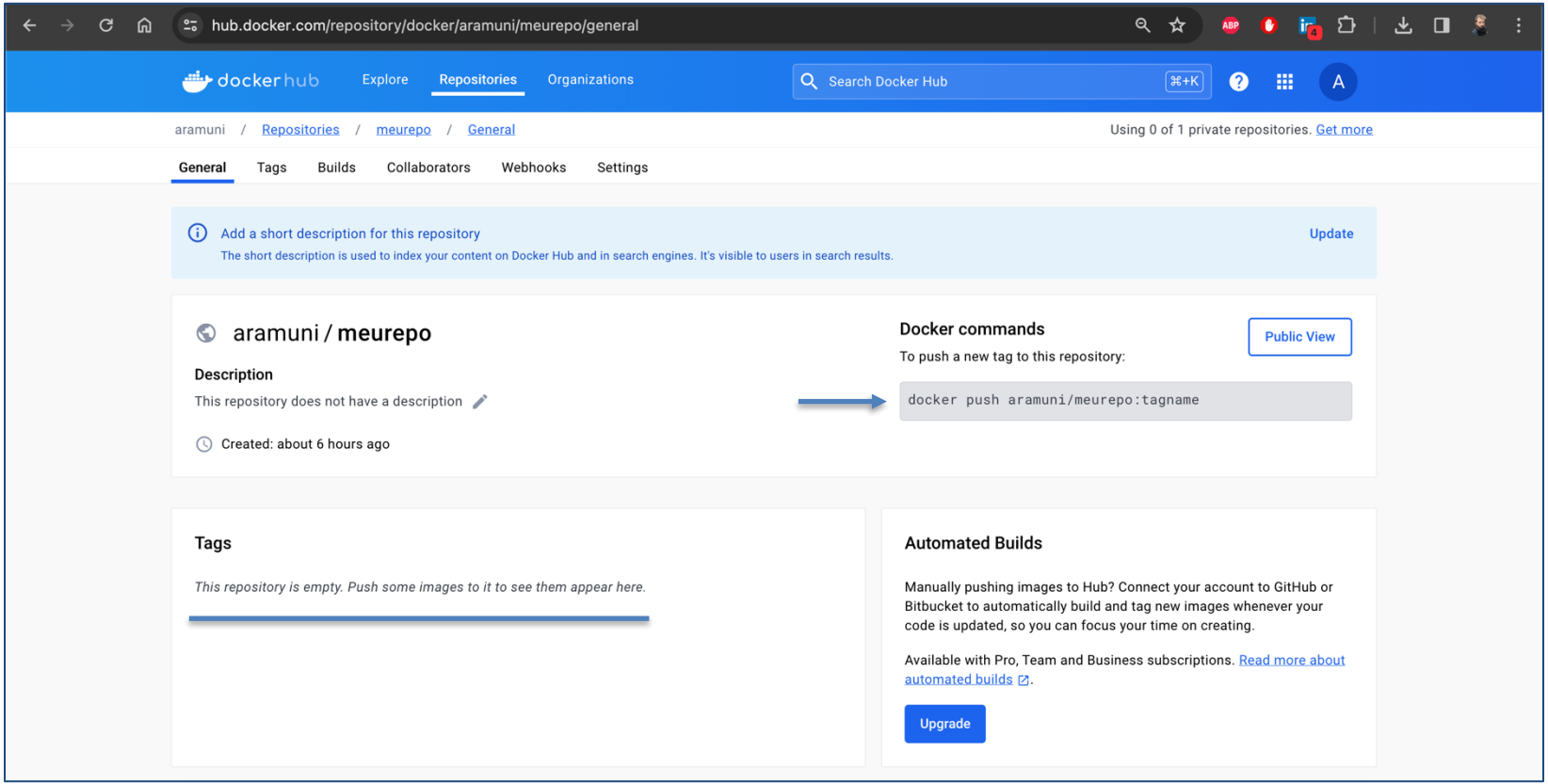
- **Docker Hub:**


ubuntu
1B+ · ☆10K+
Updated 12 days ago
Ubuntu is a Debian-based Linux operating system based on free software.
Linux PowerPC 64 LE IBM Z 386 riscv64 x86-64 ARM ARM 64
Pulls: 24,537,311
Jan 14 to Jan 20
Learn more
python
1B+ · ☆9.4K
Updated 2 days ago
Python is an interpreted, interactive, object-oriented, open-source programming language.
Linux Windows IBM Z mips64le x86-64 ARM ARM 64 386 PowerPC 64 LE
Pulls: 7,809,750
Jan 14 to Jan 20
Learn more
redis
1B+ · ☆10K+
Updated 4 hours ago
Redis is an open source key-value store that functions as a data structure server.
Linux unknown Windows PowerPC 64 LE IBM Z x86-64 ARM ARM 64 unknown 386 mips64le
Pulls: 51,912,855
Jan 14 to Jan 20
Learn more
postgres
1B+ · ☆10K+
Updated 6 hours ago
The PostgreSQL object-relational database system provides reliability and data integrity.
Linux unknown ARM unknown ARM 64 386 mips64le PowerPC 64 LE IBM Z x86-64
Pulls: 49,179,547
Jan 14 to Jan 20
Learn more

- **Docker Hub**:
  - O Fly.io permite que você implante suas aplicações sem a necessidade de armazenar suas imagens Docker no Docker Hub ou em outro registro público. O Fly.io oferece a opção de enviar sua imagem diretamente para seus servidores durante o processo de implantação.
  - Apesar disso, crie seu primeiro repositório no **Docker Hub** para que você possa praticar esse processo e posteriormente publicar suas imagens aqui.



- **Docker Hub:**


hub.docker.com/repository/docker/aramuni/meurepo/general
docker hub
Explore
Repositories
Organizations
Search Docker Hub
⌘+K
A
aramuni / Repositories / meurepo / General
Using 0 of 1 private repositories. Get more
General
Tags
Builds
Collaborators
Webhooks
Settings
Add a short description for this repository
The short description is used to index your content on Docker Hub and in search engines. It's visible to users in search results.
Update
aramuni / meurepo
Description
This repository does not have a description
Created: about 6 hours ago
Docker commands
To push a new tag to this repository:
Public View
docker push aramuni/meurepo:tagname
Tags
This repository is empty. Push some images to it to see them appear here.
Automated Builds
Manually pushing images to Hub? Connect your account to GitHub or Bitbucket to automatically build and tag new images whenever your code is updated, so you can focus your time on creating.
Available with Pro, Team and Business subscriptions. Read more about automated builds.
Upgrade

Hospedagem de aplicações Web

- **Docker e Fly.io**:
  - O Fly.io simplifica o processo de desenvolvimento ao incorporar a construção da imagem Docker diretamente em seu **fluxo** de implantação.
  - Durante o processo de **deploy**, o Fly.io automaticamente executa o build da imagem Docker, eliminando a necessidade de os desenvolvedores realizarem esse passo manualmente.
  - Essa abordagem reduz a complexidade do ciclo de vida da aplicação, proporcionando uma experiência mais fluida para os desenvolvedores.

Hospedagem de aplicações Web

- **Docker e Fly.io**:
  - Ao permitir que o Fly.io gerencie a construção da imagem durante o deploy, os desenvolvedores podem se concentrar mais no desenvolvimento de código e menos na configuração de ambientes, simplificando consideravelmente o processo de implantação de aplicações.

  - Para fins de aprendizado, este seria um comando **docker build** para a construção de uma imagem Docker:
  - docker build --platform=linux/amd64 --build-arg JAR_FILE=target/*.jar -t meurepo/fly_io_view_restapi .

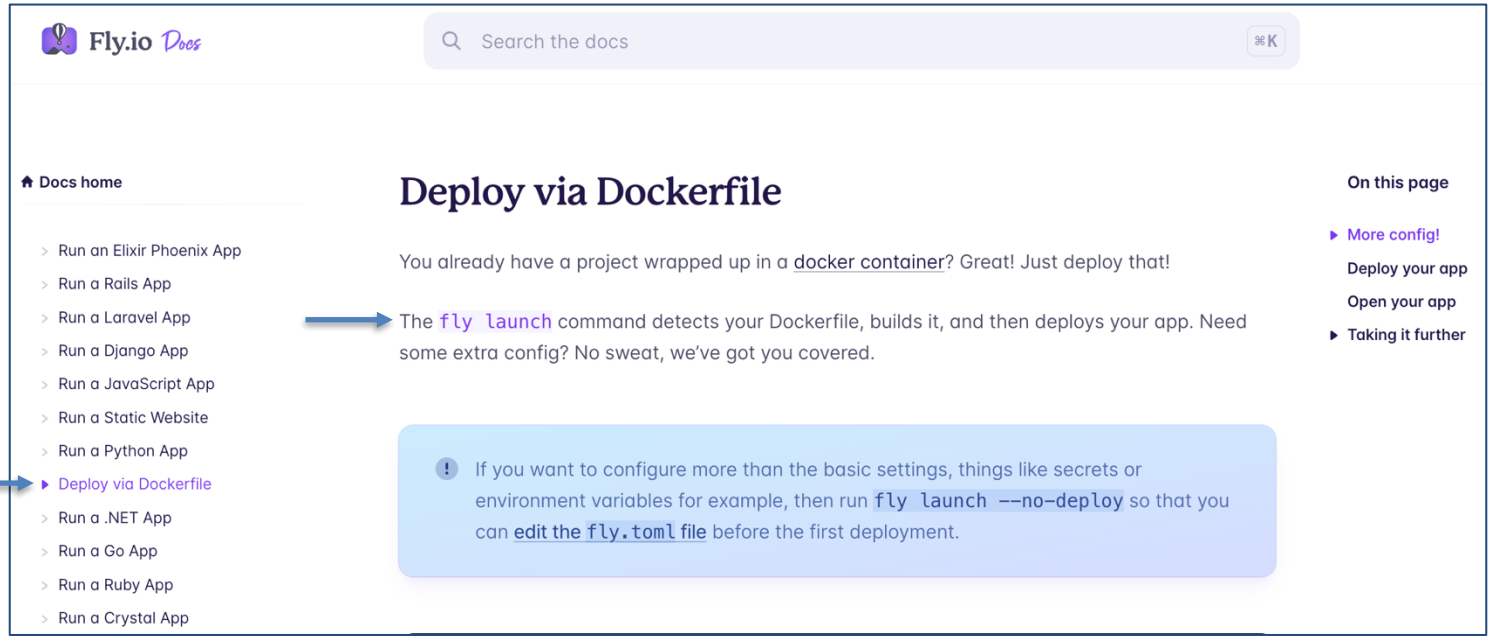
Hospedagem de aplicações Web

- **Docker e Fly.io**:
  - Para fazer o deploy no Fly.io, de uma aplicação dockerizada, basta ter um arquivo chamado **Dockerfile** na pasta do seu projeto.

  - O Dockerfile é um arquivo de configuração utilizado no ecossistema Docker para criar imagens de contêineres. Ele contém um conjunto de instruções que especificam as operações necessárias para configurar um ambiente dentro de um contêiner Docker.

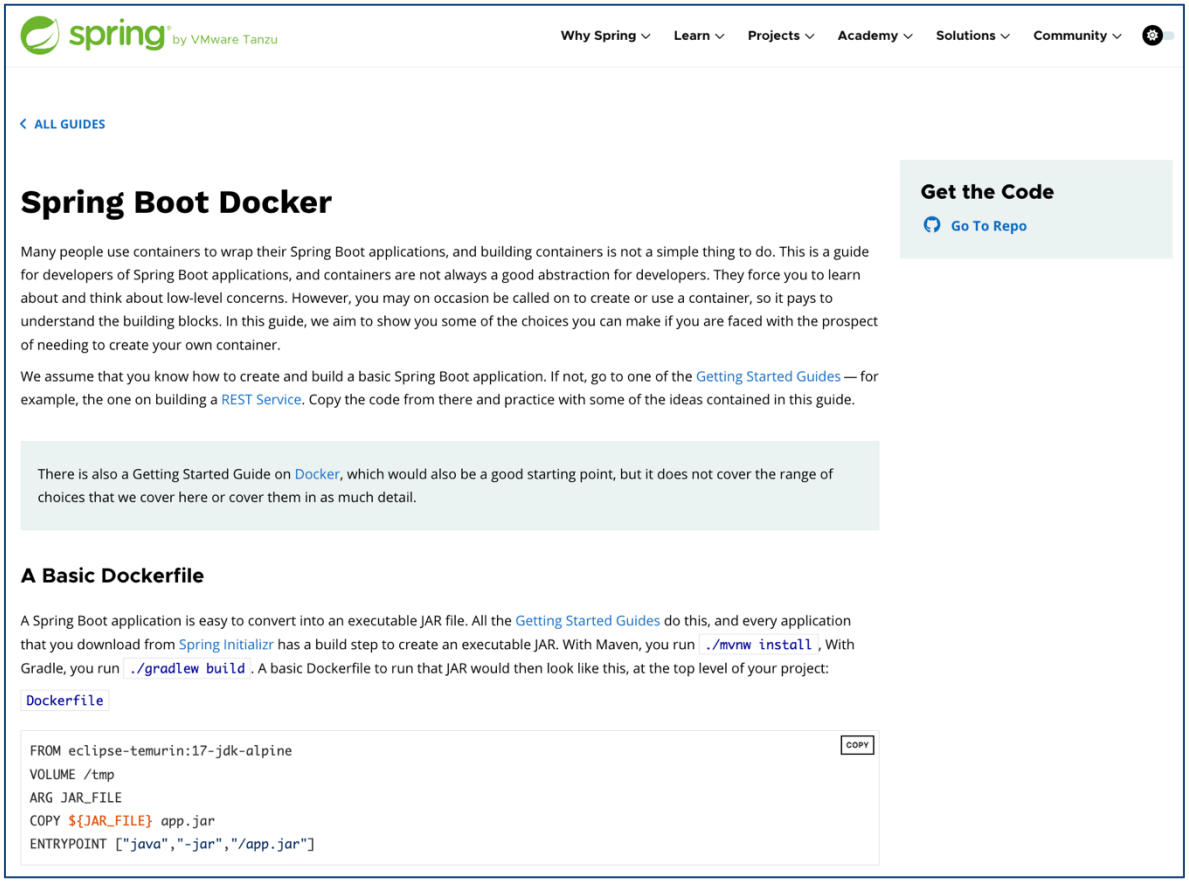
Hospedagem de aplicações Web

- **Docker e Fly.io**:
  - https://fly.io/docs/languages-and-frameworks/dockerfile/



Leitura sugerida:

- Spring Boot Docker:
  - https://spring.io/guides/topicals/spring-boot-docker/



spring by VMware Tanzu

Why Spring | Learn | Projects | Academy | Solutions | Community

< ALL GUIDES

Spring Boot Docker

Many people use containers to wrap their Spring Boot applications, and building containers is not a simple thing to do. This is a guide for developers of Spring Boot applications, and containers are not always a good abstraction for developers. They force you to learn about and think about low-level concerns. However, you may on occasion be called on to create or use a container, so it pays to understand the building blocks. In this guide, we aim to show you some of the choices you can make if you are faced with the prospect of needing to create your own container.

We assume that you know how to create and build a basic Spring Boot application. If not, go to one of the Getting Started Guides — for example, the one on building a REST Service. Copy the code from there and practice with some of the ideas contained in this guide.

There is also a Getting Started Guide on Docker, which would also be a good starting point, but it does not cover the range of choices that we cover here or cover them in as much detail.

A Basic Dockerfile

A Spring Boot application is easy to convert into an executable JAR file. All the Getting Started Guides do this, and every application that you download from Spring Initializr has a build step to create an executable JAR. With Maven, you run `./mvnw install`. With Gradle, you run `./gradlew build`. A basic Dockerfile to run that JAR would then look like this, at the top level of your project:

`Dockerfile`

```
FROM eclipse-temurin:17-jdk-alpine
VOLUME /tmp
ARG JAR_FILE
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

COPY

Get the Code

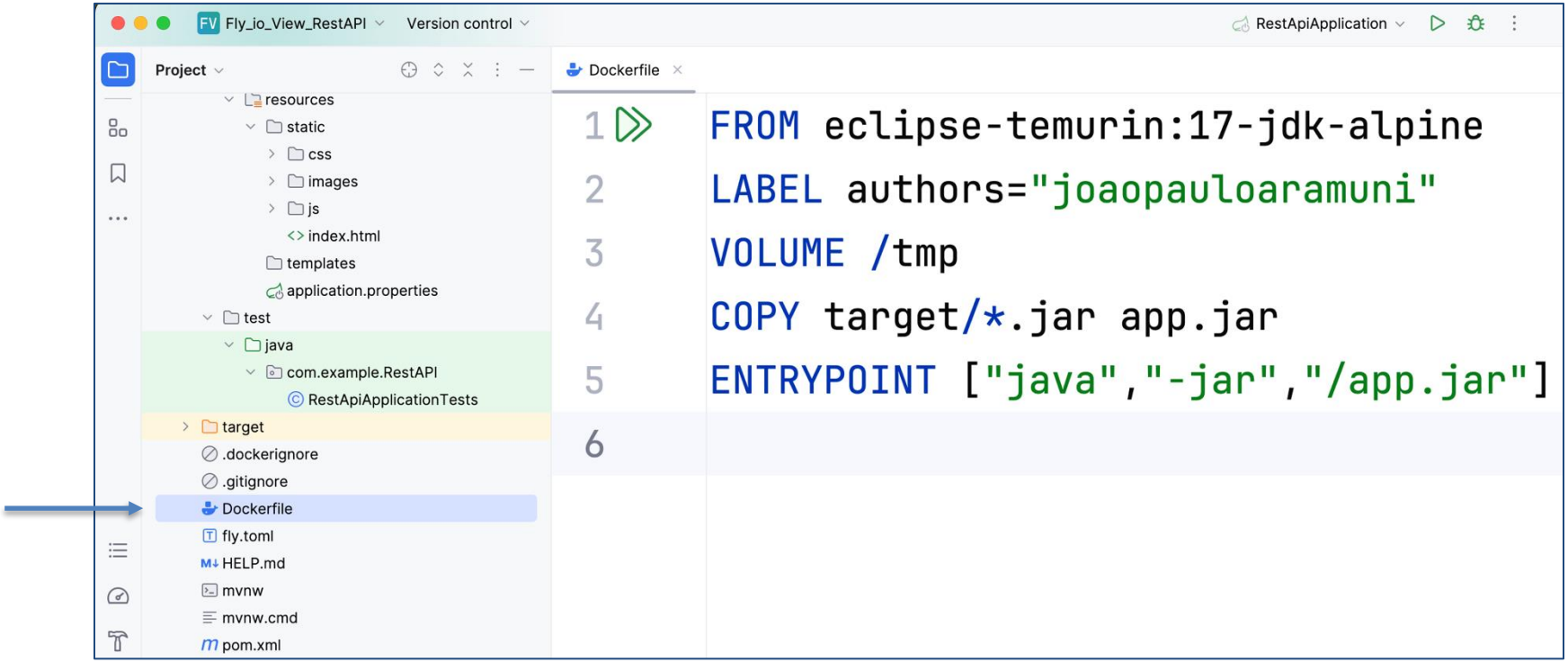
Go To Repo

Hospedagem de aplicações Web

- **Docker e Fly.io**:
  - Arquivo **Dockerfile**:
    - Vejamos como fica o nosso arquivo Dockerfile.

    - Usaremos como base o projeto **View_RestAPI** com apenas um endpoint **/exemplo** e um arquivo resources/static/**index.html** com um <h1>Hello World</h1>.

- Arquivo **Dockerfile**:
  - Vamos analisar o que cada linha significa.



- Arquivo **Dockerfile**:
  - Vamos analisar o que cada linha significa.

  - **FROM eclipse-temurin:17-jdk-alpine**: Especifica a imagem base a ser utilizada como ponto de partida para a construção da imagem Docker. Neste caso, a imagem base é eclipse-temurin com a tag 17-jdk-alpine, que contém o **JDK 17** (Java Development Kit) e é baseada no **Alpine Linux**, uma distribuição Linux leve.

  - **LABEL authors="joaopauloaramuni"**: Adiciona uma etiqueta (label) à imagem Docker para fornecer informações adicionais. Neste caso, a etiqueta "authors" é definida com o valor "joaopauloaramuni".

- Arquivo **Dockerfile**:
  - Vamos analisar o que cada linha significa.

  - **VOLUME /tmp**: Cria um ponto de montagem (volume) no sistema de arquivos da imagem Docker. O diretório /tmp é designado como um volume, permitindo que dados sejam persistidos fora do contêiner e compartilhados com outros contêineres, se necessário.

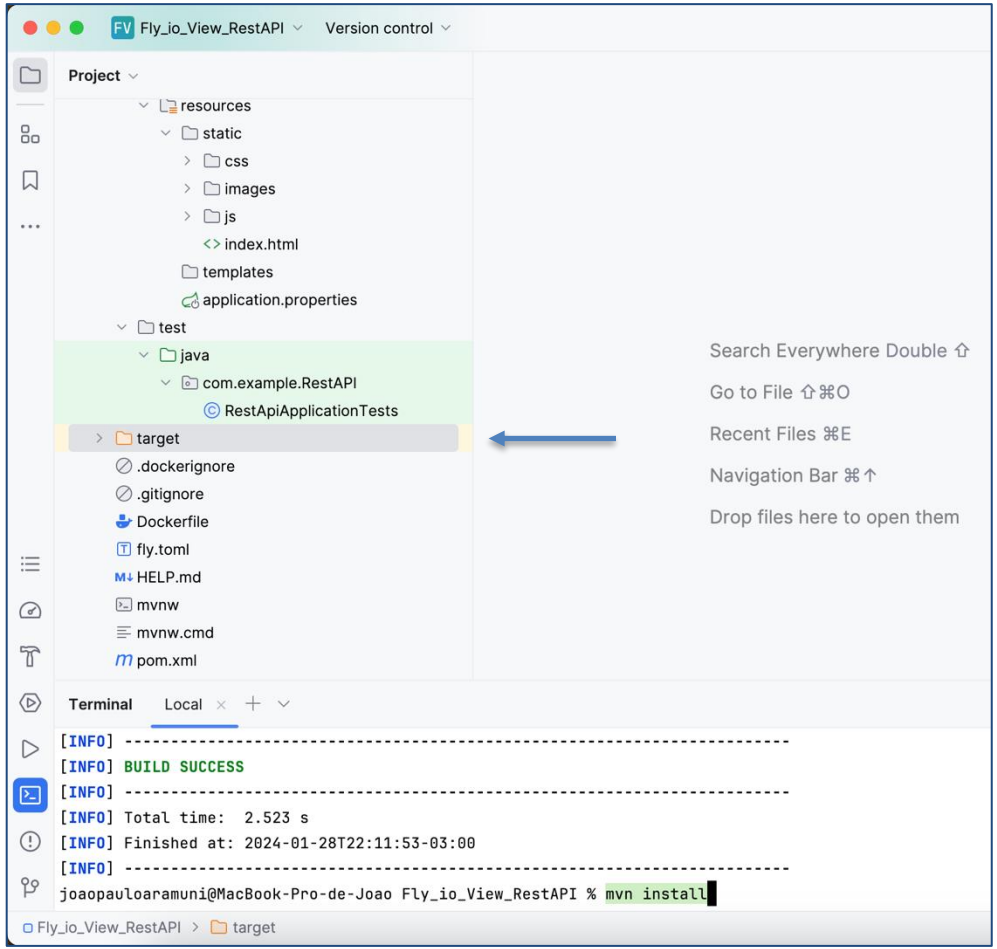
  - **COPY target/*.jar app.jar**: Copia os arquivos JAR gerados durante a compilação do projeto para dentro da imagem Docker. Esses arquivos geralmente contêm o código Java compilado e as dependências necessárias para executar a aplicação.

- Arquivo **Dockerfile**:
  - Vamos analisar o que cada linha significa.

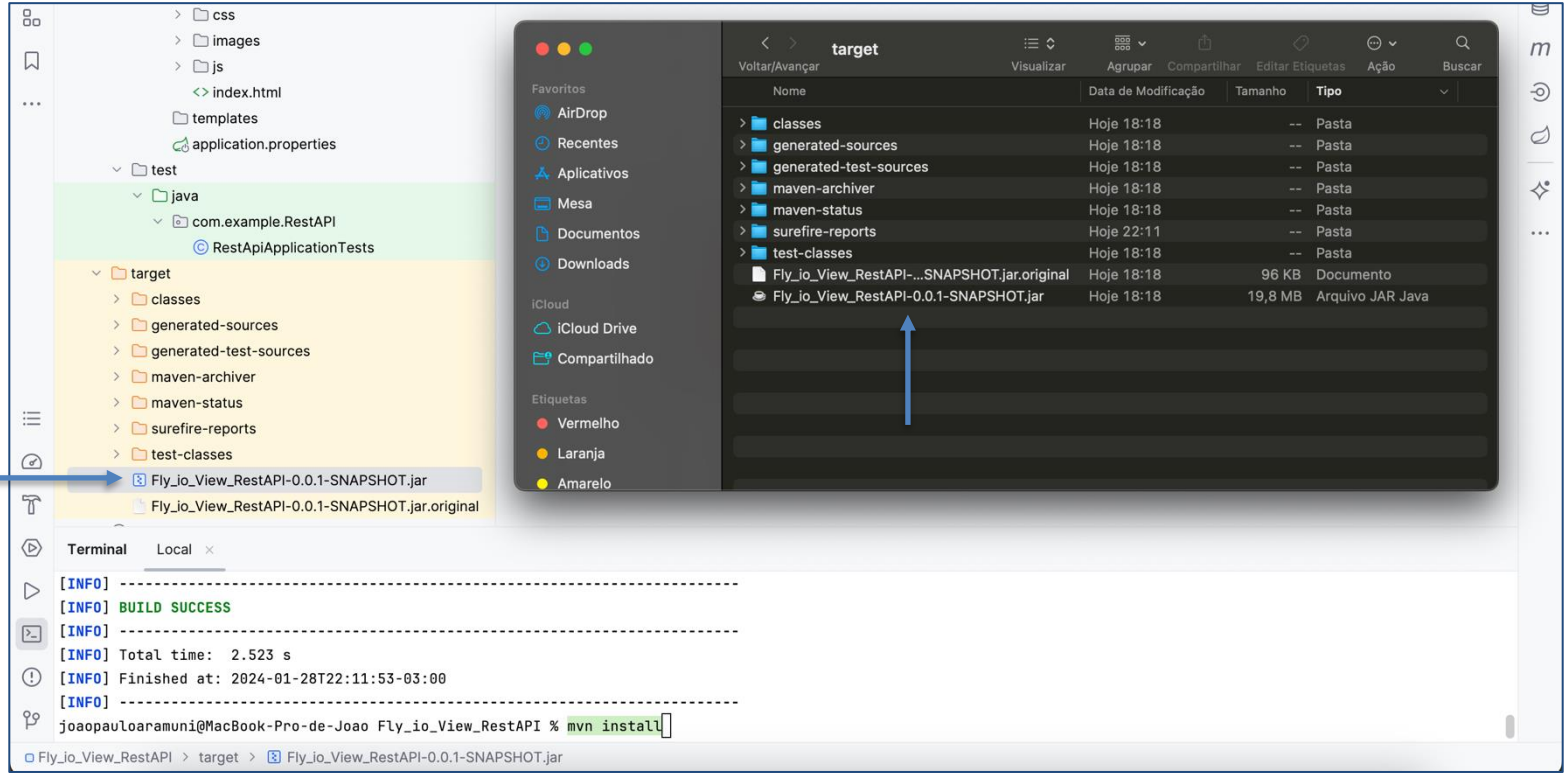
  - **ENTRYPOINT ["java","-jar","/app.jar"]**: Especifica o comando que será executado quando o contêiner for iniciado. Neste caso, o comando Java é utilizado para iniciar a execução do aplicativo JAR (app.jar).

  - O formato de arquivo **JAR** (Java Archive) é um formato de arquivo compactado usado para distribuir e armazenar aplicativos Java. Ele foi projetado para ser uma maneira conveniente de empacotar vários arquivos Java em um único arquivo, facilitando a distribuição e a execução de aplicativos Java.

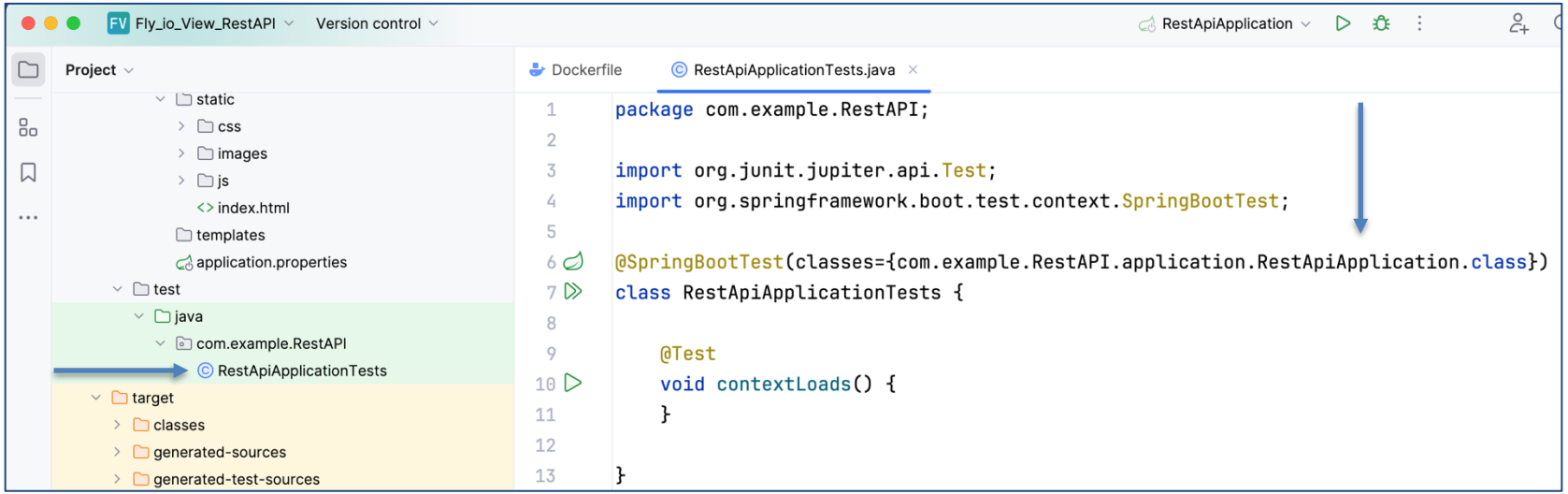
- Arquivo **.jar**:
  - O JAR file é como se fosse um executável **.exe** do Windows.
  - Vejamos como gerá-lo:
    - No terminal do IntelliJ IDEA execute o comando maven:
      - **mvn install**
    - A pasta target será criada.



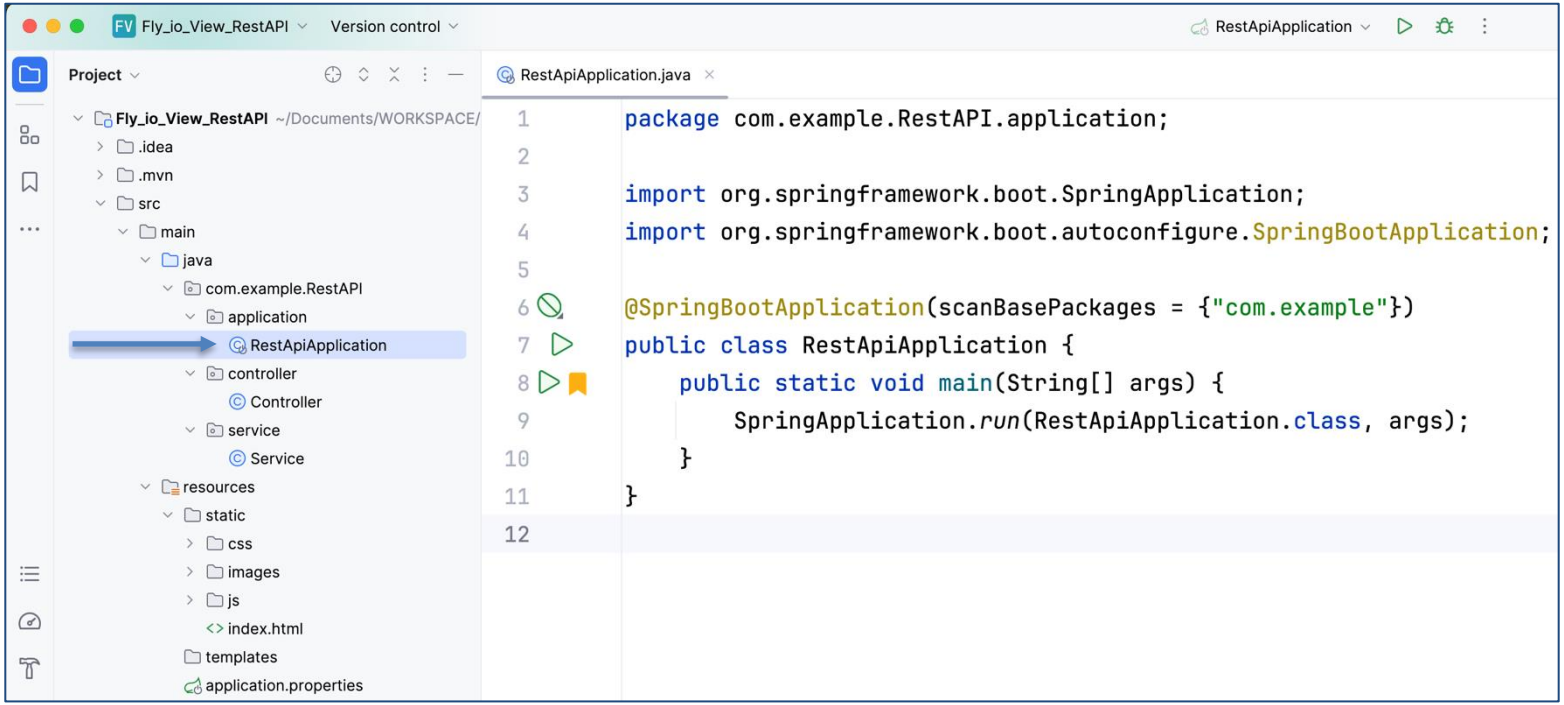
- Arquivo **.jar**:
  - Para deletar a pasta target: **mvn clean**



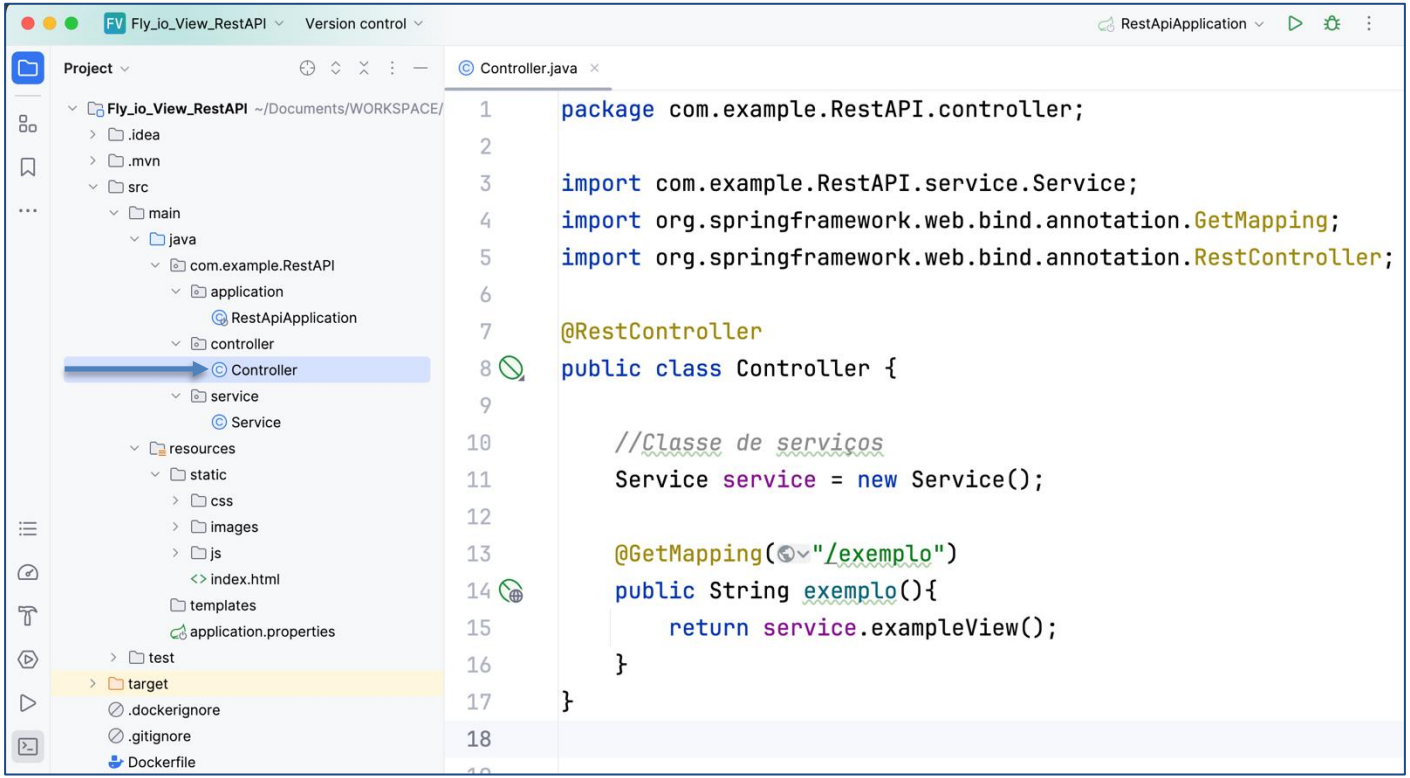
- Arquivo **RestApiApplicationTests**:
  - Um detalhe importante para não bugar o build (mvn install) do projeto:
    - Na classe de testes, especifique a classe de configuração principal da aplicação com a opção classes={com.example.RestAPI.application.RestApiApplication.class}.
    - Como alteramos a estrutura de pastas/pacotes (application, controller e service) isto se faz necessário.



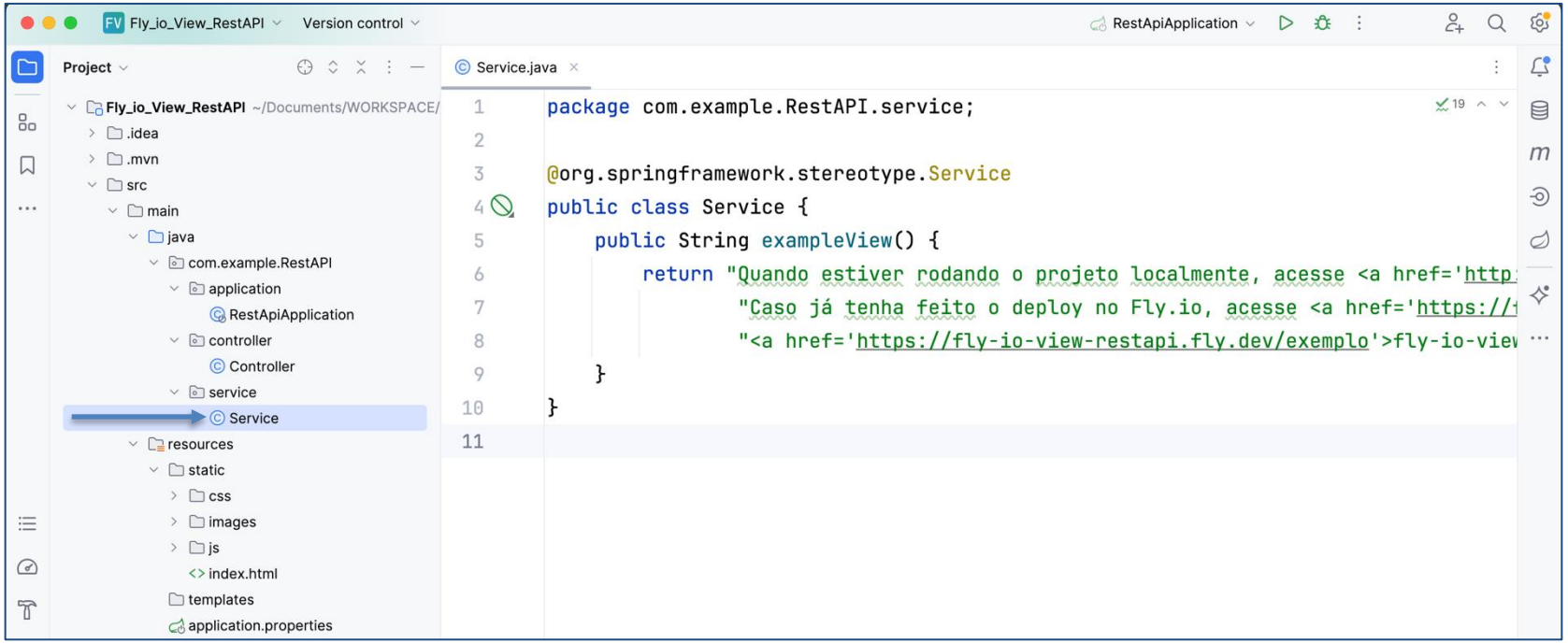
- Arquivo **RestApiApplication**:
  - Antes de partirmos para o lançamento (launch) da aplicação no Fly.io, vejamos como estão os demais arquivos do projeto.



- Arquivo **Controller**:
  - Possui apenas um endpoint **/exemplo**.



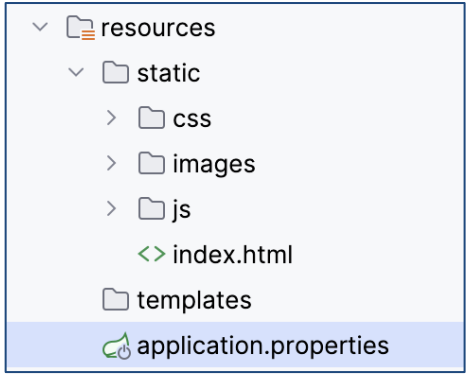
- Arquivo **Service**:
  - Possui apenas um serviço, uma mensagem simples.



- Arquivo **index.html**:
  - Dentro de resources/static/

- Arquivo **pom.xml**:
  - Dependências e plugins.
  - Não faremos nenhuma mudança aqui.

- Arquivo **application.properties**:
  - Ficará vazio, sem nada.




```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

Sumário

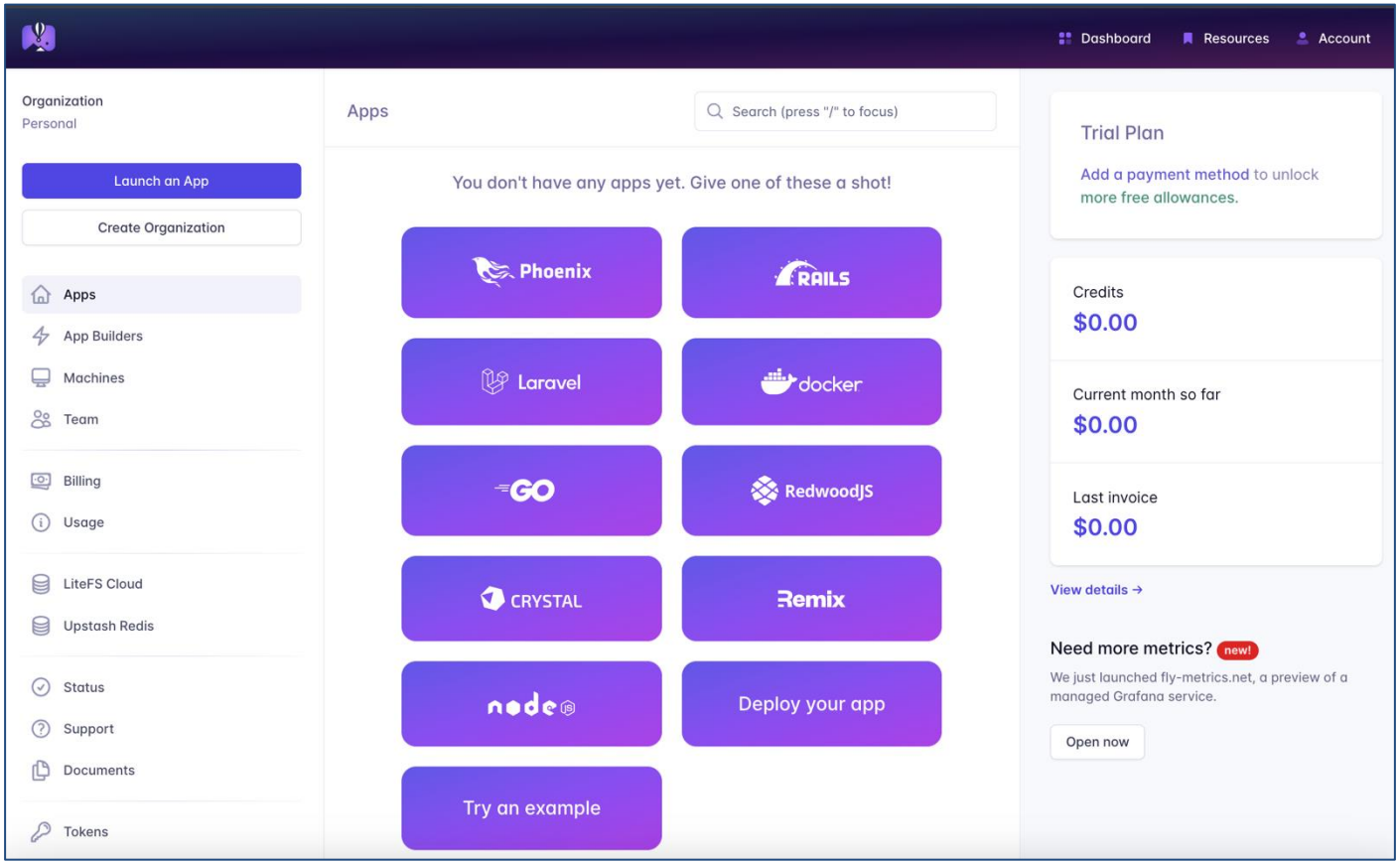
- Hospedagem de aplicações Web
  - Plataformas em nuvem para hospedagem
  - Preparando a aplicação
  - **Publicando a aplicação em uma solução PaaS (Platform as a Service)**

Hospedagem de aplicações Web

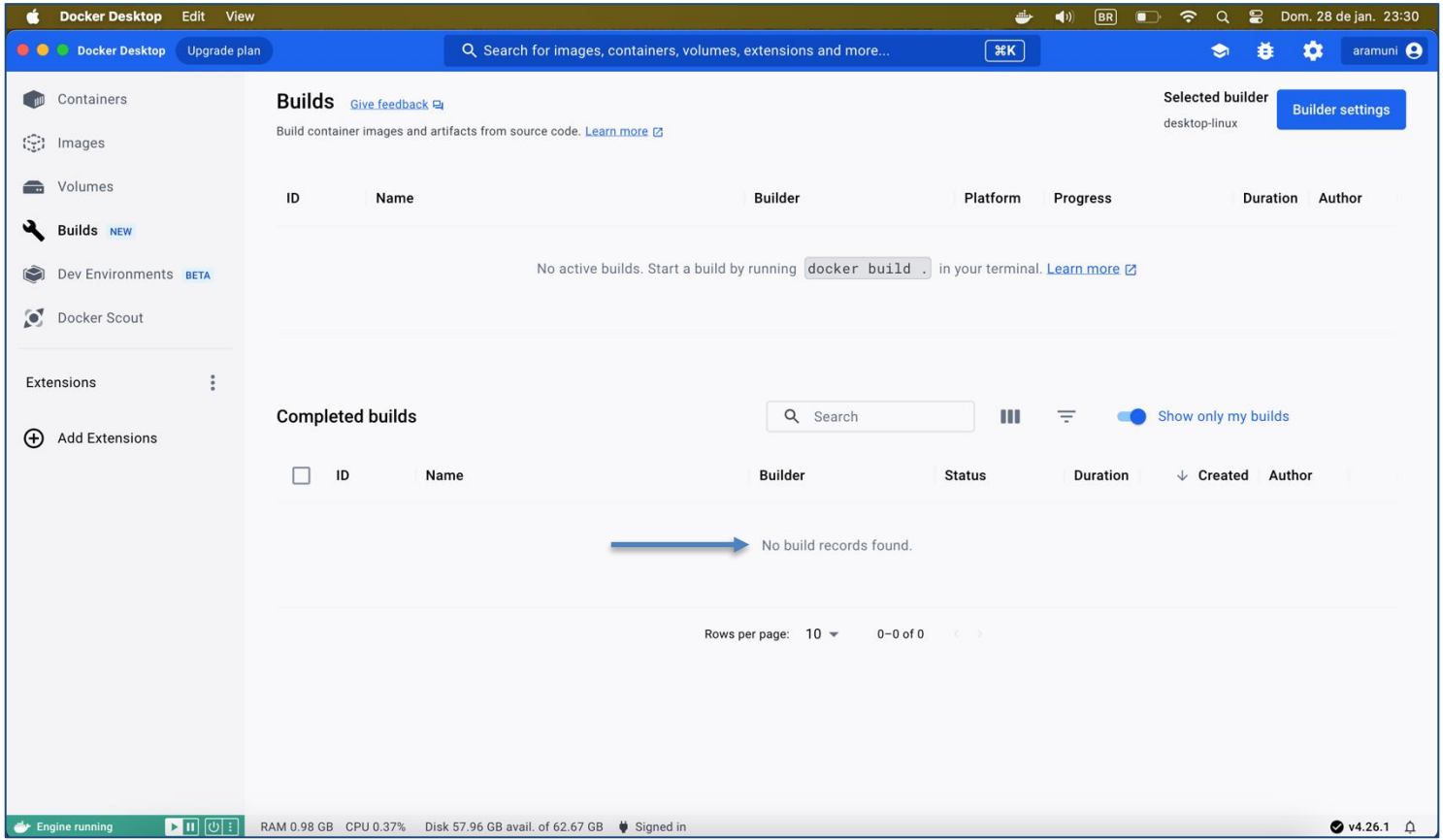
- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Finalmente, vamos deployar a nossa aplicação Spring Boot no Fly.io.
    - Primeiramente, acesse o dashboard do Fly.io onde a aplicação irá aparecer:
      - https://fly.io/dashboard

  - Leituras sugeridas:
    - https://fly.io/docs/apps/launch/
    - https://fly.io/docs/reference/fly-launch/

- https://fly.io/dashboard



- Docker Desktop sem nenhum build:



Hospedagem de aplicações Web

- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Precisaremos basicamente de 3 comandos do Fly.io:
    - fly launch --no-deploy
    - fly deploy --local-only --ha=false
    - fly status

    - https://fly.io/docs/flyctl/launch/
    - https://fly.io/docs/flyctl/deploy/
    - https://fly.io/docs/flyctl/status/

Hospedagem de aplicações Web

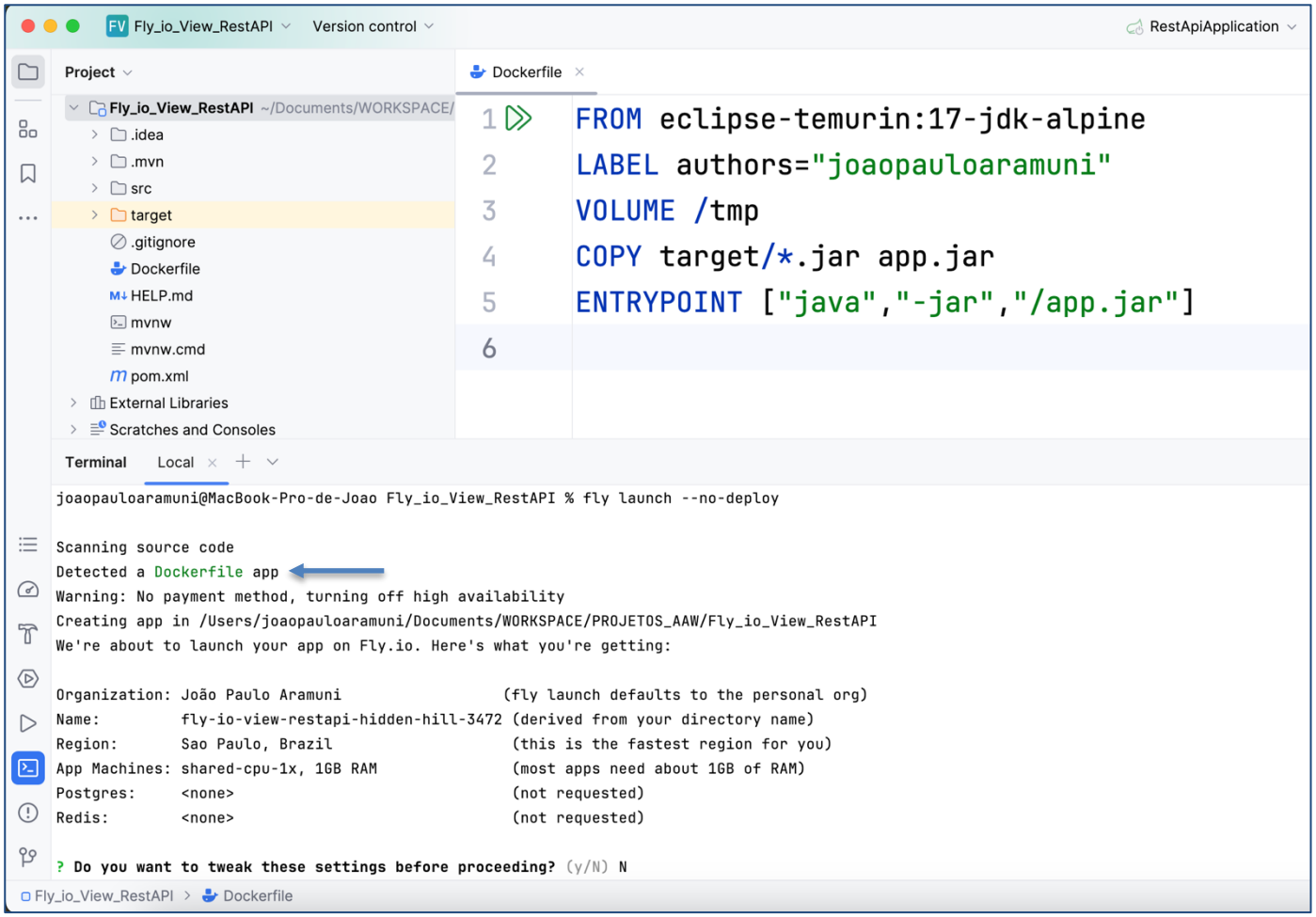
- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Execute o primeiro comando:
    - fly launch --no-deploy
      - O **flyctl** tem um comando específico para lançamento da aplicação, o **launch**.
      - Contudo, ao final dele, o **deploy** da aplicação é executado.
      - Para evitar o deploy no primeiro momento, pois ainda faremos uma pequena configuração, vamos executá-lo com a opção **--no-deploy**.

Fly.io

Para a pergunta:

Você deseja ajustar essas configurações antes de prosseguir?

Responda: N

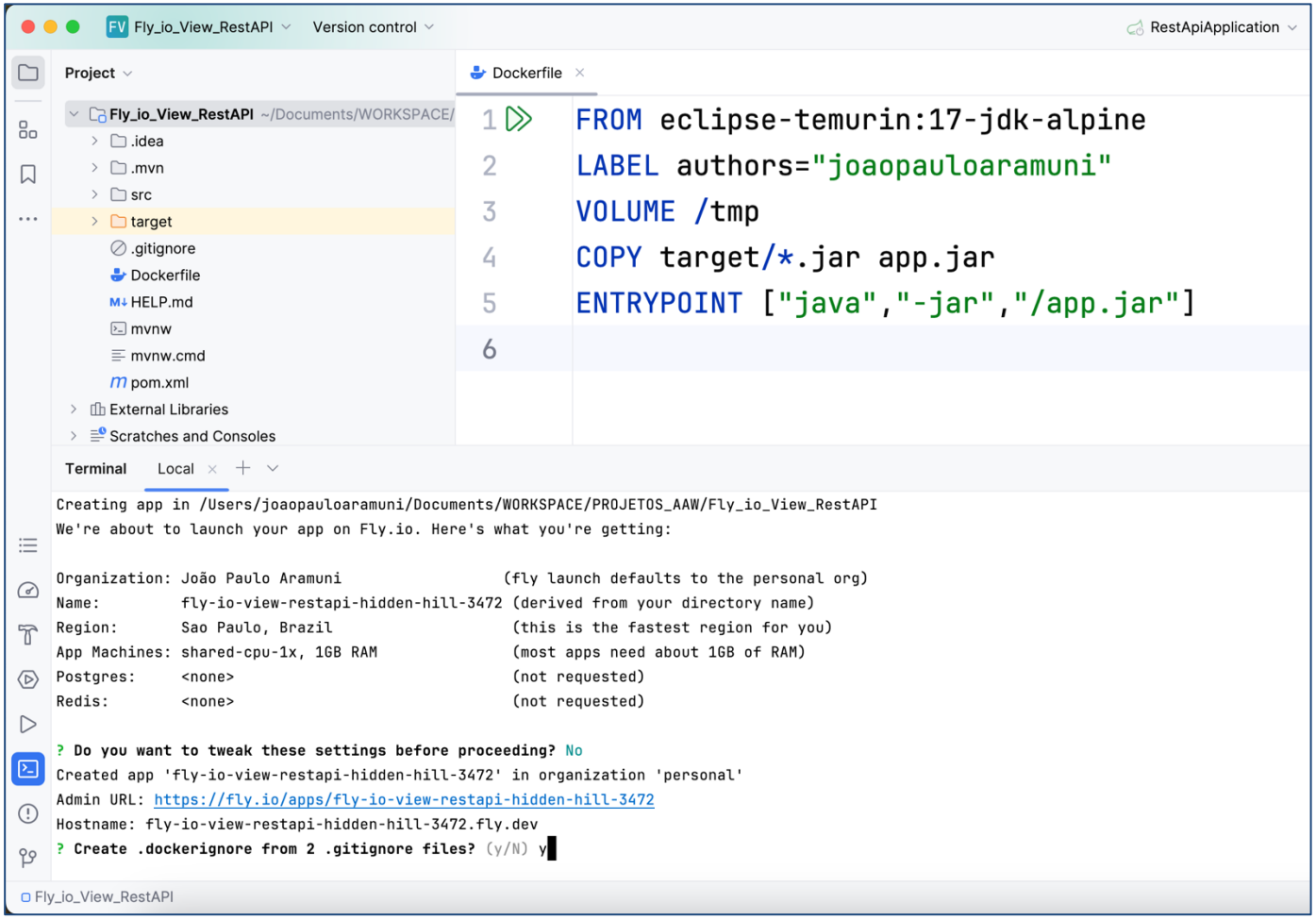


Fly.io

Para a pergunta:

Criar .dockerignore file a partir do .gitignore file?

Responda: y



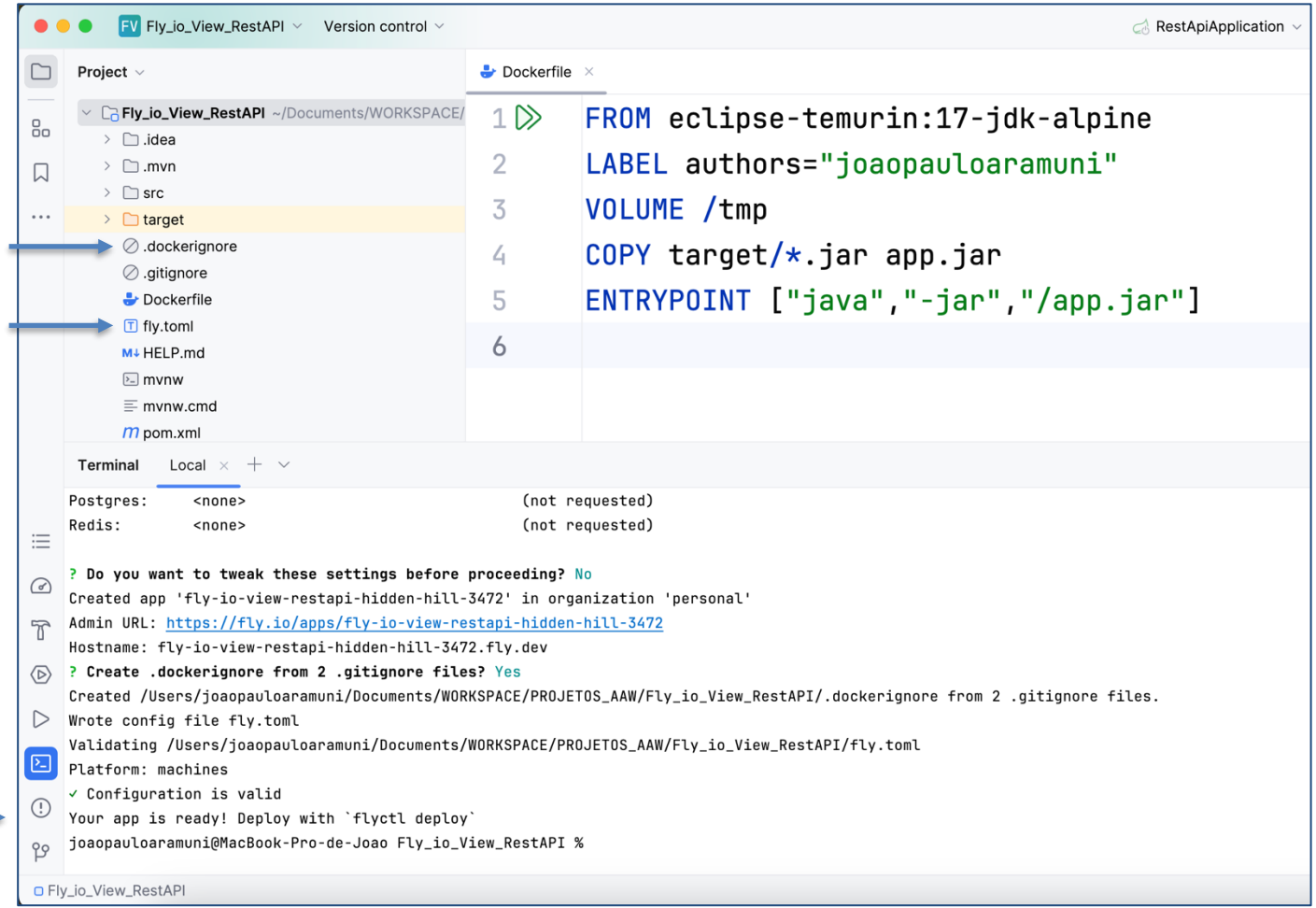
Fly.io

Repare que dois novos arquivos foram criados:

.dockerignore

fly.toml

Seu app está pronto para o deploy =D



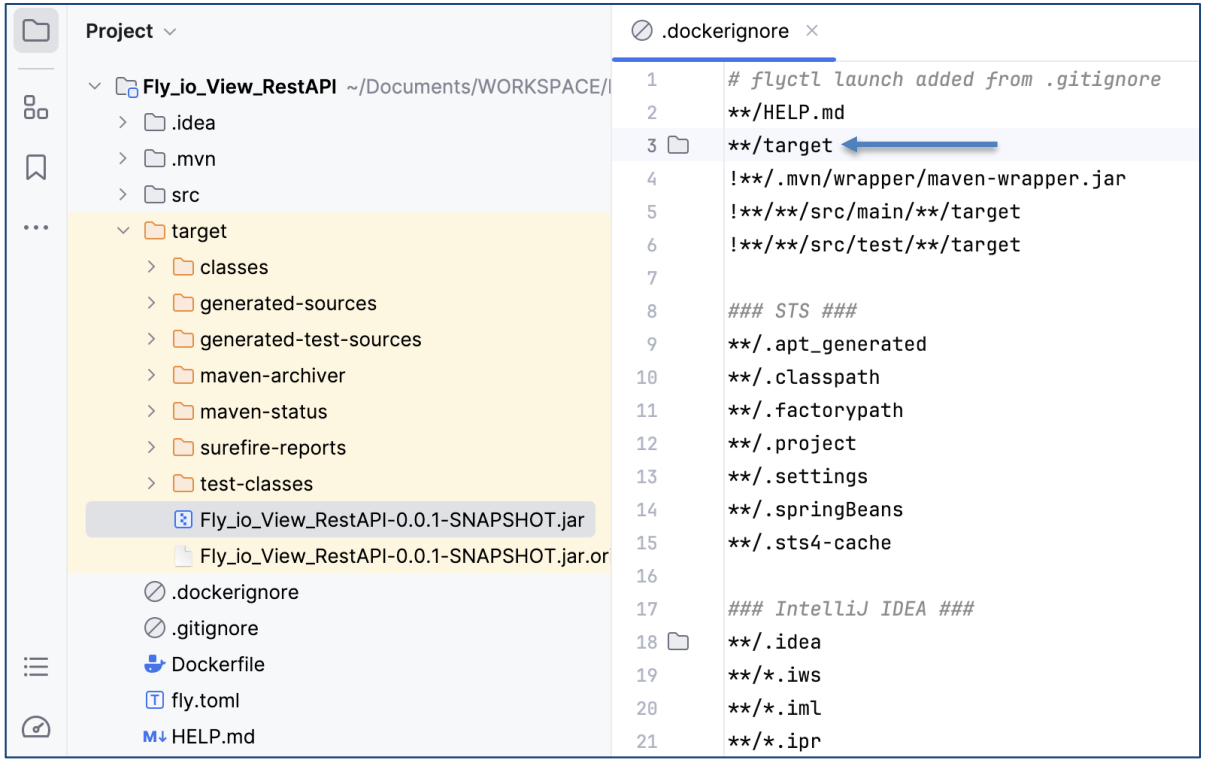
Fly.io

Abra o arquivo:

.dockerignore

Veja que a pasta **target** está configurada para ser ignorada durante o build da imagem.

Comente essa linha.



- O arquivo .dockerignore é usado no contexto do Docker para especificar quais arquivos e diretórios não devem ser incluídos no processo de construção da imagem Docker.

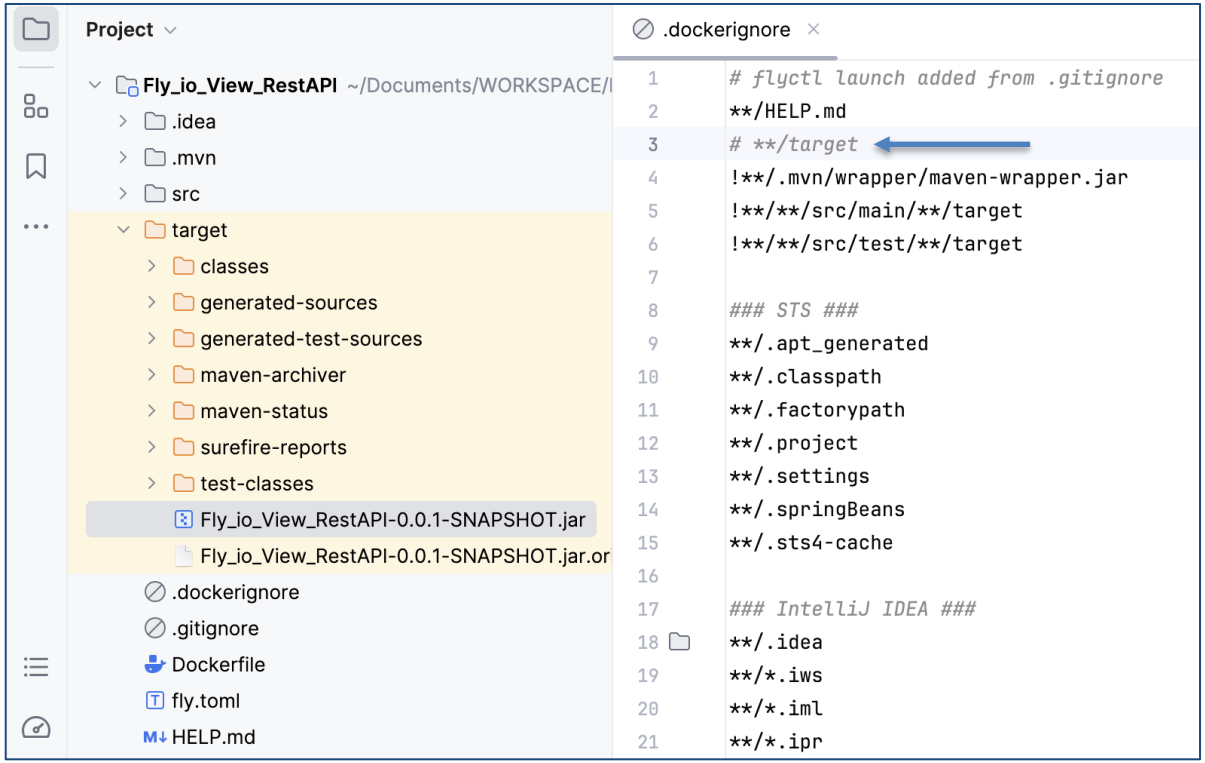
Fly.io

Abra o arquivo:

.dockerignore

Veja que a pasta **target** está configurada para ser ignorada durante o build da imagem.

Comente essa linha.



- Outra opção aqui é **não** ignorar somente o jar file:
  - !target/*.jar

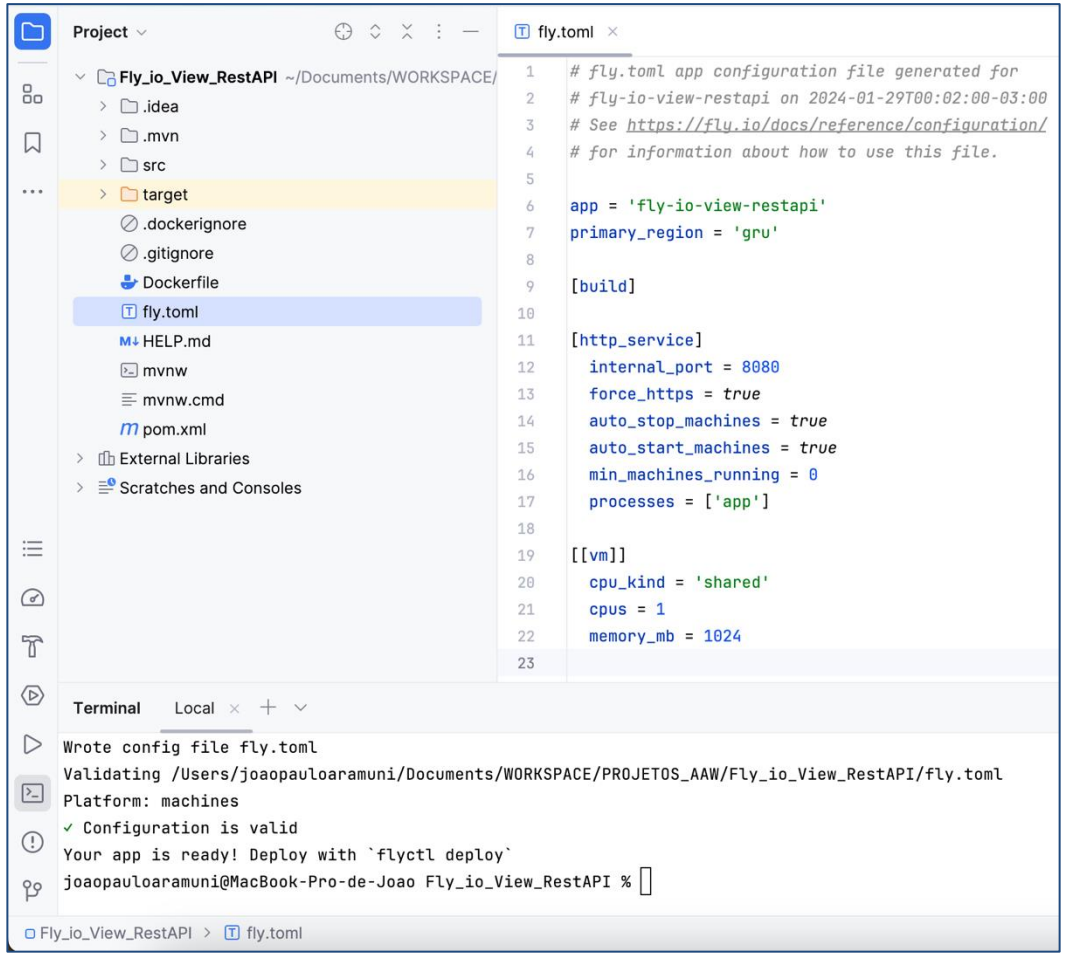
Fly.io

Abra o arquivo:

fly.toml

TOML é um formato de arquivo de configuração simples.

TOML
[Tom's Obvious Minimal Language]



Fly Launch configuration (fly.toml)

- https://fly.io/docs/reference/configuration/

TOML

- https://github.com/toml-lang/toml
- https://npf.io/2014/08/intro-to-toml/
- https://toml.io/pt/

Hospedagem de aplicações Web

- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Execute o segundo comando:
    - fly deploy --local-only --ha=false
      - O Fly.io pode fazer o deploy de duas formas:
        - Copiar os arquivos e fazer o build da imagem Docker na nuvem;
        - ou fazer o build localmente e subir apenas o container para um repositório disponível no Fly.io.

Hospedagem de aplicações Web

- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Execute o segundo comando:
    - fly deploy **--local-only** --ha=false
      - Optaremos por fazer o build localmente para não serem alocadas duas máquinas em nossa aplicação.
      - No plano gratuito existe uma limitação de máquinas disponíveis por aplicação. Quando usamos mais de uma máquina, temos que ter um plano pago, por esse motivo, faremos o build localmente.
      - Para executar o build localmente usamos a flag **--local-only**.

Hospedagem de aplicações Web

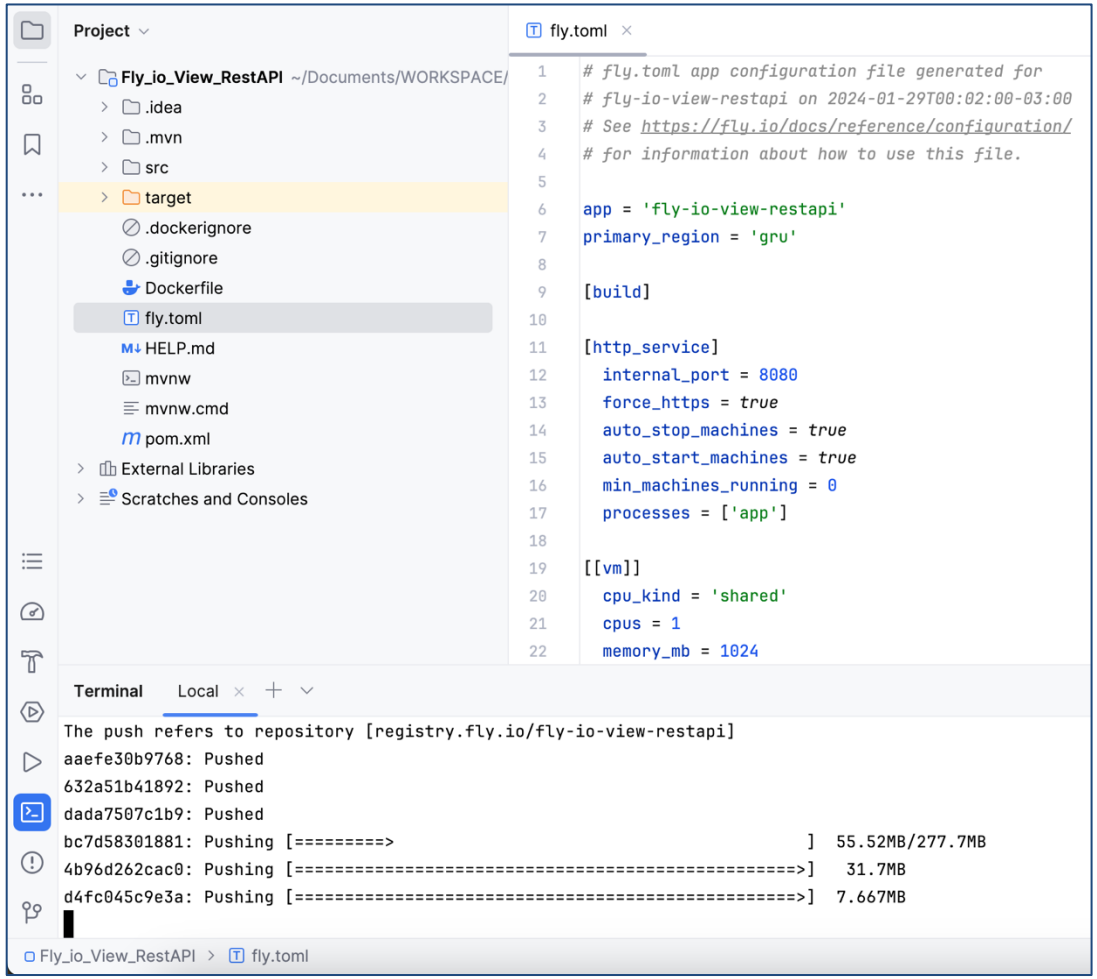
- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Execute o segundo comando:
    - fly deploy --local-only **--ha=false**
      - O Fly sobe duas instâncias por padrão da nossa aplicação para melhorar a disponibilidade do app.
      - Como vamos nos basear no uso gratuito, adicionaremos a flag **--ha=false** ao deploy para desativamos a alta disponibilidade (High Availability).

Fly.io

O deploy pode levar alguns minutos.

Aguarde.

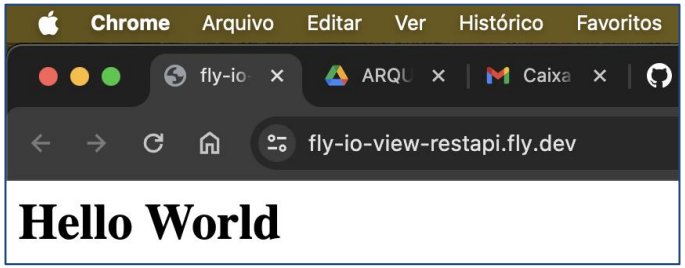
Essa é a hora da verdade =D

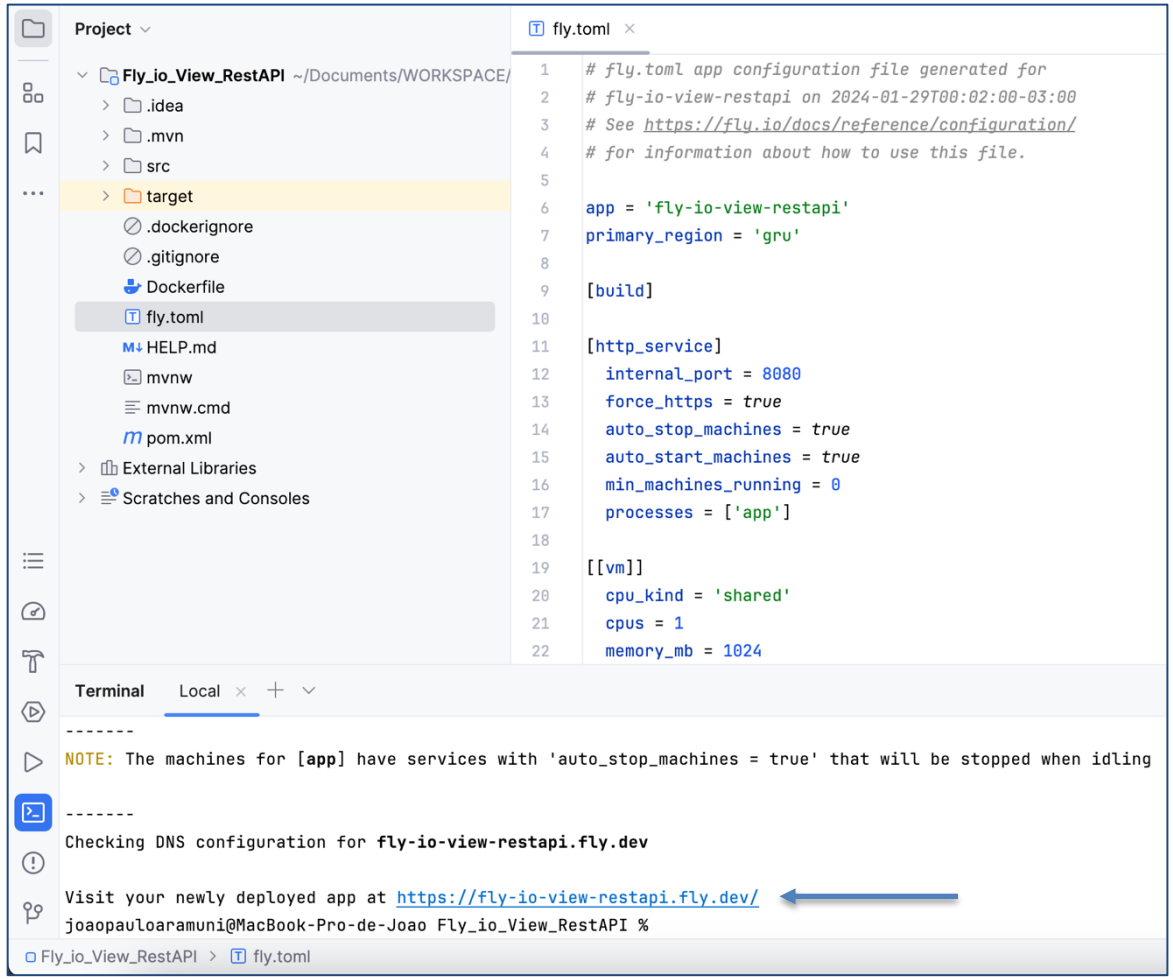


Fly.io

O deploy foi realizado com **sucesso**.

Visite: https://fly-io-view-restapi.fly.dev/

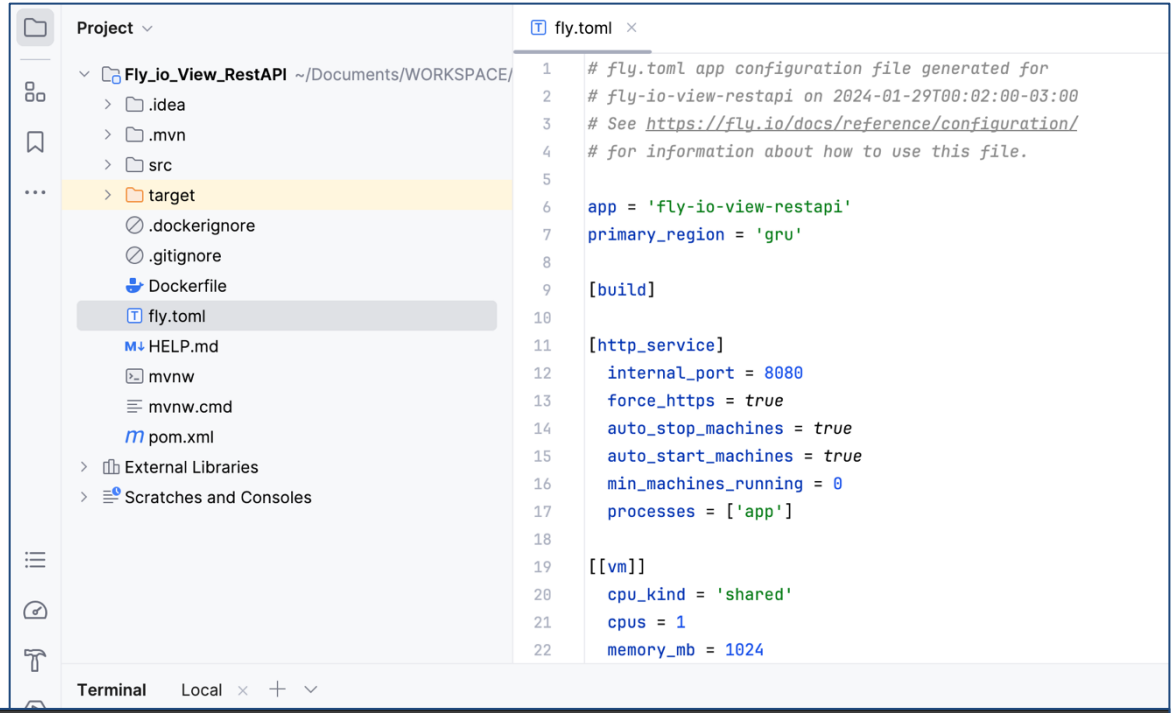




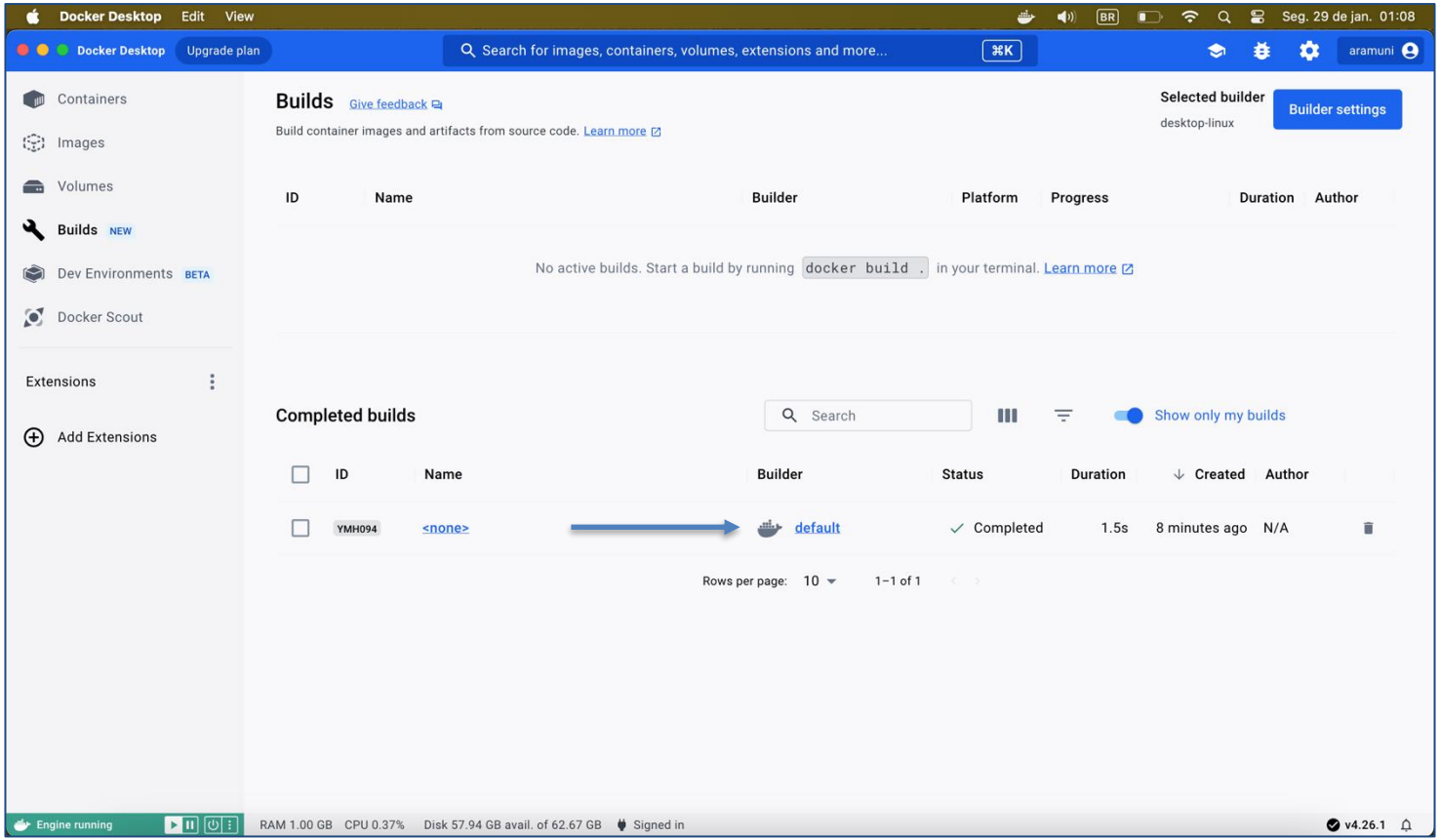
Fly.io

O deploy foi realizado com **sucesso**.

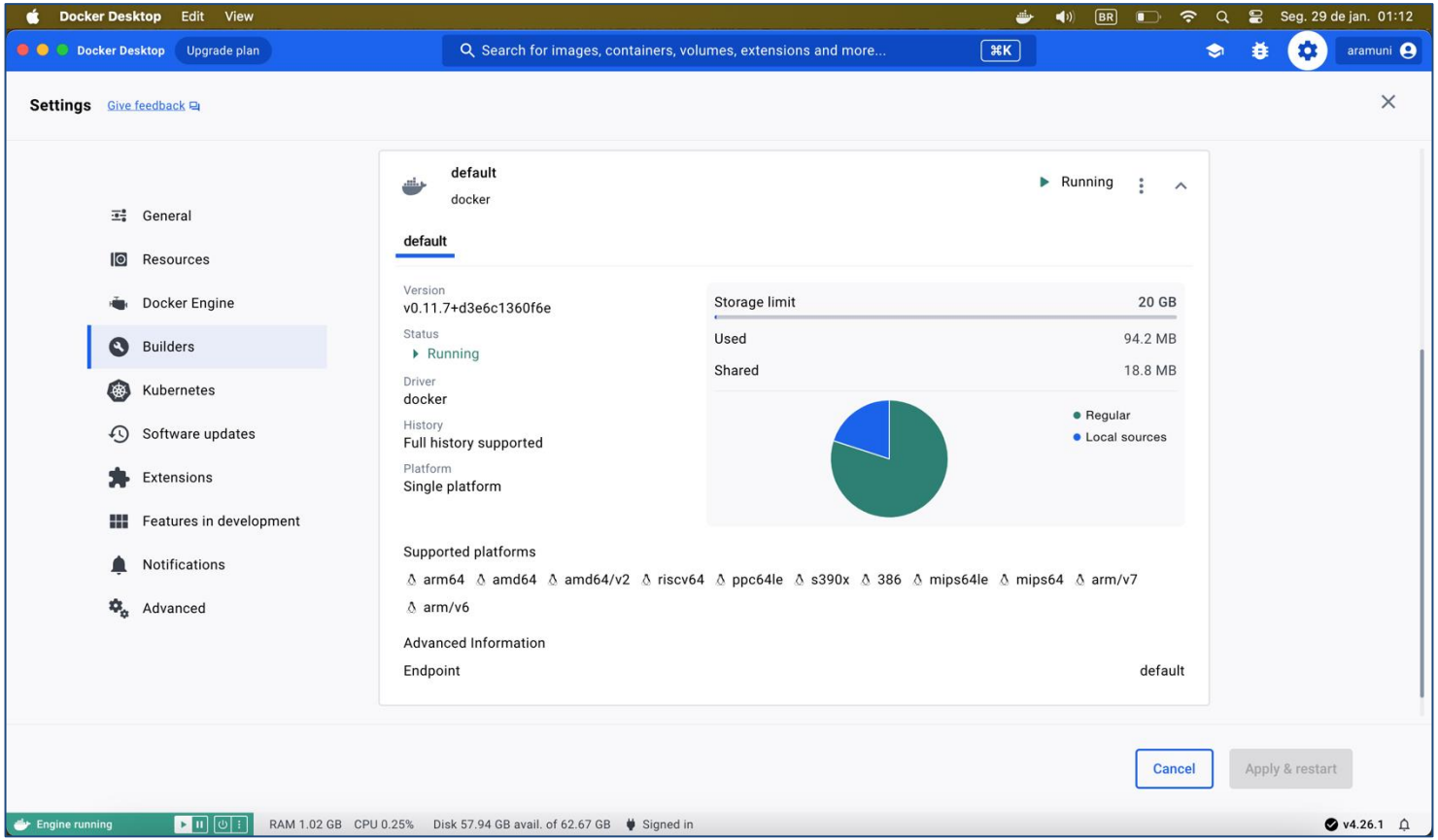
Visite:
https://fly-io-view-restapi.fly.dev/exemplo



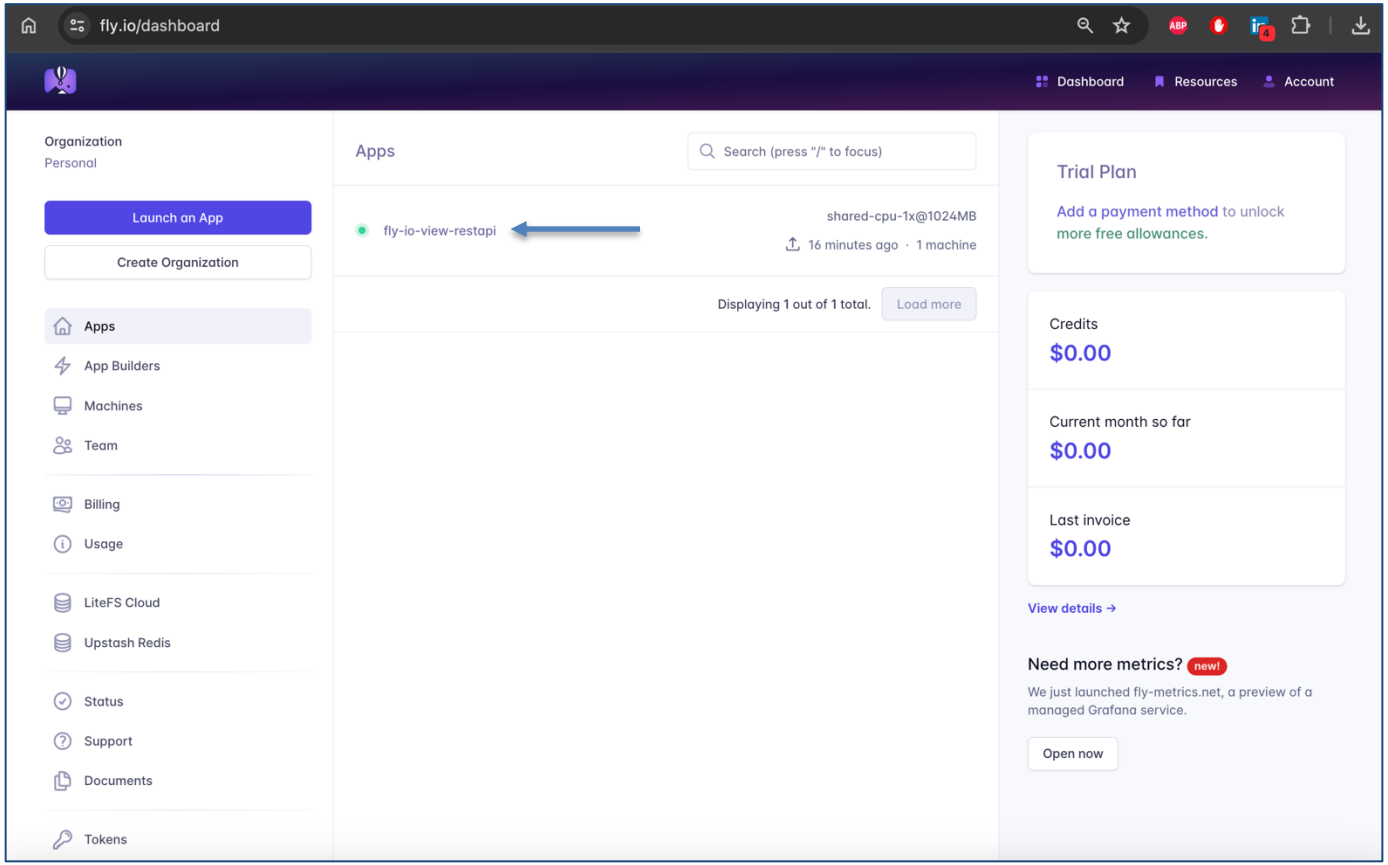
- Docker Desktop atualizou:



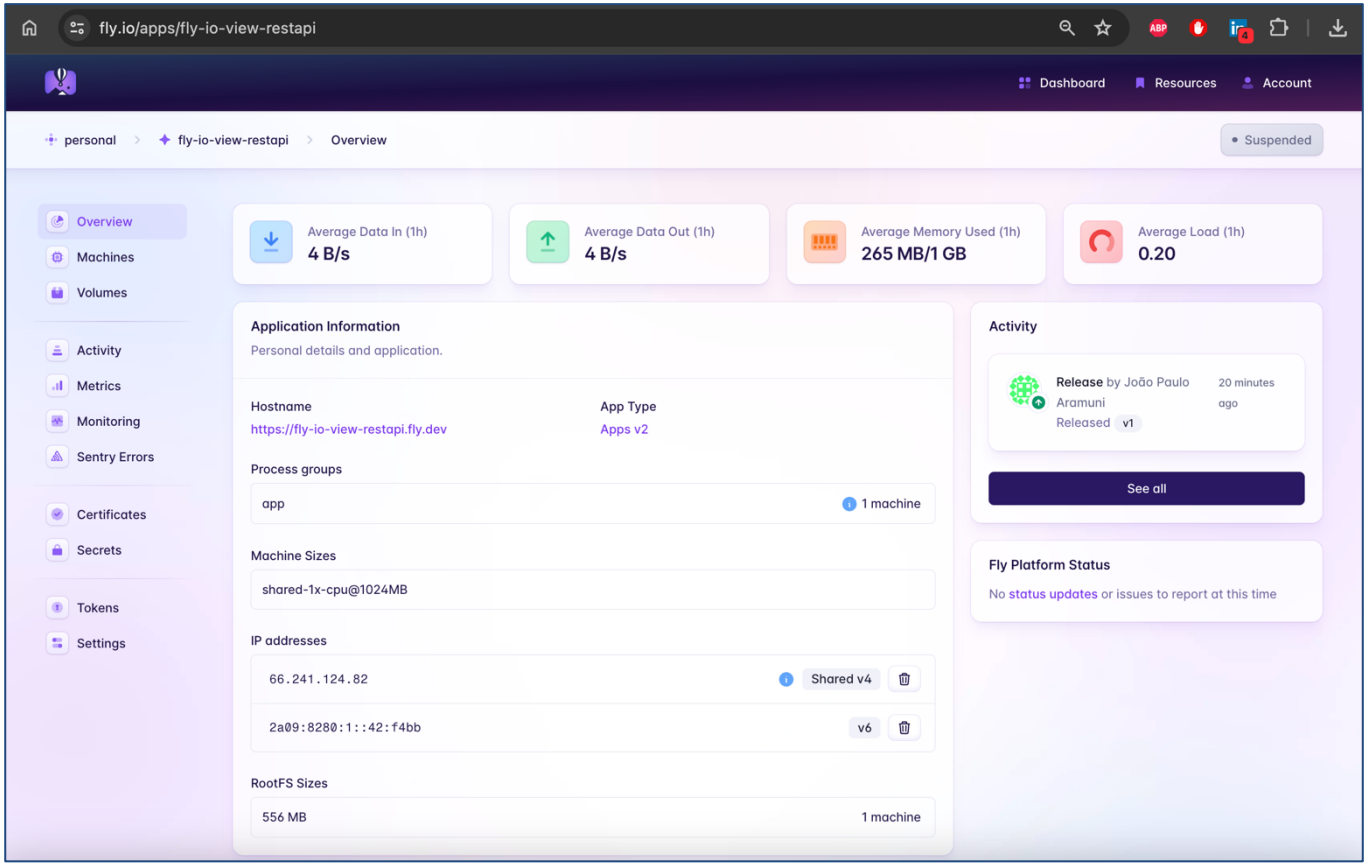
- Docker Desktop atualizou:



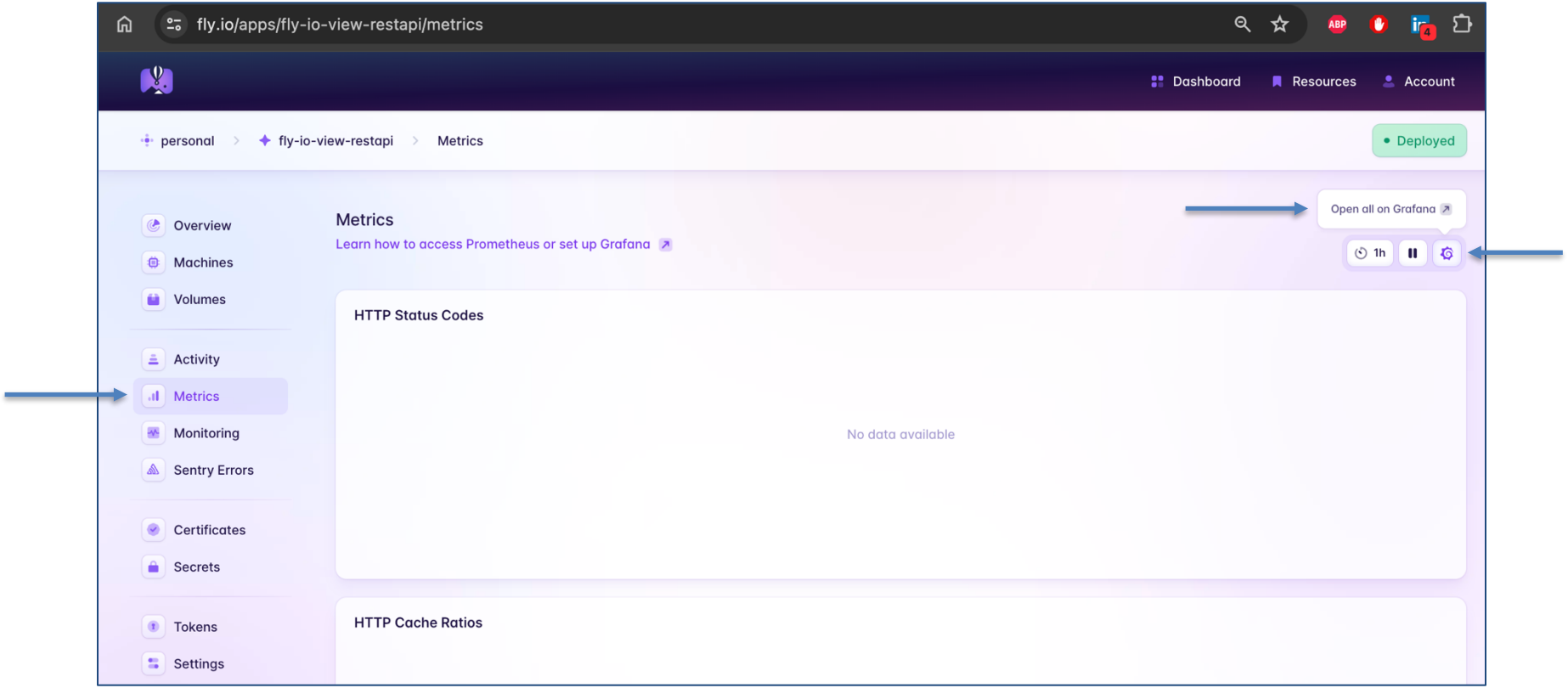
- Atualize o dashboard do Fly.io: https://fly.io/dashboard



- https://fly.io/apps/fly-io-view-restapi



- https://fly.io/apps/fly-io-view-restapi/metrics



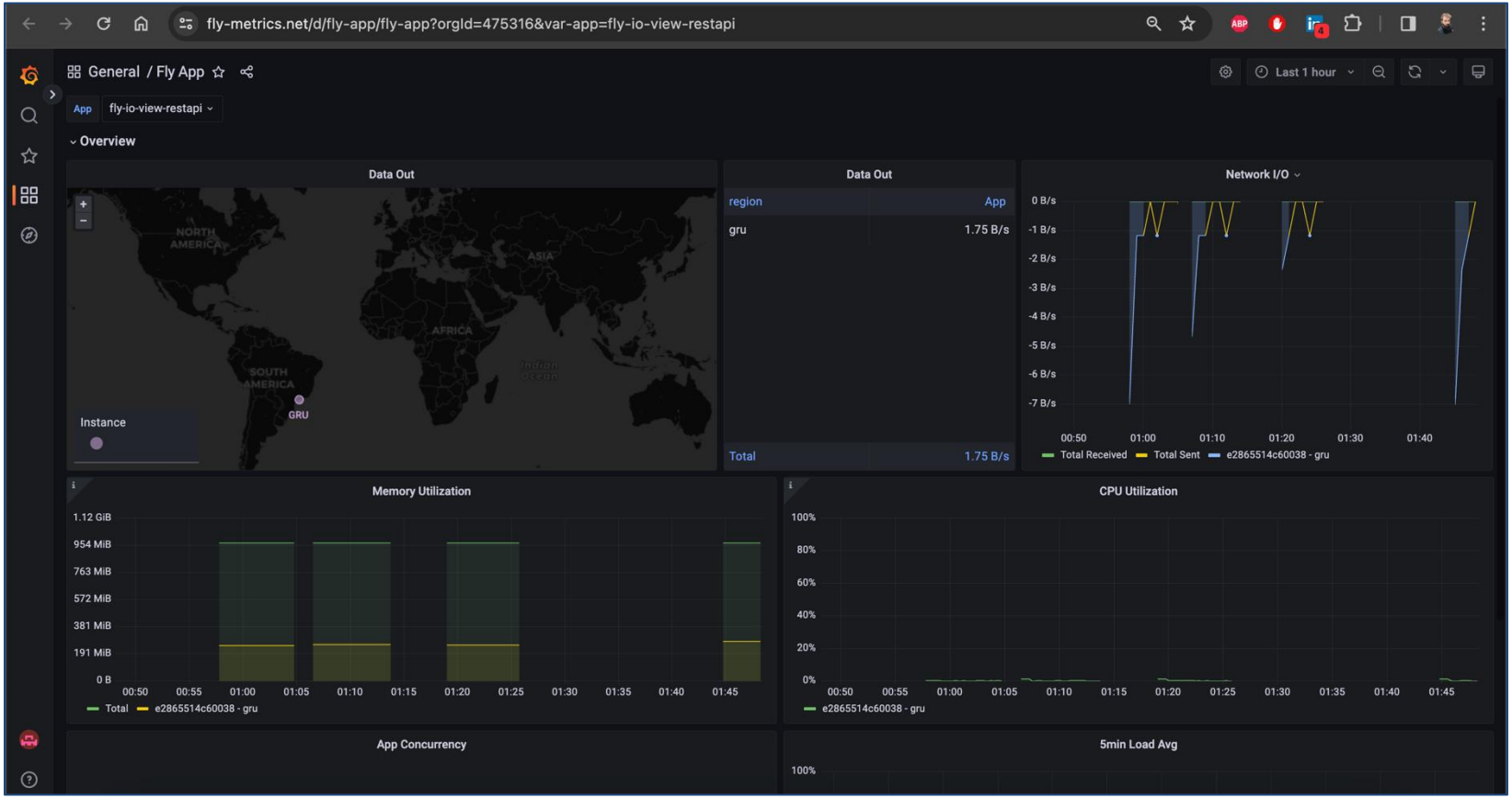
Grafana e Fly.io

- Você se lembra do Grafana da aula de **Monitoramento**?
  - Veja a quantidade de dashboards que o Fly.io gera automaticamente para você.

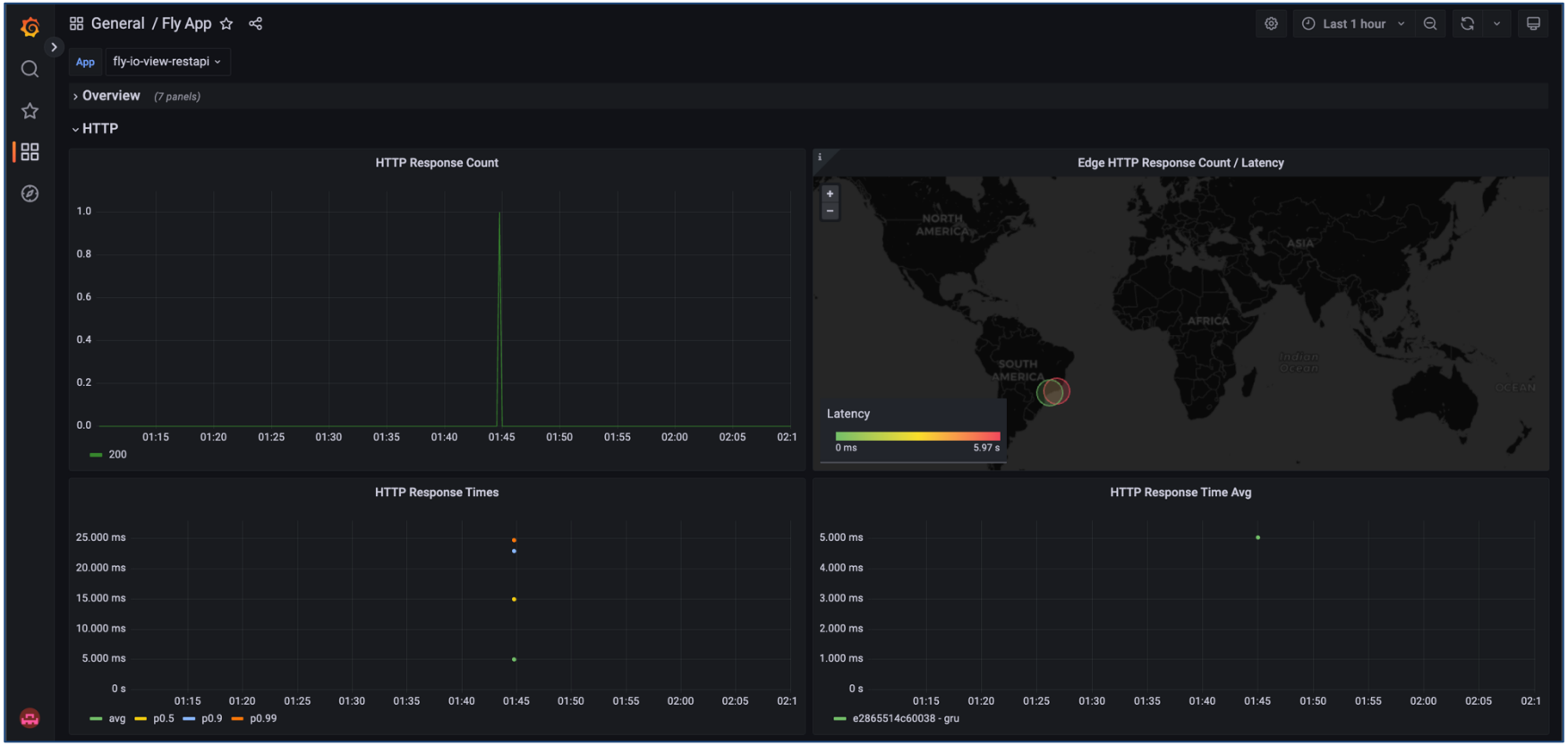
- Grafana - General - **Fly App**
  - https://fly-metrics.net/d/fly-app/fly-app?orgId=475316&var-app=fly-io-view-restapi
  - O Dashboard **Fly App** fornece uma visão ampla e geral do desempenho e métricas de toda a aplicação.

- Fly.io métricas:
  - https://fly.io/docs/reference/metrics/

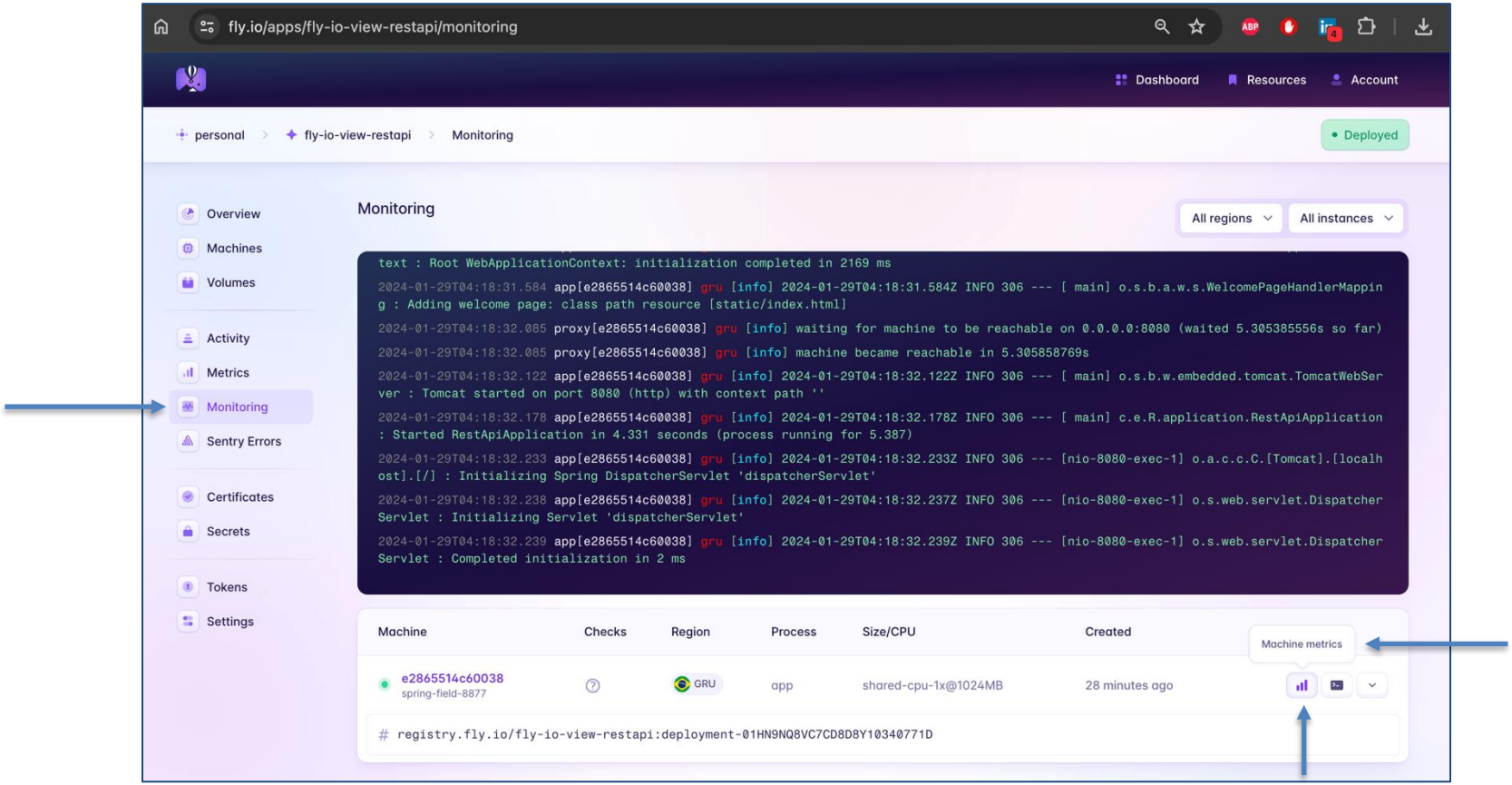
- Grafana - General / **Fly App** - Overview



- Grafana - General / **Fly App** - HTTP



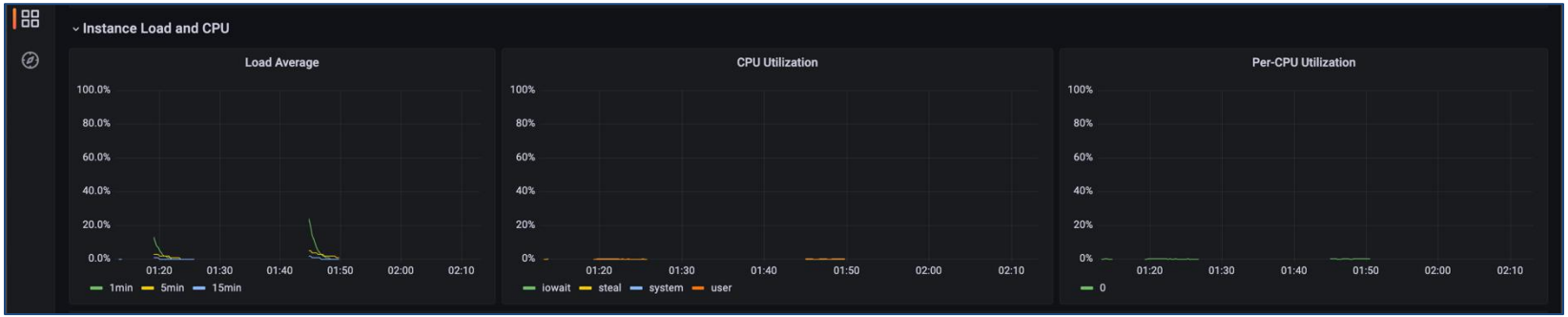
- https://fly.io/apps/fly-io-view-restapi/monitoring



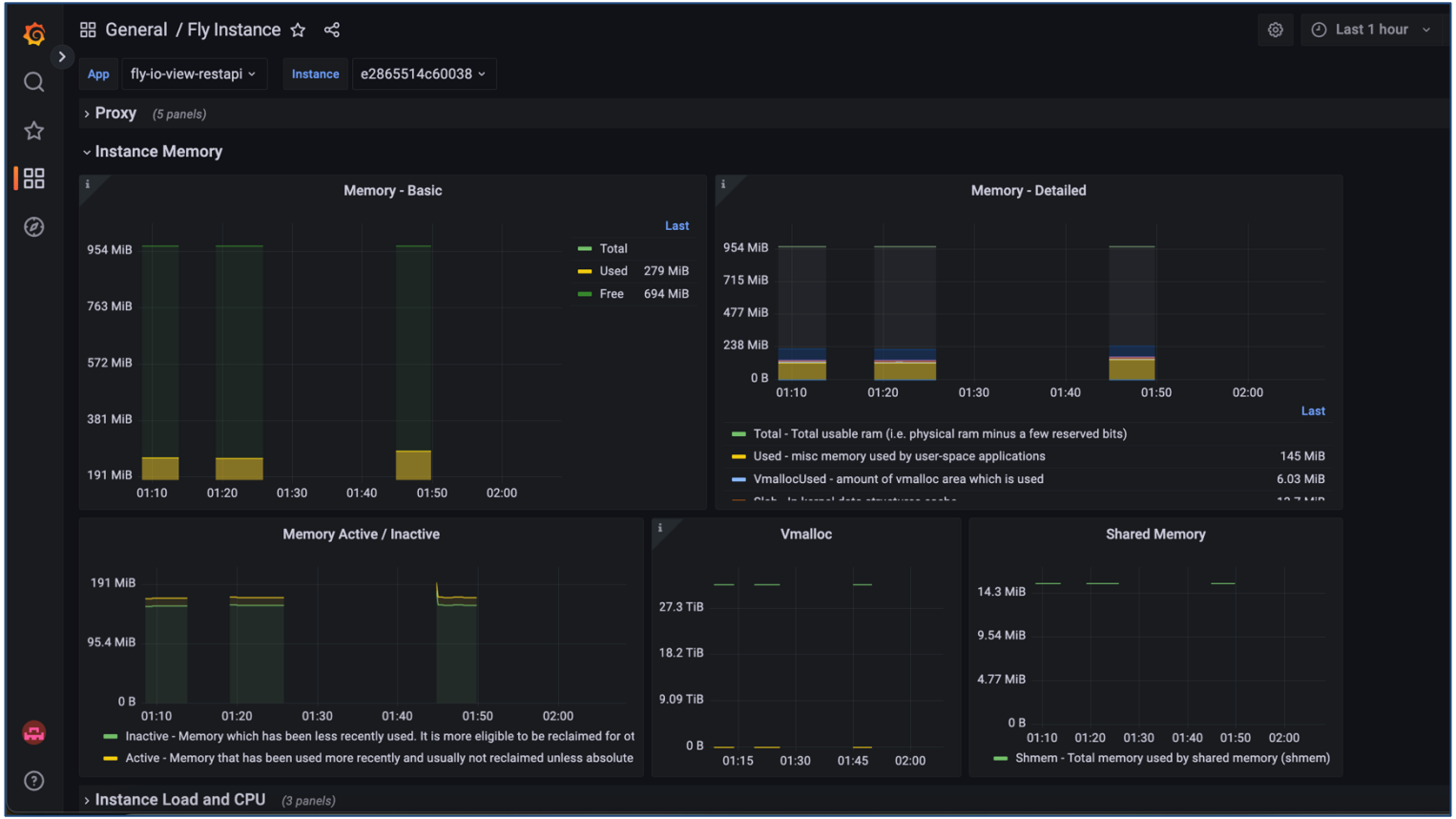
- Grafana - General / **Fly instance**
  - https://fly-metrics.net/d/fly-instance/fly-instance?from=now-1h&to=now&var-app=fly-io-view-restapi&orgId=475316&var-instance=e2865514c60038

- Este tipo de dashboard é mais específico e focado em uma **instância individual** da aplicação. Geralmente exibe métricas específicas relacionadas a uma instância, como uso de CPU, memória, tráfego de rede, etc.

- Grafana - General / **Fly instance** - Proxy e Instance Load and CPU

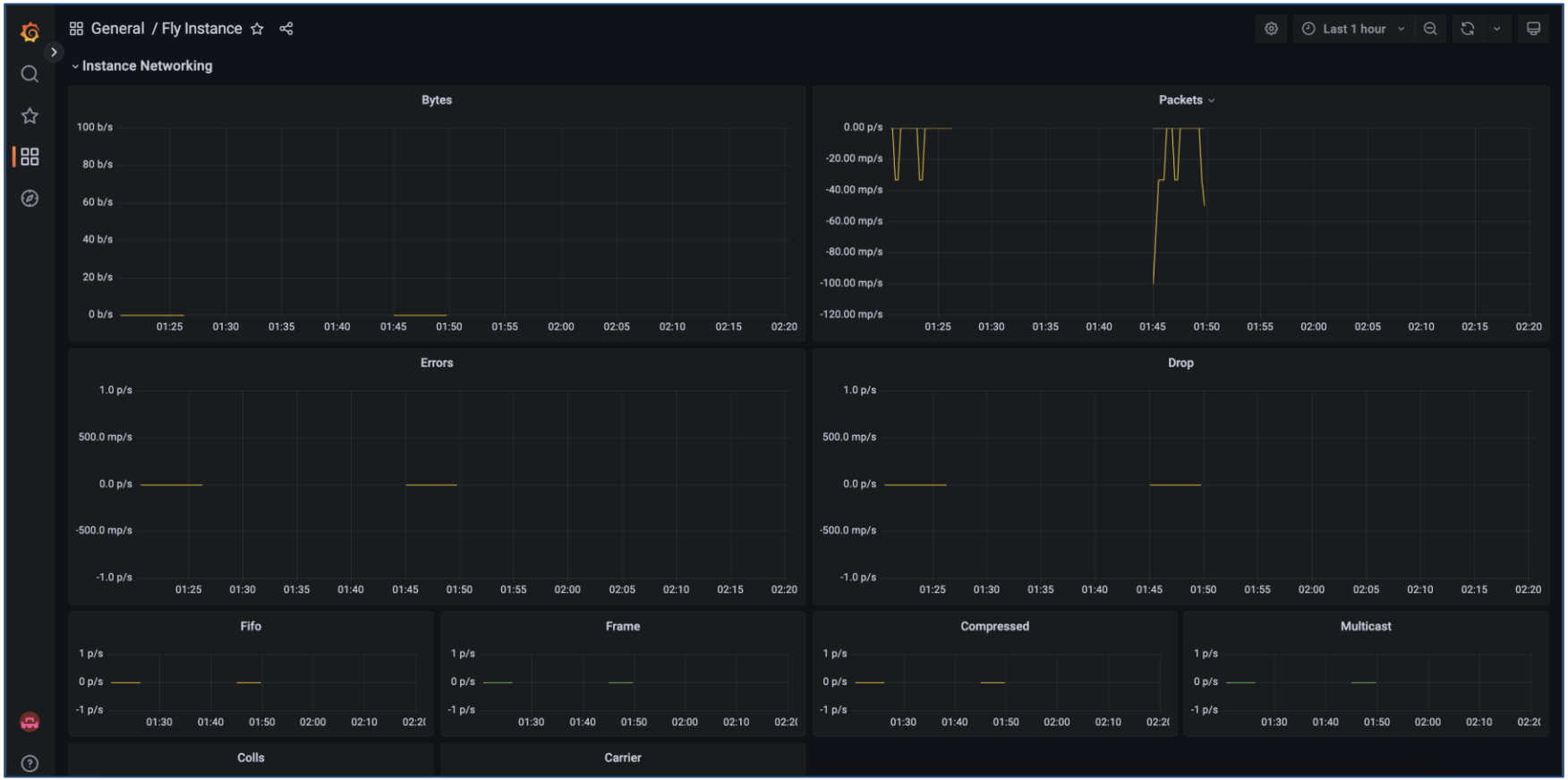


- Grafana - General / **Fly instance** - Instance Memory

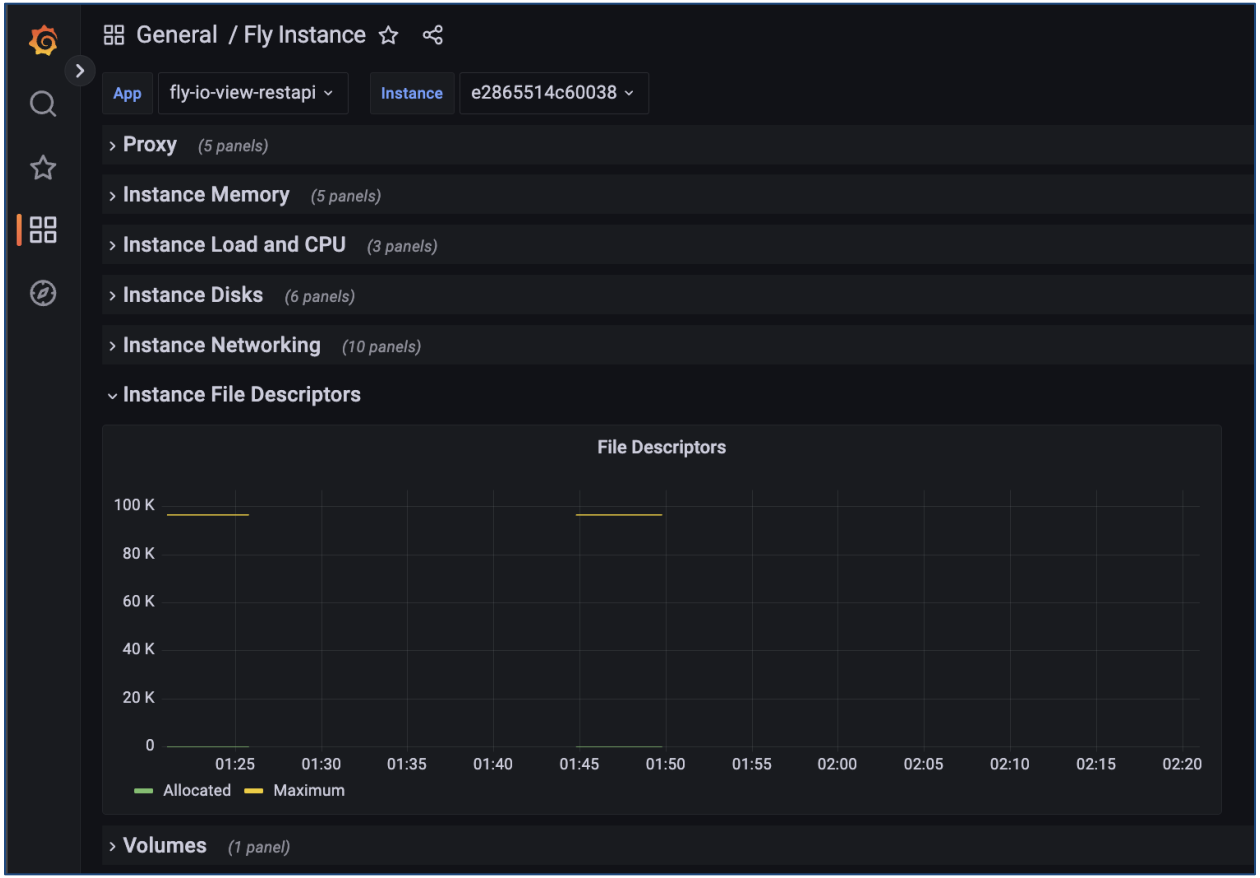


- Grafana - General / **Fly instance** - Instance Disks

- Grafana - General / **Fly instance** - Instance Networking



- Grafana - General / **Fly instance** - Instance File Descriptors



Hospedagem de aplicações Web

- **Publicando a aplicação em uma solução PaaS (Platform as a Service)**
  - Leituras sugeridas:
    - https://fastapidozero.dunossauro.com/11/
    - https://coder.com/blog/remote-developer-environments-on-fly-io
    - https://medium.com/@vergil333/deploy-spring-boot-to-fly-iof-d54d5ca05243
    - https://fly.io/blog/free-postgres/
    - https://fly.io/blog/fly-with-alpine/

Exercício 1

- Utilize a plataforma de hospedagem gratuita do Fly.io para disponibilizar o projeto JWT_RestAPI na nuvem.

- Ao final da publicação do projeto, você deverá ter um link semelhante à esse:
  - https://jwt-restapi.fly.dev

- Utilize o Postman ou o Insomnia para enviar as requisições HTTP para a sua API hospedada no Fly.io. Lembre-se de substituir as URLs locais pelas URLs na nuvem.
  - POST http://localhost:8080/login --> POST https://jwt-restapi.fly.dev/login

Bônus

- **Spring Cloud**
  - O Spring Cloud é um framework desenvolvido pela Pivotal Software, uma divisão da VMware, que oferece suporte para a construção de sistemas distribuídos em ambientes de microsserviços.

  - Ele é construído sobre o ecossistema do Spring Framework, fornecendo soluções para vários desafios comuns encontrados ao desenvolver e implantar arquiteturas baseadas em microsserviços.

Bônus

- Principais características do **Spring Cloud**:
  - **Configuração distribuída**: O Spring Cloud oferece ferramentas para a gestão de configurações distribuídas, permitindo que configurações específicas de cada serviço sejam gerenciadas centralmente e distribuídas conforme necessário.

  - **Descoberta de serviços**: A descoberta de serviços é facilitada por meio de ferramentas como a **Eureka**, que permite aos serviços se registrarem e descobrirem outros serviços em um ambiente distribuído.

- Principais características do **Spring Cloud**:
  - **Balanceamento de carga**: O Spring Cloud inclui suporte para balanceamento de carga através do **Ribbon**, que distribui as solicitações entre instâncias de serviços disponíveis, melhorando a escalabilidade e a resiliência.

  - **Chamadas de serviço**: Facilita a comunicação entre serviços por meio de abstrações como **Feign**, que simplifica a criação de clientes REST.

  - **Gateway API**: O Spring Cloud fornece uma solução para criar gateways de API eficientes, como o **Spring Cloud Gateway**, que gerencia o roteamento e a filtragem de requisições.

Bônus

- Vantagens em ambientes de **microsserviços**:
  - **Flexibilidade e escalabilidade**: O Spring Cloud possibilita a construção de sistemas flexíveis, permitindo que os microsserviços sejam desenvolvidos, implantados e escalados independentemente.

  - **Resiliência**: Ferramentas como o **Hystrix** fornecem mecanismos para lidar com falhas em serviços, ajudando a criar sistemas mais robustos e resilientes.

Bônus

- Vantagens em ambientes de **microsserviços**:
  - **Manutenção simplificada**: A capacidade de gerenciar configurações centralmente e de forma distribuída facilita a manutenção e atualização de microsserviços.

  - **Descoberta dinâmica**: O suporte à descoberta de serviços simplifica a adição e remoção de microsserviços em execução, sem a necessidade de configuração estática.

- Spring Cloud:
  - https://spring.io/cloud

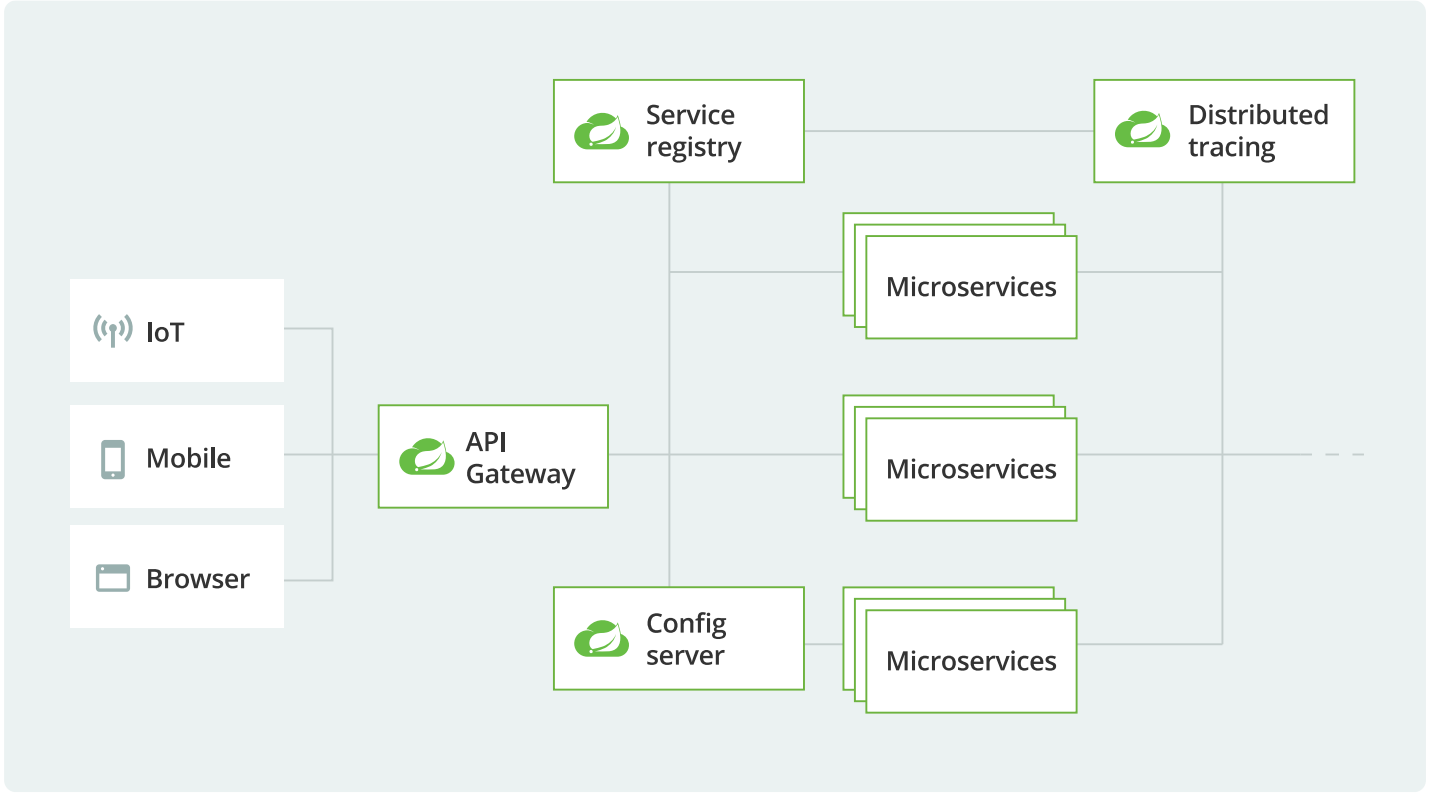
spring by VMware Tanzu

Why Spring | Learn | Projects | Academy | Solutions | Community

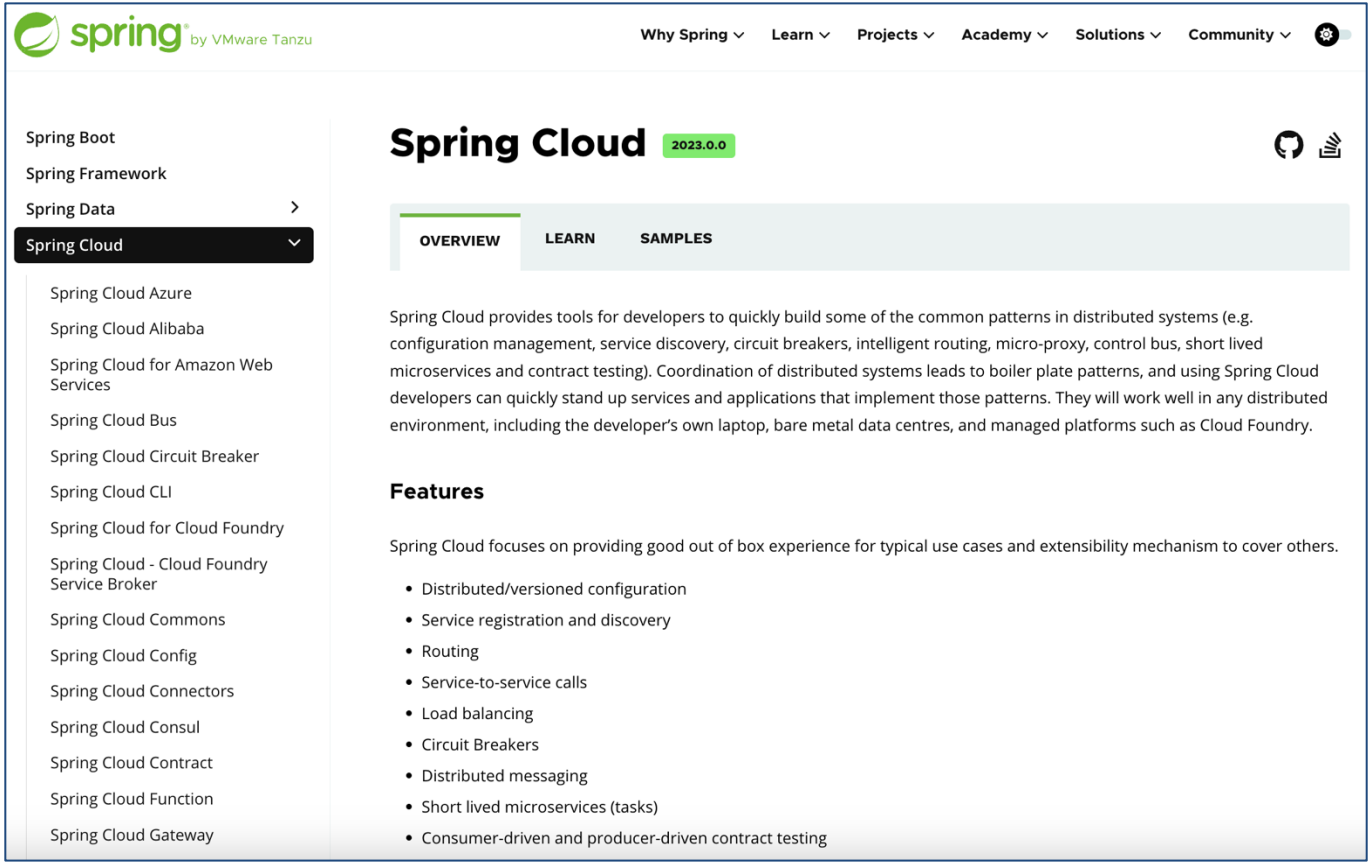
Cloud

Developing distributed systems can be challenging. Complexity is moved from the application layer to the network layer and demands greater interaction between services. Making your code 'cloud-native' means dealing with 12-factor issues such as external configuration, statelessness, logging, and connecting to backing services. The Spring Cloud suite of projects contains many of the services you need to make your applications run in the cloud.

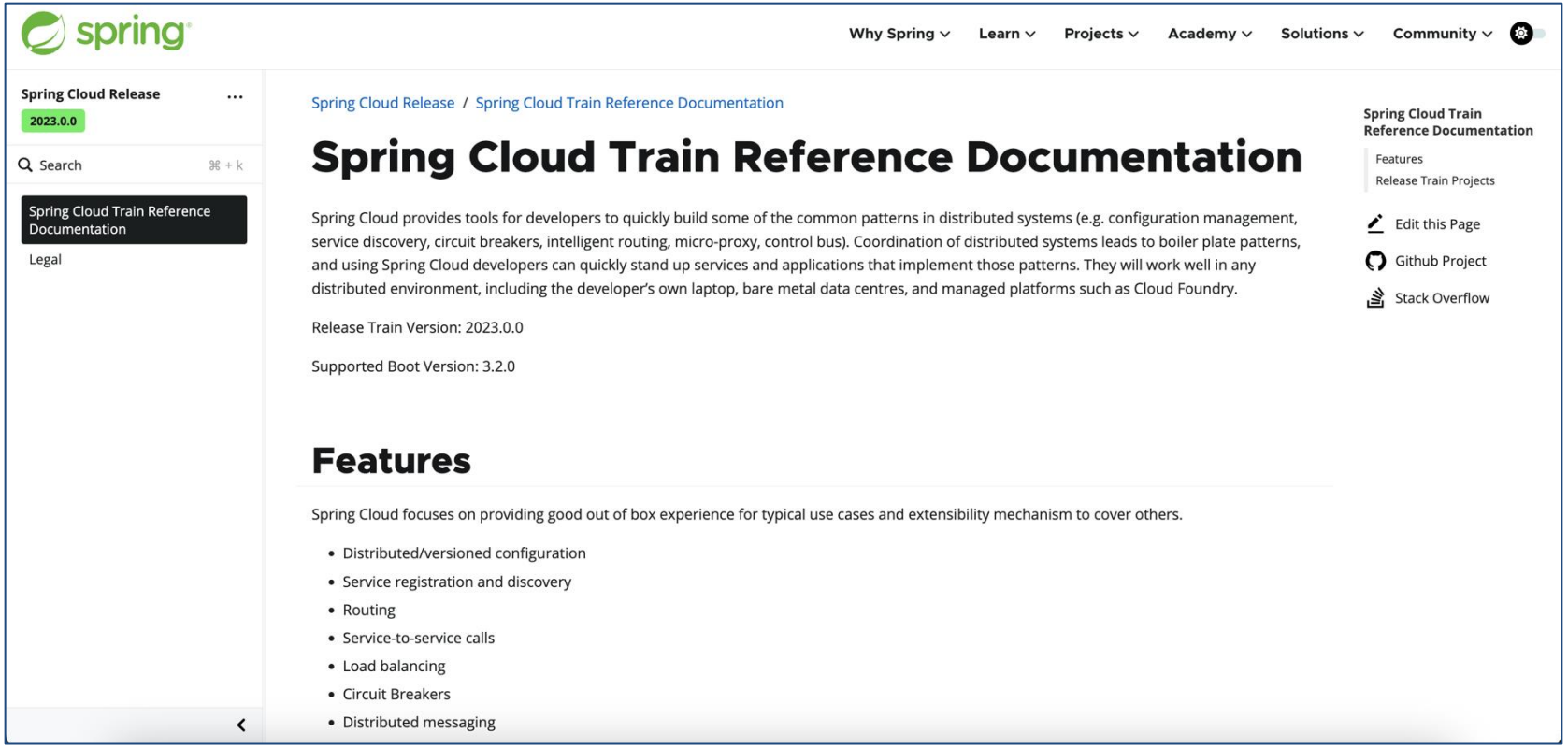
- Spring Cloud architecture highlights:
  - https://spring.io/cloud



- https://spring.io/projects/spring-cloud



- Documentação do Spring Cloud:
  - https://docs.spring.io/spring-cloud-release/reference/index.html



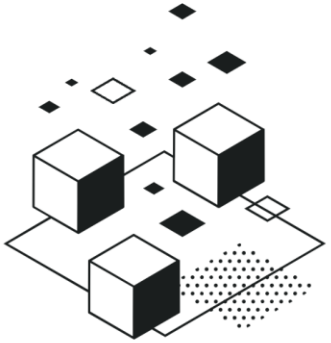
- https://spring.io/microservices

spring by VMware Tanzu

Why Spring | Learn | Projects | Academy | Solutions | Community

Microservices

Microservice architectures are the 'new normal'. Building small, self-contained, ready to run applications can bring great flexibility and added resilience to your code. Spring Boot's many purpose-built features make it easy to build and run your microservices in production at scale. And don't forget, no microservice architecture is complete without Spring Cloud - easing administration and boosting your fault-tolerance.



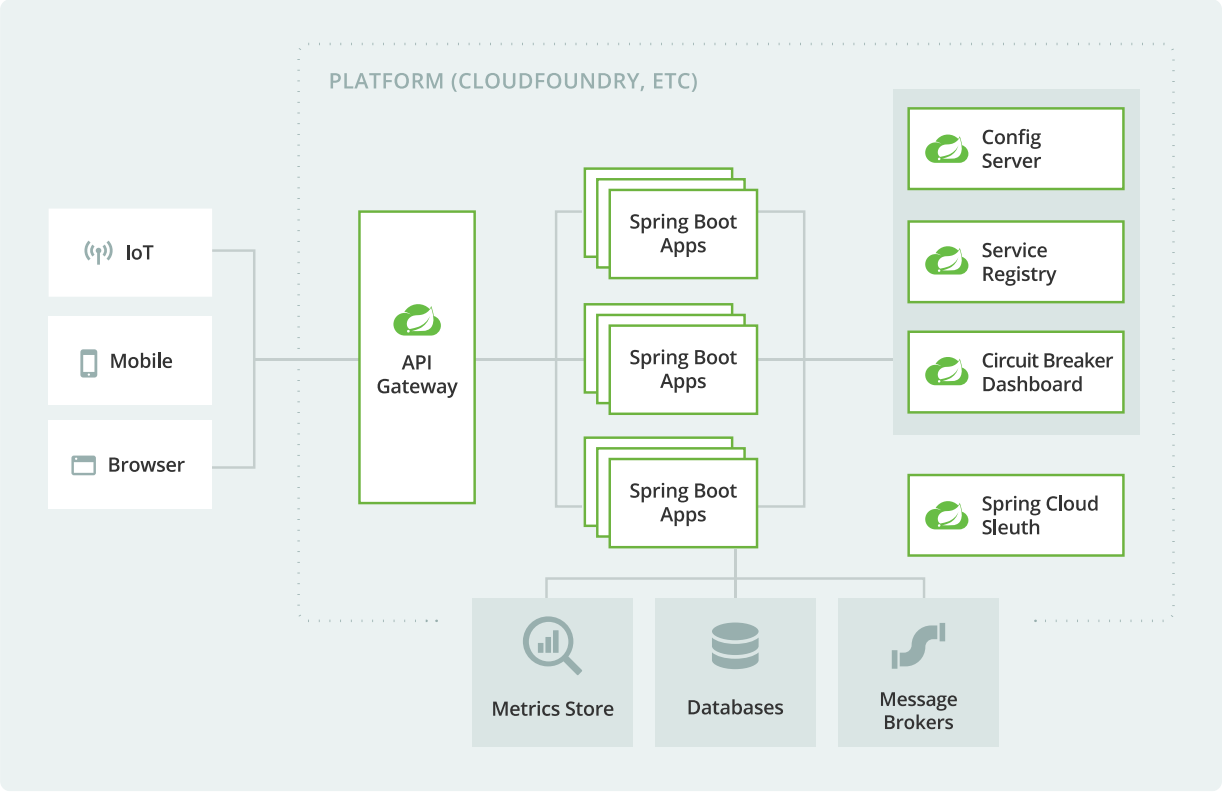
What are microservices?

Microservices are a modern approach to software whereby application code is delivered in small, manageable pieces, independent of others.

Why build microservices?

Their small scale and relative isolation can lead to many additional benefits, such as easier maintenance, improved productivity, greater fault tolerance, better business alignment, and more.

- Microservice resilience with Spring Cloud
  - https://spring.io/microservices

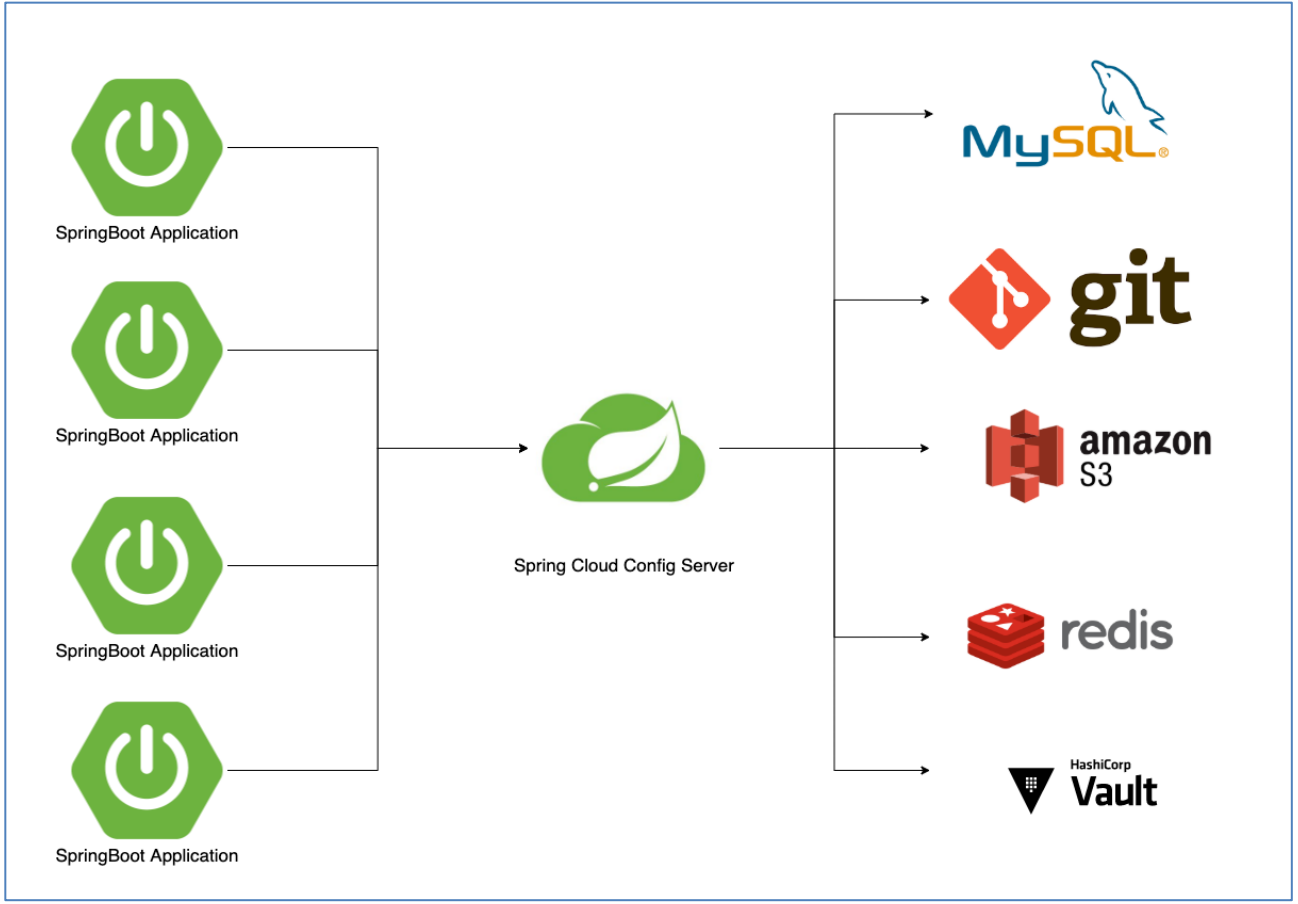


Leituras sugeridas:

https://medium.com/javarevisited/centralized-configuration-for-microservices-using-spring-cloud-config-c7845cbccc34

https://dev.to/womakerscode/microsservicos-com-spring-cloud-introducao-3cn6

https://www.linkedin.com/pulse/introdução-ao-spring-cloud-yara-oliveira/



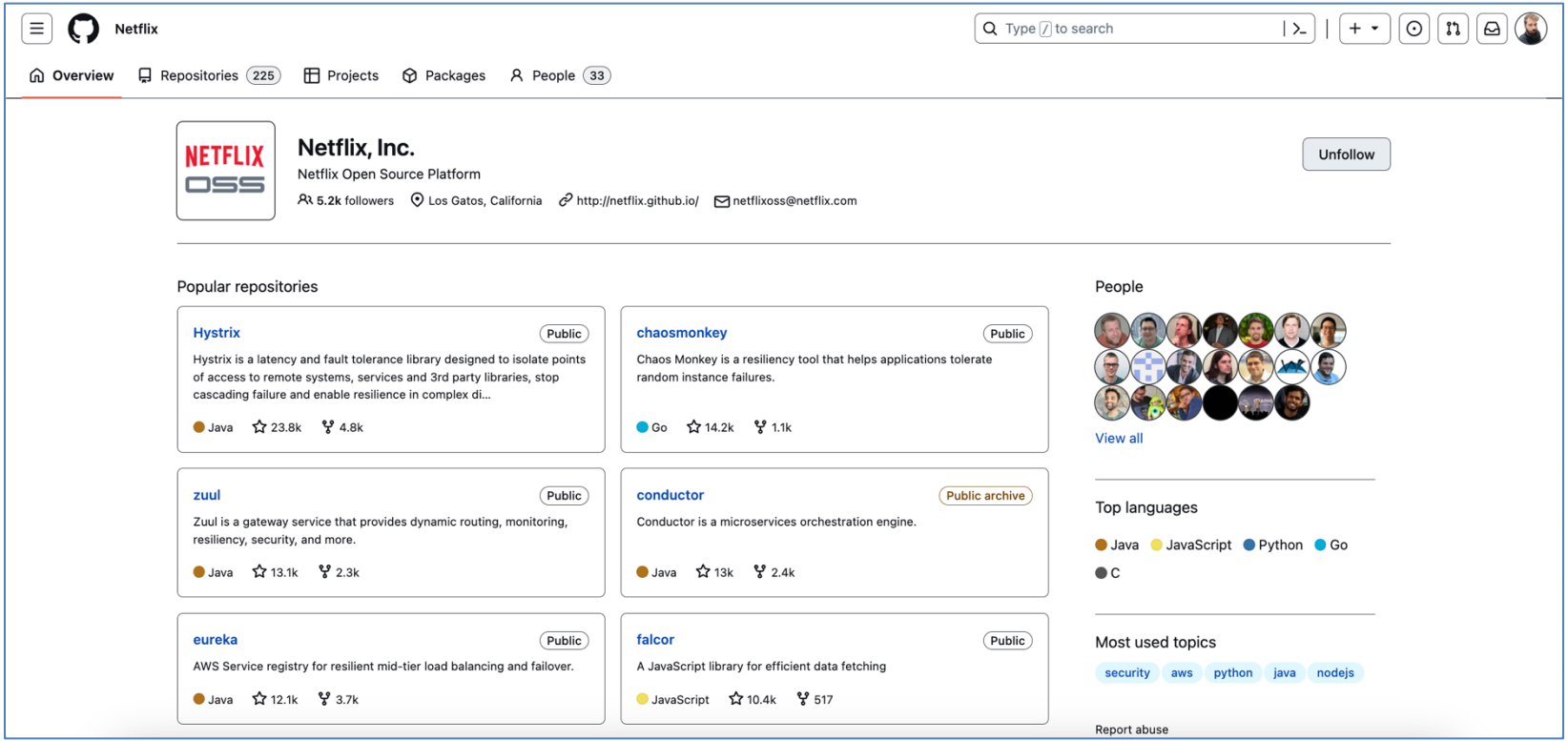
Bônus

- Relação com a **Netflix**:
  - O Spring Cloud tem uma relação estreita com a Netflix, pois inicialmente muitas das ferramentas desenvolvidas pela Netflix para gerenciar **arquiteturas de microsserviços** foram incorporadas ao ecossistema do Spring Cloud.

  - A Netflix OSS (*Open Source Software*) é um conjunto de ferramentas e bibliotecas de código aberto desenvolvidas pela Netflix para suportar sua arquitetura baseada em microsserviços.

- Netflix OSS:
  - https://netflix.github.io/

- Netflix OSS GitHub:
  - https://github.com/Netflix



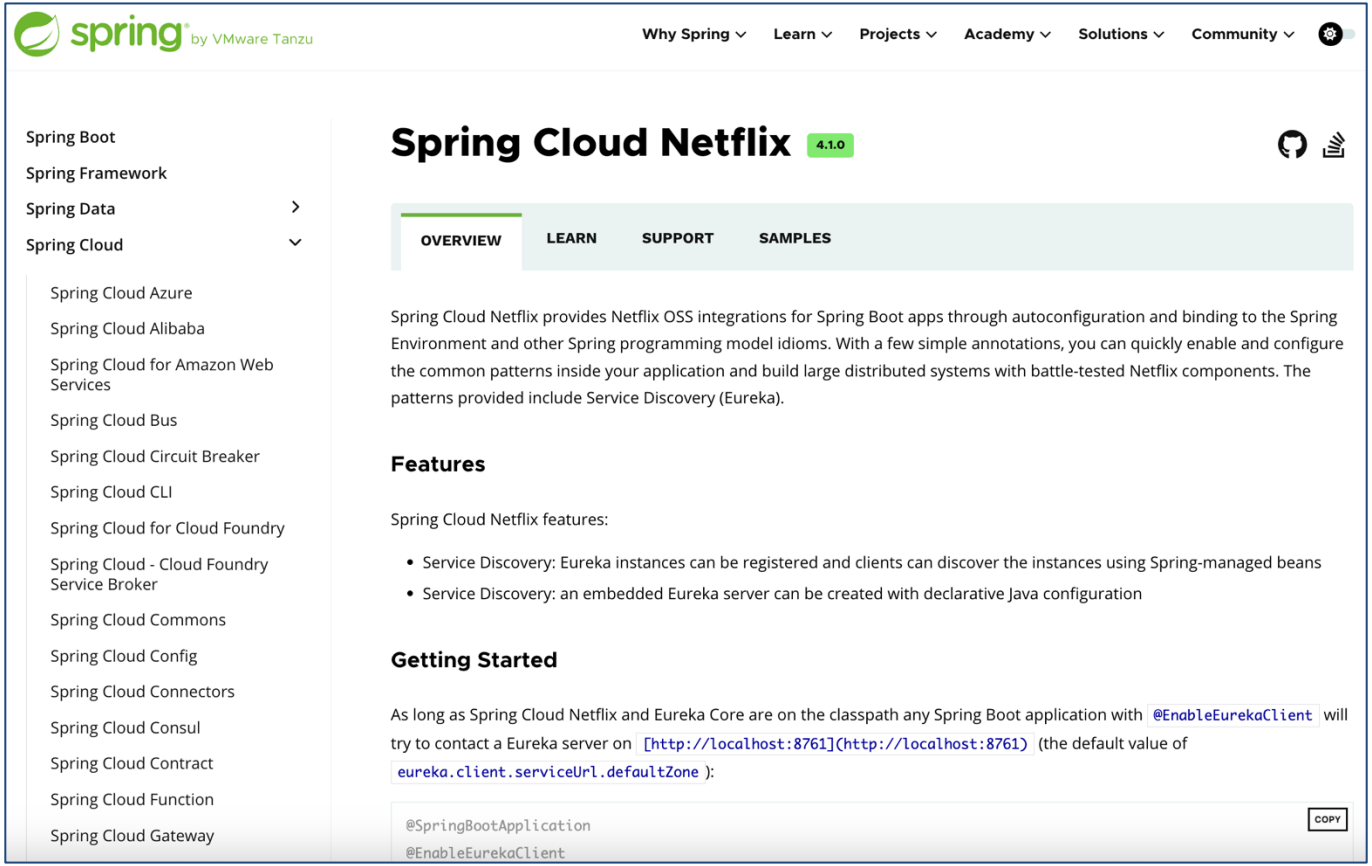
Bônus

- Relação com a **Netflix**:
  - Ferramentas como **Eureka**, que proporciona serviços de descoberta, e **Hystrix**, focada em tolerância a falhas, foram desenvolvidas pela Netflix como parte do conjunto de ferramentas de código aberto conhecido como Netflix OSS.

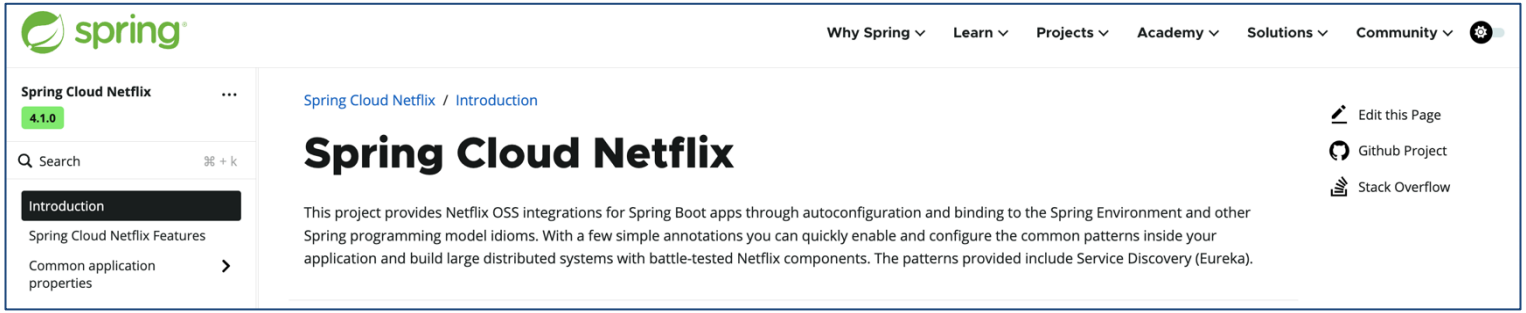
  - Essas tecnologias foram posteriormente integradas ao ecossistema do Spring Cloud para oferecer soluções robustas no desenvolvimento e operação de sistemas distribuídos baseados em **microsserviços**.

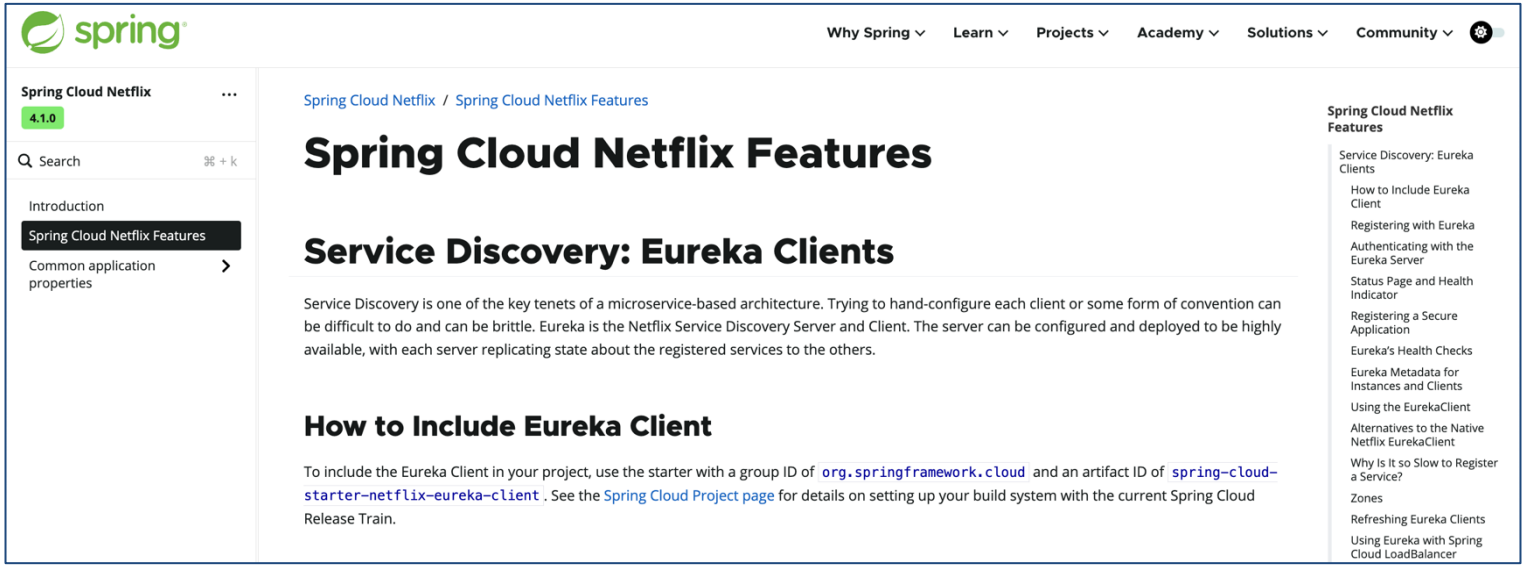
NETFLIX | OSS

- https://spring.io/projects/spring-cloud-netflix

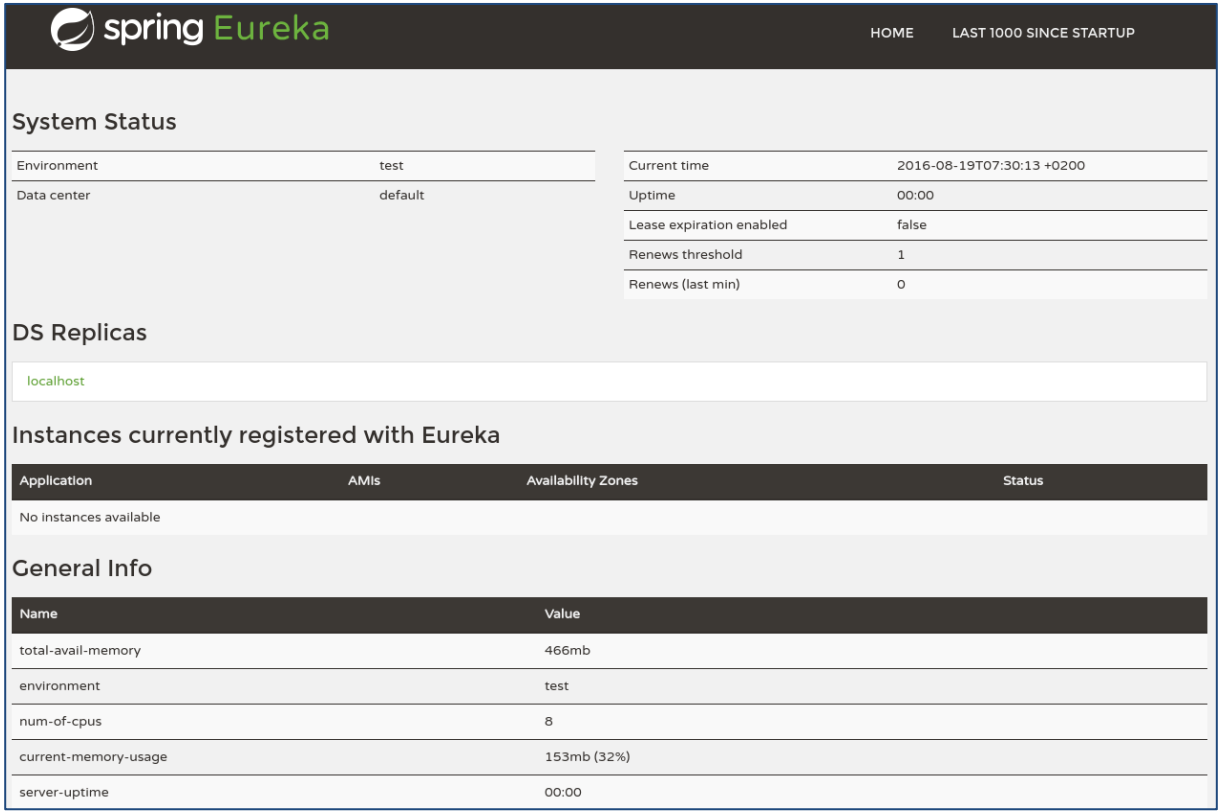


- Documentação do Spring Cloud Netflix: https://docs.spring.io/spring-cloud-netflix/reference/index.html





- Leitura sugerida:
  - https://www.baeldung.com/spring-cloud-netflix-eureka

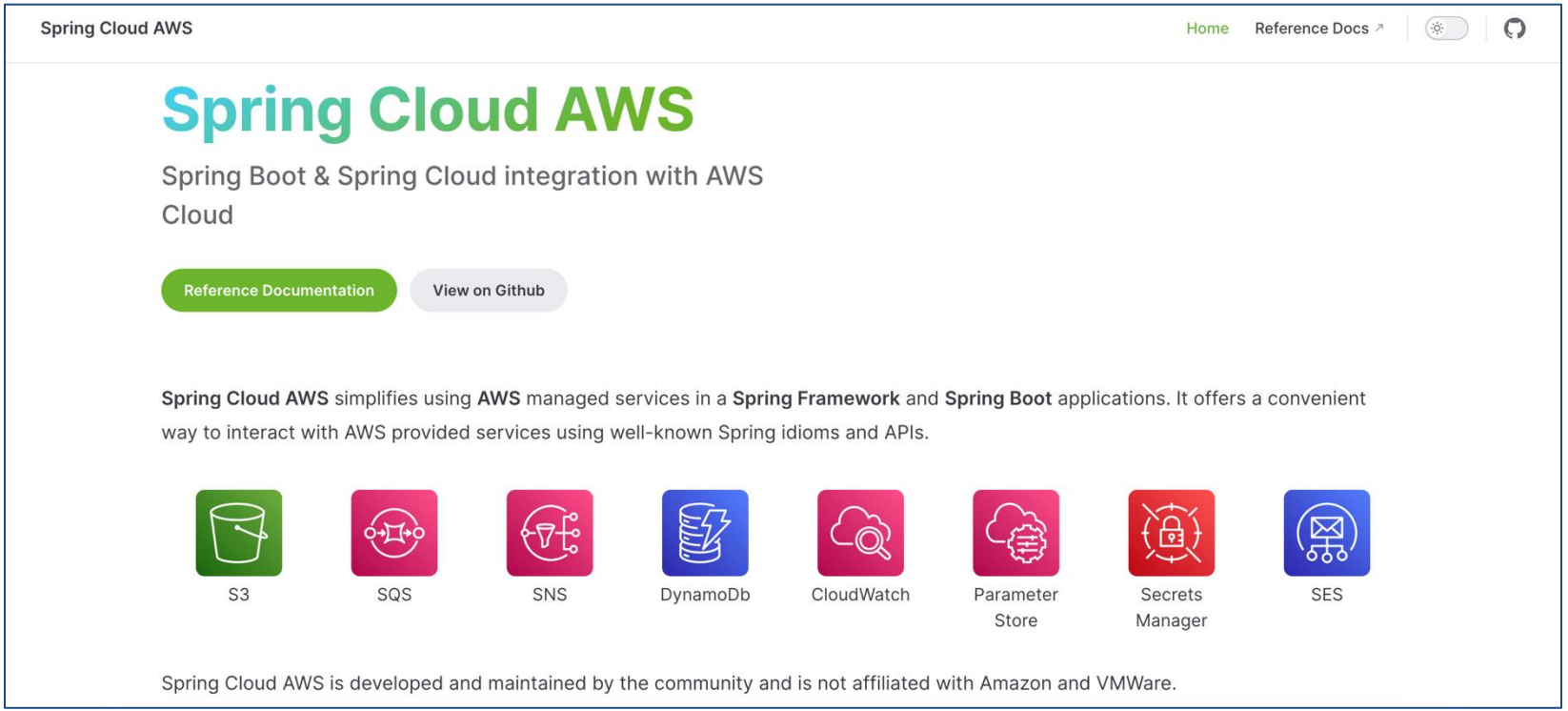


Bônus

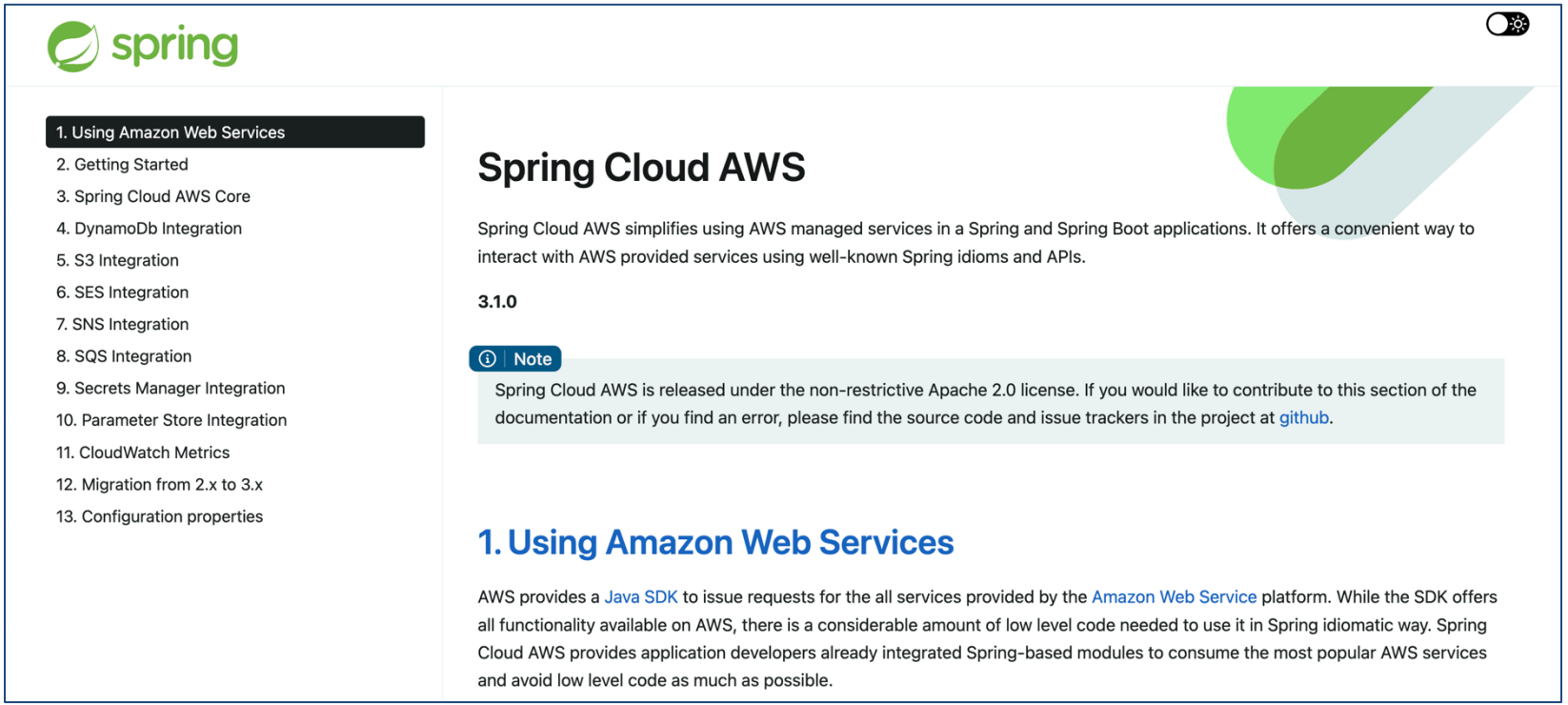
- **Spring Cloud AWS**:
  - O projeto "Spring Cloud AWS" é uma iniciativa que visa facilitar a integração de aplicações Spring Boot e Spring Cloud com os serviços da Amazon Web Services (AWS).

  - Ele oferece uma série de recursos e bibliotecas para simplificar o desenvolvimento, a implantação e a execução de aplicativos Java na nuvem da AWS.

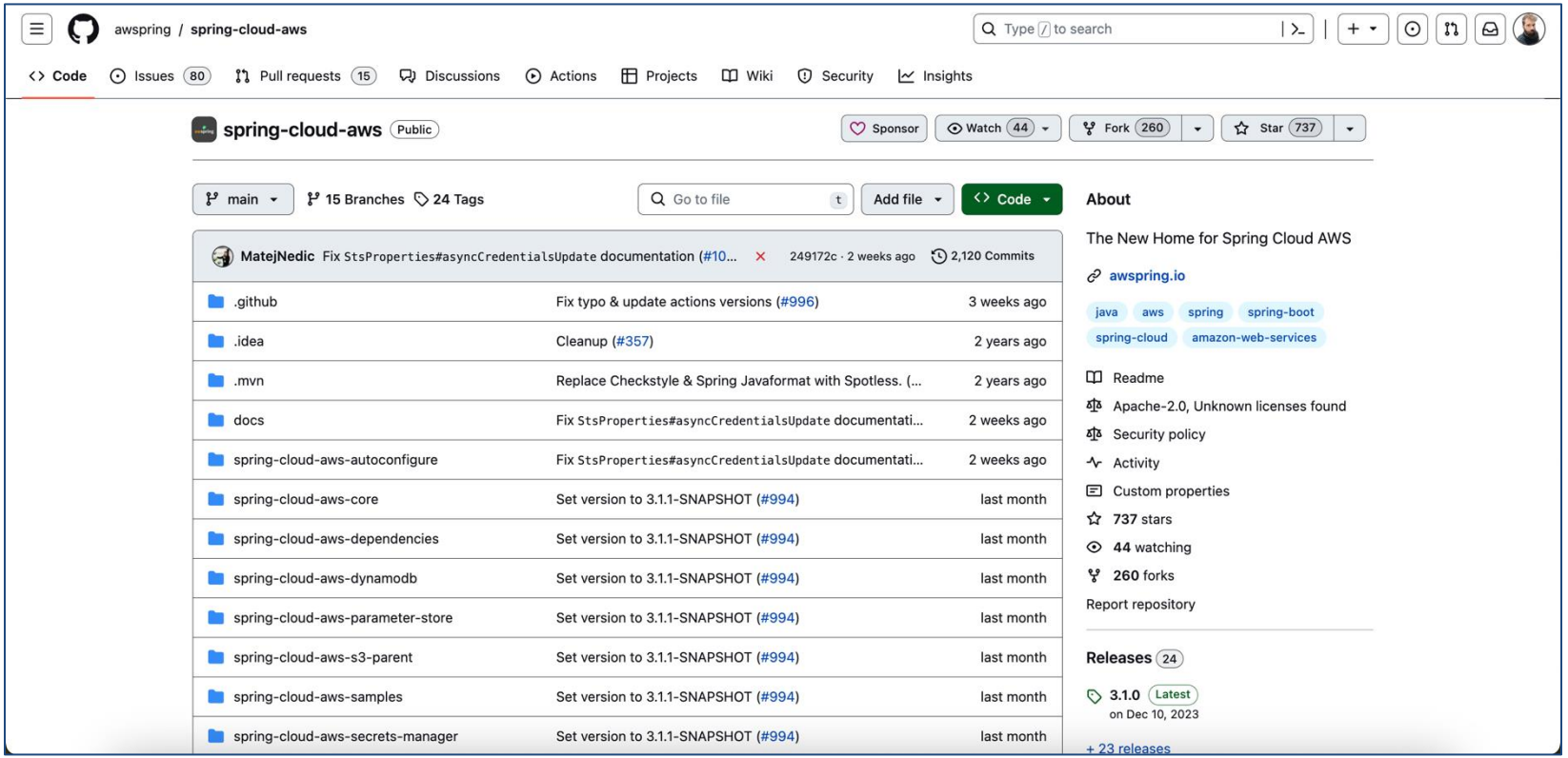
- Spring Cloud AWS:
  - https://awspring.io/



- Documentação do Spring Cloud AWS:
  - https://docs.awspring.io/spring-cloud-aws/docs/3.1.0/reference/html/index.html



- Spring Cloud AWS GitHub:
  - https://github.com/awspring/spring-cloud-aws



Bônus

- **Spring Cloud Azure**:
  - O Spring Cloud Azure é uma extensão do ecossistema Spring que oferece suporte a serviços e integração específicos da plataforma Microsoft Azure.

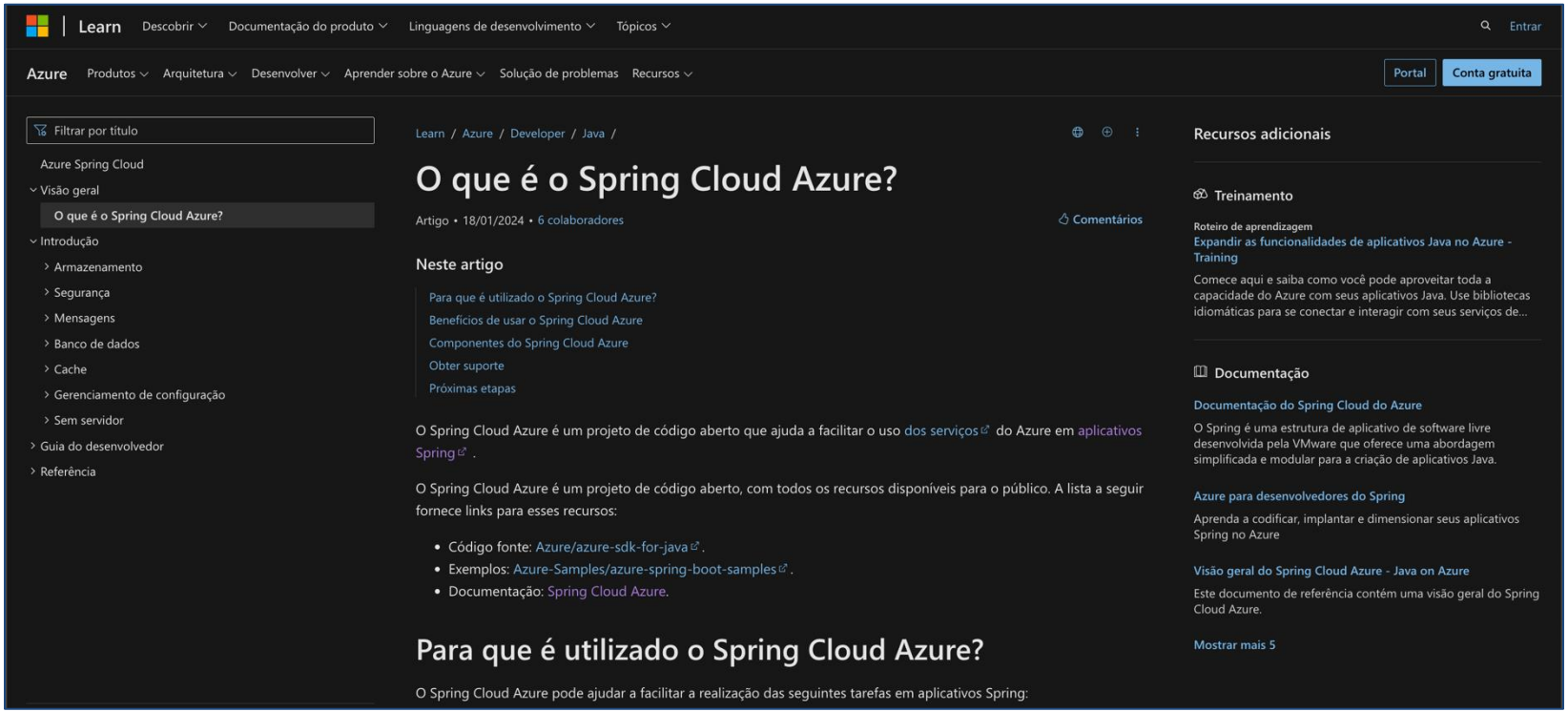
  - Ele facilita o desenvolvimento de aplicativos Java Spring que podem ser implantados e integrados de maneira eficiente na infraestrutura e nos serviços fornecidos pelo Azure.

Bônus

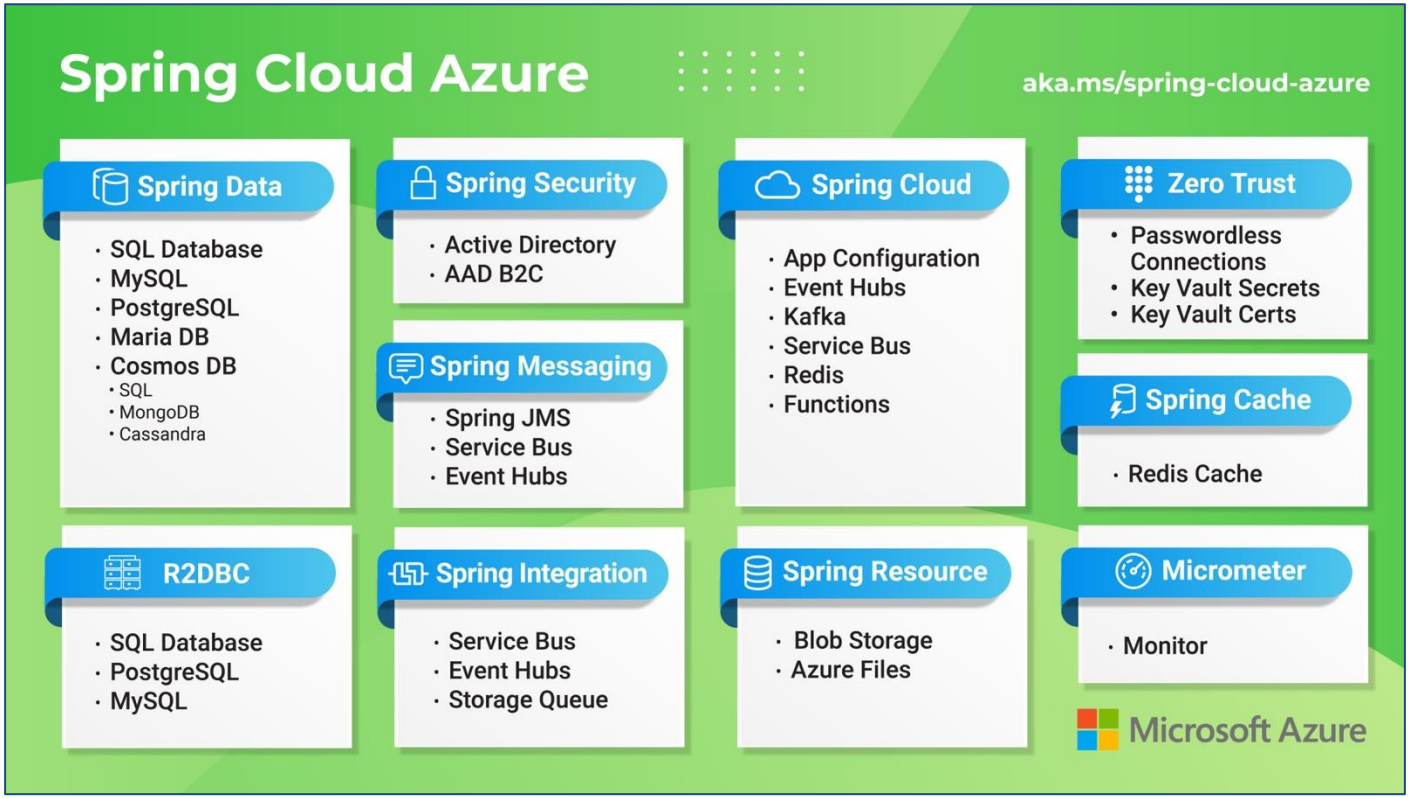
- **Spring Cloud Azure**:
  - O Spring Cloud Azure fornece abstrações e ferramentas para simplificar tarefas comuns, como gerenciamento de configurações, descoberta de serviços, e acesso a serviços do Azure, permitindo que os desenvolvedores se concentrem mais na lógica de negócios de suas aplicações.

  - Isso promove uma integração mais suave entre o ecossistema Spring e as soluções oferecidas pela Microsoft Azure.

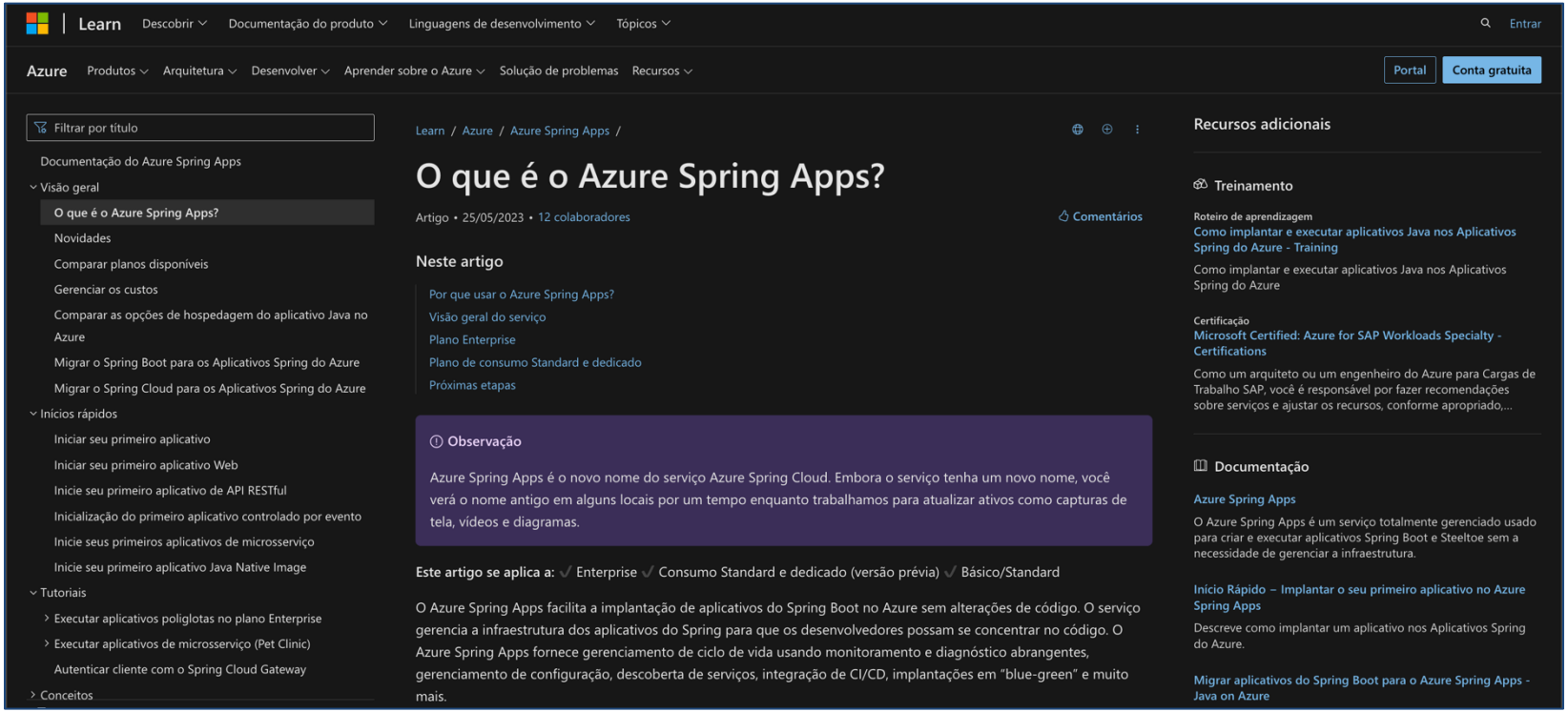
- Spring Cloud Azure:
  - https://learn.microsoft.com/pt-br/azure/developer/java/spring-framework/spring-cloud-azure-overview



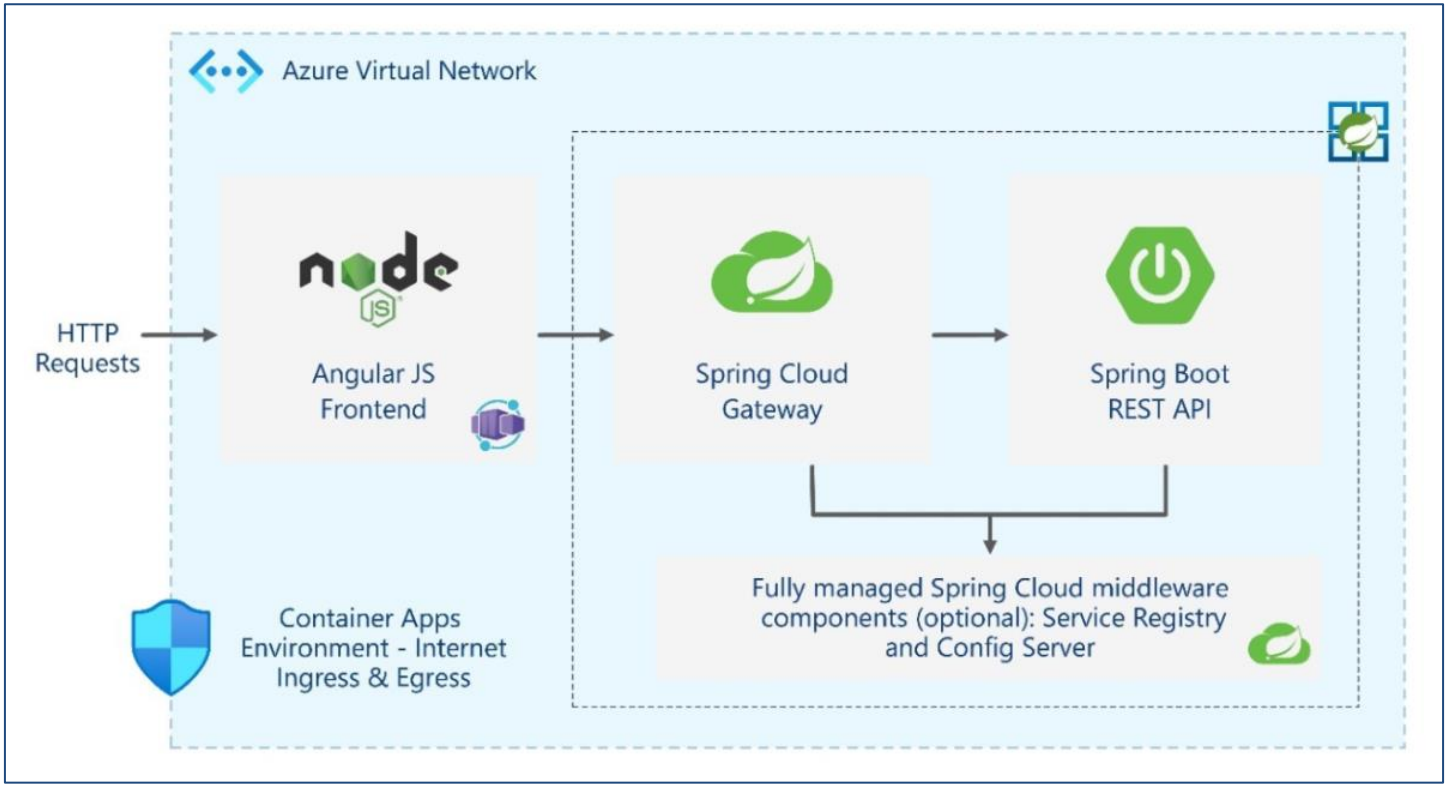
- Spring Cloud Azure:
  - https://learn.microsoft.com/pt-br/azure/developer/java/spring-framework/spring-cloud-azure-overview



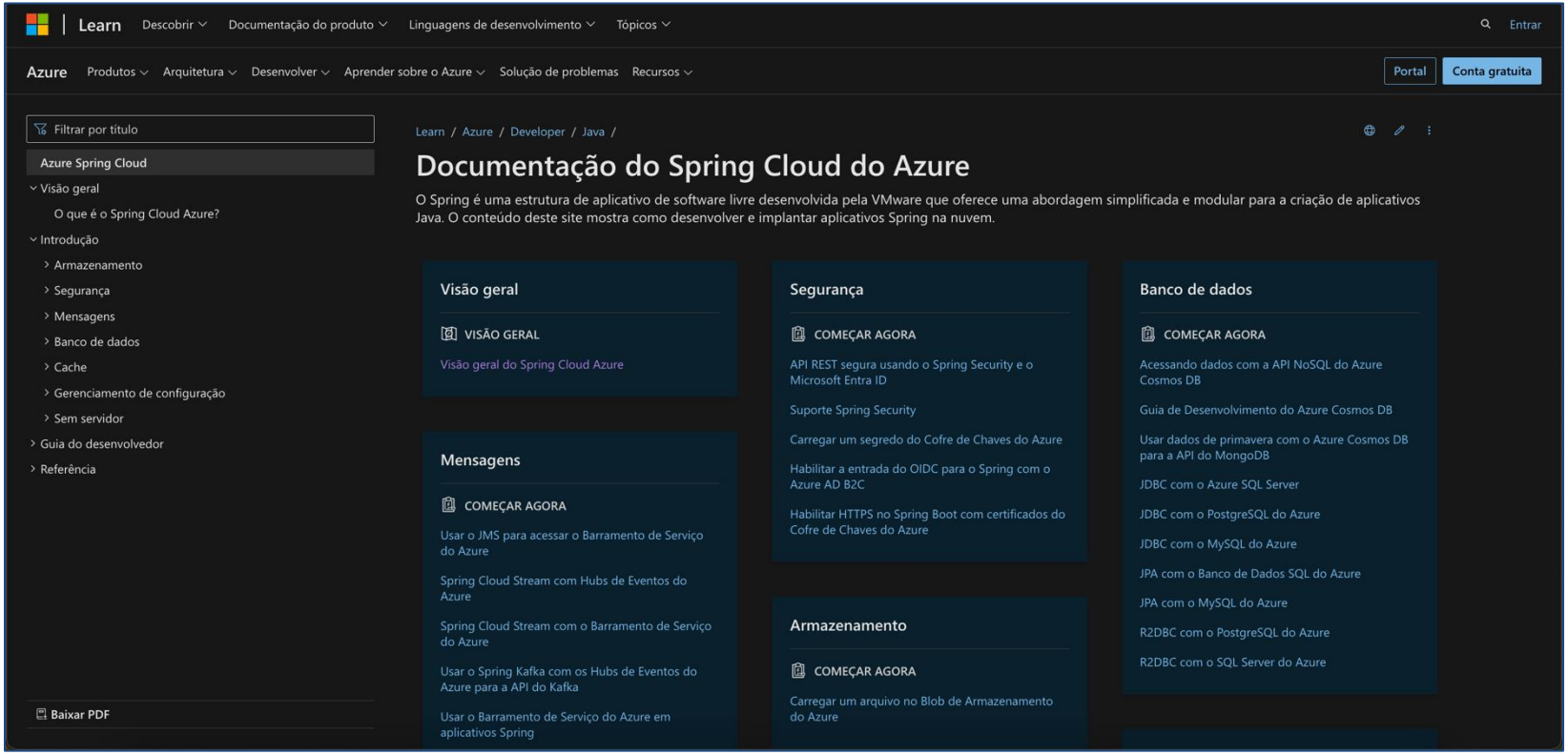
- Azure Spring Apps:
  - https://learn.microsoft.com/pt-br/azure/spring-apps/overview



- Arquitetura de uma rede virtual nos Aplicativos Spring do Azure:
  - https://learn.microsoft.com/pt-br/azure/spring-apps/overview



- Documentação do Spring Cloud Azure:
  - https://learn.microsoft.com/pt-br/azure/developer/java/spring-framework/



Bônus

- **Vamos à prática!**
  - Configuração distribuída com Spring Cloud =D

  - Você foi designado(a) para criar uma arquitetura de configuração distribuída utilizando Spring Cloud em dois projetos distintos:
    - **SpringCloudServer** e **SpringCloudClient**.
    - Cada projeto desempenha um papel específico na troca de configurações entre servidor e cliente.

Bônus

- **Vamos à prática!**
  - No projeto **SpringCloudServer**, você implementará um controlador (**ServerHelloController**) que responde à requisições na rota "/hello-server".

  - Este controlador busca uma mensagem de saudação a partir de configurações centralizadas, especificamente configuradas no arquivo config/application.properties. O servidor utiliza a dependência do **Spring Cloud Config Server** para fornecer essas configurações centralizadas.
    - hello.message=Oi, sou uma mensagem do "Config Server"

Bônus

- **Vamos à prática!**
  - Já no projeto **SpringCloudClient**, você implementará um controlador (**ClientHelloController**) responsável por lidar e responder solicitações na rota "/hello-client".

  - Este controlador utiliza configurações provenientes do servidor configurado. A aplicação do cliente usa a dependência do **Spring Cloud Starter Config** para acessar as configurações distribuídas.

Bônus

- O exercício consiste em:
  - **Compreensão dos projetos**:
    - Analise o código nos dois projetos (**SpringCloudServer** e **SpringCloudClient**) para compreender como a comunicação entre o servidor de configuração e o cliente é estabelecida.

    - Observe as dependências presentes em cada projeto e compreenda o papel de cada uma na configuração distribuída.

Bônus

- O exercício consiste em:
  - **Exploração do Spring Cloud Config**:
    - Pesquise sobre as anotações e configurações utilizadas nos controladores e nos arquivos de propriedades (**application.properties**) para entender como as configurações são injetadas nos controladores.

    - Documentação do Spring Cloud Config:
      - https://docs.spring.io/spring-cloud-config/docs/current/reference/html/

- Documentação do **Spring Cloud Config Server**:
  - https://docs.spring.io/spring-cloud-config/docs/current/reference/html/#_spring_cloud_config_server

- Documentação do **Spring Cloud Config Client**:
  - https://docs.spring.io/spring-cloud-config/docs/current/reference/html/#_spring_cloud_config_client

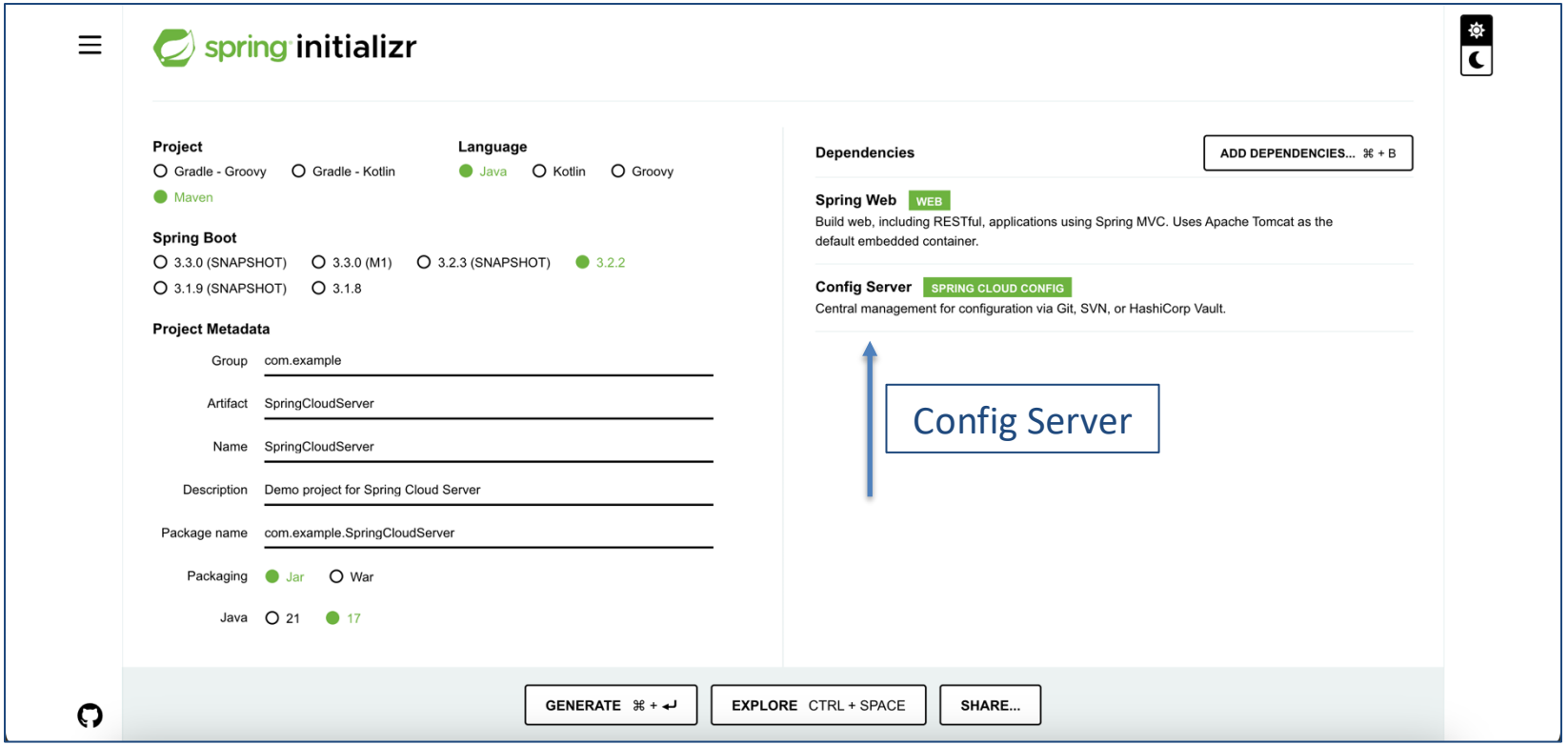
Bônus

- O exercício consiste em:
  - **Configuração adicional**:
    - Experimente adicionar novas configurações ao servidor e observe como essas alterações afetam a resposta do cliente.
    - Tente modificar as rotas e as mensagens nos controladores para personalizar a interação entre o servidor e o cliente.

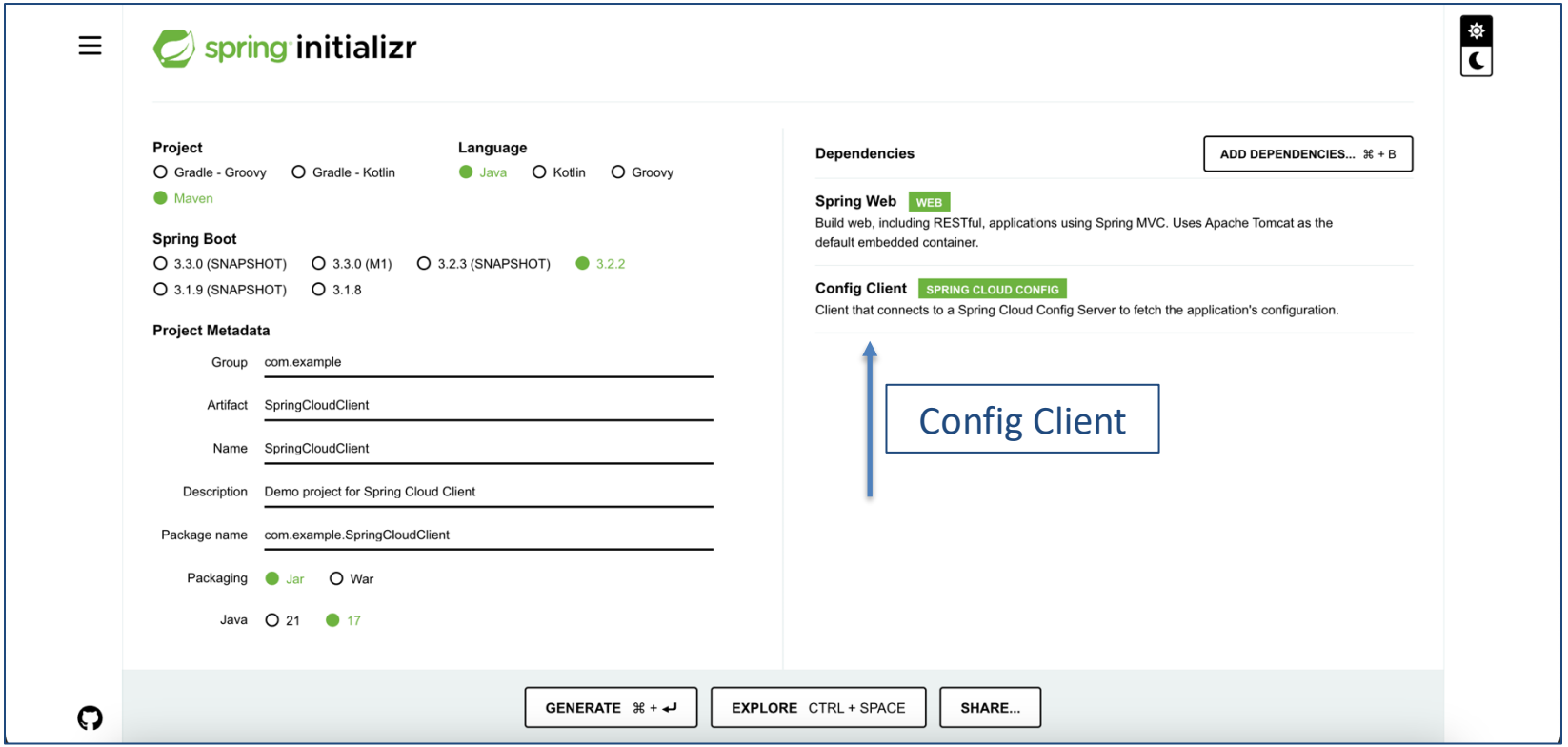
Bônus

- O exercício consiste em:
  - **Resenha**:
    - Elabore uma breve resenha sobre as vantagens de se ter um servidor de configuração centralizado em um ambiente distribuído.
    - Considere situações em que essa abordagem pode ser benéfica e elenque possíveis desafios que podem surgir.

- Primeiramente, crie a aplicação **SpringCloudServer**:
  - https://start.spring.io/



- Em seguida, crie a aplicação **SpringCloudClient**:
  - https://start.spring.io/



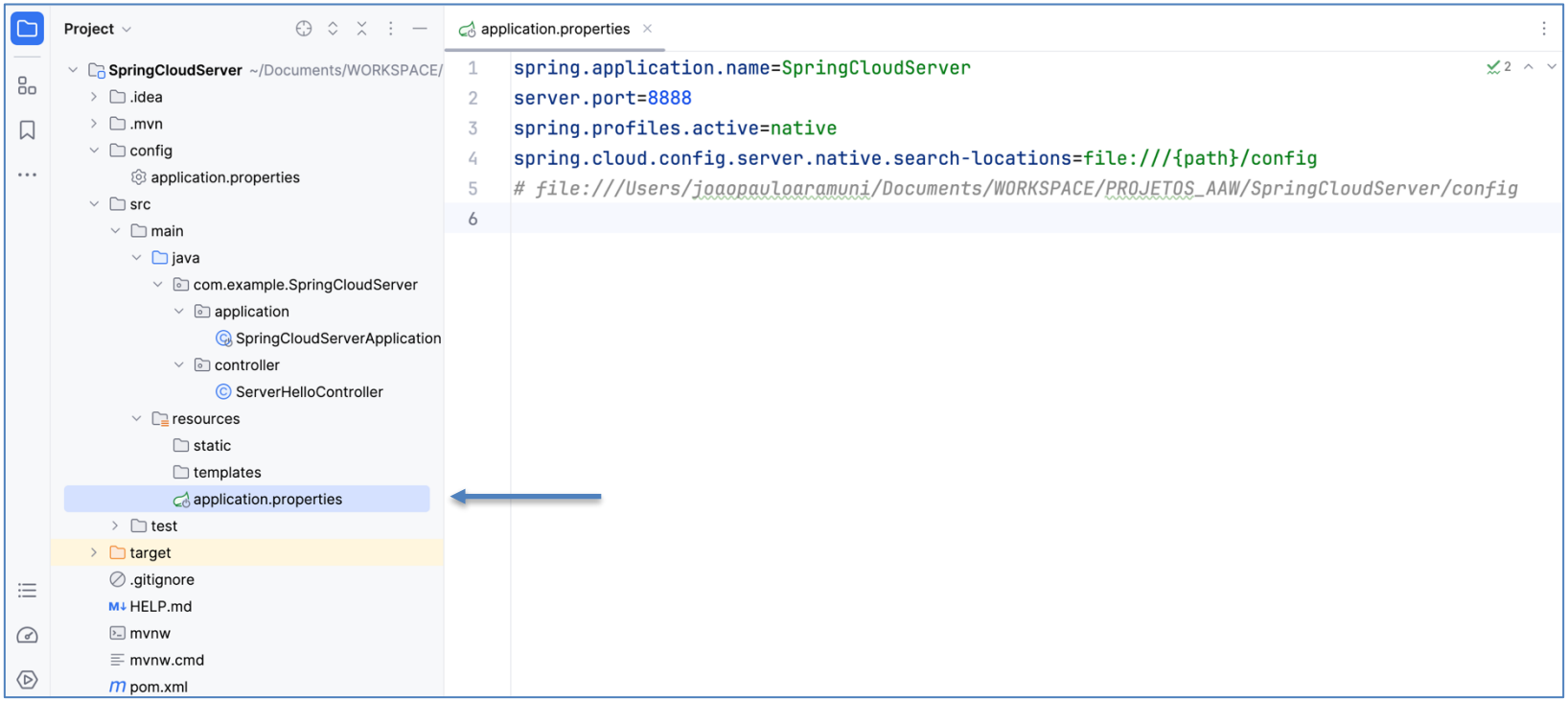
- Projeto **SpringCloudServer**
  - Arquivo **pom.xml**
  - spring-cloud-config-server

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
        <!--
            spring-cloud-config-server: Usado em um aplicativo que será um servidor de configuração.
            spring-cloud-starter-config: Usado em um aplicativo que será um cliente de configuração.
        -->
    </dependency>
</dependencies>
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

- Projeto **SpringCloudClient**
  - Arquivo **pom.xml**
  - spring-cloud-starter-config

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-config</artifactId>
        <!--
            spring-cloud-config-server: Usado em um aplicativo que será um servidor de configuração.
            spring-cloud-starter-config: Usado em um aplicativo que será um cliente de configuração.
        -->
    </dependency>
</dependencies>
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

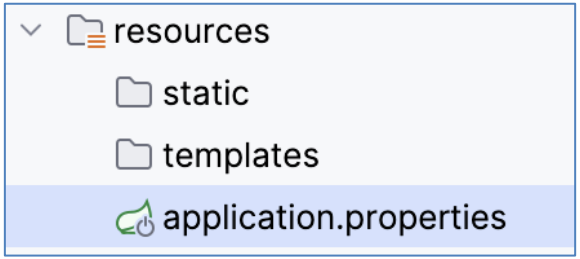
- Projeto **SpringCloudServer**
  - Arquivo resources/**application.properties**



- Projeto **SpringCloudServer**
  - Arquivo resources/**application.properties**

  - **spring.application.name=SpringCloudServer**: Define o nome da aplicação Spring Boot. Neste contexto, pode ser interpretado como o nome do serviço de configuração que está sendo executado.

  - **server.port=8888**: Define a porta em que o serviço de configuração será executado. Neste caso, outras aplicações buscarão suas configurações nesse serviço na porta 8888.

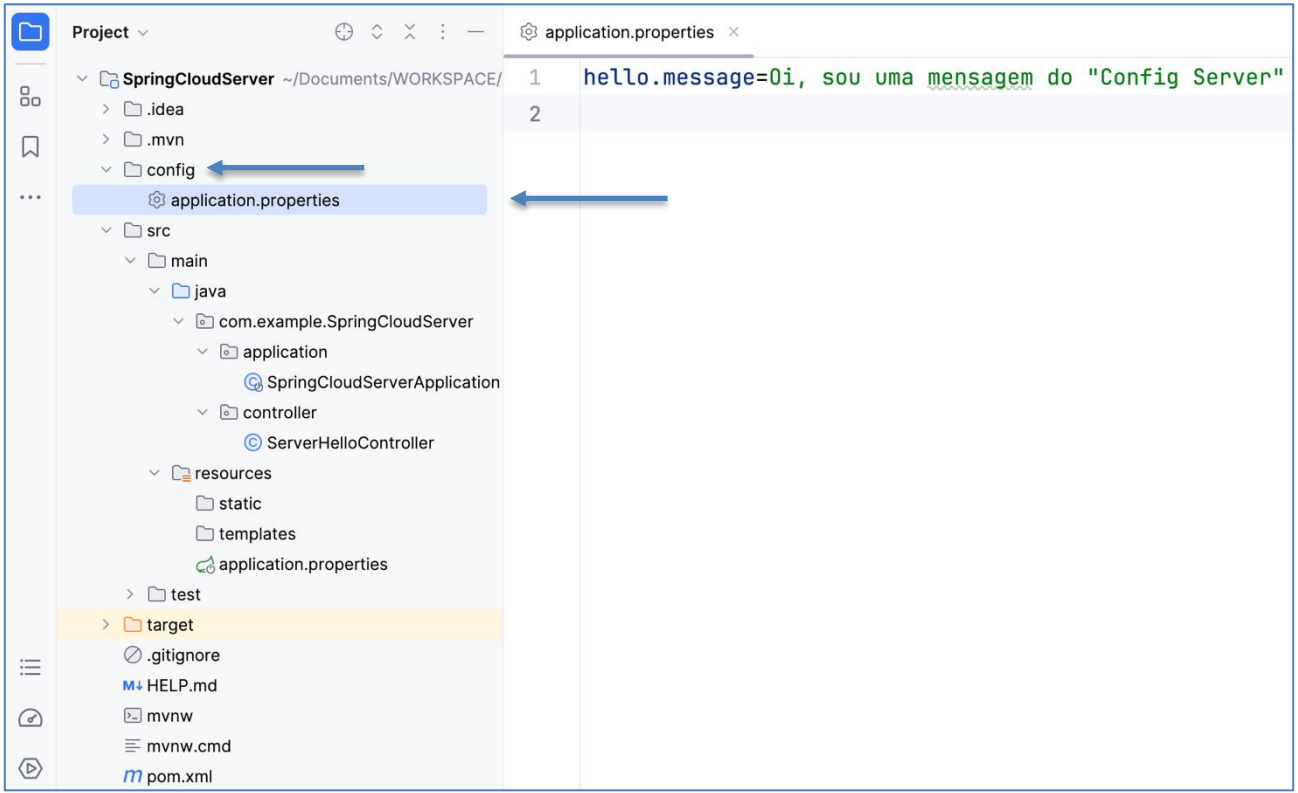


- Projeto **SpringCloudServer**
  - Arquivo resources/**application.properties**

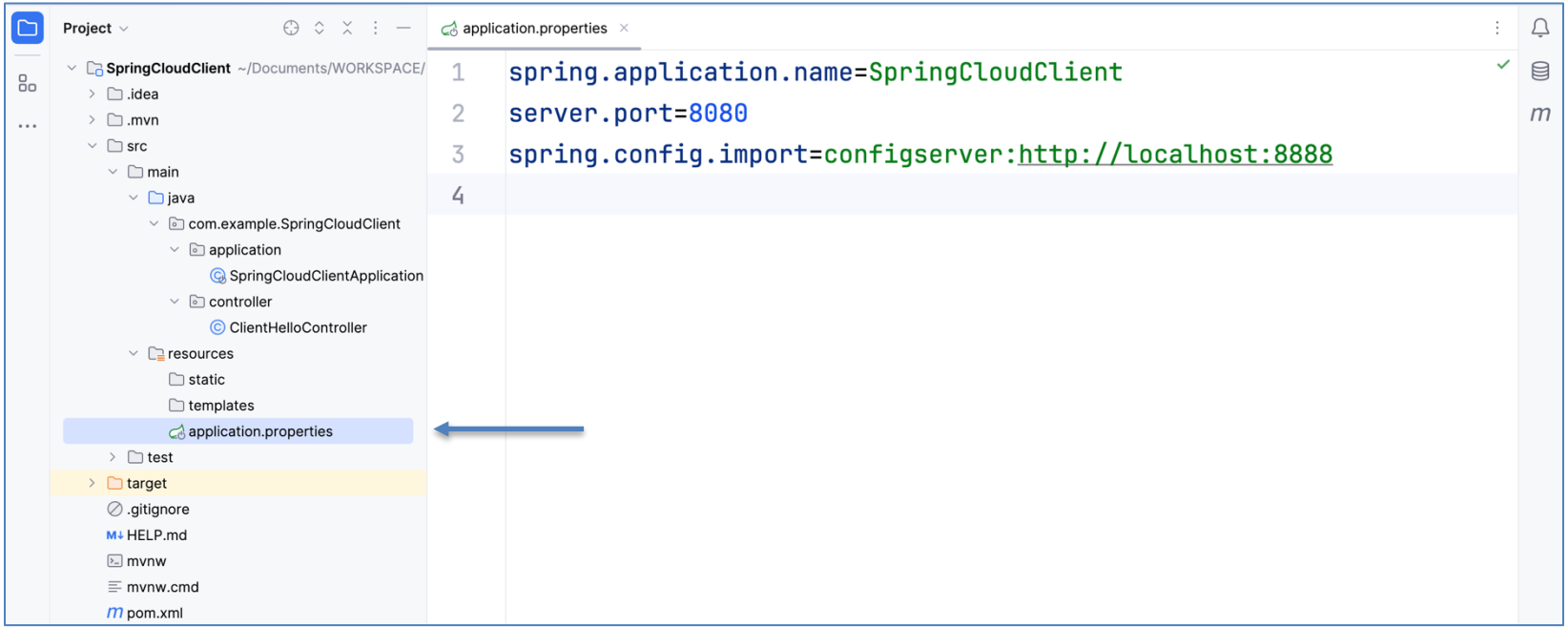
  - **spring.profiles.active=native**: Define o perfil ativo da aplicação. No contexto de um servidor de configuração, o perfil pode influenciar de onde as configurações são carregadas. O perfil "native" geralmente significa que as configurações serão carregadas a partir do sistema de arquivos local.

  - **spring.cloud.config.server.native.search-locations=file:///{path}/config**: Especifica o local onde o servidor de configuração deve procurar pelos arquivos de configuração nativos. Neste caso, o servidor está configurado para procurar no diretório local fornecido. O {path}: Indica que os arquivos de configuração estão localizados no caminho absoluto especificado.

- Projeto **SpringCloudServer**
  - Arquivo config/**application.properties**



- Projeto **SpringCloudClient**
  - Arquivo resources/**application.properties**



- Projeto **SpringCloudClient**
  - Arquivo resources/**application.properties**

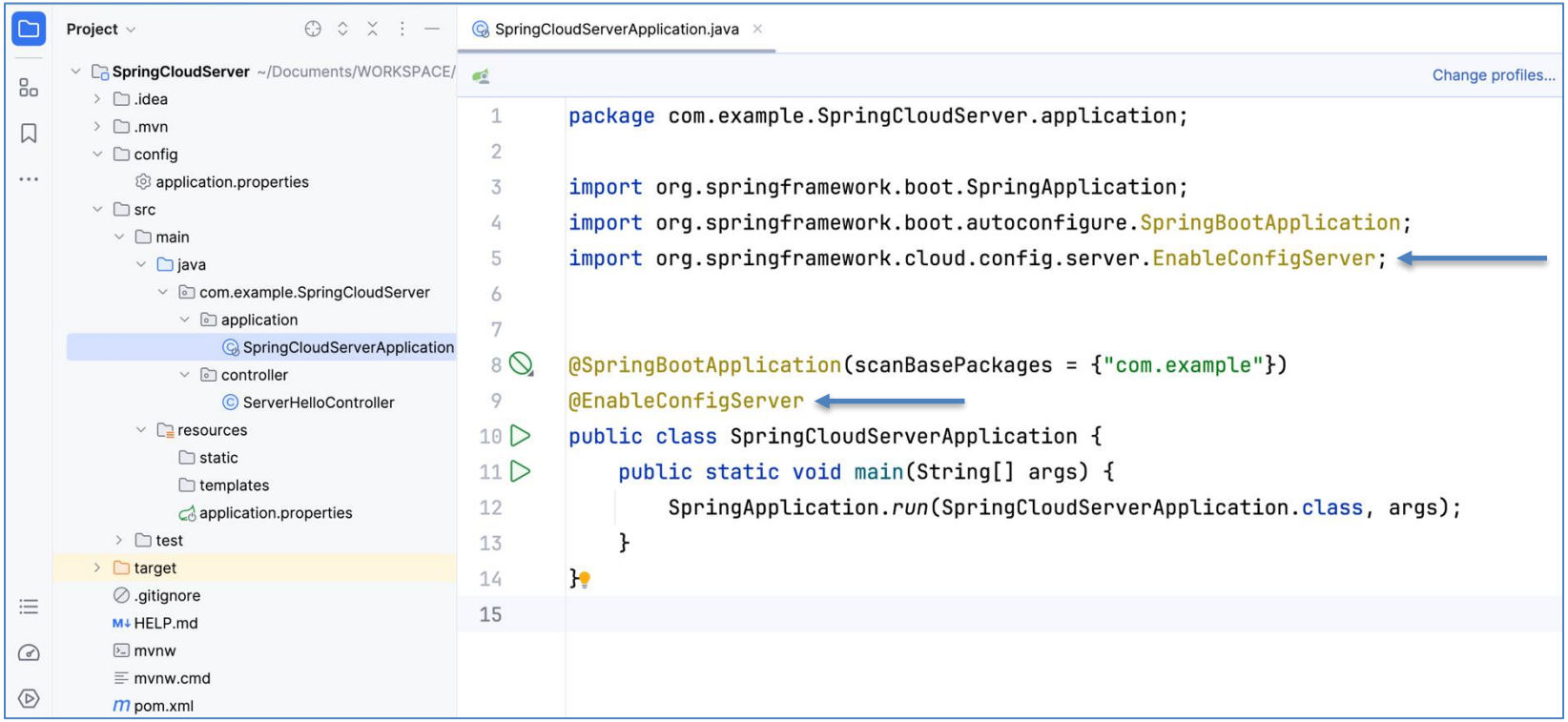
  - **spring.application.name=SpringCloudClient**: Define o nome da aplicação Spring Boot. Isso é usado como identificador único para a aplicação no ambiente distribuído. No contexto do Spring Cloud, isso é frequentemente usado para localizar configurações específicas para essa aplicação no servidor de configuração.

  - **server.port=8080**: Especifica a porta em que a aplicação estará disponível para receber solicitações. Neste caso, a aplicação estará acessível e “escutando” requisições por meio da porta **8080**.

- Projeto **SpringCloudClient**
  - Arquivo resources/**application.properties**

  - **spring.config.import=configserver:http://localhost:8888**: Indica que a aplicação Spring Boot deve importar suas configurações do servidor de configuração, que é o Spring Cloud Config Server.
    - **configserver**: é o protocolo de importação que o Spring Boot usa para integrar-se ao servidor de configuração.
    - **http://localhost:8888** é a URL do servidor de configuração. O cliente buscará suas configurações nesse servidor. Aqui, está configurado para procurar o servidor de configuração localmente na porta 8888. Isso significa que o servidor de configuração deve estar em execução em **http://localhost:8888**.

- Projeto **SpringCloudServer**
  - Arquivo **SpringCloudServerApplication**



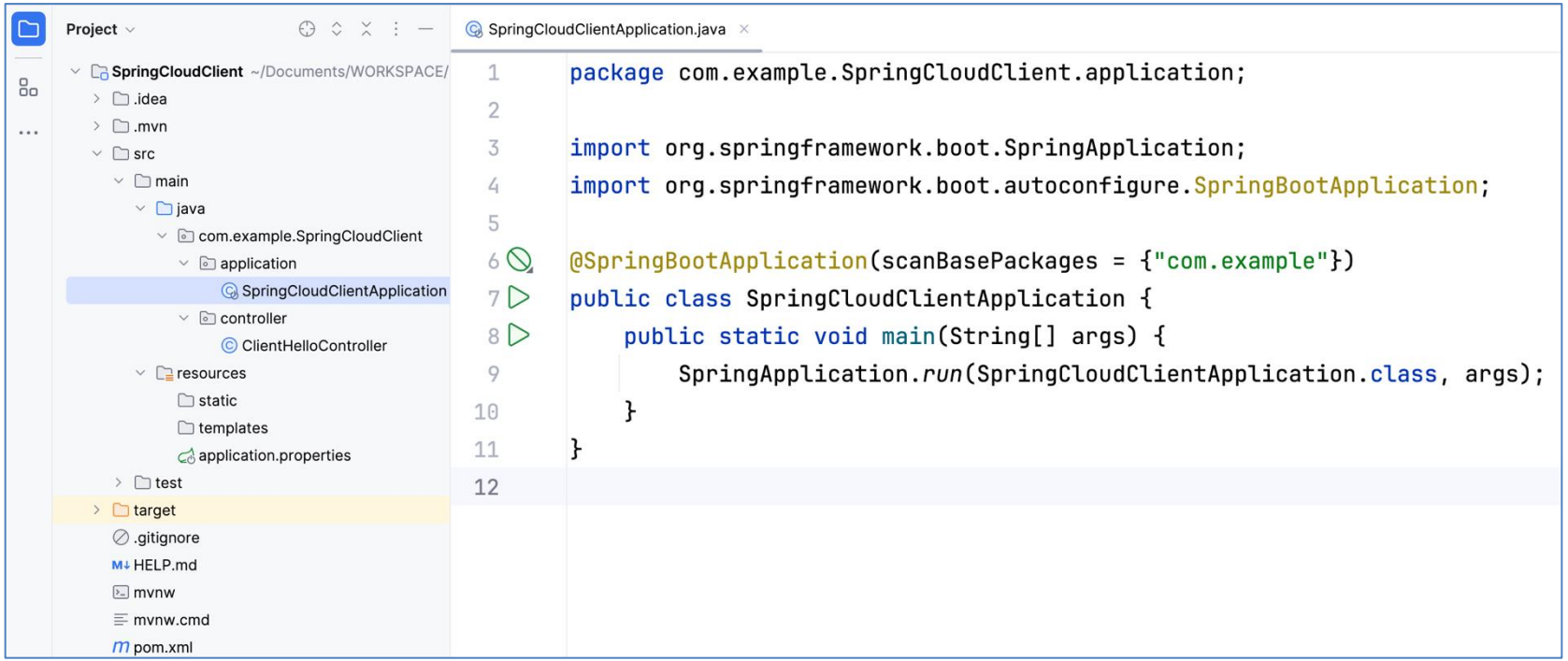
- Projeto **SpringCloudServer**
  - Arquivo **SpringCloudServerApplication**

  - A anotação **@EnableConfigServer** é usada para habilitar e configurar o servidor de configuração do Spring Cloud Config Server.

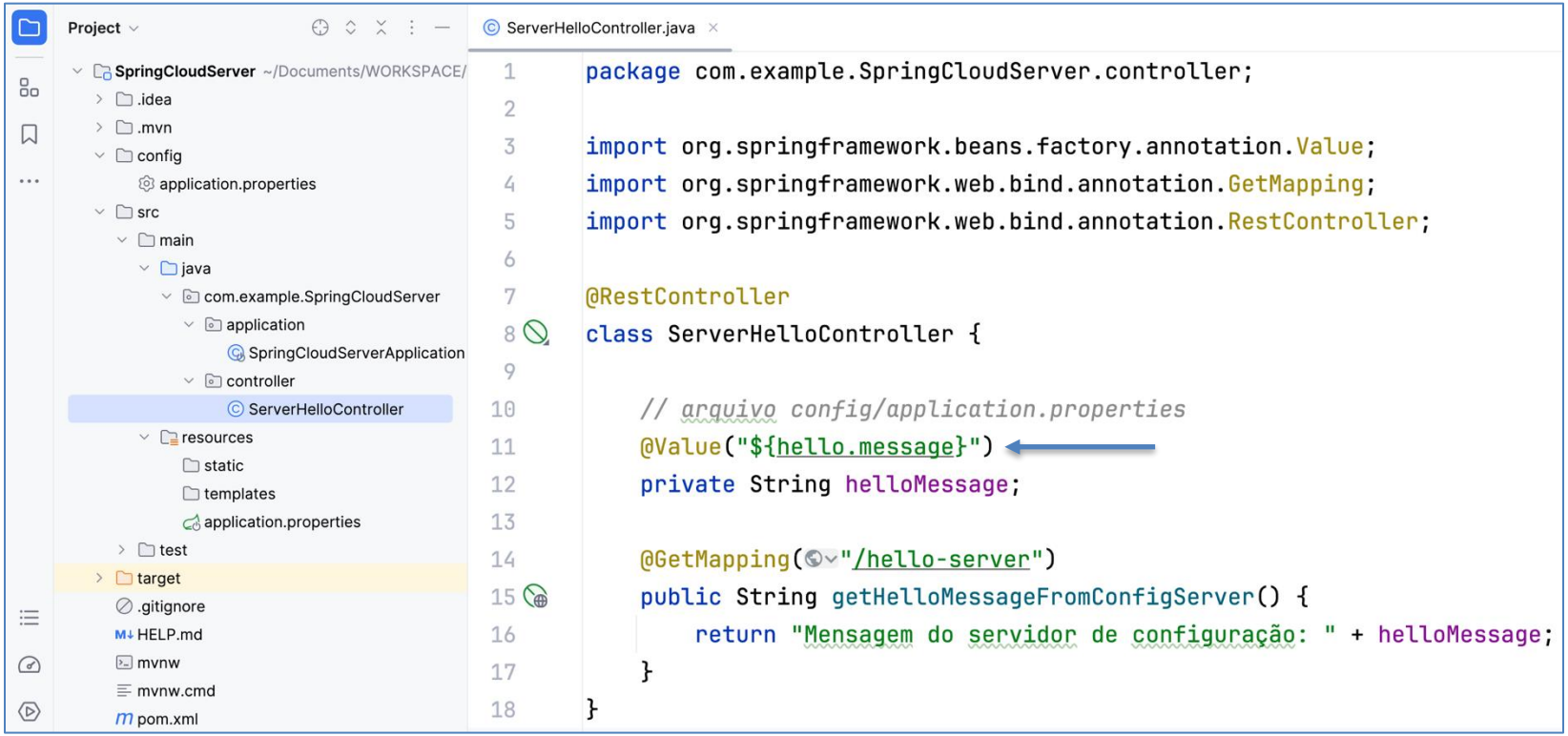
  - **@EnableConfigServer** é uma anotação que sinaliza ao Spring que a aplicação deve atuar como um servidor de configuração. Isso significa que o aplicativo irá fornecer configurações centralizadas para outros aplicativos (SpringCloudClient) em um ambiente distribuído.

  - O principal propósito do servidor de configuração é centralizar as configurações de vários serviços em um local. Isso facilita a gestão, atualização e manutenção das configurações em ambientes de microsserviços.

- Projeto **SpringCloudClient**
  - Arquivo **SpringCloudClientApplication**



- Projeto **SpringCloudServer**
  - Arquivo **ServerHelloController**



- Projeto **SpringCloudServer**
  - Arquivo **ServerHelloController**

  - A anotação **@Value** é usada no Spring Framework para injetar valores diretamente em variáveis, métodos ou construtores.

  - Neste exemplo, a anotação @Value("${hello.message}") é usada para injetar o valor da **propriedade** hello.message do arquivo de configuração config/application.properties na **variável** helloMessage.

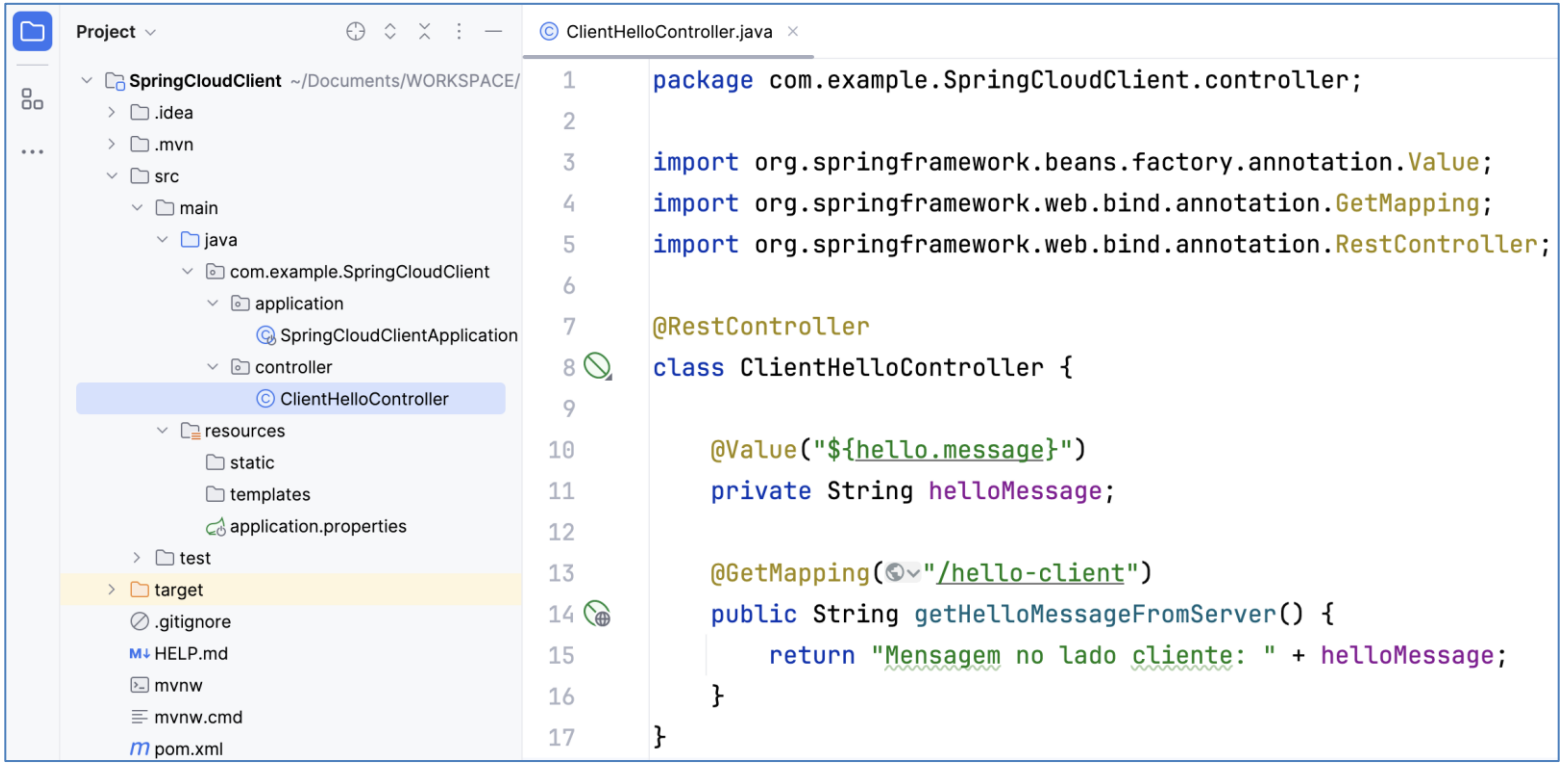
```
@RestController
class ServerHelloController {

    // arquivo config/application.properties
    @Value("${hello.message}")
    private String helloMessage;
```

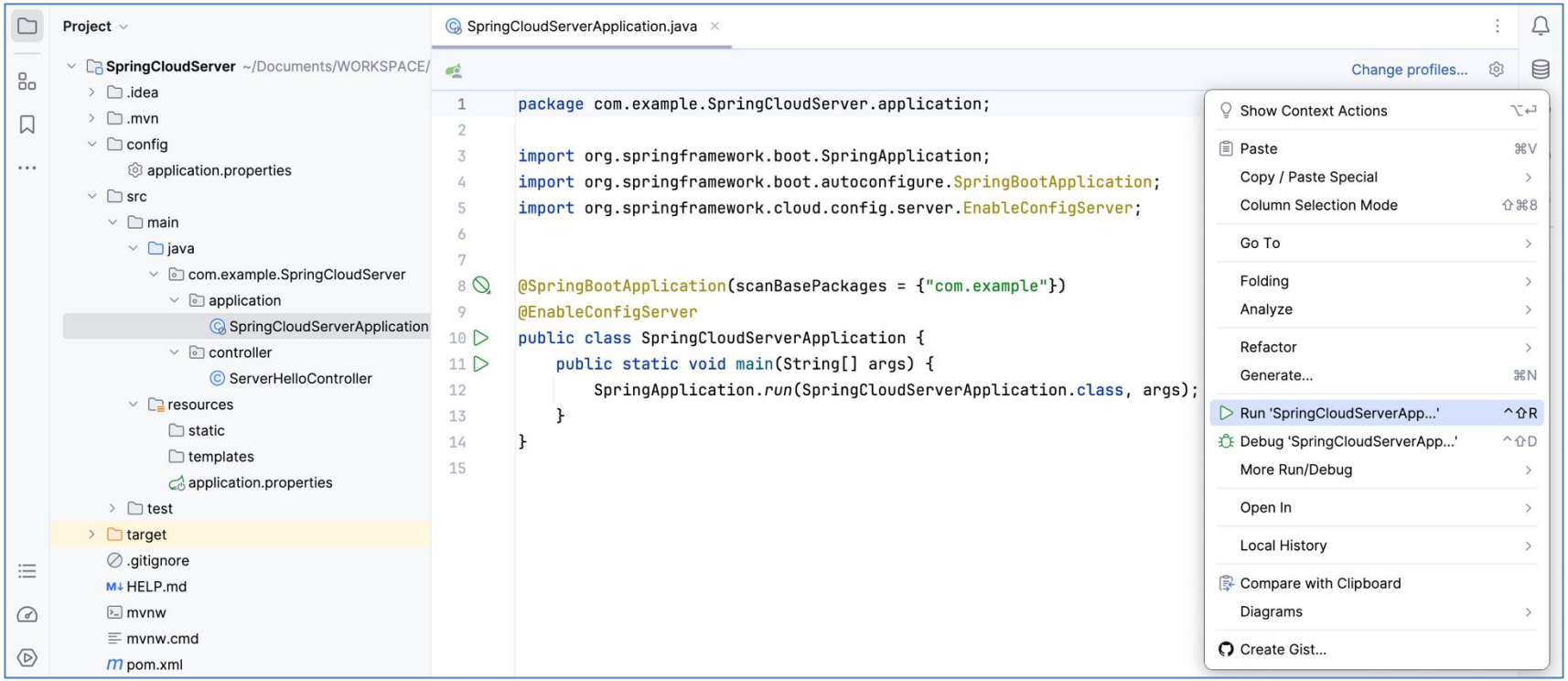
```
@RestController
class ClientHelloController {

    @Value("${hello.message}")
    private String helloMessage;
```

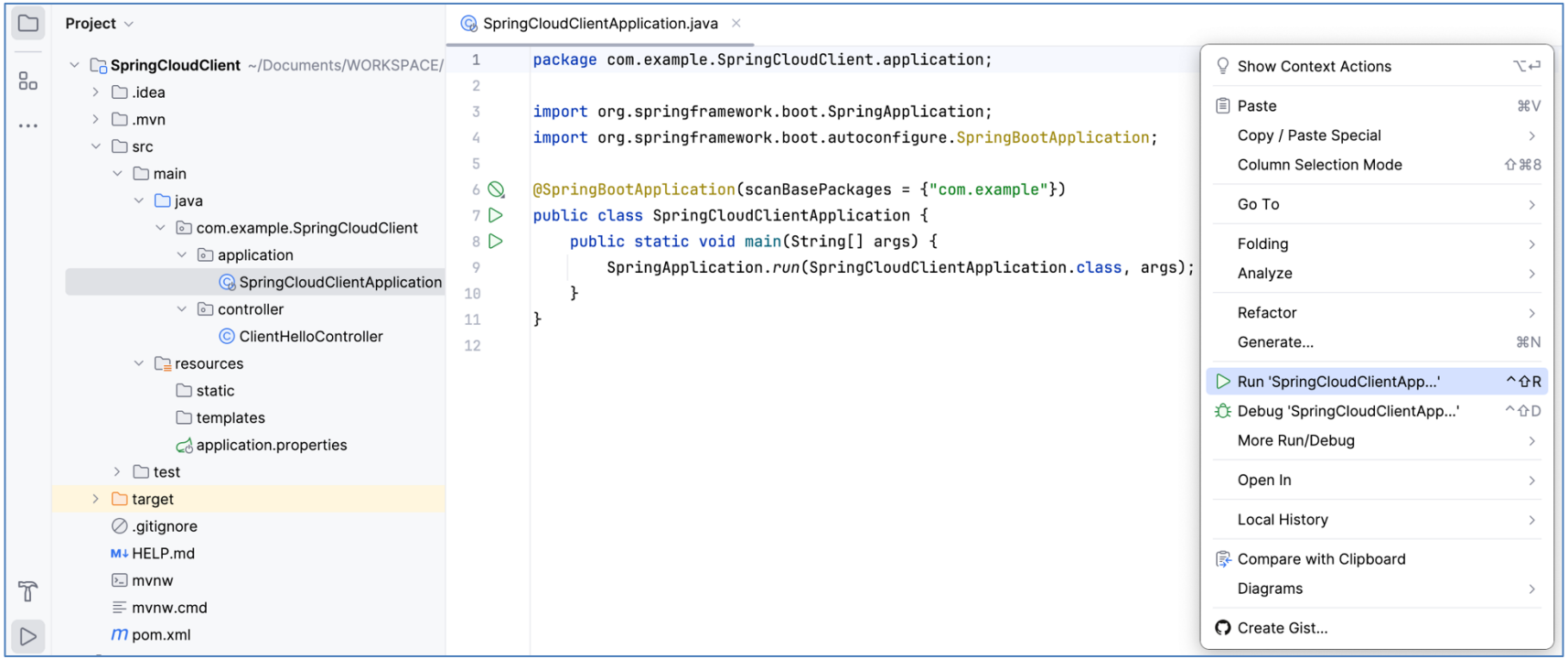
- Projeto **SpringCloudClient**
  - Arquivo **ClientHelloController**



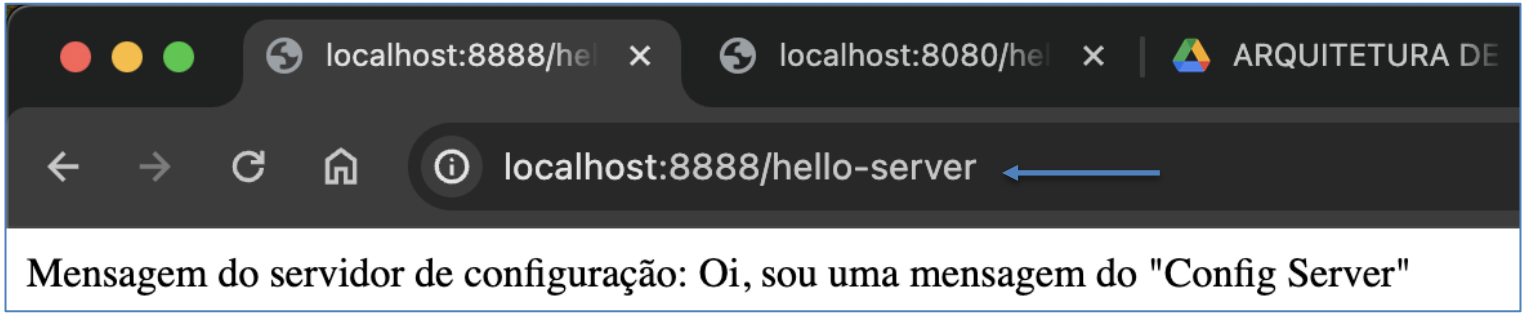
- Projeto **SpringCloudServer**
  - Execute primeiro o **SpringCloudServerApplication**

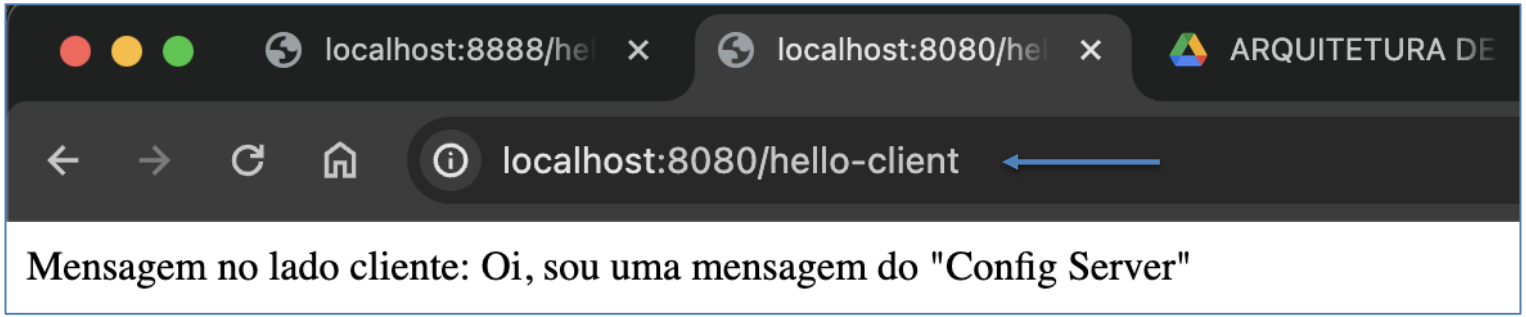


- Projeto **SpringCloudClient**
  - Em seguida, execute o **SpringCloudClientApplication**



- Acesse as URLs (atente-se para as portas):
  - http://localhost:8888/hello-server
  - http://localhost:8080/hello-client





Exercício 2

- Incorpore o Spring Cloud ao projeto JWT RestAPI.

- Este projeto visa aprimorar uma API RESTful existente, que já possui autenticação baseada em tokens JWT (JSON Web Token), integrando o Spring Cloud para otimizar a escalabilidade, modularidade e eficiência operacional do sistema.

Exercício 2

- Principais tarefas:
  - **Spring Cloud Config**: Utilize o Spring Cloud Config para gerenciar as configurações da aplicação de forma centralizada.
  - **Spring Cloud Eureka**: Implemente o serviço de descoberta de instâncias para facilitar a escalabilidade e a descoberta de microsserviços.
  - **Spring Cloud Gateway**: Implemente um gateway para direcionar e rotear as requisições de forma eficiente.
  - **Spring Cloud Security**: Utilize o Spring Cloud Security para reforçar a segurança entre os microsserviços.

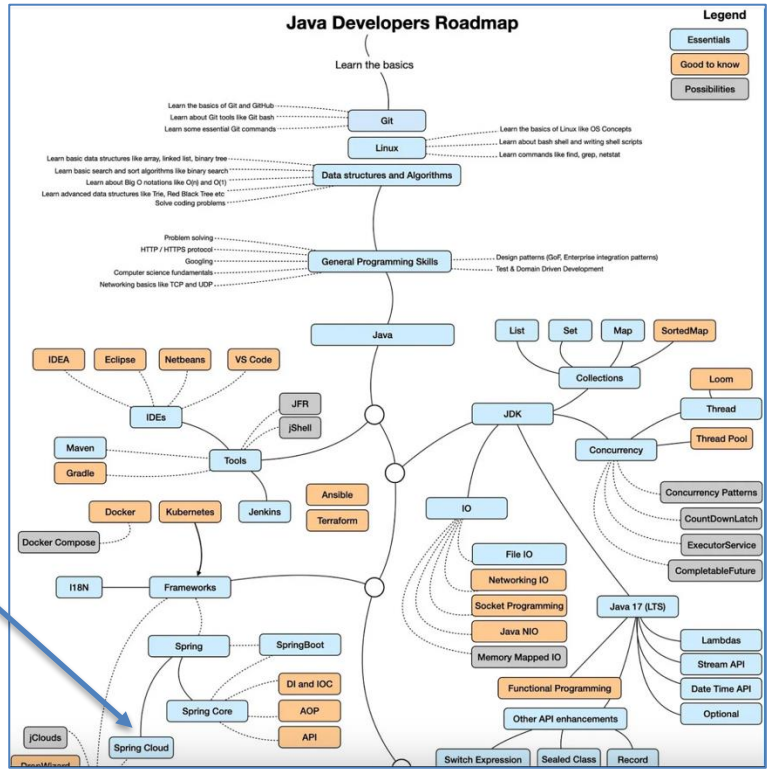
Exercício 2

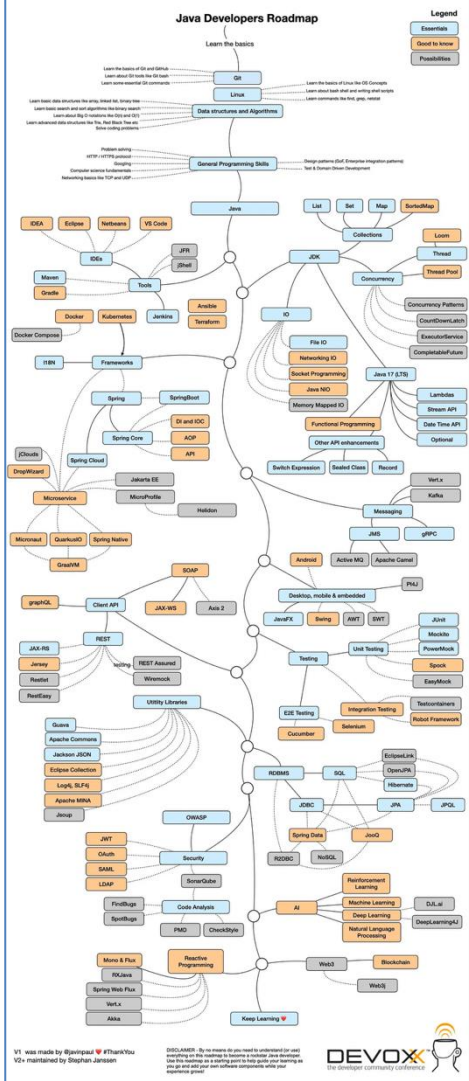
- Principais tarefas:
  - **Spring Cloud Config**: https://spring.io/projects/spring-cloud-config
    - Implemente a configuração centralizada de propriedades e parâmetros da aplicação.
    - Utilize o suporte a ambientes específicos para gerenciar configurações em diferentes cenários.
  - **Spring Cloud Eureka**: https://spring.io/projects/spring-cloud-netflix
    - Implemente o registro dinâmico de instâncias de microsserviços.
    - Forneça uma interface de descoberta para facilitar a comunicação entre serviços.
    - Explore as estratégias de balanceamento de carga oferecidas pelo Eureka.

Exercício 2

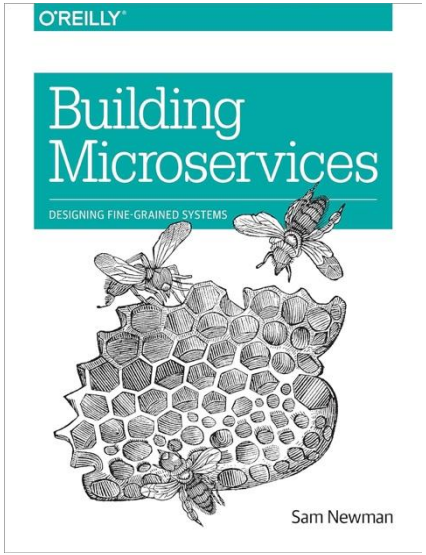
- Principais tarefas:
  - **Spring Cloud Gateway**: https://spring.io/projects/spring-cloud-gateway
    - Implemente o roteamento eficiente de requisições para serviços específicos.
    - Utilize a implementação de filtros para manipulação de requisições e respostas.
    - Integre com o Eureka para descoberta dinâmica de serviços.
  - **Spring Cloud Security**: https://spring.io/projects/spring-cloud-security
    - Implemente a autenticação e autorização entre os microsserviços.
    - Utilize tokens JWT para garantir a segurança nas comunicações.
    - Configure políticas de segurança específicas para cada serviço.

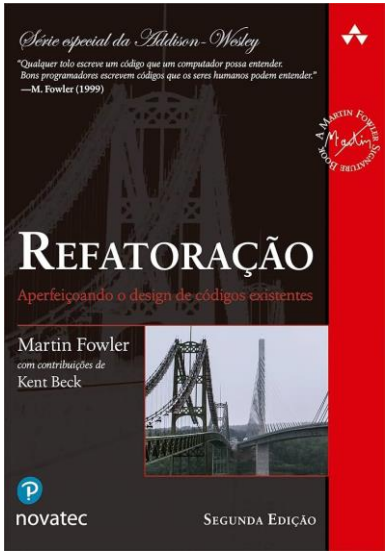
- Leitura sugerida:
  - https://medium.com/javarevisited/the-java-programmer-roadmap-f9db163ef2c2

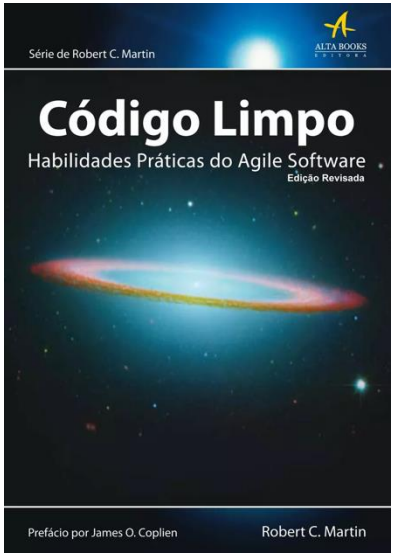




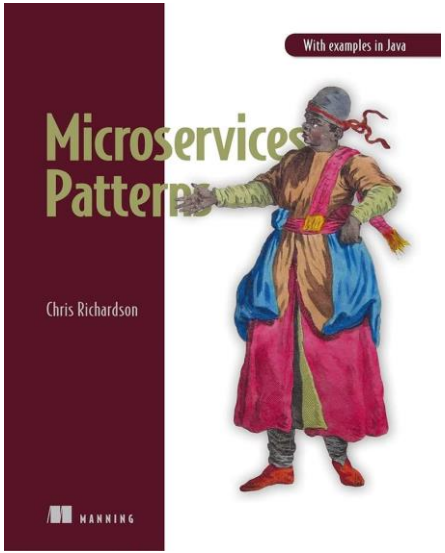
Referências básicas

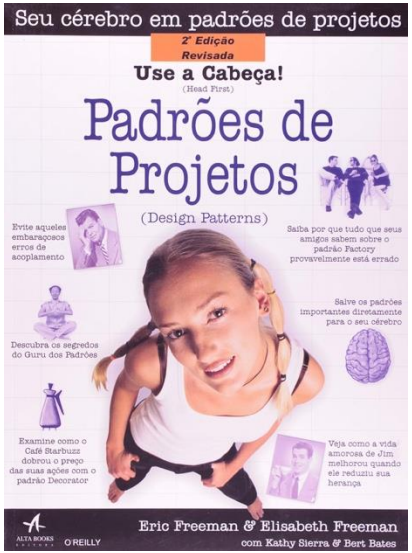


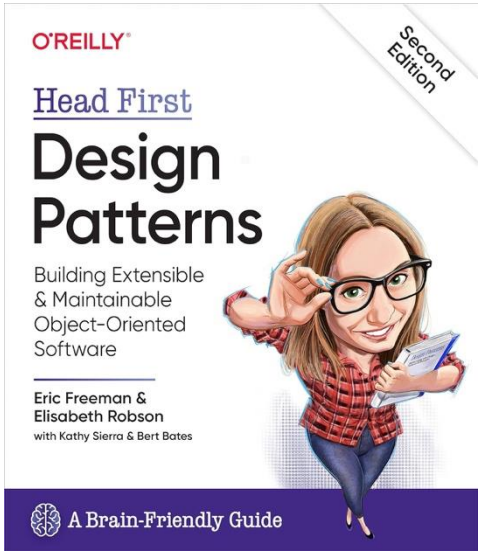


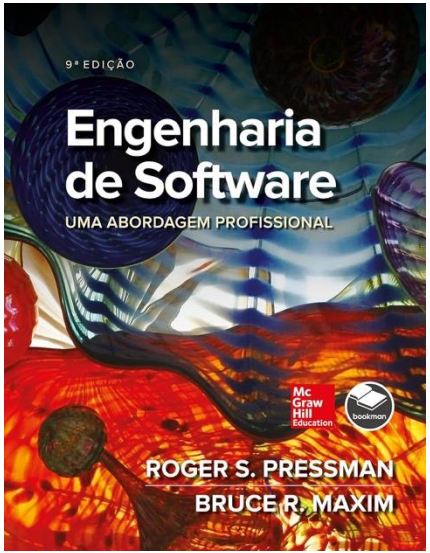


Referências complementares

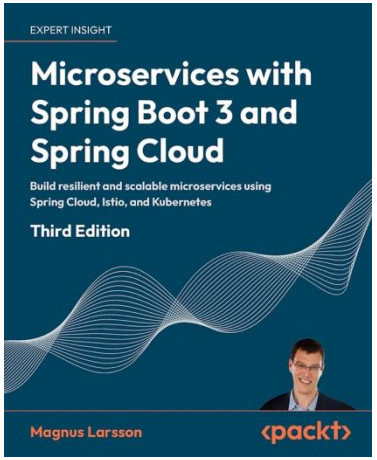


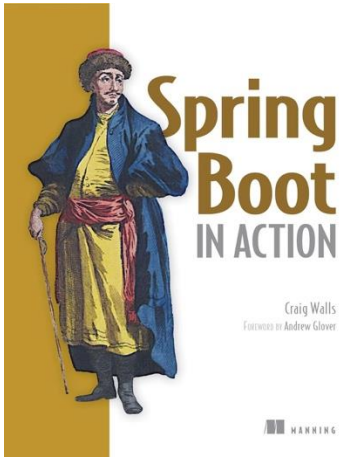


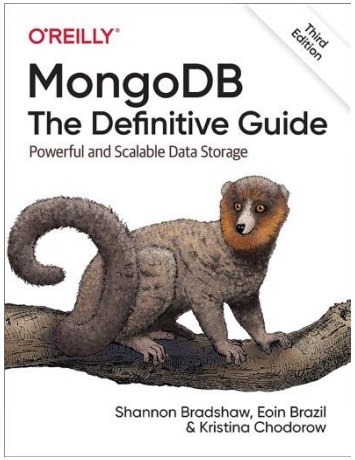


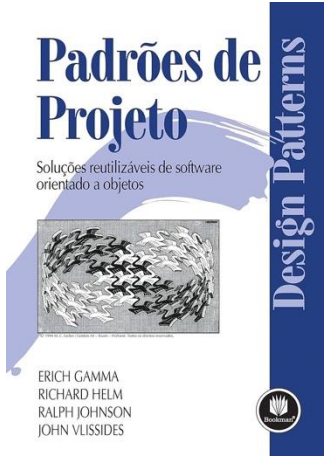


Outras referências









Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com

GitHub LinkedIn Lattes